



AN0400

Ameba-D Application Note



Realtek Semiconductor Corp.

No. 2, Innovation Road II, Hsinchu Science Park, Hsinchu 300, Taiwan

Tel.: +886-3-578-0211. Fax: +886-3-577-6047

www.realtek.com

COPYRIGHT

©2021 Realtek Semiconductor Corp. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form or by any means without the written permission of Realtek Semiconductor Corp.

DISCLAIMER

Please Read Carefully:

Realtek Semiconductor Corp., (Realtek) reserves the right to make corrections, enhancements, improvements and other changes to its products and services. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

Reproduction of significant portions in Realtek data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Realtek is not responsible or liable for such reproduced documentation. Information of third parties may be subject to additional restrictions.

Buyers and others who are developing systems that incorporate Realtek products (collectively, “Customers”) understand and agree that Customers remain responsible for using their independent analysis, evaluation and judgment in designing their applications and that Customers have full and exclusive responsibility to assure the safety of Customers' applications and compliance of their applications (and of all Realtek products used in or for Customers' applications) with all applicable regulations, laws and other applicable requirements. Designer represents that, with respect to their applications, Customer has all the necessary expertise to create and implement safeguards that (1) anticipate dangerous consequences of failures, (2) monitor failures and their consequences, and (3) lessen the likelihood of failures that might cause harm and take appropriate actions. Customer agrees that prior to using or distributing any applications that include Realtek products, Customer will thoroughly test such applications and the functionality of such Realtek products as used in such applications.

Realtek's provision of technical, application or other design advice, quality characterization, reliability data or other services or information, including, but not limited to, reference designs and materials relating to evaluation kits, (collectively, “Resources”) are intended to assist designers who are developing applications that incorporate Realtek products; by downloading, accessing or using Realtek's Resources in any way, Customer (individually or, if Customer is acting on behalf of a company, Customer's company) agrees to use any particular Realtek Resources solely for this purpose and subject to the terms of this Notice.

Realtek's provision of Realtek Resources does not expand or otherwise alter Realtek's applicable published warranties or warranty disclaimers for Realtek's products, and no additional obligations or liabilities arise from Realtek providing such Realtek Resources. Realtek reserves the right to make corrections, enhancements, improvements and other changes to its Realtek Resources. Realtek has not conducted any testing other than that specifically described in the published documentation for a particular Realtek Resource.

Customer is authorized to use, copy and modify any individual Realtek Resource only in connection with the development of applications that include the Realtek product(s) identified in such Realtek Resource. No other license, express or implied, by estoppel or otherwise to any other Realtek intellectual property right, and no license to any technology or intellectual property right of Realtek or any third party is granted herein, including but not limited to any patent right, copyright, mask work right, or other intellectual property right relating to any combination, machine, or process in which Realtek products or services are used. Information regarding or referencing third-party products or services does not constitute a license to use such products or services, or a warranty or endorsement thereof. Use of Realtek Resources may require a license from a third party under the patents or other intellectual property of the third party, or a license from Realtek under the patents or other Realtek's intellectual property.

Realtek's Resources are provided “as is” and with all faults. Realtek disclaims all other warranties or representations, express or implied, regarding resources or use thereof, including but not limited to accuracy or completeness, title, any epidemic failure warranty and any implied warranties of merchantability, fitness for a particular purpose, and non-infringement of any third party intellectual property rights.

Realtek shall not be liable for and shall not defend or indemnify Customer against any claim, including but not limited to any infringement claim that related to or is based on any combination of products even if described in Realtek Resources or otherwise. In no event shall Realtek be liable for any actual, direct, special, collateral, indirect, punitive, incidental, consequential or exemplary damages in connection with or arising out of Realtek's Resources or use thereof, and regardless of whether Realtek has been advised of the possibility of such damages. Realtek is not responsible for any failure to meet such industry standard requirements.

Where Realtek specifically promotes products as facilitating functional safety or as compliant with industry functional safety standards, such products are intended to help enable customers to design and create their own applications that meet applicable functional safety standards and requirements. Using products in an application does not by itself establish any safety features in the application. Customers must ensure compliance with safety-related requirements and standards applicable to their applications. Designer may not use any Realtek products in life-critical medical equipment unless authorized officers of the parties have executed a special contract specifically governing such use. Life-critical medical equipment is medical equipment where failure of such equipment would cause serious bodily injury or death. Such equipment includes, without limitation, all medical devices identified by the U.S.FDA as Class III devices and equivalent classifications outside the U.S.

Customers agree that it has the necessary expertise to select the product with the appropriate qualification designation for their applications and that proper product selection is at Customers' own risk. Customers are solely responsible for compliance with all legal and regulatory requirements in connection with such selection.

Customer will fully indemnify Realtek and its representatives against any damages, costs, losses, and/or liabilities arising out of Designer's non-compliance with the terms and provisions of this Notice.

TRADEMARKS

Realtek is a trademark of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

USING THIS DOCUMENT

Though every effort has been made to ensure that this document is current and accurate, more information may have become available subsequent to the production of this guide.

Contents

Contents.....	4
Convention	13
1 Building Environment	14
1.1 Introduction.....	14
1.2 Preparing GCC Environment.....	14
1.2.1 Windows.....	14
1.2.2 Linux	15
1.3 Building Code.....	16
1.3.1 Normal Image.....	16
1.3.2 MP Image	18
1.4 Setting Debugger	18
1.4.1 Probe	18
1.4.2 J-Link.....	23
1.5 Downloading Image to Flash	28
1.6 Entering Debug Mode.....	28
1.7 Command List.....	29
1.8 GDB Debugger Basic Usage	30
1.9 Q & A	30
1.9.1 “Error 127” for Building Code	30
1.9.2 “Permission denied” Error under Linux.....	31
1.9.3 How to Reset KM0/KM4 under Debug Mode?	31
2 SDK Architecture	33
2.1 component	33
2.1.1 common.....	33
2.1.2 os	34
2.1.3 soc.....	34
2.2 doc.....	34
2.3 tools.....	34
2.4 GCC Project for KM4.....	35
2.5 Critical Header Files	35
3 GCC Makefile	36
3.1 KM4 Makefile Architecture	36
3.2 How to Build Code into Flash?.....	36
3.3 How to Build Code into SRAM?	37
3.4 How to Use Section Attribute?	38
3.5 How to Build Library?	39
3.6 How to Add Library?	40
4 C++ Standards Supported in GCC	41
4.1 Introduction.....	41
4.2 To Compile C++ Codes	41
5 GCC Standard Library.....	43

5.1	Introduction	43
5.2	Default Use of Library Function	43
5.3	To Use Configurable Function in GCC Standard Library	44
5.4	Tips	45
6	IAR Build Environment Setup.....	46
6.1	Requirement.....	46
6.1.1	IAR Embedded Workbench	46
6.1.2	J-Link or RLX Probe.....	46
6.2	Hardware Configuration	46
6.2.1	Connecting with J-Link.....	47
6.2.2	Connecting with RLX Probe.....	48
6.3	How to Use IAR SDK?.....	49
6.3.1	IAR Project Introduction	49
6.3.2	IAR Build	50
6.3.3	IAR Download	54
6.3.4	IAR Debug.....	58
6.3.5	IAR Memory Configuration.....	63
6.4	How to Build Sample Code?	64
6.5	Used Memory Size Calculation	65
6.5.1	Memory Section.....	65
6.5.2	Memory Size	66
7	Demo Board	68
7.1	PCB Layout Overview.....	68
7.2	Pin Out.....	69
7.3	DC Power Supply	70
7.4	USB Interface Configuration	71
7.5	LOGUART	71
7.6	SWD	72
7.7	VBAT ADC.....	72
8	ImageTool.....	73
8.1	Introduction.....	73
8.2	Environment Setup.....	73
8.2.1	Hardware Setup.....	73
8.2.2	Software Setup	74
8.3	Download	74
8.3.1	Image Download	74
8.3.2	Flash Erase.....	75
8.3.3	Flash Status Operation	76
8.4	Generate.....	79
8.4.1	Merge Images.....	79
8.4.2	Generate Cloud OTA Image	79
9	Memory Layout	80
9.1	Flash Program Layout	80
9.1.1	Image Header	81
9.1.2	System Data (4K)	82
9.1.3	User Data.....	82
9.1.4	4M Flash Usage Example.....	83
9.2	SRAM Layout	84
9.2.1	KM4 SRAM Layout	84

9.2.2	KM0 SRAM Layout	85
9.3	PSRAM Layout	86
9.4	Retention SRAM	87
9.4.1	Retention SRAM Layout	87
9.4.2	User Area	87
9.5	OTA Layout	88
9.5.1	OTA Program Layout	88
9.5.2	OTA Header Layout	89
10	File System	89
10.1	Introduction	89
10.2	FatFs on Flash	89
10.2.1	Software Setup	89
10.2.2	FatFs Bin File Generation	90
10.3	FatFs on SD Card	90
10.3.1	Hardware Setup	90
10.3.2	Software Setup	91
11	Inter Processor Communication (IPC)	92
11.1	Introduction	92
11.2	How to Use IPC?	92
11.3	IPC Program APIs	93
11.3.1	ipc_table_init	93
11.3.2	ipc_send_message	93
11.3.3	ipc_get_message	93
11.4	IPC Driver Code	93
11.4.1	IPC_INTConfig	93
11.4.2	IPC_IERSet	93
11.4.3	IPC_IERGet	94
11.4.4	IPC_INTRRequest	94
11.4.5	IPC_INTClear	94
11.4.6	IPC_INTGet	94
11.4.7	IPC_CPUID	94
11.4.8	IPC_SEMGet	95
11.4.9	IPC_SEMFree	95
11.4.10	IPC_INTHandler	95
11.4.11	IPC_INTUserHandler	95
12	OTA Firmware Update	96
12.1	Introduction	96
12.2	RSIP-MMU	96
12.3	OTA Upgrade from Local Server	97
12.3.1	Firmware Format	98
12.3.2	OTA Flow	98
12.3.3	Boot Option	99
12.3.4	Address Remapping	100
12.3.5	How to Use OTA Demo?	100
12.3.6	OTA Firmware Swap	102
12.4	User Configuration	103
13	eFuse	105
13.1	Introduction	105
13.2	Power Requirement	105

13.3	eFuse Auto-load	106
13.4	Physical eFuse	106
13.5	Logical eFuse	106
13.5.1	Logical eFuse Layout	106
13.5.2	Wi-Fi 2.4G Power Index	107
13.5.3	Wi-Fi 5G Power Index	108
13.5.4	Wi-Fi Channel Plan	109
13.5.5	Wi-Fi Crystal Calibration	110
13.5.6	Wi-Fi Thermal Meter	110
13.5.7	Wi-Fi MAC Address	111
13.5.8	Cap-Touch	111
13.5.9	BLE	111
13.5.10	HCI USB	112
13.6	eFuse PG APIs	112
13.6.1	Low Level APIs	112
13.6.2	Mbed APIs	113
13.7	eFuse PG Command	114
14	Power Save	115
14.1	Power Save Mode	115
14.1.1	Summary	115
14.1.2	Sleep Mode	116
14.1.3	Sleep Mode Configuration	117
14.1.4	Deepsleep Mode	126
14.1.5	Deepsleep Mode Configuration	127
14.1.6	GPIO Pull Control	132
14.2	Power Save Related APIs	132
14.2.1	<i>pmu_register_sleep_callback</i>	132
14.2.2	<i>pmu_unregister_sleep_callback</i>	132
14.2.3	<i>pmu_acquire_wakelock</i>	132
14.2.4	<i>pmu_release_wakelock</i>	133
14.2.5	<i>pmu_set_sysactive_time</i>	133
14.2.6	<i>pmu_set_max_sleep_time</i>	133
14.3	Power Consumption Measurement	133
14.3.1	Test Command	133
14.3.2	Hardware Preparation	133
15	Hardware Crypto Engine	135
15.1	Introduction	135
15.2	Hardware Crypto Engine APIs	135
15.2.1	Crypto Engine Initialization API	135
15.2.2	Hash APIs	135
15.2.3	Cipher APIs	140
15.3	Hardware Crypto Engine APIs Usage	143
15.3.1	Starting Hardware Crypto Engine	143
15.3.2	Starting Crypto Engine Calculation	143
15.4	Demo Code Path	144
16	User Configuration	145
16.1	Configuration File List	145
16.2	flashcfg.c	145
16.2.1	Flash Classification	146
16.2.2	FlashClass1 ~ FlashClass6	150

16.3	pinmapcfg.c	153
16.4	sleepcfg.c	154
16.5	bootcfg.c	155
16.6	ipccfg.c	157
16.7	lp_intfcfg.c	158
16.8	hp_intfcfg.c	158
17	Flash Operation	159
17.1	Functional Description	159
17.2	Protection Method.....	159
17.3	Flash Raw APIs	160
17.3.1	Read.....	160
17.3.2	Write.....	161
17.3.3	Erase	161
17.3.4	Receive/Transmit Command	161
17.4	Flash Mbed APIs	162
17.5	User Configuration	162
18	Battery Measurement	163
18.1	Functional Description	163
18.2	Calibration.....	163
19	Wi-Fi.....	165
19.1	Wi-Fi Data Structures	165
19.2	Wi-Fi APIs	165
19.2.1	System APIs	165
19.2.2	Scan APIs	166
19.2.3	Connection APIs.....	167
19.2.4	Channel APIs.....	169
19.2.5	Power APIs.....	170
19.2.6	AP Mode APIs	171
19.2.7	Custom IE APIs.....	173
19.2.8	Wi-Fi Setting APIs	173
19.2.9	Wi-Fi Indication APIs.....	177
19.2.10	eFuse Writing APIs	178
19.3	Fast Connection	178
19.3.1	Implement	178
19.3.2	APIs.....	179
19.4	WPS APIs	179
19.4.1	wps_start.....	179
19.4.2	wps_stop	179
20	Liquid Crystal Display Controller (LCDC)	180
20.1	Interface.....	180
20.2	Resolution	180
20.3	Pinmux	181
20.4	LCDC APIs	181
20.4.1	MCU Function	181
20.4.2	RGB Function	182
20.4.3	LED Function	183
20.4.4	Common Function.....	183
20.5	How to Use LCDC	185
20.5.1	MCU Interface	185

20.5.2	RGB Interface.....	187
20.5.3	LED Interface	187
20.6	GUI	188
20.6.1	Authorization.....	188
20.6.2	emWin Software.....	188
20.6.3	How to Adapt to LCM	189
20.6.4	How to Adapt to Touch Panel.....	190
20.6.5	How to Use emWin in SDK.....	191
21	PSRAM	192
21.1	Throughput	192
21.2	Power Management	193
21.3	How to Use PSRAM?	193
21.3.1	Initializing PSRAM.....	193
21.3.2	Adding BSS/TEXT/DATA Section into PSRAM Region.....	193
21.3.3	Allocating Heap from PSRAM.....	193
21.4	PSRAM Cache “Write Back” Policy Change Note	194
21.4.1	Cache Policy Change.....	194
21.4.2	Scope	194
21.4.3	Notice	194
22	MPU and Cache	196
22.1	Functional description	196
22.1.1	MPU.....	196
22.1.2	Cache	196
22.2	MPU APIs.....	197
22.2.1	mpu_init	197
22.2.2	mpu_set_mem_attr.....	197
22.2.3	mpu_region_cfg	197
22.2.4	mpu_entry_free.....	197
22.2.5	mpu_entry_alloc.....	198
22.3	Cache APIs.....	198
22.3.1	ICache_Enable	198
22.3.2	ICache_Disable	198
22.3.3	ICache_Invalidate	198
22.3.4	DCache_IsEnabled	198
22.3.5	DCache_Enable.....	198
22.3.6	DCache_Disable.....	199
22.3.7	DCache_Invalidate.....	199
22.3.8	DCache_Clean.....	199
22.3.9	DCache_CleanInvalidate.....	199
22.4	How to Define a Non-cacheable Data Buffer	199
23	Audio Codec Controller Guide	200
23.1	Audio Codec	200
23.1.1	Diagram.....	200
23.1.2	Features.....	201
23.1.3	Application Mode	201
23.2	Audio Codec Controller	204
23.2.1	Features.....	204
23.2.2	Control Interface.....	204
23.3	Audio PLL.....	204
23.3.1	Diagram.....	205

23.3.2	Operation Mode	206
23.4	Audio Codec APIs	206
23.4.1	PLL APIs.....	206
23.4.2	SPORT APIs	207
23.4.3	SI APIs	210
23.4.4	Codec APIs	210
23.5	How to Use AC APIs?.....	212
23.5.1	Audio Play.....	212
23.5.2	Audio Record	213
23.5.3	Example List.....	213
23.6	Hardware Design Guide	214
23.6.1	Line-out.....	214
23.6.2	AMIC-in.....	214
23.6.3	Power.....	215
23.7	Performance of Encoding & Decoding	215
23.7.1	AC3 Format.....	215
23.7.2	OPUS Format	215
23.7.3	FLAC Format	216
23.7.4	AAC Format	216
23.7.5	MP3 Format.....	216
23.8	Q & A	217
23.8.1	How to Connect the Output of DAC to A Power Amplifier?	217
23.8.2	What's the Difference between Single-ended MIC Input and Differential MIC Input?	217
23.8.3	How to Play Local Audio Files?	217
24	Cap-Touch	218
24.1	Pinmux	218
24.2	APIs	218
24.2.1	CapTouch_StructInit	218
24.2.2	CapTouch_Init.....	218
24.2.3	CapTouch_Cmd.....	218
24.2.4	CapTouch_INTConfig	219
24.2.5	CapTouch_GetISR	219
24.2.6	CapTouch_INTClearPendingBit.....	219
24.3	How to Use CTC.....	219
24.3.1	CTC Initialization.....	219
24.3.2	CTC Calibration	220
24.4	Cap-Touch Schematic Design and PCB Layout Guidelines	220
24.4.1	Cap-Touch Schematic Design.....	220
24.4.2	PCB Layout.....	221
25	Infrared Radiation (IR).....	224
25.1	Pinmux	224
25.2	Data Format	224
25.2.1	IR Tx.....	224
25.2.2	IR Rx.....	225
25.3	APIs	225
25.3.1	IR Setting APIs.....	225
25.3.2	IR Tx APIs	227
25.3.3	IR Rx APIs	228
25.4	IR Usage	230
25.4.1	Sending.....	230
25.4.2	Receiving	230

25.5	Tx Compensation Mechanism	231
25.5.1	Introduction	231
25.5.2	Tx Compensation Mechanism Application	231
25.6	IR Schematic Design Guideline	232
25.6.1	Leakage	232
25.6.2	Carrier Problem in Rx Learning	233
26	Brownout Detector (BOD)	234
26.1	Recommended Threshold Parameter	234
26.2	BOD APIs	235
26.2.1	BOR_ModeSet	235
26.2.2	BOR_ThresholdSet	235
26.2.3	BOR_ClearINT	235
26.2.4	BOR_DbncSet	235
27	Flash Translation Layer (FTL)	237
27.1	Overview	237
27.2	Features	238
27.3	FTL APIs	238
27.3.1	ftl_init	238
27.3.2	ftl_load_from_storage	238
27.3.3	ftl_save_to_storage	239
27.4	How to Use FTL	239
27.4.1	Precautions	239
27.4.2	General Steps	239
28	Audio Signal Generation and Analysis	241
28.1	Introduction	241
28.1.1	Compilation	241
28.1.2	Generating a Signal of a Certain Frequency	241
28.1.3	Analyzing Signals Collected by DMIC	241
28.1.4	DAAD	241
28.1.5	Command	242
29	Key-Scan	243
29.1	Pinmux	243
29.2	APIs	243
29.2.1	keyscan_array_pinmux	243
29.2.2	keyscan_getdatanum	243
29.2.3	keyscan_read	243
29.2.4	keyscan_init	244
29.2.5	keyscan_set_irq	244
29.2.6	keyscan_clear_irq	244
29.2.7	keyscan_get_irq_status	244
29.2.8	keyscan_set_irq_handler	244
29.2.9	keyscan_enable	244
29.2.10	keyscan_disable	245
29.2.11	keyscan_clearfifodata	245
29.3	Key-Scan Usage	245
29.3.1	Using Default Configuration	245
29.3.2	Using APIs	246
30	Bluetooth	247

30.1	Overview	247
30.2	BT Examples	247
30.2.1	<i>GCC Project</i>	247
30.2.2	<i>IAR Project</i>	248
30.3	BT Debug	248
30.3.1	<i>Introduction</i>	248
30.3.2	<i>Hardware & Software Preparation</i>	248
30.3.3	<i>Prerequisite – Opening the Log Switch</i>	249
30.3.4	<i>Capturing Log Files</i>	250
30.3.5	<i>FAQ</i>	253
Revision History.....		255

Convention

Ameba-D consists of two MCUs:

- Real-M300: a high performance MCU (Armv8-M, Cortex-M33 instruction set compatible) called KM4 thereafter
- Real-M200: a low power MCU (Armv8-M, Cortex-M23 instruction set compatible) called KM0 thereafter

1 Building Environment

1.1 Introduction

This chapter illustrates how to build Realtek Wi-Fi SDK under GCC environment. It focuses on both Windows platform and Linux distribution. The build and download procedure are quite similar between Windows and Linux operating system.

- For Windows, Windows 7 64-bit is used as platform.
- For Linux distribution, Ubuntu 18.04 64-bit is used as platform.

1.2 Preparing GCC Environment

1.2.1 Windows

On Windows, you can use Cygwin as the GCC environment. Cygwin is a large collection of GNU and open source tools which provide functionality similar to a Linux distribution on Windows.

Click <http://cygwin.com> and download the Cygwin package-setup-x86.exe for your Windows platform.

Note:

- Only 32-bit Cygwin is supported both for 32-bit Windows and 64-bit Windows.
- During the installation of Cygwin package, include 'Devel -> make' and 'Math -> bc' utilities on the **Select Packages** step, as Fig 1-1 and Fig 1-2 shows.

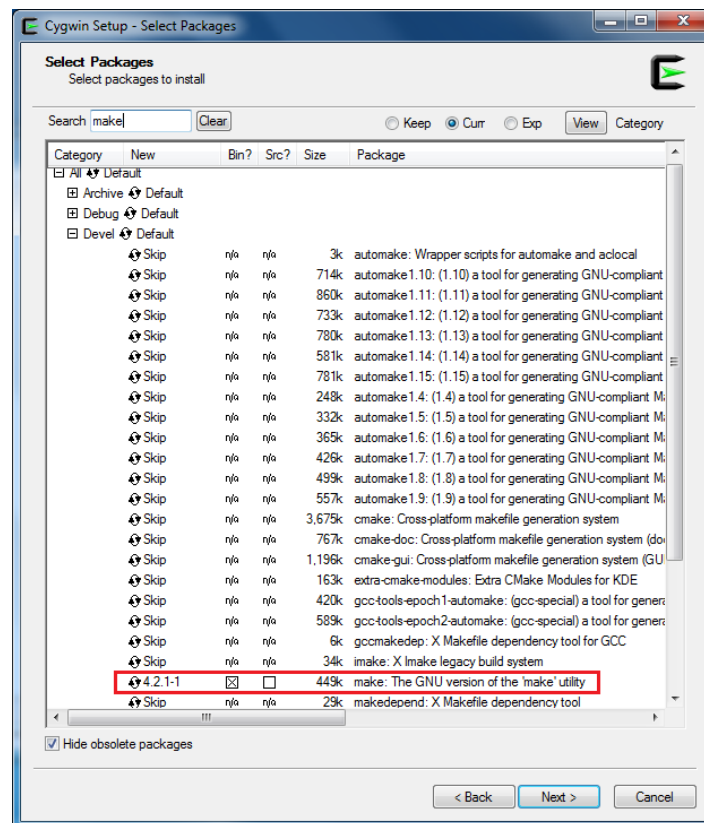


Fig 1-1 'Devel' setting during Cygwin installation

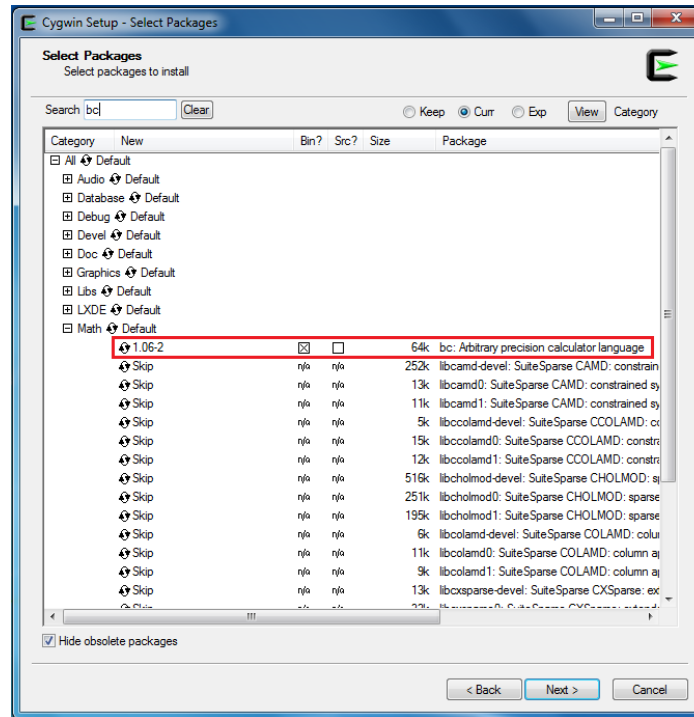


Fig 1-2 'Math' setting during Cygwin installation

1.2.2 Linux

On Linux, some packages must be installed for the GCC environment.

- **libc6-i386** (GNU C library for 64-bit platform. If you are using 32-bit platform, install **libc6** instead)
- **lib32ncurses5** (32-bit terminal handling for 64-bit platform. If you are using 32-bit platform, install **libncurses5** instead)
- **make**
- **bc**
- **gawk**
- **ncurses**

Some of these packages may have been pre-installed in your operating system. Use package manager to check and decide whether to install. For the last three packages, you can also type the corresponding version command on terminal like the following figures to check whether it exists. If not, make these packages installed.

- **\$ make -v**

If **make** isn't installed, type **apt-get install make** to install **make**.

```
karst@realtek:~$ make -v
GNU Make 4.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2014 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

- **\$ bc -v**

If **bc** isn't installed, type **apt-get install bc** to install **bc**.

```
karst@realtek:~$ bc -v
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2012-2017 Free Software
Foundation, Inc.
```

- **\$ gawk --v**

If **gawk** isn't installed, type **apt-get install gawk** to install **gawk**.

```

Hrm@realtek:~$ gawk --v
GNU Awk 4.1.4, API: 1.1 (GNU MPFR 4.0.1, GNU MP 6.1.2)
Copyright (C) 1989, 1991-2016 Free Software Foundation.

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see http://www.gnu.org/licenses/.

```

- **ncurses**

ncurses is needed if you want to use **make menuconfig** command. Type **apt-get install ncurses-dev** to install **ncurses**.

1.3 Building Code

This section illustrates how to build SDK. First, you need to switch to GCC project directory.

- For Windows, open Cygwin terminal and use **\$ cd** command to change directory to KM0 or KM4 project directory of Ameba-D SDK.
Note: You need to replace the **{path}** to your own SDK location, and add “cygdrive” prefix in front of the SDK location, so that Cygwin can access your file system.
 - **\$ cd /cygdrive/{path}/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp**
 - **\$ cd /cygdrive/{path}/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp**
- For Linux, open its own terminal and use **\$ cd** command to change directory to KM0 or KM4 project directory of Ameba-D SDK.
 - **\$ cd /{path}/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp**
 - **\$ cd /{path}/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp**

1.3.1 Normal Image

To build SDK for normal image, simply use **\$ make all** command under the corresponding project directories on Cygwin (Windows) or terminal (Linux).

1.3.1.1 KM0 Project

For KM0 project, if the terminal contains “km0_image2_all.bin” and “Image manipulating end” output message, it means that the image has been built successfully, as Fig 1-3 shows.

```

/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/xip_image2_all.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/flash_text_start
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/target_img2.map
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/ram_retention.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/ram_retention_entry_func
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/target_img2.map
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/xip_image2_prepend.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/ram_retention_prepend.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/ram_retention_prepend.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/km0_image2_all.bin
./project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image/km0_image2_all.bin
Image manipulating end
make[1]: Leaving directory /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk
/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp
$

```

Fig 1-3 KM0 project make all

If somehow it is built failed, type **\$ make clean** to clean and then redo the make procedure. After successfully built, the image file is located in **project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/image**, as Fig 1-4 shows.

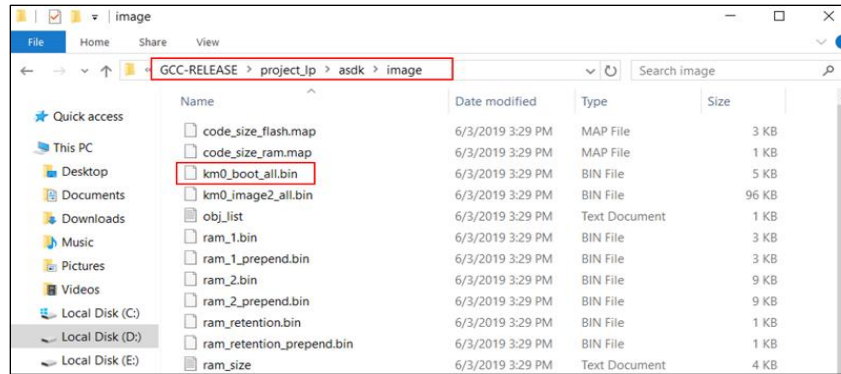


Fig 1-4 KM0 project bin generation

1.3.1.2 KM4 Project

For KM4 project, if the terminal contains “km4_image2_all.bin” and “Image manipulating end” output message, it means that the image has been built successfully, as Fig 1-5 shows.

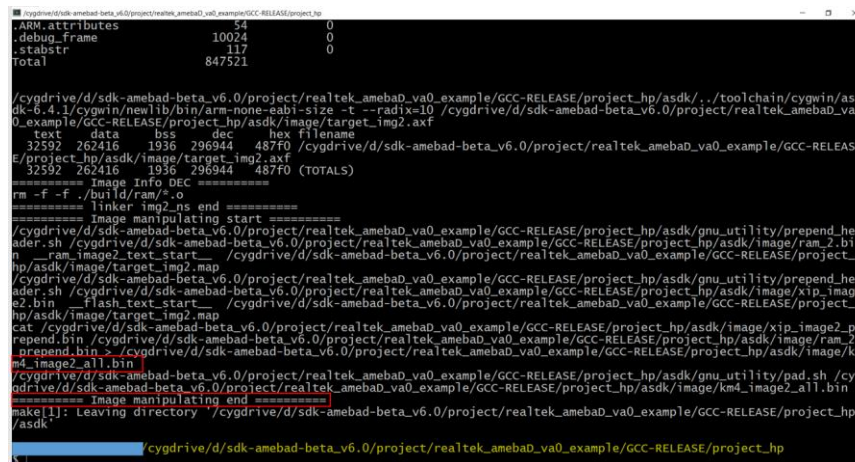


Fig 1-5 KM4 project make all

If somehow it built failed, type **\$ make clean** to clean and then redo the make procedure. After built successfully, the image file is located in **project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/image**, as Fig 1-6 shows.

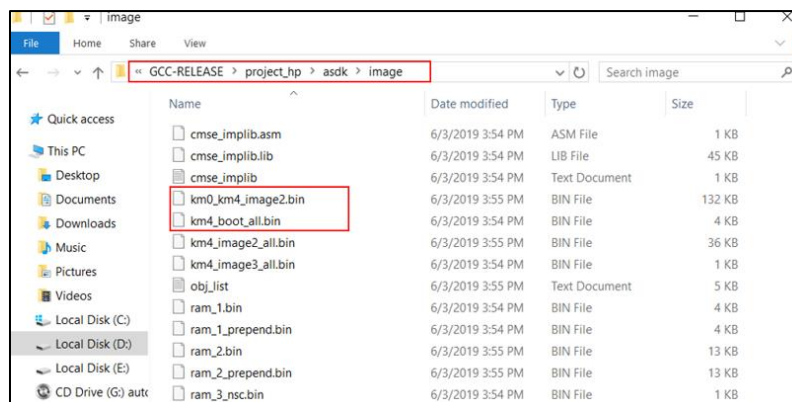


Fig 1-6 KM4 project bin generation

1.3.2 MP Image

Use **make mp** command under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp** to generate MP image. After successful compilation, you will find the generated images under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/image**, as shown in Fig 1-7.

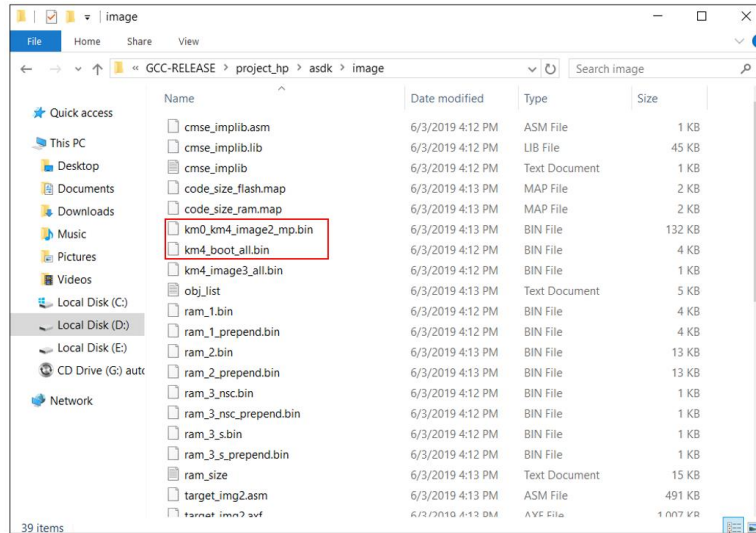


Fig 1-7 MP image generation

1.4 Setting Debugger

This section illustrates how to enter GDB debugger mode.

Note: Considering KM4 is powered-on by KM0, make sure that KM0 has boot up already before accessing KM4 by J-Link or Probe. Clicking Reset button on demo board is a recommended way to boot up KM0.

1.4.1 Probe

1.4.1.1 Windows

Ameba-D Device Board supports Probe debugger. We can use Probe to download the software and enter GBD debugger mode under GCC environment. Refer to Fig 1-8 to connect Probe debugger to the SWD of Ameba-D.

(1) Install Probe driver

Before using the Probe, its driver is required to be installed. Obtain the **RLX_Probe_Driver_2.3.11p22_Setup.exe** under tools/probe, then install it correctly. Refer to Fig 1-8 to connect Probe debugger to the SWD of Ameba-D, which is connecting TCK pin of Probe to SWD CLK pin of Ameba-D, and TMS pin of Probe to SWD DATA pin of Ameba-D. What's more, a common ground is needed between Probe Board and Ameba-D Device Board.

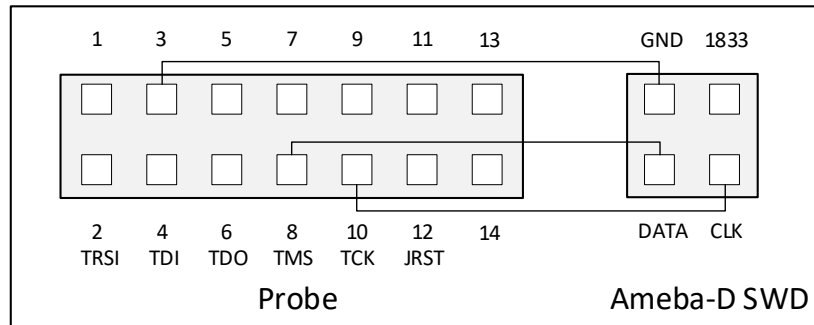


Fig 1-8 Wiring diagram of connecting Probe to SWD

(2) Execute the **cm0_RTL_Probe.bat**

After the installation of the software pack, execute the **cm0_RTL_Probe.bat** under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script**.

Note: The default path of Probe driver in **cm0_RTL_Probe.bat** file is **C:\RLX\PROBE\rlx_probe_driver.exe**, you may have to change the path according to your own settings.

The started Probe server looks like Fig 1-9. This window should NOT be closed if you want to download the image or enter debug mode.

```

C:\Windows\system32\cmd.exe
system reset-halted, DHCSR 0x02030003
This Core is implemented by Realtek Semiconductor Corporation.
This is a KM0 processor! KM0 found CPUID = 0x721cd200, Reversion is r1p0!
DCB_DHCSR 0x00030003;
NUIC_DFSR 0x00000009;
NUIC_AIRCR 0xfa052000; little-endian;
DCB_DCSR 0x00000000;
DCB_DCRDR 0x00000000;
DWT_CYCCNT 0x00000000;
DWT_MASK0 0x00000000;
DWT_FUNCTION0 0x50000000;
This MCU has has a Flash Patch Breakpoint version 2!
This MCU has 2 of code comparators and 0 literal comparators in FPB!
Renapping not supported!
DWT_CTRL 0x1b000000;
DWT_CYCCNT 0x00000000;
DWT_COMP0 0x00000000;
This MCU has 1 comparators!
This MCU supports Trace sampling and exception tracing!
This MCU does not include external match signals, CMPMATCH[N]!
This MCU supports a cycle counter!
This MCU supports the profiling counters!
Program flow prediction disabled! Instruction caches disabled! Data and unified ca
ches disabled!
Cache is implemented at level 1, and is Separate Instruction & Data Caches!
Level 1 I-Cache infomation:
I-Cache supports Write-Through!
I-Cache supports Write-Back!
I-Cache supports Read-Allocation!
I-Cache does not support Write-Allocation!
The processor's I-cache number of sets is 256!
The processor's I-cache number of ways is 2!
The processor's I-cache number of linesize is 8 Words
The processor's I-cache size is 16 KB!
Level 1 D-Cache infomation:
D-Cache supports Write-Through!
D-Cache supports Write-Back!
D-Cache supports Read-Allocation!
D-Cache supports Write-Allocation!
The processor's D-cache number of sets is 64!
The processor's D-cache number of ways is 2!
The processor's D-cache number of linesize is 8Words
The processor's D-cache size is 4KB!
=====
IAP 1 Core 0 Initialize Over !
SUCCESS to Trigger this Core
=====

```

Fig 1-9 KM0 Probe server connection under Windows

(3) Setup Probe for KM0

On the Cygwin terminal, type **\$ make setup GDB_SERVER=probe** command to select Probe debugger, as Fig 1-10 shows.

```

/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_
example/GCC-RELEASE/project_lp
$ make setup GDB_SERVER=probe
make -C asdk setup
make[1]: Entering directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_
amebaD_va0_example/GCC-RELEASE/project_lp/asdk'
-----
Setup probe
-----
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug_probe.txt /cygdr
ive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/proje
ct_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write_probe.txt /
cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE
/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com_probe.txt
/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEA
SE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_a
mebaD_va0_example/GCC-RELEASE/project_lp/asdk'

```

Fig 1-10 KM0 Probe setup under Windows

- (4) Execute the **cm4_RTL_Probe.bat**
 Execute the **cm4_RTL_Probe.bat** under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script** the same as executing the **cm0_RTL_Probe.bat**. The started Probe server looks like Fig 1-11. This window should NOT be closed if you want to enter debug mode.

```

C:\Windows\system32\cmd.exe
This Core supports Non-Secure Invasive Debug !
DHCSR = 0x00130003
This Core is in NON-Secure state, concurrent write to CDS would be ignored!
Now Access by Secure Request!
KM4 halted!
This Core is implemented by Realtek Semiconductor Corporation.
This is a KM4 processor! KM4 found CPUID = 0x721fd220, Revision is r1p0!
DCB_DHCSR 0x00130003;
NUIC_DFSR 0x00000001;
NUIC_AIBCR 0xfa052000; little-endian;
DCB_DCSR 0x00000000;
DCB_DCRDR 0x00000000;
DWT_CYCCNT 0x00000000;
DWT_MASK0 0x00000000;
DWT_FUNCTION0 0x58000000;
SAU information:
SAU ATTR: Memory is marked as Secure and is not Non-secure callable!
The MPU is Enabled!
MPU Region 0: 0x100268c0 ~ 0x1002c63f is marked as privileged access per
mitted only, Non-shareable, Read/write by any privilege level, Execution only pe
rmitted if read permitted, Normal memory, Outer Non-cacheable, Normal memory, In
ner Non-cacheable
MPU Region 1: 0x1010a000 ~ 0x1014001f is marked as privileged access per
mitted only, Non-shareable, Read/write by any privilege level, Execution only pe
rmitted if read permitted, Normal memory, Outer Non-cacheable, Normal memory, In
ner Non-cacheable
MPU Region 2: 0x101c0000 ~ 0x101d401f is marked as privileged access per
mitted only, Non-shareable, Read/write by any privilege level, Execution only pe
rmitted if read permitted, Normal memory, Outer Non-cacheable, Normal memory, In
ner Non-cacheable
MPU Region 3: 0x000c0000 ~ 0x000c041f is marked as privileged access per
mitted only, Non-shareable, Read/write by any privilege level, Execution only pe
rmitted if read permitted, Normal memory, Outer Non-cacheable, Normal memory, In
ner Non-cacheable
This MCU has a Flash Patch Breakpoint version 2!
This MCU has 2 of code comparators and 0 literal comparators in FPB!
Remapping not supported!
DWT_CTRL 0x1b000000;
DWT_CYCCNT 0x00000000;
DWT_COMP0 0x00000000;
This MCU has 1 comparators!
This MCU supports Trace sampling and exception tracing!
This MCU does not include external match signals, CMPMATCH[1]!
This MCU supports a cycle counter!
This MCU supports the profiling counters!
Program flow prediction disabled! Instruction caches enabled! Data and unified cac
hes enabled!
There is no D-cache in this processor!
/*****
TAP 1 Core 0 Initialize Over !
SUCCESS to Trigger this Core
*****/

```

Fig 1-11 KM4 Probe server connection under Windows

Note: When **cm4_RTL_Probe.bat** is running, the connection built by running **cm0_RTL_Probe.bat** will be closed. If you want to connect the Probe to KM0 and KM4 simultaneously, follow the steps below:

- Make a copy of **rlx_probe0.cfg** under the folder **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script** and rename it as **rlx_probe1.cfg**.

- b) Cut **rlx_probe1.cfg** and move it to **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script**.
- c) Connect the Probe to both KM0 and KM4 by executing **cm0_RTL_Probe.bat** under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script**.

(5) Setup Probe for KM4

On the Cygwin terminal, type **\$ make setup GDB_SERVER=probe** command to select Probe debugger, as Fig 1-12 shows.

```

/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_
$ make setup GDB_SERVER=probe
make -C asdk setup
make[1]: Entering directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_
p/asdk'
-----
Setup probe
-----
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
script/rtl_gdb_debug_probe.txt /cygdrive/d/sdk-amebad-beta_v6.0/project/realte
_hp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
script/rtl_gdb_flash_write_probe.txt /cygdrive/d/sdk-amebad-beta_v6.0/project/
project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
script/rtl_gdb_jtag_boot_com_probe.txt /cygdrive/d/sdk-amebad-beta_v6.0/projec
/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_a
/asdk'

```

Fig 1-12 KM4 Probe setup under Windows

1.4.1.2 Linux

Before using Probe, its driver is required to be installed. Obtain the **RLX_Probe_Linux_Driver_v2.3.11p22** under **tools/probe** and install it correctly. Then connect TCK pin of Probe to SWD CLK pin, and TMS pin of Probe to SWD DATA pin. What's more, a common ground is needed between Probe Board and Ameba-D Device Board.

After the installation of the software pack, use **ln -s** command in both folder **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script** and **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script** to create a soft link to link the **rlx_probe_driver.elf**.

```

wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script
$ ln -s /home/wlan5/RLX_Probe_driver/RLX_Probe_Linux_Driver/rlx_probe_driver.elf /home/wlan5/sdk-amebad-beta
_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script/
wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script
$ ln -s /home/wlan5/RLX_Probe_driver/RLX_Probe_Linux_Driver/rlx_probe_driver.elf /home/wlan5/sdk-amebad-beta
_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script/

```

Fig 1-13 Creating soft link to elf

Open a new terminal under the corresponding project folder and type the following command to start probe server. This terminal should NOT be closed if you want to download the image or enter debug mode.

- KM0 project

For KM0, open a new terminal under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script**, and type **\$ sudo ./rlx_probe_driver.elf**. If connection is successful, the log is shown as Fig 1-14.

```
wlan5@RS-wlan5: ~/V03/V03_bak/project/realtek_amebaD_cm0_gcc_verification/jlink_script
File Edit View Search Terminal Help
DAURHSTATUS = 0x00000001
This Core does not support Secure Non-Invasive Debug !
This Core does not support Secure Invasive Debug !
This Core supports Non-Secure Non-Invasive Debug !
This Core supports Non-Secure Invasive Debug !
DHCSR = 0x00030003
This Core is in NON-Secure state, concurrent write to CDS would be ignored!
Now Access by Non-Secure Request!
KM0 halted!
This core is implemented by Realtek Semiconductor Corporation.
This is a KM0 processor! KM0 found CPUID = 0x721cd200, Revision is r1p0!
UCB_DCSR 0x00030003;
NVIC_DFSR 0x00000001;
NVIC_AIRCR 0xfa052000; little-endian;
DCB_DCSR 0x00000000;
DCB_DCRDR 0x00000000;
DWT_CYCNT 0x00000000;
DWT_MASK0 0x00000000;
DWT_FUNCTION0 0x50000000;
This MCU has has a Flash Patch Breakpoint version 2!
This MCU has 2 of code comparators and 0 literal comparators in FPB!
Renapping not supported!
DWT_CTRL 0x1b000000;
DWT_CYCNT 0x00000000;
DWT_COMP0 0x00000000;
This MCU has 1 comparators!
This MCU supports trace sampling and exception tracing!
This MCU does not include external match signals, CMPMATCH[N]!
This MCU supports a cycle counter!
This MCU supports the profiling counters!
Program flow prediction disabled! Instruction caches enabled! Data and unified caches enabled!
Cache is implemented at level 1, and is Separate Instruction & Data Caches!
Level 1 I-Cache information:
I-Cache supports Write-Through!
I-Cache supports Write-Back!
I-Cache supports Read-Allocation!
I-Cache does not support Write-Allocation!
The processor's I-cache number of sets is 256!
The processor's I-cache number of ways is 2!
The processor's I-cache number of linesize is 8 Words
The processor's I-cache size is 16 KB!
Level 1 D-Cache information:
D-Cache supports Write-Through!
D-Cache supports Write-Back!
D-Cache supports Read-Allocation!
D-Cache supports Write-Allocation!
The processor's D-cache number of sets is 64!
The processor's D-cache number of ways is 2!
The processor's D-cache number of linesize is 8 Words
The processor's D-cache size is 4KB!
/*****
TAP 1 Core 0 Initialize Over !
SUCCESS to Trigger this Core
*****/
```

Fig 1-14 KM0 probe GDB server connection success under Linux

Open a new terminal under the same project folder (**/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp**), type **\$ make setup GDB_SERVER=probe** command to select Probe debugger.

- KM4 project

For KM4, open a new terminal under **/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script**, and type **\$ sudo./rlx_probe_driver.elf**. If connection is successful, the log is shown as Fig 1-15.

```
wlan5@RS-wlan5: ~/V03/V03_bak/project/realtek_amebaD_cm4_gcc_verification/jlink_script
File Edit View Search Terminal Help
This Core supports Non-Secure Non-Invasive Debug !
This Core supports Non-Secure Invasive Debug !
DHCSR = 0x00130003
This Core is in Secure state, concurrent write to CDS would be ignored!
Now Access by Secure Request!
KM4 halted!
This Core is implemented by Realtek Semiconductor Corporation.
This is a KM4 processor! KM4 found CPUID = 0x721fd220, Reversion is r1p0!
DCE_DMCSR 0x00130003;
NVIC_DFSR 0x00000009;
NVIC_AIRCR 0xfa050000; little-endian;
DCB_DCSR 0x00000000;
DCB_DCRDR 0x00000000;
DWT_CYCNT 0x00000000;
DWT_MASK0 0x00000000;
DWT_FUNCTION0 0x58000000;
SAU Information:
SAU ATTR: Memory is marked as Secure and is not Non-secure callable!
The MPU is disabled!
This MCU has a Flash Patch Breakpoint version 2!
This MCU has 2 of code comparators and 0 literal comparators in FPB!
Remapping not supported!
DWT_CTRL 0x1b000000;
DWT_CYCNT 0x00000000;
DWT_COMP0 0x00000000;
This MCU has 1 comparators!
This MCU supports Trace sampling and exception tracing!
This MCU does not include external match signals, CMPMATCH[N]!
This MCU supports a cycle counter!
This MCU supports the profiling counters!
Program flow prediction disabled! Instruction caches disabled! Data and unified caches disabled!
Cache is implemented at level 1, and is Separate Instruction & Data Caches!
Level 1 I-Cache information:
I-Cache supports Write-Through!
I-Cache supports Write-Back!
I-Cache supports Read-Allocation!
I-Cache does not support Write-Allocation!
The processor's I-cache number of sets is 512!
The processor's I-cache number of ways is 2!
The processor's I-cache number of linesize is 8 Words
The processor's I-cache size is 32 KB!
Level 1 D-Cache information:
D-Cache supports Write-Through!
D-Cache supports Write-Back!
D-Cache supports Read-Allocation!
D-Cache supports Write-Allocation!
The processor's D-cache number of sets is 64!
The processor's D-cache number of ways is 2!
The processor's D-cache number of linesize is 8 Words
The processor's D-cache size is 4KB!
/*****
TAP 1 Core 0 Initialize Over !
SUCCESS to Trigger this Core
*****/
```

Fig 1-15 KM4 probe GDB server connection success under Linux

Open a new terminal under the same project folder (/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp), type \$ make setup GDB_SERVER=probe command to select Probe debugger.

1.4.2 J-Link

Ameba-D also supports J-Link debugger. You need to do some hardware configuration to use J-Link debugger. Refer to Fig 1-16 to connect J-Link to the SWD of Ameba-D, which is connecting SWCLK pin of J-Link to SWD CLK pin of Ameba-D, and SWDIO pin of J-Link to SWD DATA pin of Ameba-D. After finishing these configurations, connect it to PC.

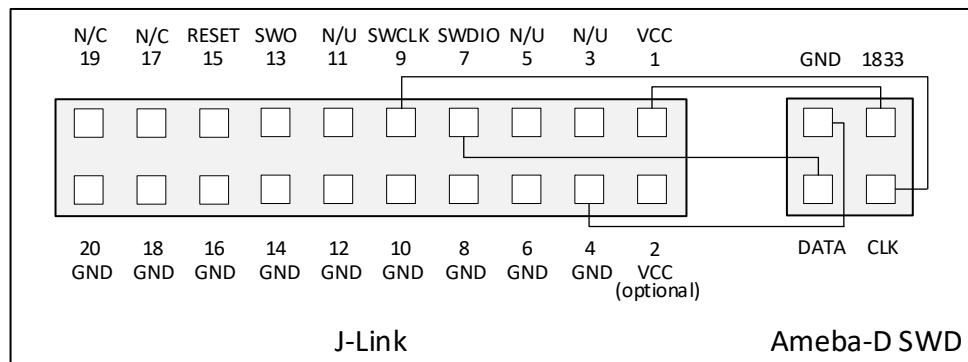


Fig 1-16 Wiring diagram of connecting J-Link to SWD

Note: For Ameba-D CPU, the J-Link version must be v9 or higher. If Virtual Machine (VM) is used as your platform, make sure the USB connection setting between VM host and client is correct, so that the VM host can detect the device.

1.4.2.1 Windows

(1) Install J-Link GDB server

Besides the hardware configuration, J-Link GDB server is also required to install. For Windows, click <https://www.segger.com/downloads/jlink> and download the software in “J-Link Software and Documentation Pack”, then install it correctly.

Note: The version of J-Link GDB server displayed in the pictures below is just an example, you can select the latest version to download.

(2) Execute the `cm0_jlink.bat`

After the installation of the software pack, execute the `cm0_jlink.bat` under `/project/realtek_amebaD_va0_example/GCCRELEASE/project_lp/jlink_script`.

Note: The default path of J-Link driver in `cm0_jlink.bat` file is `C:\Program Files (x86)\SEGGER\JLink_V634b\JLinkGDBServer.exe`, you may have to change the path according to your own settings.

The started J-Link GDB server looks like Fig 1-17. This window should NOT be closed if you want to download the image or enter debug mode.

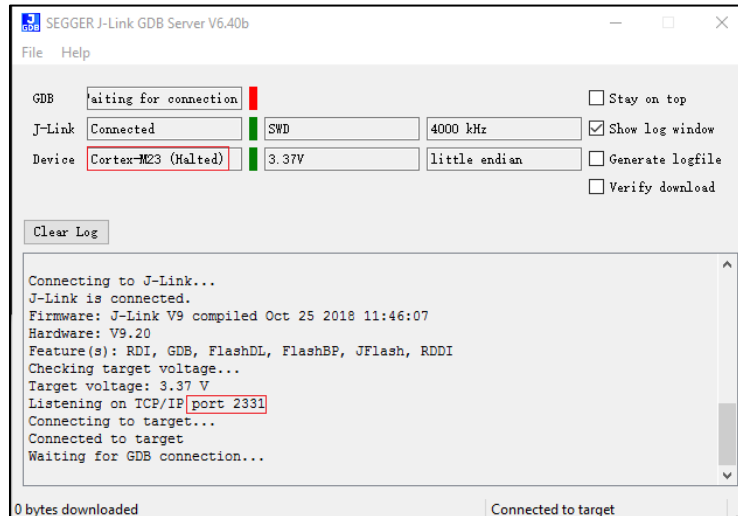


Fig 1-17 KM0 J-Link GDB server connection under Windows

(3) Setup J-Link for KM0

On the Cygwin terminal, type `$ make setup GDB_SERVER=jlink` command before to select J-Link debugger, as Fig 1-18 shows.

```
/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_
example/GCC-RELEASE/project_lp
$ make setup GDB_SERVER=jlink
make -C asdk setup
make[1]: Entering directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_
amebaD_va0_example/GCC-RELEASE/project_lp/asdk'
-----
Setup jlink
-----
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug_jlink.txt /cygdriv
e/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/proje
ct_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write_jlink.txt /
cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE
/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-
RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt /
cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEA
SE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/cygdrive/d/sdk-amebad-beta_v6.0/project/realtek_a
mebaD_va0_example/GCC-RELEASE/project_lp/asdk'
```

Fig 1-18 KM0 J-Link setup under Windows

- (4) Execute the **cm4_jlink.bat**
Execute the **cm4_jlink.bat** under **/project/realtek_amebaD_va0_example/GCCRELEASE/project_hp/jlink_script** the same as executing the **cm0_jlink.bat**. The started J-Link GDB server looks like Fig 1-19. This window should NOT be closed if you want to download the image or enter debug mode.

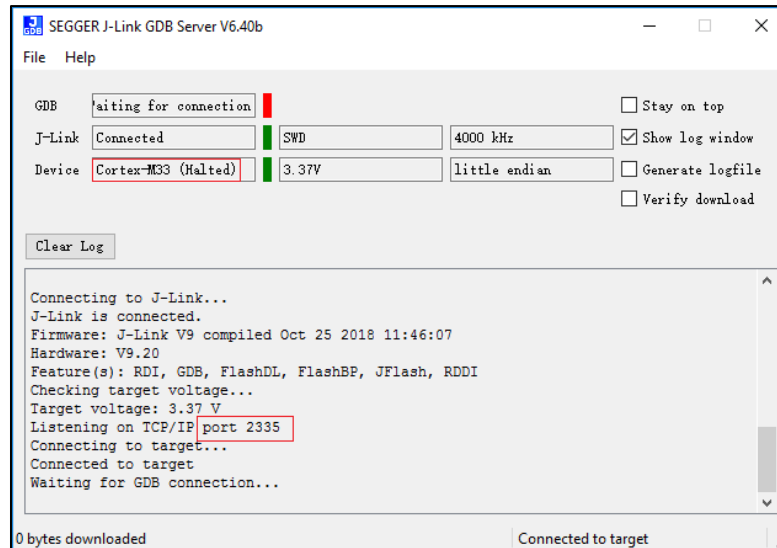


Fig 1-19 KM4 J-Link GDB server connection under Windows

- (5) Setup J-Link for KM4
On the Cygwin terminal, type **\$ make setup GDB_SERVER=jlink** command to select J-Link debugger, as Fig 1-20 shows.

```

/cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp
$ make setup GDB_SERVER=jlink
make -C asdk setup
make[1]: Entering directory '/cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk'
-----
Setup jlink
-----
cp -p /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_debug_jlink.txt /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write_jlink.txt /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com_jlink.txt /cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/cygdrive/d/sdk-amebaD-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk'

```

Fig 1-20 KM4 J-Link setup under Windows

1.4.2.2 Linux

For J-Link GDB server, click <https://www.segger.com/downloads/jlink> and download the software in “J-Link Software and Documentation Pack”. We suggest using Debian package manager to install the Debian version.

- **\$ dpkg -i jlink_6.0.7_x86_64.deb**

After the installation of the software pack, there is a tool named “JLinkGDBServer” under JLink directory. Take Ubuntu 18.04 as an example, the JLinkGDBServer can be found at **/opt/SEGGER/JLink**. Open a new terminal under the corresponding project folder and type the following command to start GDB server. This terminal should NOT be closed if you want to download the image or enter debug mode.

1.4.2.2.1 KM0 Project

For KM0, open a new terminal under `/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/jlink_script`, and type `$ /opt/SEGGER/JLink/JLinkGDBServer -device cortex-m23 -if SWD -scriptfile AP1_KM0.JLinkScript port 2331`, as Fig 1-21 shows. If connection is successful, the log is shown as Fig 1-22.

```
wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-REL
EASE/project_lp/jlink_script$ /opt/SEGGER/JLink/JLinkGDBServer -device cortex-m2
3 -if SWD -scriptfile AP1_KM0.JLinkScript -port 2331
SEGGER J-Link GDB Server V6.46b Command Line Version

JLinkARM.dll V6.46b (DLL compiled May 31 2019 17:41:22)

Command line: -device cortex-m23 -if SWD -scriptfile AP1_KM0.JLinkScript -port 2
331
-----GDB Server start settings-----
GDBInit file:      none
GDB Server Listening port: 2331
SWO raw output listening port: 2332
Terminal I/O port: 2333
Accept remote connection: yes
Generate logfile:  off
Verify download:   off
Init regs on start: off
Silent mode:       off
Single run mode:   off
Target connection timeout: 0 ms
-----J-link related settings-----
```

Fig 1-21 KM0 J-Link GDB server connection setting under Linux

```
File Edit View Search Terminal Help
Silent mode:      off
Single run mode:  off
Target connection timeout: 0 ms
-----J-Link related settings-----
J-Link Host interface: USB
J-Link script:    AP1_KM0.JLinkScript
J-Link settings file: none
-----Target related settings-----
Target device:    cortex-m23
Target interface: SWD
Target interface speed: 4000kHz
Target endian:    little

Connecting to J-Link...
J-Link is connected.
Firmware: J-Link V9 compiled Apr 20 2018 16:47:26
Hardware: V9.40
S/N: 59425868
Feature(s): RDI, GDB, FlashDL, FlashBP, JFlash, RDDI
Checking target voltage...
Target voltage: 3.38 V
Listening on TCP/IP port 2331
Connecting to target... Connected to target
Waiting for GDB connection...
```

Fig 1-22 KM0 J-Link GDB server connection success under Linux

Open a new terminal under the same project folder, type `$ make setup GDB_SERVER=jlink` command to select J-Link debugger, as Fig 1-23 shows.

```
wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp$ make setup GDB_SERVER=jlink
make -C asdk setup
make[1]: Entering directory '/home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk'

Setup jlink

cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug_jlink.txt /ho
e/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write_jlink.t
t /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com_jlink
txt /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk'
```

Fig 1-23 KM0 J-Link terminal setup under Linux

1.4.2.2.2 KM4 Project

For KM4, open a new terminal under `/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script`, and type `$ /opt/SEGGER/JLink/JLinkGDBServer -device cortex-m33 -if SWD -scriptfile AP2_KM4.JLinkScript -port 2335`, as Fig 1-24 shows. If connection is successful, the log is shown as Fig 1-25.

```
wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/jlink_script$ /opt/SEGGER/JLink/JLinkGDBServer -device cortex-m33 -if SWD -scriptfile AP2_KM4.JLinkScript -port 2335
SEGGER J-Link GDB Server V6.46b Command Line Version

JLinkARM.dll V6.46b (DLL compiled May 31 2019 17:41:22)

Command line: -device cortex-m33 -if SWD -scriptfile AP2_KM4.JLinkScript -port 2335
-----GDB Server start settings-----
GDBInit file: none
GDB Server Listening port: 2335
SWO raw output listening port: 2332
Terminal I/O port: 2333
Accept remote connection: yes
Generate logfile: off
Verify download: off
Init regs on start: off
Silent mode: off
Single run mode: off
Target connection timeout: 0 ms
-----J-Link related settings-----
```

Fig 1-24 KM4 J-Link GDB server connection setting under Linux

```
File Edit View Search Terminal Help
Silent mode: off
Single run mode: off
Target connection timeout: 0 ms
-----J-Link related settings-----
J-Link Host interface: USB
J-Link script: AP2_KM4.JLinkScript
J-Link settings file: none
-----Target related settings-----
Target device: cortex-m33
Target interface: SWD
Target interface speed: 4000kHz
Target endian: little

Connecting to J-Link...
J-Link is connected.
Firmware: J-Link V9 compiled Apr 20 2018 16:47:26
Hardware: V9.40
S/N: 59425868
Feature(s): RDI, GDB, FlashDL, FlashBP, JFlash, RDDI
Checking target voltage...
Target voltage: 3.37 V
Listening on TCP/IP port 2335
Connecting to target...Connected to target
Waiting for GDB connection...
```

Fig 1-25 KM4 J-Link GDB server connection success under Linux

Open a new terminal under the same project folder, type `$ make setup GDB_SERVER=jlink` command to select J-Link debugger, as Fig 1-26 shows.

```
wlan5@RS-wlan5:~/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp$ make setup GDB_SERVER=jlink
make -C asdk setup
make[1]: Entering directory '/home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk'
Setup jlink
cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_debug_jlink.txt /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write_jlink.txt /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_flash_write.txt
cp -p /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com_jlink.txt /home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk/gnu_utility/gnu_script/rtl_gdb_jtag_boot_com.txt
make[1]: Leaving directory '/home/wlan5/sdk-amebad-beta_v6.0/project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp/asdk'
```

Fig 1-26 KM4 J-Link terminal setup under Linux

1.5 Downloading Image to Flash

This section illustrates how to download image to Flash.

To download software into Ameba-D Device Board, make sure steps mentioned in section 1.3 to 1.4 are done and then type **\$ make flash** command on Cygwin (Windows) or terminal (Linux).

Both KM0 and KM4 download codes by this command. This command downloads the software into Flash and it will take several seconds to finish, as shown in Fig 1-27.

```
Breakpoint 1, RtlFlashProgram () at /cygdrive/e/v03newest/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/flashloader/rtl_flash_d
88      asm("nop");
flash_write FileName:2
flash_write FileSize:0
flash_write FlashAddrForWrite:6000
FileSize: 0
Loopnumber = 0
TailSize = 0
global variables
FlashDatSrc:822ac
FlashBlockWriteSize:800
FlashAddrForWrite:6000Flash write start...
dump for check
Breakpoint 2, RtlFlashProgram () at /cygdrive/e/v03newest/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk/flashloader/rtl_flash_d
120      if (FlashWriteComplete == 1) {
make[1]: Leaving directory '/cygdrive/e/v03newest/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp/asdk'
fiona.shen@Y38471517 /cygdrive/e/v03newest/project/realtek_amebaD_va0_example/GCC-RELEASE/project_lp
```

17:05:21.123	#ROM:[V1.2]	
17:05:21.183	FLASHRATE:1	
17:05:21.194	=====	
17:05:21.259	Flash Downloader Build Time: 2019/07/05-16:54:02	
17:05:21.276	=====	
17:05:21.291	Flash init start	
17:05:21.309	Flash init done	
17:05:21.322	Flash download start	
17:05:21.712	Flash download done	log in trace tool

Fig 1-27 Download codes success log

After downloaded successfully, press the **Reset** button and you can see the device is booted with new image.

Note: For Probe downloading,

- Make chip enter download mode before downloading code to Flash
- Download KM4 project code also through **cm0_RTL_Probe.bat**
- Probe uses USB 1.0 interface, so its download rate is limited by the USB 1.0 protocol.

1.6 Entering Debug Mode

To enter GDB debugger mode, make sure steps mentioned in section 1.3 to 1.5 are finished and then reset the device first. After resetting the chip, type **\$ make debug** command on Cygwin (Windows) or terminal (Linux).

- For Windows, a popup window will display, as shown in Fig 1-28.
- For Linux, the log is shown in Fig 1-29.

```

E:\SDK\sdm-amebad-beta_v5.2\project\realtek_amebaD_cm4_gcc_verification\toolchain\...
GNU gdb (Realtek ASDK-6.4.1 Build 2778) 7.12.50.20170111-git
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-cygwin --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x100075da in ?? ()
Notification of completion for asynchronous execution commands is off.
(gdb)

```

Fig 1-28 Debug window under Windows

```

File Edit View Search Terminal Help
tion/asdk/./toolchain/linux/asdk-6.4.1/linux/newlib/bin/arm-none-eabi-gdb -x /h
ome/derek/work/sdk_release_wjy/v03/project/realtek_amebaD_cm0_gcc_verificati
on/asdk/gnu_utility/gnu_script/rtl_gdb_debug.txt
GNU gdb (Realtek ASDK-6.4.1 Build 2778) 7.12.50.20170111-git
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-linux --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0008215e in ?? ()
Notification of completion for asynchronous execution commands is off.
Resets the core only, not peripherals.
Sleep 20ms
(gdb)

```

Fig 1-29 Debug window under Linux

1.7 Command List

The commands used above are listed in Table 1-1.

Table 1-1 Command list

Usage	Command	Description
all	\$ make all	Compile project to generate ram_all.bin
setup	\$ make setup GDB_SERVER= jlink	Select GDB_SERVER
flash	\$ make flash	Download ram_all.bin to Flash
clean	\$ make clean	Remove compile file (*.bin, *.o, ...)
clean_all	\$ make clean_all	Remove compile result and toolchains
debug	\$ make debug	Enter debug mode

1.8 GDB Debugger Basic Usage

GDB, the GNU project debugger, allows you to examine the program while it executes, and it is helpful for catching bugs. Section 1.6 has described how to enter GDB debugger mode, this section illustrates some basic usage of GDB commands. For further information about GDB debugger and its commands, click <https://www.gnu.org/software/gdb/> and <https://sourceware.org/gdb/current/onlinedocs/gdb/>.

Table 1-2 GDB debugger command list

Usage	Command	Description
Breakpoint	\$ break	Breakpoints are set with the <i>break</i> command (abbreviated <i>b</i>). The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Set-Breaks.html
Watchpoint	\$ watch	You can use a watchpoint to stop execution whenever the value of an expression changes. The related commands include <i>watch</i> , <i>rwatch</i> , and <i>awatch</i> . The usage of these commands can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Set-Watchpoints.html Note: Keep the range of watchpoints less than 20 bytes.
Print breakpoints & watchpoints	\$ info	To print a table of all breakpoints, watchpoints set and not deleted, use the <i>info</i> command. You can simply type <i>info</i> to know its usage.
Delete breakpoints	\$ delete	To eliminate the breakpoints, use the <i>delete</i> command (abbreviated <i>d</i>). The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Delete-Breaks.html .
Continue	\$ continue	To resume program execution, use the <i>continue</i> command (abbreviated <i>c</i>). The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Continuing-and-Stepping.html .
Step	\$ step	To step into a function call, use the <i>step</i> command (abbreviated <i>s</i>). It will continue running your program until control reaches a different source line. The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Continuing-and-Stepping.html .
Next	\$ next	To step through the program, use the <i>next</i> command (abbreviated <i>n</i>). The execution will stop when control reaches a different line of code at the original stack level. The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Continuing-and-Stepping.html .
Quit	\$ quit	To exit GDB debugger, use the <i>quit</i> command (abbreviated <i>q</i>), or type an end-of-file character (usually <i>Ctrl-d</i>). The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Quitting-GDB.html .
Backtrace	\$ backtrace	A backtrace is a summary of how your program got where it is. You can use <i>backtrace</i> command (abbreviated <i>bt</i>) to print a backtrace of the entire stack. The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Backtrace.html .
Print source lines	\$ list	To print lines from a source file, use the <i>list</i> command (abbreviated <i>l</i>). The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/List.html .
Examine data	\$ print	To examine data in your program, you can use <i>print</i> command (abbreviated <i>p</i>). It evaluates and prints the value of an expression. The usage can be found at https://sourceware.org/gdb/current/onlinedocs/gdb/Data.html .

1.9 Q & A

1.9.1 “Error 127” for Building Code

If you use \$ **make all** command to build code but encounter “Error 127” error message like Fig 1-30, it’s caused by incorrect Cygwin package. Both 32-bit and 64-bit operating system need to install 32-bit installation Cygwin Package – setup-x86.exe.

```

../component/common/application/apple/homekit/crypto/poly1305 -I/cygdrive/f/v03
/project/realtek_amebaD_cm0_gcc_verification/asdk/../../../../component/common/appl
ication/apple/homekit/crypto/ed25519 -I/cygdrive/f/v03/project/realtek_amebaD_cm
0_gcc_verification/asdk/../../../../component/common/application/apple/homekit/cryp
to/ed25519/core -I/cygdrive/f/v03/project/realtek_amebaD_cm0_gcc_verification/as
dk/../../../../component/common/application/apple/homekit/crypto/sha512 -DCONFIG_PL
ATFORM_8721D /cygdrive/f/v03/project/realtek_amebaD_cm0_gcc_verification/asdk/f
lashloader/rtl_flash_download.c -o rtl_flash_download.o
make[2]: *** [/cygdrive/f/v03/project/realtek_amebaD_cm0_gcc_verification/asdk/f
lashloader/rtl_flash_download.o] Error 127
make[2]: Leaving directory '/cygdrive/f/v03/project/realtek_amebaD_cm0_gcc_verif
ication/asdk/make/flashloader'
make[1]: *** [Makefile:87: make_subdirs_flashloader] Error 2
make[1]: Leaving directory '/cygdrive/f/v03/project/realtek_amebaD_cm0_gcc_verif
ication/asdk'
make: *** [Makefile:15: all] Error 2

```

Fig 1-30 “Error 127” error message

1.9.2 “Permission denied” Error under Linux

If you use **\$ make all** command to compile project but encounter “Permission denied” error message like Fig 1-31, it’s caused by file access permission. You can enter **chmod -R 777 \$SDK_FILENAME** under the path of SDK first to change the permission of files in SDK, and then continue compile operation.

```

===== Image manipulating start =====
/home/jesse/sdk-amebaD-beta_v4_20180914/project/realtek_amebaD_cm0_gcc
_verification/asdk/gnu_utility/prepend_header.sh /home/jesse/sdk-ameba
D-beta_v4_20180914/project/realtek_amebaD_cm0_gcc_verification/asdk/im
age/ram_1.bin __ram_start_table_start__ /home/jesse/sdk-amebaD-beta
_v4_20180914/project/realtek_amebaD_cm0_gcc_verification/asdk/image/tar
get_loader.map
make[1]: execvp: /home/jesse/sdk-amebaD-beta_v4_20180914/project/realte
k_amebaD_cm0_gcc_verification/asdk/gnu_utility/prepend_header.sh: Per
mission denied
Makefile:163: recipe for target 'linker_loader' failed
make[1]: *** [linker_loader] Error 127
make[1]: Leaving directory '/home/jesse/sdk-amebaD-beta_v4_20180914/pr
oject/realtek_amebaD_cm0_gcc_verification/asdk'
Makefile:15: recipe for target 'all' failed
make: *** [all] Error 2

```

Fig 1-31 “Permission denied” error message under Linux

1.9.3 How to Reset KM0/KM4 under Debug Mode?

Steps to reset KM0 and KM4 are different.

- For KM0, you need to use the “monitor reset” instruction in corresponding rtl_gdb_debug.txt to reset it under debug. The default path of this file is under \project\realtek_amebaD_va0_example\GCC-RELEASE\project_lp\asdk\gnu_utility\gnu_script\rtl_gdb_debug.txt. Please change the default setting which is “#monitor reset 1” to “monitor reset”.
- For KM4, you need to use the “monitor reset” instruction in corresponding rtl_gdb_debug.txt first (this process is similar to the process of KM0). Then, set the bit[25] of memory address 0x4800_03f8 to 1 because the boot process of KM4 is controlled by the KM0. Without this operation, the KM4 can't jump out of the boot function. Details of this operation are as follows: After executing “make debug” instruction, a debug window pops up. In this window. Set bit[25] to 1, then type “c”, the KM4 is reset. refer to Fig 1-32 for more information.

Note: Only resetting KM0 may cause KM4 to work in an abnormal way because KM0 reboot will change some settings of KM4. If you find KM4 doesn't work after KM0 reboot, you should reset KM4.

```

D:\Project\Ameba-D\Task\release\sdk-amebad-beta_v5.2\sdk-amebad-beta_v5.2\project\r...
GNU gdb (Realtek ASDK-6.4.1 Build 2778) 7.12.50.20170111-git
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later (http://gnu.org/licenses/gpl.html)
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-cygwin --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0e007564 in ?? ()
Notification of completion for asynchronous execution commands is off.
Resetting target.
(gdb) x/iwx 0x480003f8
0x480003f8: 0x00000201
(gdb) set (unsigned)0x480003f8=0x02000201
(gdb) b main
Breakpoint 1: file main.c, line 211.
(gdb) c
Continuing.

Breakpoint 1, main () at main.c:211
211
(gdb) n
212         if (wifi_config.wifi_ultra_low_power &&
(gdb) n
216         InterruptRegister(IPC_INTHandler, IPC_IRQ, IPCM0_DEU, 10);
(gdb) n
217         InterruptEn(IPC_IRQ, 10);
(gdb) c
Continuing.

```

Fig 1-32 KM4 “monitor reset” setting

2 SDK Architecture

The architecture of SDK is shown in Fig 1-1.

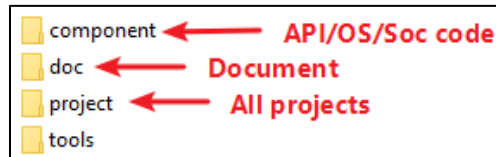


Fig 2-1 SDK architecture

2.1 component

2.1.1 common

Items	Description
api	● AT command ● Platform_stdlib.h: standard library header ● Wi-Fi driver interface ● Wi-Fi promisc mode interface ● Wi-Fi simple configuration
application	● DuerOS ● MQTT
audio	Audio related
bluetooth	Internal BT driver
drivers	● alc5616/ alc5640/ alc5651/ alc5660/ alc5679/ alc5680/ sgtl5000 drivers ● IR NEC driver ● SDIO host driver ● Ameba-D internal codec rl6548 driver ● USB host and device drivers ● WLAN driver
example	Utility examples: wlan_fast_connect/ssl_download/ fatfs/mdns/media_geo_mp4 ...
file_system	File system
grahpic	JPEG decoder
mbed	mbed API source code
media	Multi-media framework
network	● coap ● dhcp ● http ● lwip ● mDNS ● rtsp ● sntp ● ssl (MBEDTLS) ● tftp ● websocket
test	● WLAN test file
ui	● emWin ● littlevGL
utilities	● cJSON ● http_client ● ssl_client

	● tcpecho ● udpecho ● webserver/xml
video	Video related

2.1.2 os

Items	Description
freertos	FreeRTOS source code
os_dep	● osdep_service.c: Realtek encapsulating interface for FreeRTOS ● osdep_service.h: Realtek encapsulating interface header

2.1.3 soc

Items	Description
app	monitor and shell
bootloader	Bootloader
cmsis	ARM headers, include ARM CPU registers and operations
cmsis-dsp	ARM CMSIS-DSP source code
fwlib	Low level drivers like: UART/I2C/SPI/Timer/PWM ...
fwlib/usrcfg	User configuration files, maintained by user: bootcfg/trustzonecfg/sleepcfg/flashcfg/pinmapcfg ...
img3	Files for image3
imgtool_loader	Flash loader for image tool
misc	misc file like ota/pmu...
swlib	Standard software library supported by ROM code, like: _memcpy/_memcmp ...

2.2 doc

Items	Description
AN0004	Realtek low power Wi-Fi MP user guide
AN0011	Realtek WLAN simple configuration
AN0012	Realtek secure socket layer (SSL)
AN0025	Realtek AT command
AN0075	Realtek Ameba-all at command v2.0
AN0096	Realtek Ameba-all xmodem uart update firmware
UM0150	Realtek Ameba CoAP User Guide.pdf

2.3 tools

Items	Description
ImageTool	tools\AmebaD\Image_Tool: image tool for Ameba-D
autopatch	Automatically patched script
bluetooth	Tools for BT
DownloadServer	Used to send image to device based on socket by OTA function.
DownloadServer(HTTP)	Used to send image to device based on socket by OTA function.
file_check_sum	File used to check sum for Flash
probe	Tools for RLX Probe debug
serial_to_usb	Driver for serial to usb
simple_config_wizard	Tools for Wi-Fi simple configuration
simple_config_wizard_3.4b	Tools for Wi-Fi simple configuration
iperf.exe	iperf for Wi-Fi performance test

2.4 GCC Project for KM4

The architecture of KM4 project is shown in Fig 2-2.

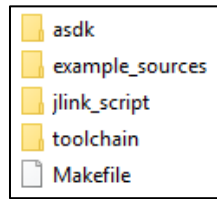


Fig 2-2 Architecture of KM4

Items	Description
asdk	● Menuconfig ● Link script ● Library: lib_wlan.a/lib_wps.a/rom_symbol ... ● Makefile
example_sources	Peripherals driver demo code: ADC/Audio/LCD/UART/I2C ...
jlink_script	J-Link script for KM4
toolchain	GCC & Cygwin toolchain
Makefile	Top Makefile

2.5 Critical Header Files

Items	Description
basic_types.h	SUCCESS/FAIL/TRUE/FALSE/NULL/u8/u16/u32/BOOL/BIT(x) ...
platform_stdlib.h	Standard library API: memcmp/strcpy/atoll/strpbrk/sscanf/printf/sprintf/snprintf/vsnprintf ...
section_config.h	Section definition used in link script: IMAGE2_RAM_TEXT_SECTION ...
mbed api headers	component/common/mbed/ ... ● Peripheral header files for mbed APIs ● If you want to use mbed APIs, related headers must be included.
ameba_soc.h	● Peripheral header files for raw APIs ● Raw APIs have more features than mbed APIs, mbed APIs just have basic features. ● If you want to use raw APIs, this header must be included.

3 GCC Makefile

3.1 KM4 Makefile Architecture

The architecture of KM4 makefile is shown in Fig 3-1.

```

project_hp\Makefile-->
  project_hp\asdk\Makefile-->
    project_hp\asdk\make\Makefile-->
      project_hp\asdk\make\api\Makefile
      project_hp\asdk\make\app\Makefile
      project_hp\asdk\make\application\Makefile
      project_hp\asdk\make\audio\Makefile
      project_hp\asdk\make\bootloader\Makefile
      project_hp\asdk\make\cmsis\Makefile
      project_hp\asdk\make\project\Makefile→
        project_hp\asdk\make\project\sram\Makefile: how to build code into ram
        project_hp\asdk\make\project\xip\Makefile: how to build code into flash
        project_hp\asdk\make\project\library\Makefile: how to build library
      project_hp\asdk\make\drivers\Makefile
      project_hp\asdk\make\example\Makefile
      project_hp\asdk\make\file_system\Makefile
      project_hp\asdk\make\mbed\Makefile
      project_hp\asdk\make\mbedtls\Makefile
      project_hp\asdk\make\network\Makefile
      project_hp\asdk\make\os\Makefile
      project_hp\asdk\make\rtl_bluetooth\Makefile
      project_hp\asdk\make\ssl\Makefile
      project_hp\asdk\make\target\Makefile
      project_hp\asdk\make\utilities\Makefile
      project_hp\asdk\make\utilities_example\Makefile
      project_hp\asdk\make\wlan\Makefile
      project_hp\asdk\make\wps\Makefile
      project_hp\asdk\make\utilities_example\Makefile

```

Fig 3-1 KM4 Makefile architecture

3.2 How to Build Code into Flash?

Makefile in project/xip is an example for how to build code into Flash.

```
include ${MAKE_INCLUDE_GEN}

.PHONY: all clean

#####
#                               VARIABLES                               #
#####
add your include path here
IFLAGS += -I${BASEDIR}/component/common/network/coap/include

set your source code path here
CUSTOMER_DIR = ${ROOTDIR}/make/project/xip

#####
#                               Source FILE LIST                       #
#####
add your source code here
OBJS += \
    ${CUSTOMER_DIR}/xip_test.o\

#####
#                               Include Dependency                       #
#####
include ${OBJS:.o=.d}

#####
#                               RULES TO GENERATE TARGETS               #
#####
all: CORE_TARGETS COPY_RAM_OBJS
#####
#                               GENERATE OBJECT FILE                   #
#####
CORE_TARGETS: ${OBJS}

#####
#                               CLEAN GENERATED FILES                   #
#####
clean: CLEAN_OBJS
    $(REMOVE) *.o
    $(REMOVE) *.i
    $(REMOVE) *.s
    $(REMOVE) *.d

include ${DEPS}
```

3.3 How to Build Code into SRAM?

Makefile in project/SRAM is an example for how to build code into SRAM.

```

include $(MAKE_INCLUDE_GEN)

.PHONY: all clean

#####
#                               VARIABLES                               #
#####
#add your include path here
IFLAGS += #-I$(BASEDIR)/component/common/network/coap/include

#set your source code path here
CUSTOMER_DIR = $(ROOTDIR)/make/project/sram

#####
#                               Source FILE LIST                       #
#####
#add your source code here
OBJS += \
    $(CUSTOMER_DIR)/ram_test.o\

#####
#                               Include Dependency                       #
#####
-include $(OBJS:.o=.d)

#####
#                               RULES TO GENERATE TARGETS               #
#####
all: CORE_TARGETS RENAME_CODE2SRAM COPY_RAM_OBJS
#####
#                               GENERATE OBJECT FILE                   #
#####
CORE_TARGETS: $(OBJS)

%.o:%.c
    $(CC) $(GLOBAL_CFLAGS) $(MODULE_IFLAGS) $< -o $@

#####
#                               CLEAN GENERATED FILES                 #
#####
clean: CLEAN_OBJS
    $(REMOVE) *.o
    $(REMOVE) *.i
    $(REMOVE) *.s
    $(REMOVE) *.d

-include $(DEPS)

```

3.4 How to Use Section Attribute?

Because SRAM space is limited, we suggest you build critical code/data into SRAM and let other code/data left in Flash. You should use section attribute "IMAGE2_RAM_TEXT_SECTION" to locate some of your functions into SRAM.

```
#include <basic_types.h>
#include <section_config.h>

/* const is read only, it will build into flash */
const u32 xip_test_read_only_data = 0;

/* non read only data will build into SRAM */
u32 xip_test_data = 0;

/* code in in SRAM */
IMAGE2_RAM_TEXT_SECTION
void test_critical_code(void)
{
}

/* code in in Flash */
void test_non_critical_code(void)
{
}
```

Note:

- You should include "section_config.h"
- const/read-only data will build into Flash
- Non read-only data will build into SRAM (If you need read only data build into SRAM to run faster, you should not use "const")
- Functions limited by IMAGE2_RAM_TEXT_SECTION will build into SRAM
- Functions not limited by IMAGE2_RAM_TEXT_SECTION will build into Flash

3.5 How to Build Library?

Makefile in project/library is an example for build user library.

```
include $(MAKE_INCLUDE_GEN)

.PHONY: all clean

#####
#                               VARIABLES                               #
#####
#add your include path here
IFLAGS += #-I$(BASEDIR)/component/common/network/coap/include

#set your source code path here
CUSTOMER_DIR = $(ROOTDIR)/make/project/library

#####
#                               Source FILE LIST                       #
#####
#add your source code here
OBJS += \
        $(CUSTOMER_DIR)/lib_user_test.o\

#####
#                               Dependency                               #
#####
#include $(OBJS:.o=.d)

#####
#                               RULES TO GENERATE TARGETS              #
#####
# Define the Rules to build the core targets
all: CORE_TARGETS COPY_RAM_OBJS
    $(AR) rvs lib_user.a $(OBJS) $(OBJS_SRAM)
    $(MOVE) -f lib_user.a $(ROOTDIR)/lib/application

#####
#                               GENERATE OBJECT FILE                   #
#####
CORE_TARGETS: $(OBJS)

#####
#                               RULES TO CLEAN TARGETS                 #
#####
clean: CLEAN_OBJS
    $(REMOVE) *.o
    $(REMOVE) *.i
    $(REMOVE) *.s
    $(REMOVE) *.d
```

3.6 How to Add Library?

Open `project_hp\asdk\Makefile`, and add `lib_user.a` into `LINK_APP_LIB`.

```
LINK_APP_LIB += $(ROOTDIR)/lib/application/lib_user.a
```

4 C++ Standards Supported in GCC

4.1 Introduction

This chapter mainly introduces how to build C++ codes in Ameba-D GCC project.

Note: “iostream” is not available currently.

4.2 To Compile C++ Codes

The following steps are necessary to support C++ in current GCC project.

- (1) Modify the link script: `rlx8721d_img2_is.ld` (non-security) or `rlx8721d_img2_tz.ld` (security).

```
.xip_image2.text :
{
    __flash_text_start__ = .;

    *(.img2_custom_signature*)

    *(.text*)
    *(.rodata*)

    /* Add This for C++ support */
    . = ALIGN(4);
    __preinit_array_start = .;
    KEEP(*(SORT(.preinit_array)))
    __preinit_array_end = .;
    . = ALIGN(4);
    __init_array_start = .;
    KEEP(*(SORT(.init_array.*)))
    KEEP(*(SORT(.init_array)))
    __init_array_end = .;
    . = ALIGN(4);
    __fini_array_start = .;
    KEEP(*(SORT(.fini_array.*)))
    KEEP(*(SORT(.fini_array)))
    __fini_array_end = .;
    /*-----*/

    . = ALIGN(4);
    __cmd_table_start__ = .;
    KEEP(*(SORT(.cmd.table.data*)))
    __cmd_table_end__ = .;

    __flash_text_end__ = .;

    . = ALIGN(16);
} > KM4_IMG2

/* Add This for C++ support */
.ARM.extab :
{
    *(.ARM.extab* .gnu.linkonce.armextab.*)
} > KM4_IMG2

.ARM.exidx :
{
    __exidx_start = .;
    *(.ARM.exidx* .gnu.linkonce.armexidx.*)
    __exidx_end = .;
    end = .;
} > KM4_IMG2

__wrap_printf = 0x1010a3f5;
__wrap_sprintf = 0x1010a471;
__wrap_strcat = 0x10111635;
__wrap_strchr = 0x10111675;
__wrap_strcmp = 0x10111745;
__wrap_strncmp = 0x101118f9;
__wrap_strlen = 0x10111839;
__wrap_strnlen = 0x10111a05;
__wrap_strncat = 0x1011189d;
__wrap_strpbrk = 0x10111a39;
__wrap_strstr = 0x10111d25;
__wrap_strtok = 0x1011201d;
__wrap_strsep = 0x10111a65;
__wrap_strtoll = 0x10111ff0;
__wrap_strtoul = 0x101122e9;
__wrap_strtoull = 0x10111f3d;
__wrap_atoi = 0x101115e1;
__wrap_strcpy = 0x101117b9;
__wrap_strncpy = 0x1011199d;
__wrap_memset = 0x10110ea1;
__wrap_memcpy = 0x10110d2d;
__wrap_memcmp = 0x10110cc9;
__wrap_memmove = 0x10110dd9;
__wrap_snprintf = 0x1010a49d;
__wrap_malloc = puPortMalloc;
__wrap_realloc = puPortReAlloc;
__wrap_free = vPortFree;
/*-----*/
```

- (2) Modify startup code: `rtl8721dhp_app_start.c`.

```
#if defined ( __GNUC__ )
/* Add This For C++ support to avoid compile error */
void _init(void){}
#endif

// The Main App entry point
void app_start(void)
{
    simulation_stage_set(SIMULATION_KM4_CPUID, BIT_KM4_APP_ENTER);
    SOCPS_InitSYSIRQ_HP();

    _NVIC_SetVector(SUCCall_IRQn, (VOID*)vPortSUCHandle);
    _NVIC_SetVector(PendSV_IRQn, (VOID*)xPortPendSVHandler);
    _NVIC_SetVector(SysTick_IRQn, (VOID*)xPortSysTickHandler);

#if defined ( __ICCARM__ )
    __iar_cstart_call_ctors(NULL);
#endif

#if defined ( __GNUC__ )
/* Add This for C++ support */
__libc_init_array();
#endif

    // force SP align to 8 byte not 4 byte (initial SP is 4 byte align)
    __asm(
        "mov r0, sp\n"
        "bic r0, r0, #7\n"
        "mov sp, r0\n"
    );

    main();
}
```

- (3) Modify makefile.
 - project\realtek_amebaD_va0_example\GCC-RELEASE\project_hp\asdk\Makefile

```
linker_image2_ns:
@echo "===== linker_img2_ns start ====="
$(LD) $(LD_ARG) target_img2.axf $(RAM_OBJS_LIST) $(LINK_ROM_LIB_NS) $(LINK_APP_LIB) $(IMAGE_TARGET_FOLDER)/cns_e_implib.lib
-lm -lc -lnosys -lgcc -lstdc++
```

- `project\realtek_amebaD_va0_example\GCC-RELEASE\project_hp\asdk\Makefile.include.gen`

```
#GLOBAL_CFLAGS += -specs nosys.specs
#GLOBAL_CFLAGS += -fno-short-enums
GLOBAL_CFLAGS += -Wextra

GLOBAL_CFLAGS += $(IFLAGS)
GLOBAL_CFLAGS += -DCONFIG_PLATFORM_8721D
GLOBAL_CFLAGS += -DCONFIG_USE_MBEDTLS_ROM_ALG
GLOBAL_CFLAGS += -DCONFIG_FUNCTION_00_OPTIMIZE
GLOBAL_CFLAGS += -DDM_ODM_SUPPORT_TYPE=32

CFLAGS = $(GLOBAL_CFLAGS)
CFLAGS += -Wstrict-prototypes

CPPFLAGS = $(GLOBAL_CFLAGS)
CPPFLAGS += -std=c++11 -fno-use-cxa-atexit
```

```
LD_ARG += -nostartfiles
LD_ARG += -specs nosys.specs
LD_ARG += -Wl,--gc-sections
LD_ARG += -Wl,--warn-section-align
LD_ARG += -Wl,-Map=text.map
LD_ARG += -Wl,--cref
LD_ARG += -Wl,-build-id=none
LD_ARG += -save-temps

LD_ARG += -Wl,-wrap,strcat      -Wl,-wrap,strcmp      -Wl,-wrap,strcmp
LD_ARG += -Wl,-wrap,strcmpcmp -Wl,-wrap,strcpy     #-Wl,-wrap,strcmpcmp
LD_ARG += -Wl,-wrap,strncmppy -Wl,-wrap,strlen       #-Wl,-wrap,strcpy
LD_ARG += -Wl,-wrap,strlenlen -Wl,-wrap,strtocat    -Wl,-wrap,strupbrk
LD_ARG += -Wl,-wrap,strstr     -Wl,-wrap,strtok      #-Wl,-wrap,strspn
LD_ARG += -Wl,-wrap,strcmp     #-Wl,-wrap,strcmpxfrm -Wl,-wrap,strtod
LD_ARG += -Wl,-wrap,strtoll    #-Wl,-wrap,strtouf   -Wl,-wrap,strtold
LD_ARG += -Wl,-wrap,strtoul    -Wl,-wrap,strtoull   -Wl,-wrap,atoi
LD_ARG += #-Wl,-wrap,atoui     -Wl,-wrap,atol       -Wl,-wrap,atoul
LD_ARG += #-Wl,-wrap,atoull    -Wl,-wrap,atof
LD_ARG += -Wl,-wrap,malloc     -Wl,-wrap,free        -Wl,-wrap,realloc
LD_ARG += -Wl,-wrap,memcmp     -Wl,-wrap,memcpy
LD_ARG += -Wl,-wrap,memmove    -Wl,-wrap,memset
LD_ARG += -Wl,-wrap,printf     -Wl,-wrap,sprintf
LD_ARG += -Wl,-wrap,sprintf    #-Wl,-wrap,vsnprintf
```

```
#####
#                               RULES TO GENERATE OBJECT FILE FROM .CPP FILE                               #
#####
%.o:%.cpp
$(CC) $(CPPFLAGS) $< -o $@
```

- (4) Add C++ files into protect.
We have tested a small program and provide it as an example, which can be found in **main.cpp**.
- (5) Use command **make** in console to compile project with C++ files.

5 GCC Standard Library

5.1 Introduction

This chapter mainly introduces how to use GCC standard library in Ameba-D.

To save flash memory and improve efficiency, ROM code has implemented a standard software library, like `_memcpy` and `_memcmp`; and some functions are simplified version in ROM software library, such as `printf` and `sprintf`.

There are two methods when using standard library functions such as `printf` and `sprintf` in GCC:

- Using ROM software library.
- Using GCC standard library.

5.2 Default Use of Library Function

In current SDK, the default use of library function is illustrated in Fig 5-1 and Table 5-1

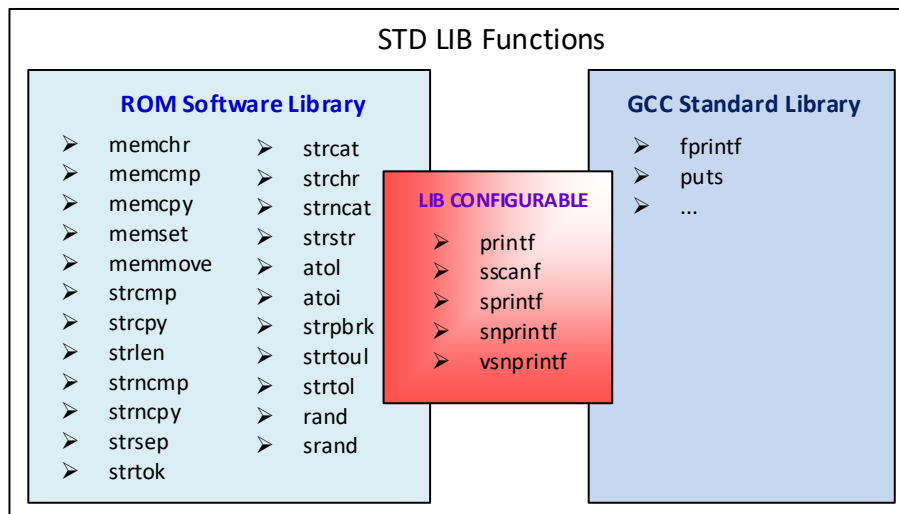


Fig 5-1 Library function guide

Table 5-1 Default use of library function

Item	Function
Using ROM software library	<code>memchr/memcmp/memcpy/memset/memmove/strcmp/strcpy/strlen/strncmp/strncpy/strsep/strtok/strcat/strchr/strncat/strstr/atoll/atoi/strpbrk/strtoul/strtol/rand/srand</code>
Using GCC standard library	<code>fprintf/puts/...</code>
Library is configurable	<code>printf/sprintf/snprintf/vsnprintf/scanf</code> Note: The default library is ROM software library; you can switch to GCC standard library by defining macro STD_PRINTF .

Because `printf/sprintf/snprintf/vsnprintf/scanf` in ROM software library are a simplified version compared with these in GCC standard library, they only support a few formats. In order to remind users that unsupported format occurs when using these functions in ROM library, SDK wraps up these functions.

Table 5-2 Wrapper functions

Item	Wrapper Function
<code>printf</code>	<code>_rtl_printf</code>

sprintf	_rtl_sprintf
snprintf	_rtl_snprintf
vsprintf	_rtl_vsnprintf
sscanf	_rtl_sscanf

When using printf/sprintf/snprintf/vsnprintf/sscanf in GCC standard library, the memory size will increase 40KB compared with using these functions in ROM software library.

5.3 To Use Configurable Function in GCC Standard Library

Compared to printf/sprintf/snprintf/vsnprintf/sscanf functions in GCC library, they only support a few formats in ROM library. The following content takes printf as an example.

Table 5-3 printf supported format

Library	printf Supported Format
ROM software library	%s, %x, %X, %p, %P, %d, %c
GCC standard library	%s, %x, %X, %p, %P, %d, %c, %f, %L, %l, %u, %U, %e, %E, ...

For printf/sprintf/snprintf/vsnprintf/sscanf, ROM library is linked by default. If “format not support!” log dumps out in trace tool, it means that unsupported format occurs, you should link these functions to GCC standard library.

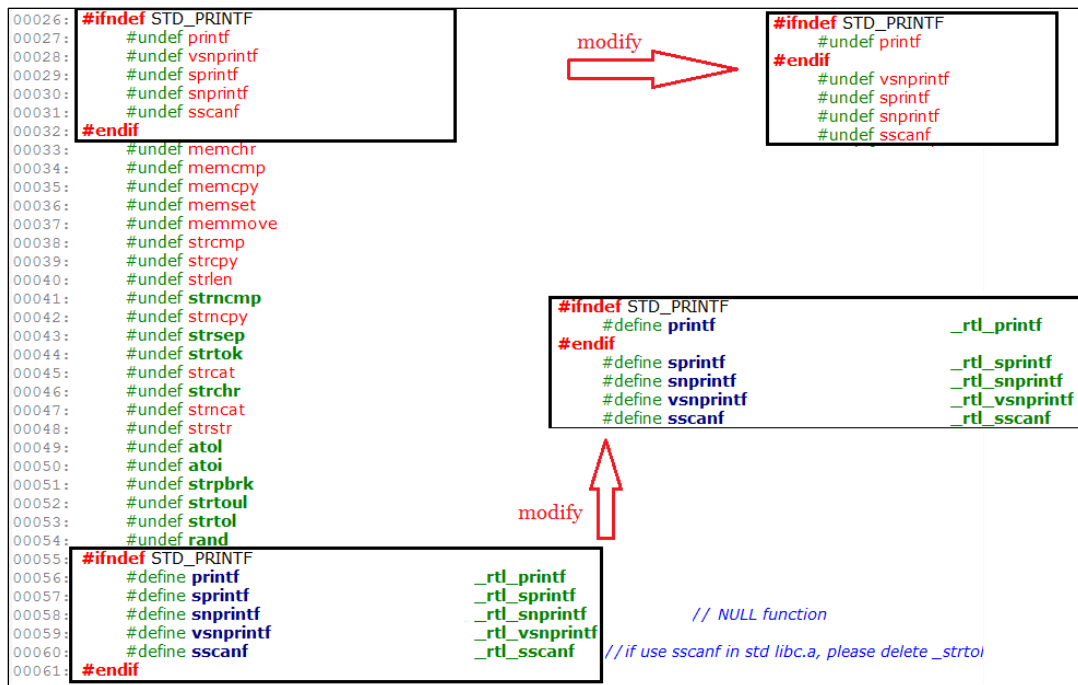
To use printf/sprintf/snprintf/vsnprintf/sscanf in GCC standard library, follow these steps:

(1) Add **#define STD_PRINTF** in the front.

There are two cases:

Item	Operation	Description
case 1	Add #define STD_PRINTF in the front of c files, and include “ platform_stdlib.h ”	Change printf/sprintf/snprintf/vsnprintf/sscanf of some files’ link to GCC standard library. Example: <pre>// main.c #define STD_PRINTF #include "main.h" #include "platform_stdlib.h" int main(void) { printf("%f\n", 1.2); printf("%u\n", 20); }</pre>
case 2	Add #define STD_PRINTF in the front of “ platform_stdlib_rtl8721d.h ”	Change printf/sprintf/snprintf/vsnprintf/sscanf of all SDK’s link to GCC standard library.

(2) Modify “**platform_stdlib_rtl8721d.h**” file if you don’t want to switch to use all the five functions in GCC standard library.
In SDK, macro **STD_PRINTF** controls printf/sprintf/snprintf/vsnprintf/sscanf at the same time. If you only want to configure some but not all, for example, you only want to use printf in GCC standard library, and other four functions sprintf/snprintf/vsnprintf/sscanf maintain the default state, you need to modify “**platform_stdlib_rtl8721d.h**” file.



5.4 Tips

- If you want to use GCC standard library, we recommend only link some user specified c files to GCC standard library instead of link all SDK to GCC standard library, because printf/sprintf/snprintf/vsnprintf/sscanf in our SDK should link to ROM standard library.
- If using printf in GCC standard library libc.a, and there is no "\n" in the end, use fflush(stdout) after printf to dump log in cache. For example:

```
printf("hello");
fflush(stdout);
```
- In current GCC project, KM0 does not support floating-point temporarily, so printf cannot dump %f in KM0 in default setting.
- If using sscanf in GCC standard library libc.a, delete _strtol_r and _strtoul_r symbols in rlx8721d_rom_symbol_acut.ld.

6 IAR Build Environment Setup

This chapter illustrates how to setup IAR development environment for Realtek Ameba-D SDK, including building projects, downloading images and debugging.

6.1 Requirement

6.1.1 IAR Embedded Workbench

IAR provides an IDE environment for code building, downloading, and debugging. Check “IAR Embedded Workbench” on <http://www.iar.com/>, and a trial version is available for 30 days.

Note: To support ARMv8-M with Security Extension (Ameba-D HS CPU, also called KM4), IAR version must be 8.30 or higher.

6.1.2 J-Link or RLX Probe

If you need to download images or debug code for Ameba-D with IAR, then a J-Link adapter or a RLX Probe is necessary.

Note: For Ameba-D CPU, the J-Link version must be v9 or higher.

6.2 Hardware Configuration

The hardware block diagram of Ameba-D demo board is shown in Fig 6-1.

- USB TO UART: supply power and print logs, baud rate is 115200 bps.
- SWD: SWD interface, used for image downloading and debugging with IAR.
- Reset button: reset Ameba-D to run firmware after IAR completes downloading.

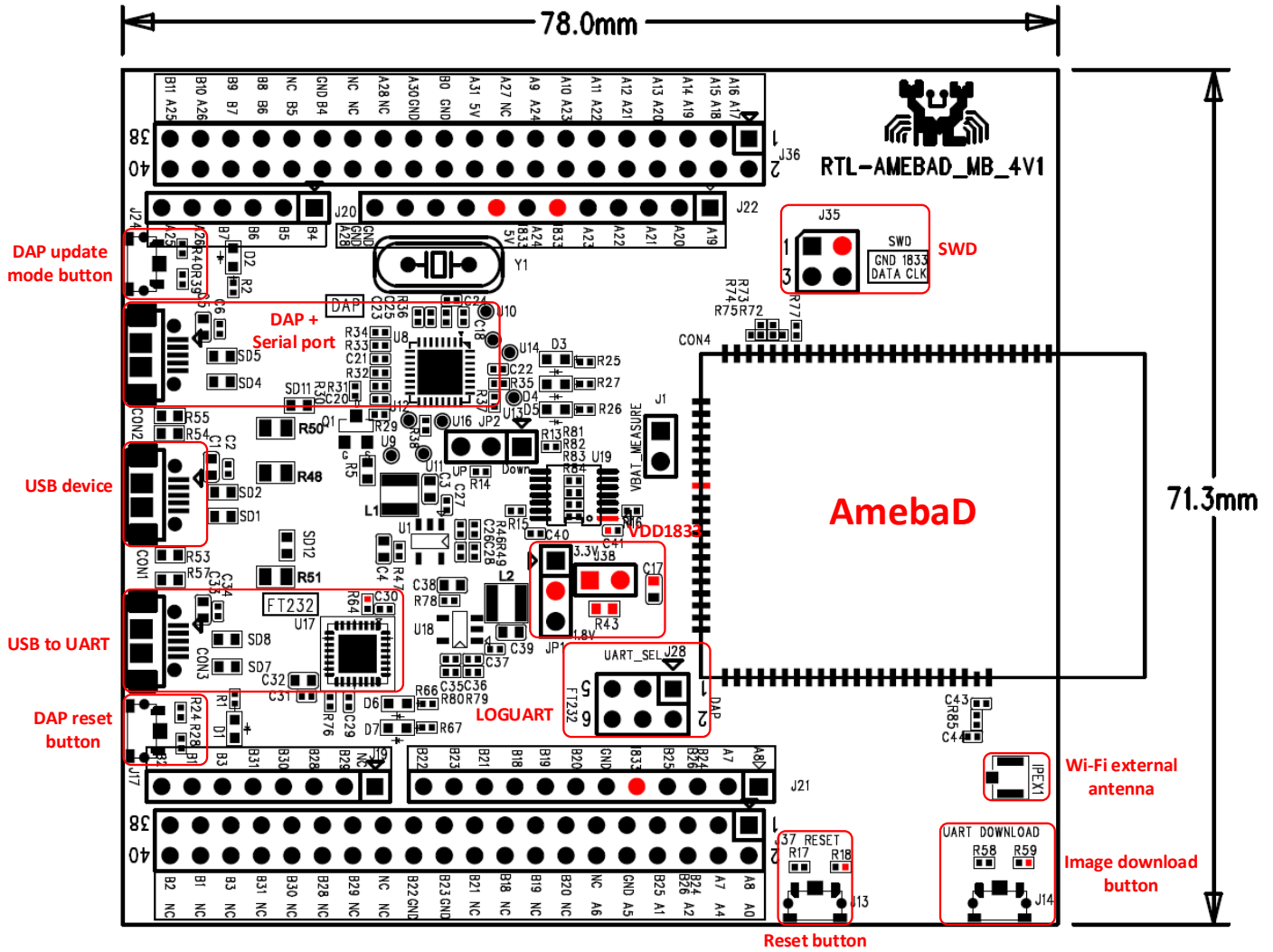


Fig 6-1 Ameba-D demo board

6.2.1 Connecting with J-Link

Refer to Fig 6-2 and Fig 6-3 to connect Ameba-D SWD interface with J-Link.

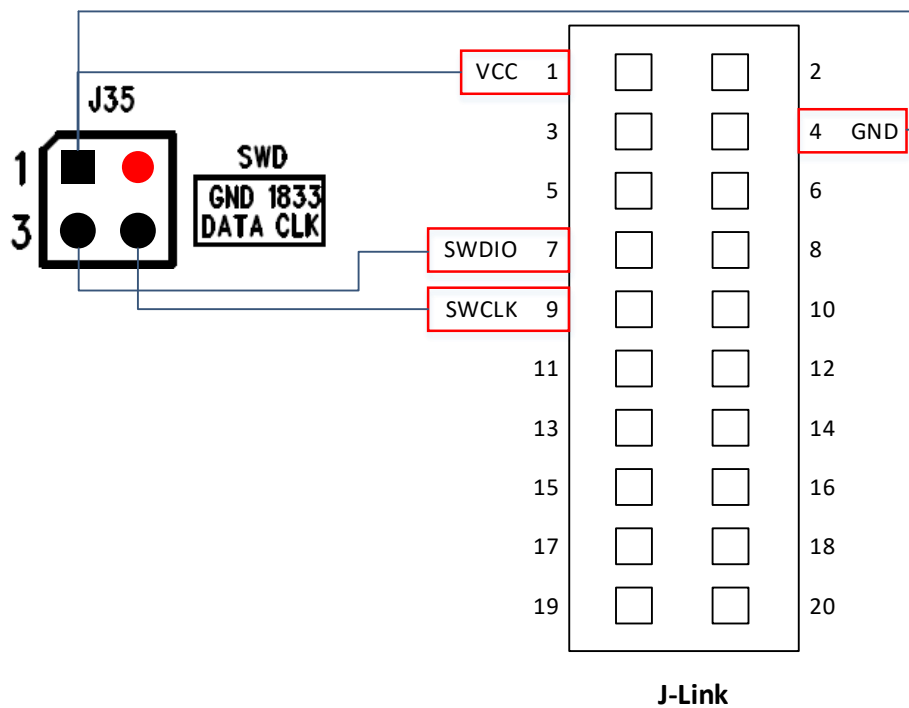


Fig 6-2 J-Link and SWD connection diagram

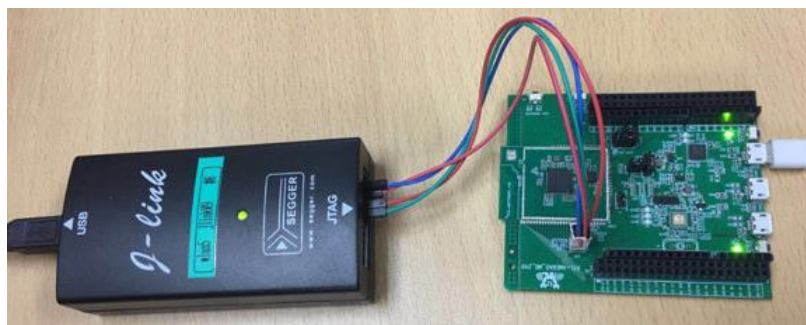


Fig 6-3 J-Link and Ameba-D SWD connection

6.2.2 Connecting with RLX Probe

Refer to Fig 6-4 and Fig 6-5 to connect Ameba-D SWD interface with RLX Probe.

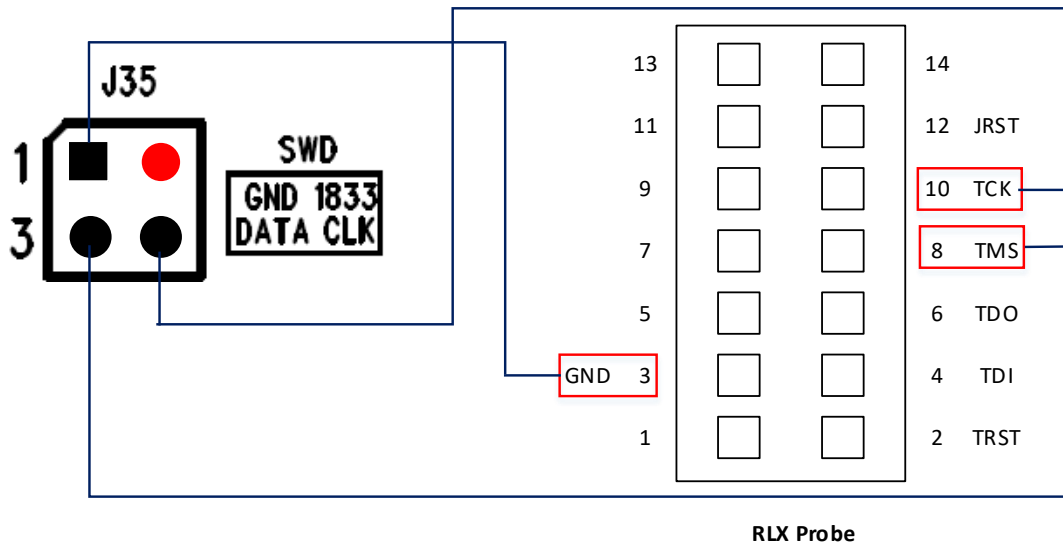


Fig 6-4 RLX Probe and SWD connection diagram

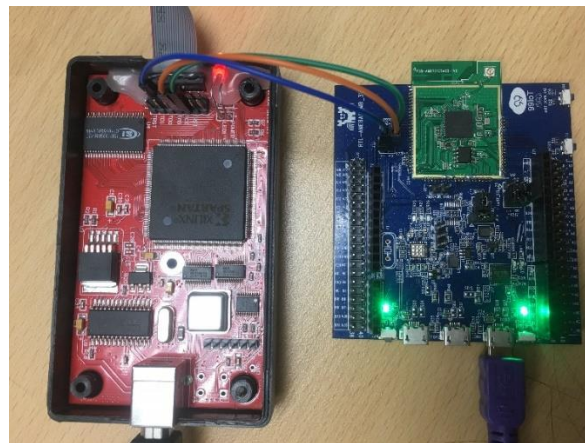


Fig 6-5 RLX Probe and Ameba-D SWD connection

6.3 How to Use IAR SDK?

6.3.1 IAR Project Introduction

Because Ameba-D is a dual-core CPU platform, two workspaces are provided to build for each core in **project\realtek_amebaD_va0_example\EWARM-RELEASE**.

- Project_lp_release.eww (KM0 workspace) contains the following projects:
 - km0_bootloader
 - km0_application
- Project_hp_release.eww (KM4 workspace) contains the following projects:
 - km4_bootloader
 - km4_application
 - km4_secure

Each project in KM4 workspace has different build configurations, as Table 6-1 shows.

Table 6-1 Build configurations for KM4 project

Project	Build Configuration	Configure TrustZone	Enable MP
km4_bootloader	km4_bootloader - is ¹	N	N
	km4_bootloader - tz ²	Y	N
km4_application	km4_application - is	N	N
	km4_application - tz	Y	N
	km4_application - is (mp) ³	N	Y
	km4_application - tz (mp)	Y	Y
km4_secure	km4_secure - tz	Y	N
	km4_secure - tz (mp)	Y	Y

Note:

1. The configuration items with “-is” are ignore secure configuration, which are designed for applications that do not use TrustZone.
 2. The configuration items with “-tz” are TrustZone configuration, which are designed for applications that use TrustZone.
 3. The configuration items with “mp” are mass production configuration, which are designed for generating MP image.
- For applications that do not use TrustZone, users should apply ignore secure configurations as Table 6-2 shows. The km4_secure project which contains Trustzone-protected code, is not used.
 - For applications that use TrustZone, users should apply TrustZone configurations as Table 6-2 shows.

Table 6-2 Configurations for project with/without TrustZone

Project	TrustZone	Normal Image	MP Image
km4_bootloader	N	km4_bootloader - is	km4_bootloader - is
	Y	km4_bootloader - tz	km4_bootloader - tz
km4_application	N	km4_application - is	km4_application - is (mp)
	Y	km4_application - tz	km4_application - tz (mp)
km4_secure	Y	km4_secure - tz	km4_secure - tz (mp)

At the top of the Workspace window, there is a drop-down list where you can choose a build configuration for a specific project.

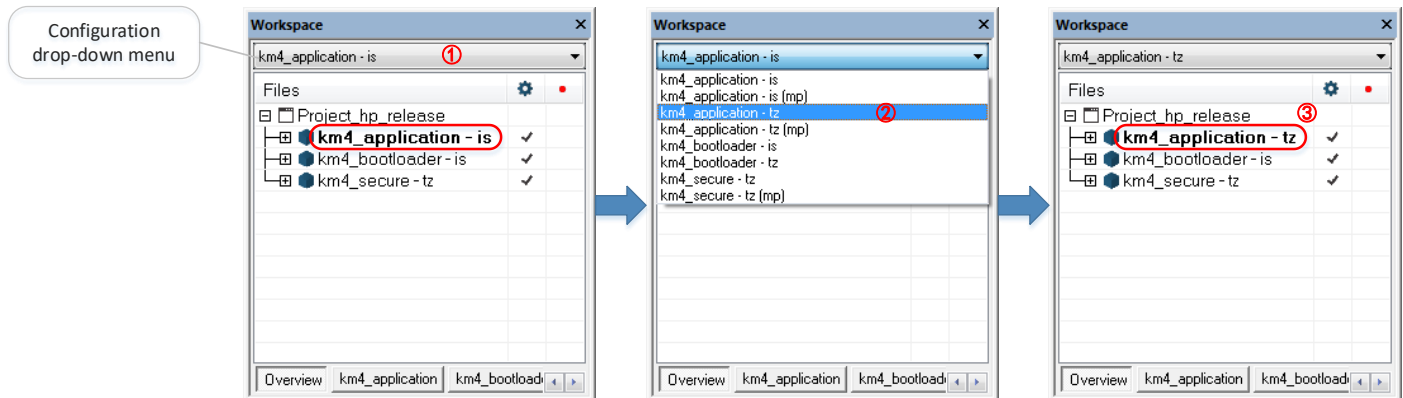


Fig 6-6 How to choose a build configuration

6.3.2 IAR Build

When building SDK for the first time, you should build both KM0 project and KM4 project. Other times, you only need to rebuild the modified project.

6.3.2.1 Building KM0 Project

The following steps show how to build KM0 project:

- (1) Open **project\realtek_amebaD_va0_example\EWARM-RELEASE\Project_lp_release.eww**.

- (2) Make sure km0_bootloader and km0_application are in Workspace. Click **Project > Options, General Options > Target > Processor Variant > Core**, verify the CPU configurations according to Fig 6-7.
- (3) Right click the project and choose "Rebuild All", as Fig 6-8 shows. The km0_bootloader and km0_application should compile in order.

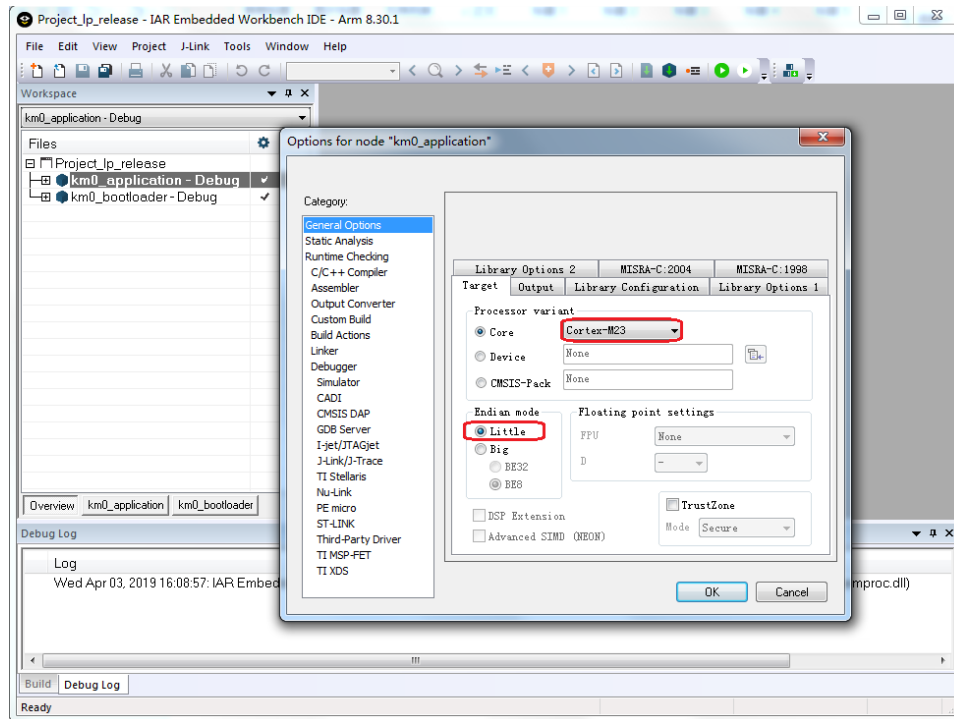


Fig 6-7 KM0 processor options

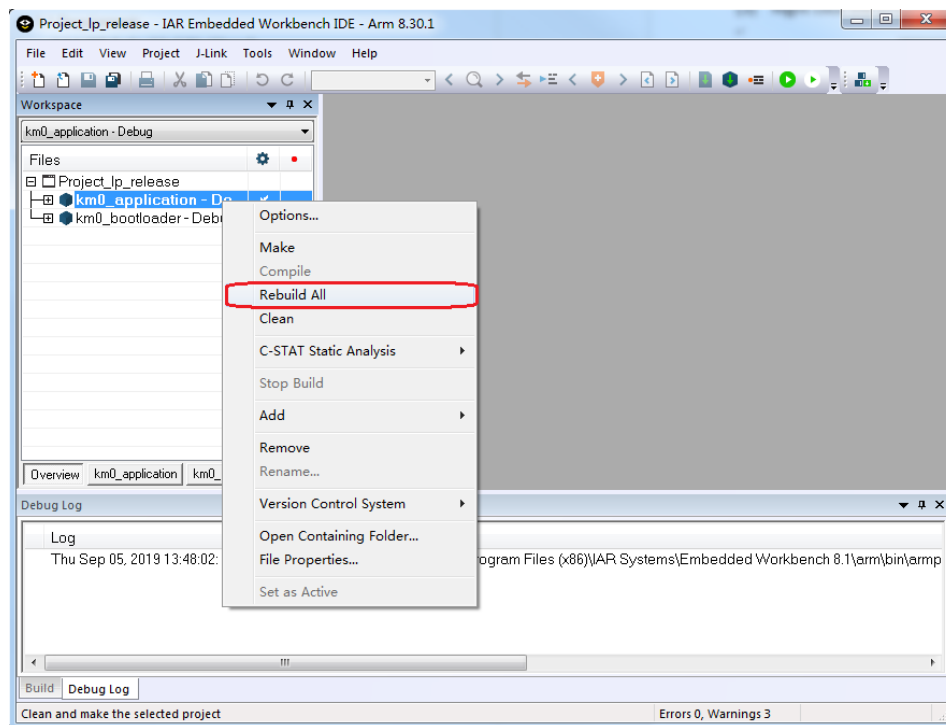


Fig 6-8 Building KM0 project

Note: After building each project, IAR will pop up a command prompt window to execute post-build action to generate images from executable files. This may takes several seconds. Don't stop it while it is in progress. After post-build action is completed, the window would disappear automatically.

```

C:\Windows\System32\cmd.exe
D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE>cmd /c
""D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE""\..\
..\component\soc\realtek\amebaD_misc\iar_utility\common\tools\nm Debug/Exe/km0
_image/km0_application.axf ! "D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0
_example\EWARM-RELEASE""\..\..\component\soc\realtek\amebaD_misc\iar_utility\c
ommon\tools\sort > Debug/Exe/km0_image/km0_application.map"

D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE>cmd /c
""D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE""\..\
..\component\soc\realtek\amebaD_misc\iar_utility\common\tools\objdump -d Debug
/Exe/km0_image/km0_application.axf > Debug/Exe/km0_image/km0_application.asm"

D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE>for /P
"delims=" %i in ('cmd /c ""D:\Code\AmebaD\03_0903\project\realtek_amebaD_va0_ex
ample\EWARM-RELEASE""\..\..\component\soc\realtek\amebaD_misc\iar_utility\comm
on\tools\grep IMAGE2 Debug/Exe/km0_image/km0_application.map ! "D:\Code\AmebaD\0
3_0903\project\realtek_amebaD_va0_example\EWARM-RELEASE""\..\..\component\soc
\realtek\amebaD_misc\iar_utility\common\tools\grep Base ! "D:\Code\AmebaD\03_09
03\project\realtek_amebaD_va0_example\EWARM-RELEASE""\..\..\component\soc\real
tek\amebaD_misc\iar_utility\common\tools\gawk '{print $1}'"') do set ran2_start=
%xi
    
```

- (4) After compile, the images km0_boot_all.bin and km0_image2_all.bin can be seen in **project\realtek_amebaD_va0_example\EWARM-RELEASE\Debug\Exe\km0_image**.

6.3.2.2 Building KM4 Project

The following steps show how to build KM4 project:

- (1) Open project\realtek_amebaD_va0_example\EWARM-RELEASE\Project_hp_release.eww.
- (2) Refer to 6.3.1 and choose the build configurations for each project according to your application.
- (3) Click **Project > Options, General Options > Target > Processor Variant > Core**, verify the CPU configurations according to Fig 6-9.

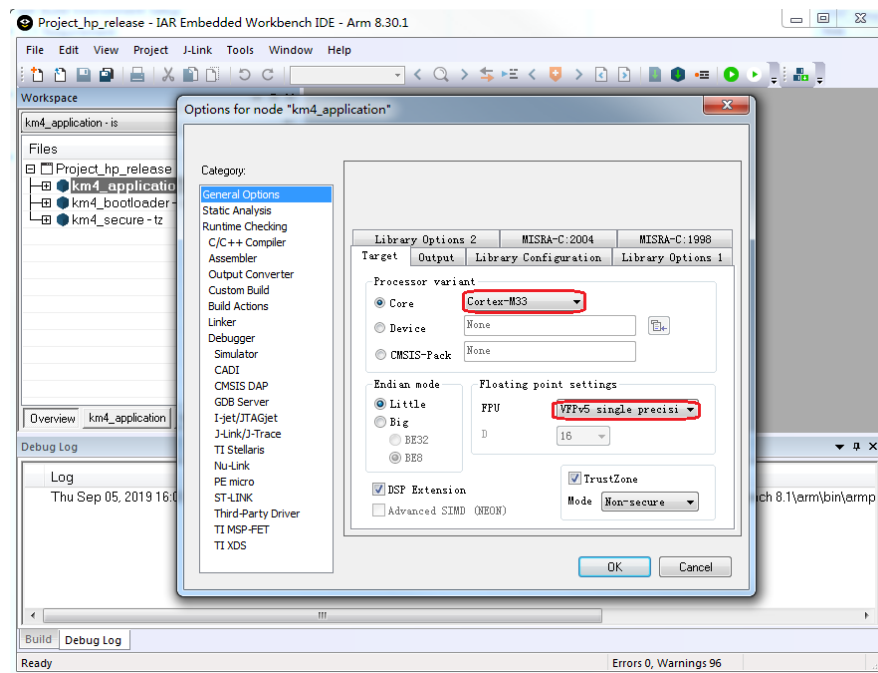


Fig 6-9 KM4 processor options

- (4) Right click the project and choose “Rebuild All”, as Fig 6-10 shows. The km4_bootloader, km4_secure and km4_application should compile in order.

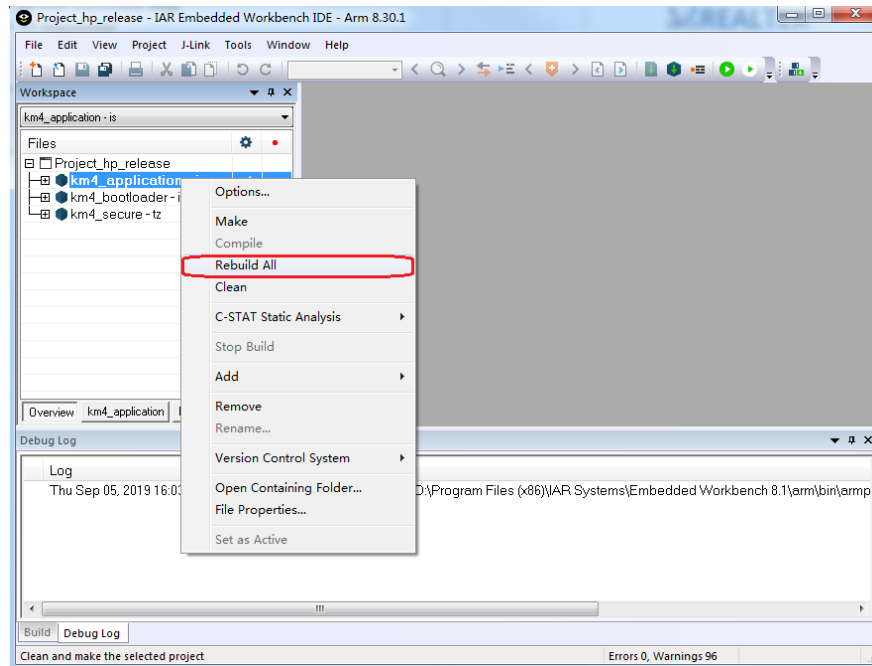
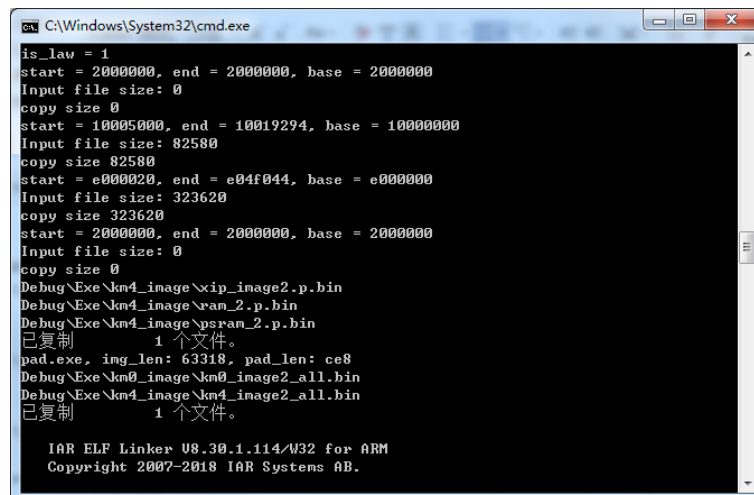


Fig 6-10 Building KM4 project

Note:

- When TrustZone is enable, the km4_secure project must be built before the km4_application project. When TrustZone is not used, there is no need to compile the km4_secure project.
- After building each project, IAR will pop up a command prompt window to execute post-build action to generate images from executable files. This may takes several seconds. Don't stop it while it is in progress. After post-build action is completed, the window would disappear automatically.



- (5) After compile, the images km4_boot_all.bin and km0_km4_image2.bin can be seen in **project\realtek_amebaD_va0_example\EWARM-RELEASE\Debug\Exe\km4_image**. For MP configurations, the km0_km4_image2_mp.bin would be generated instead.

6.3.3 IAR Download

The generated images can be downloaded in two ways:

- IAR J-Link or RLX Probe SWD (introduced in the next section)
- Ameba-D ImageTool, refer to ImageTool for more information.

Ameba-D demo board supports using J-Link and RLX Probe SWD to download and debug. Image of each project can be download individually.

Note: Considering KM4 is powered-on by KM0, you should make sure that KM0 has boot up already before downloading images to KM4. Otherwise, for J-Link, J-Link can't connect to KM4 and show the error message as Fig 6-11 shows. For RLX probe, RLX Probe driver can't be opened under KM4.

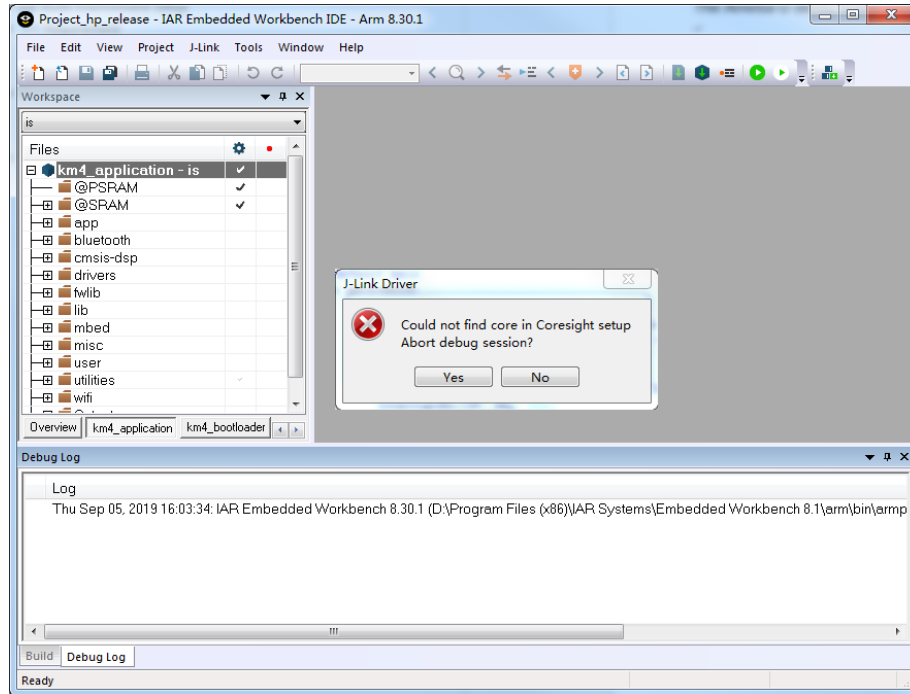


Fig 6-11 J-Link cannot find KM4

As a result, if the Flash memory is empty, the sequence to download images is:

- (1) Download for km0_bootloader and km0_application projects
- (2) Click **Reset** button on demo board to make KM0 boots up
- (3) Download for km4_bootloader and km4_application projects

During development, if Flash memory is not empty and KM0 can boot up successfully, then you can download updated images to KM4 directly, and there is no need to re-download for KM0.

The following steps show how to download image for the target project with IAR. If there is an error like Fig 6-24 displayed in IAR window, refer to 6.3.4.3.

- (1) Choose the target project display in Workspace window, for example, km4_bootloader as Fig 6-12 shows.
- (2) If using J-link debugger, check whether the J-link debugger setting is correct.
 - a) Click **Project > Options > Debugger > Setup > Driver**, and choose "J-Link/J-Trace", as Fig 6-13 shows.
 - b) Click **Debugger > J-Link/J-Trace > Connection > Interface**, and choose "SWD", as Fig 6-14 shows.

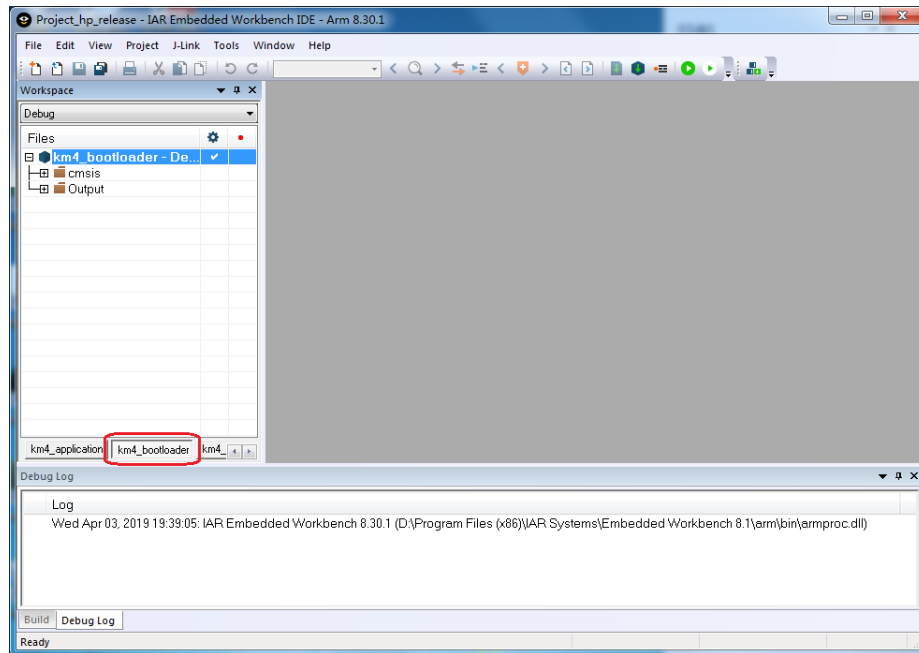


Fig 6-12 Switching to the target project view

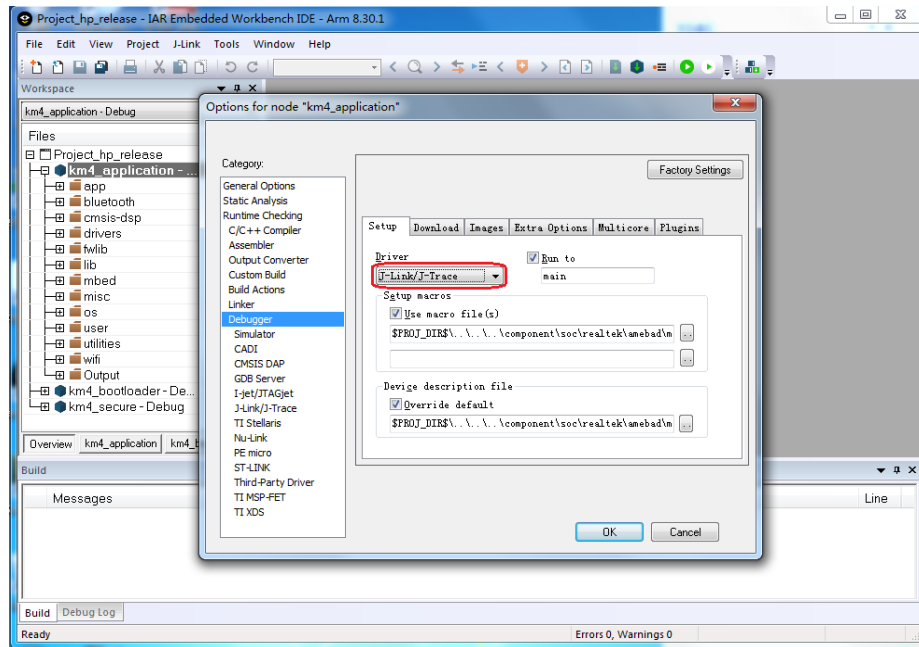


Fig 6-13 J-Link debugger setup

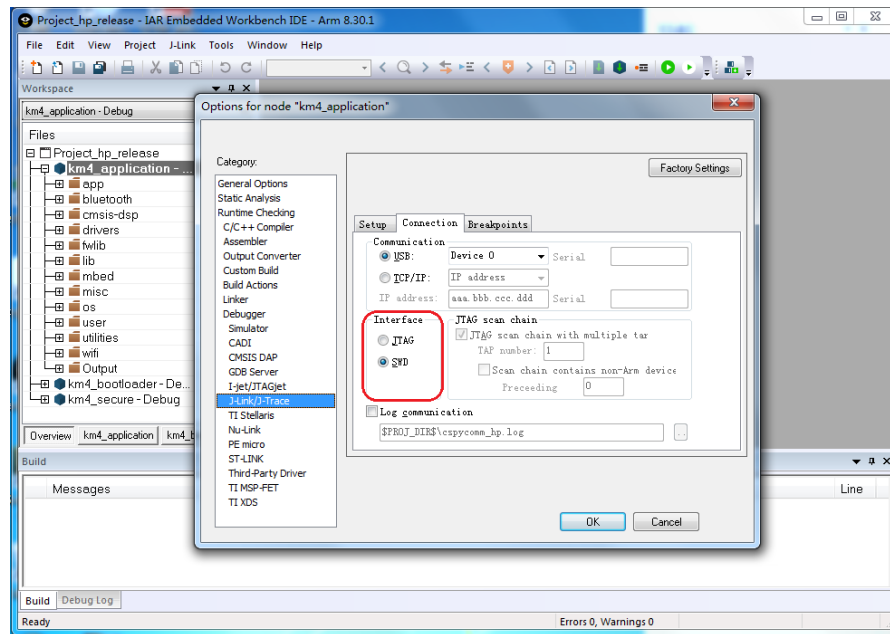


Fig 6-14 J-Link interface setup

- (3) If using RLX Probe, check whether the RLX Probe setting is correct.
 - a) Click **Project > Options > Debugger > Setup > Driver**, choose “GDB Server” and don not choose “Run to”, as Fig 6-15 shows.
 - b) Click **GDB Server > GDB Server**, and put correct value in “TCP/IP address or hostname”, as Fig 6-16 shows. The value should be: *localhost, port number*.
 - ◆ KM4: The default port numbers is 3333, which is set in **project\realtek_amebaD_va0_example\EWARM-RELEASE\probe\cm4\rlx_probe0.cfg**.
 - ◆ KM0: The default port numbers is 2331, which is set in **project\realtek_amebaD_va0_example\EWARM-RELEASE\probe\cm0\rlx_probe0.cfg**.
 - c) Open RLX Probe in **project\realtek_amebaD_va0_example\EWARM-RELEASE\probe\cm4\cm4_RTL_Probe**, as Fig 6-17 shows.
- Note:** The board must be reset before opening the RLX Probe, otherwise the connection may fail.

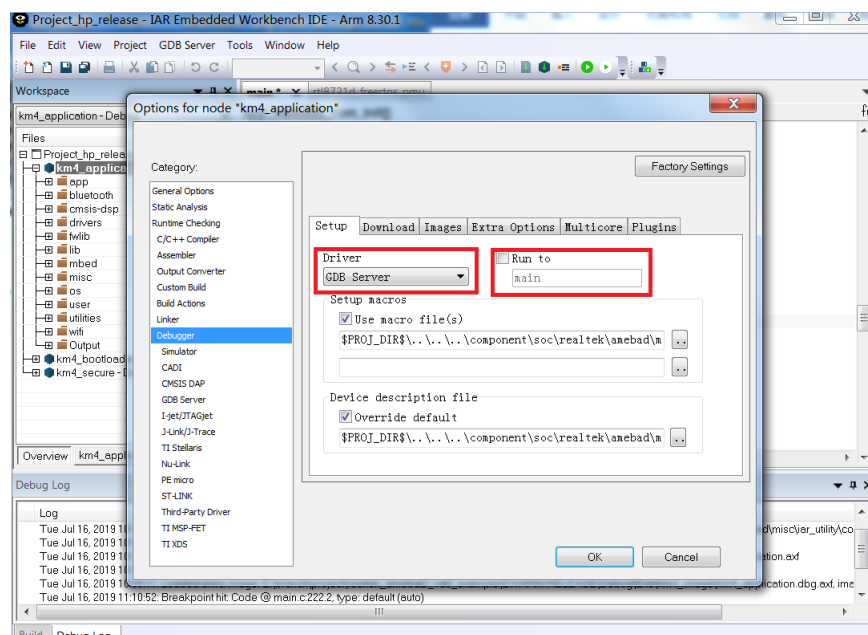


Fig 6-15 RLX Probe setup

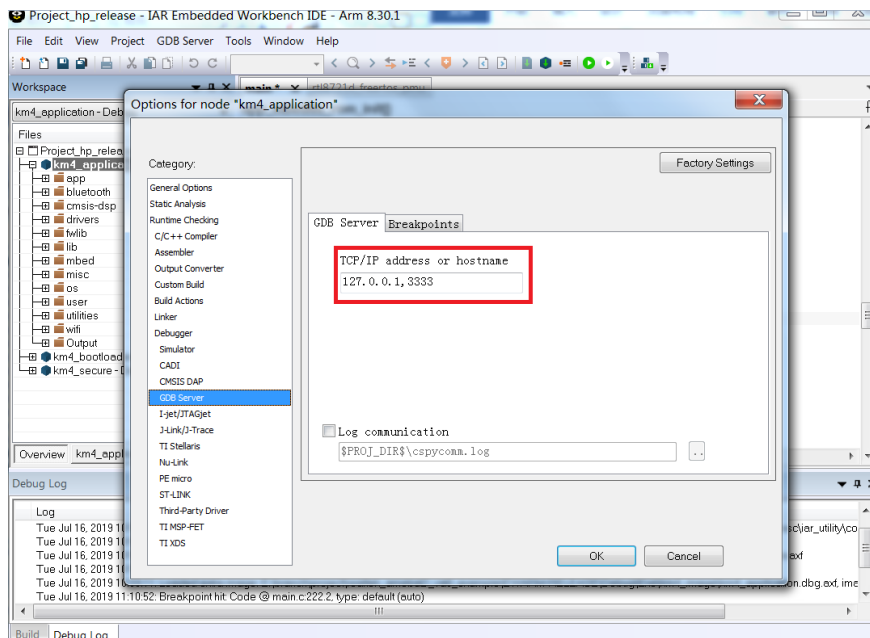


Fig 6-16 RLX Probe interface setup

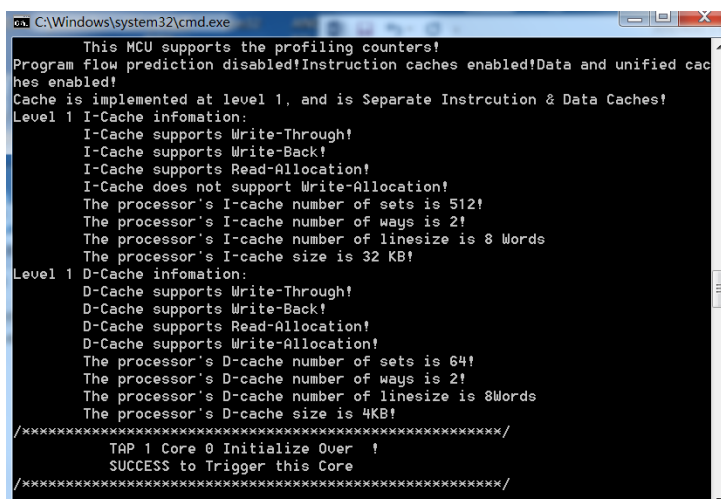


Fig 6-17 RLX Probe window

- (4) Click **Project > Download > Download active application**, image downloading starts. When downloading, Ameba-D prints the log, as Fig 6-18 shows. You can check the log to see if download is successful.

```

[FlashInit] image_size:f48, link_address:8000000, flags:0
[FlashErase] block_start:0, block_size:1000
[FlashWrite] block_start:0, offset_into_block:0, count:f48
[FlashInit] image_size:82000, link_address:8006000, flags:0
[FlashErase] block_start:6000, block_size:1000
[FlashErase] block_start:7000, block_size:1000
[FlashErase] block_start:8000, block_size:1000
[FlashErase] block_start:9000, block_size:1000
[FlashErase] block_start:a000, block_size:1000
[FlashErase] block_start:b000, block_size:1000
[FlashErase] block_start:c000, block_size:1000
[FlashErase] block_start:d000, block_size:1000
[FlashErase] block_start:e000, block_size:1000
[FlashErase] block_start:f000, block_size:1000
[FlashErase] block_start:10000, block_size:1000
[FlashErase] block_start:11000, block_size:1000
[FlashWrite] block_start:6000, offset_into_block:0, count:c000
[FlashErase] block_start:12000, block_size:1000
[FlashErase] block_start:13000, block_size:1000
[FlashErase] block_start:14000, block_size:1000
[FlashErase] block_start:15000, block_size:1000
[FlashErase] block_start:16000, block_size:1000
[FlashErase] block_start:17000, block_size:1000
[FlashErase] block_start:18000, block_size:1000
[FlashErase] block_start:19000, block_size:1000
[FlashErase] block_start:1a000, block_size:1000
[FlashErase] block_start:1b000, block_size:1000
[FlashErase] block_start:1c000, block_size:1000
[FlashErase] block_start:1d000, block_size:1000
[FlashWrite] block_start:12000, offset_into_block:0, count:c000
[FlashErase] block_start:1e000, block_size:1000
[FlashErase] block_start:1f000, block_size:1000
[FlashErase] block_start:20000, block_size:1000
[FlashErase] block_start:21000, block_size:1000
[FlashErase] block_start:22000, block_size:1000
[FlashErase] block_start:23000, block_size:1000
[FlashErase] block_start:24000, block_size:1000

```

Fig 6-18 Downloading log

- (5) You can also erase all parts of the Flash memory if necessary.

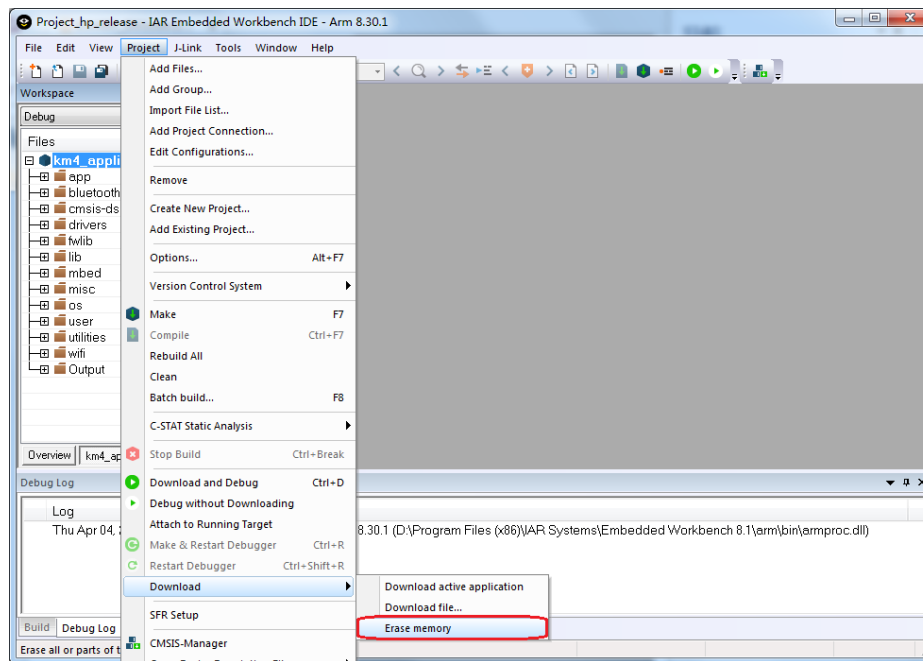


Fig 6-19 Erasing Flash memory

6.3.4 IAR Debug

You can debug or trace KM0 and KM4 system individually with J-Link or RLX Probe SWD.

Note: Considering KM4 is power-on by KM0, you should make sure that KM0 has already boot up before debug KM4. For KM0, there is no such requirement because KM0 is power-on immediately after reset.

6.3.4.1 J-Link Debug

Follow the steps to debug and trace code of target project with IAR by J-Link:

- (1) Set the target project as active project and verify the debugger configurations as step (1) and (2).
- (2) Click **Project > Download and Debug** or **Project > Debug without Downloading**.
 - Download and Debug: downloads the application and debug the project object file. If necessary, a make will be performed before download to ensure the project is up to date.
 - Debug without Downloading: debug the project object file
- (3) When starting IAR C-SPY to debug, it will firstly reset the target CPU and run to the main function, as Fig 6-20 shows.
- (4) Toggles a breakpoint at the statement or instruction that contains or is located near the cursor in the source window. The “Toggle Breakpoint” button is on the debug toolbar, as Fig 6-21 shows.

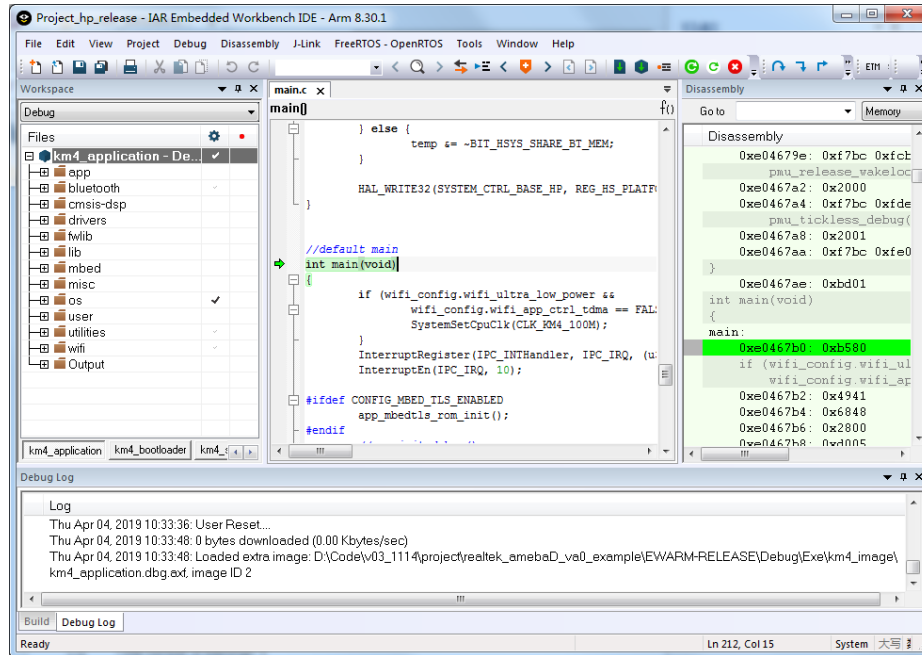


Fig 6-20 Running to main() when debug

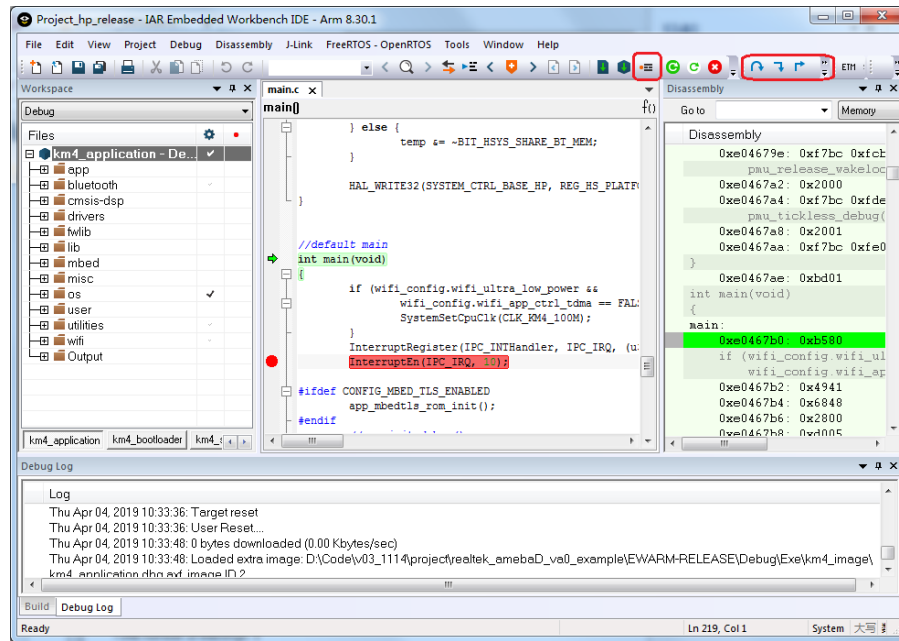


Fig 6-21 Toggle breakpoint

- (5) You can trace code step by step with “Step Into” or “Go” until triggering a breakpoint. These function buttons are available on toolbar.

6.3.4.2 RLX Probe Debug

Follow the steps to debug and trace code of target project with IAR by RLX Probe:

- (1) Set the target project as active project and verify the debugger configurations as step (1) and step (3).
- (2) Click **Project > Attach to Running Target**.
- (3) Set the target address.

When starting debug with RLX Probe, it will firstly reset the target CPU. If you want to run to a target address at first, you can set the PC address in **project\realtek_amebaD_va0_example\EWARM-RELEASE\probe\cm4\rlx_probe0.cfg**. The method is that uncomment “ew .pc = 0xxxxxxx” and change the address to the appointed value, as Fig 6-22 and Fig 6-23 shows.

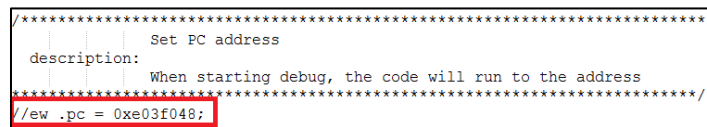


Fig 6-22 Setting the target address

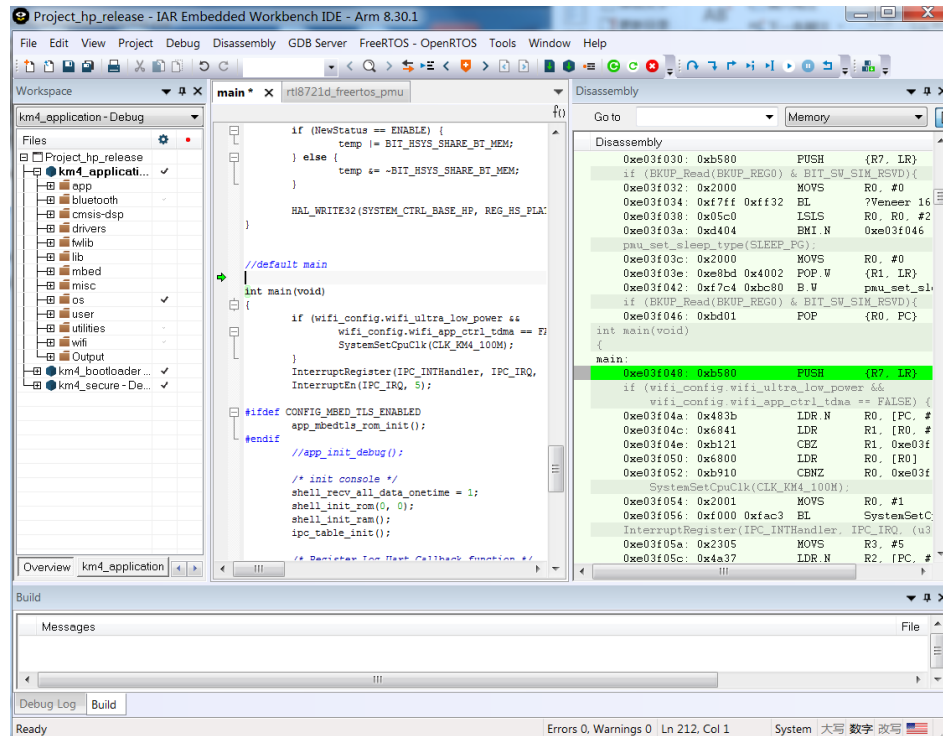


Fig 6-23 Running to the target address when debug

- (4) Toggles a breakpoint at the statement or instruction that contains or is located near the cursor in the source window. The “Toggle Breakpoint” button is on the debug toolbar, as Fig 6-21 shows.
- (5) You can trace code step by step with “Step Into”, or “Go” until triggering a breakpoint. These function buttons are available on toolbar.

6.3.4.3 IAR Debug or Download Error

Because Ameba-D has two CPU cores, and a post-build script will be run after make, sometimes the debug or download thread cannot get the correct AXF file for debug or download, the error like Fig 6-24 happens.

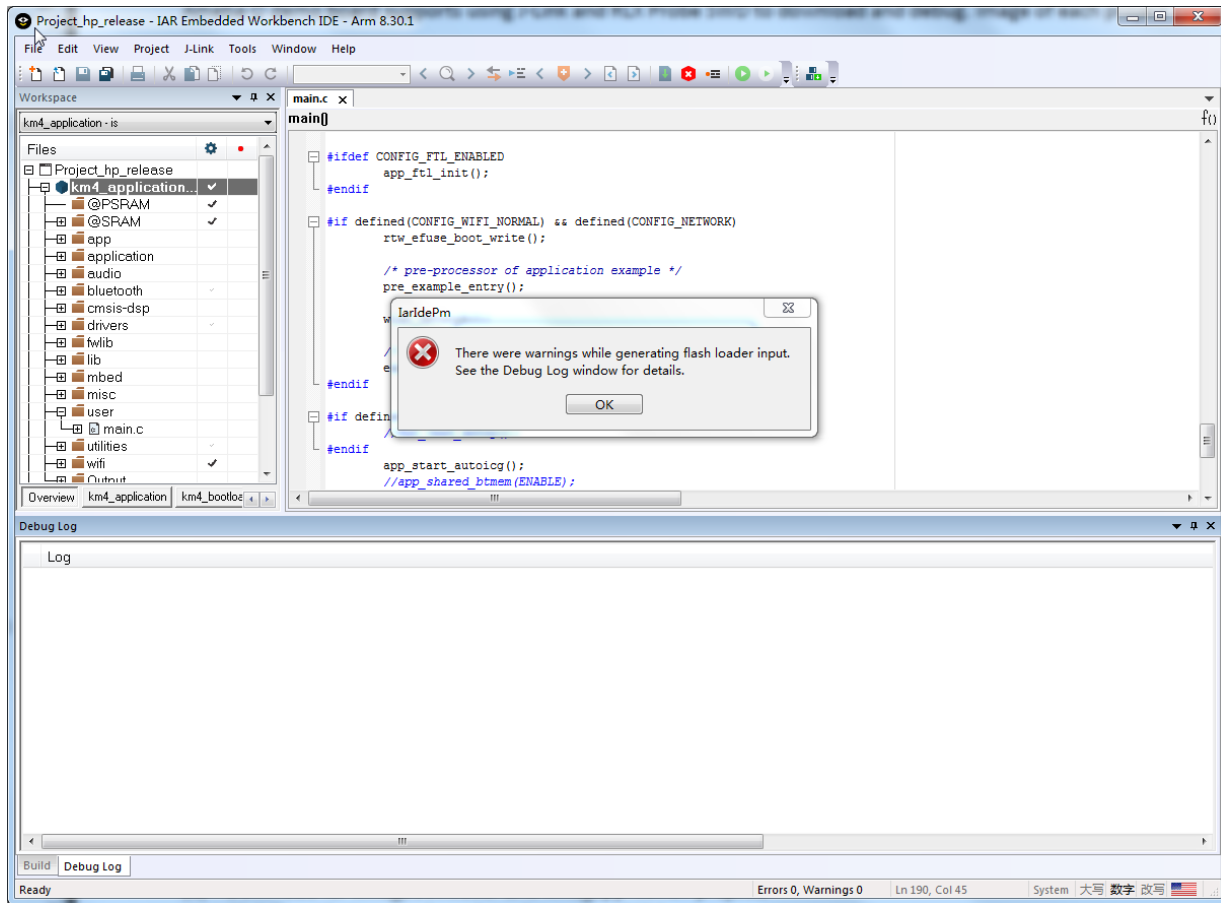


Fig 6-24 IAR debug or download error

To avoid this error, you should build manually before debug or download, and disable auto-build from **Tools > Options > Project > Make before debugging**, as Fig 6-25 shows.

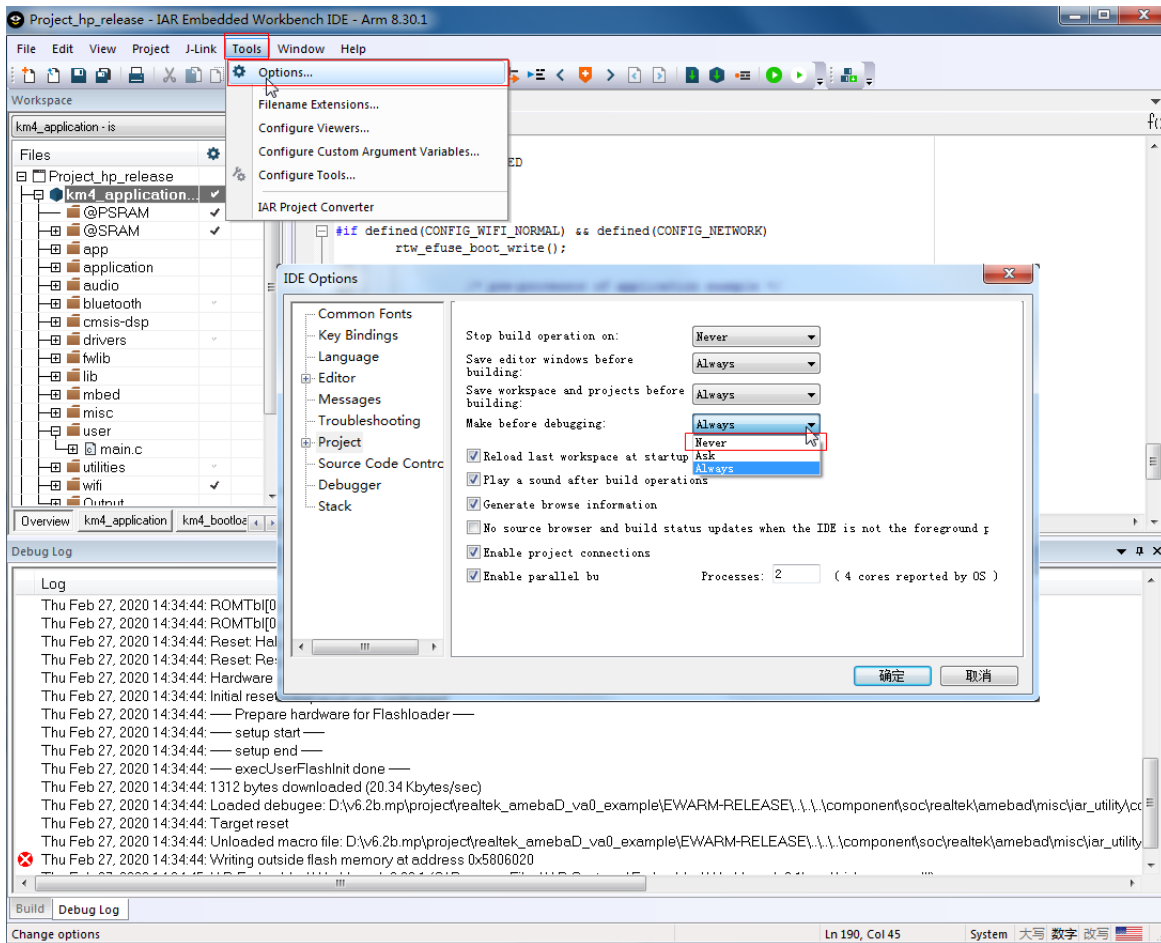


Fig 6-25 IDE options

6.3.5 IAR Memory Configuration

6.3.5.1 Configuring Memory from IAR IDE

In order to allow users to manage memory flexibly, there are some configurations to put some code into certain memory region. In IAR workspace, there are “@PSRAM” and “@SRAM” group. The code in “@PSRAM” group would be linked and loaded into PSRAM, and the code in “@SRAM” group would be linked and loaded into SRAM. The rest of code will be placed on Flash and execute in place.

Note: Considering only SRAM and PSRAM contains secure regions, the code in km4_secure project should be placed either in “@PSRAM” or in “@SRAM”. It can’t be placed outside these two groups.

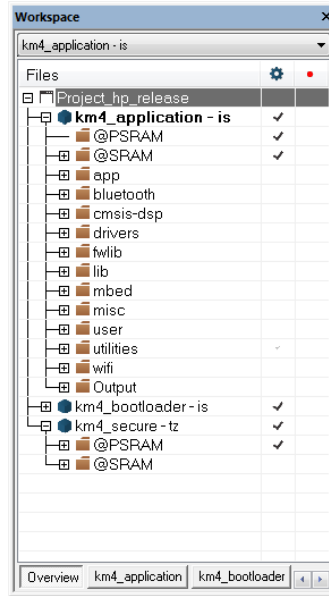


Fig 6-26 Memory location configuration

6.3.5.2 Configuring Memory from ICF File

IAR uses ICF (IAR Configuration File) to configure memory allocation, so users can configure memory allocation by ICF file.

ICF file of Ameba-D location:

- “project\realtek_amebaD_va0_example\EWARM-RELEASE\rtl8721dhp_image2_is.icf” for ignore secure project
- “project\realtek_amebaD_va0_example\EWARM-RELEASE\rtl8721dhp_image2_tz.icf” for TrustZone project

If having a good understand of the format of ICF, users can modify the section location in ICF file.

6.4 How to Build Sample Code?

The example source code is located in **project\realtek_amebaD_va0_example\example_sources**. To build sample code, you should copy the “main.c” file in the target example to **project\realtek_amebaD_va0_example\src\src_hp** and replace the original one.

For example, you can copy “main.c” from **project\realtek_amebaD_va0_example\example_sources\I2C\mbed\i2c_int_mode\src** to use i2c_int_mode example code, as Fig 6-27 shows.

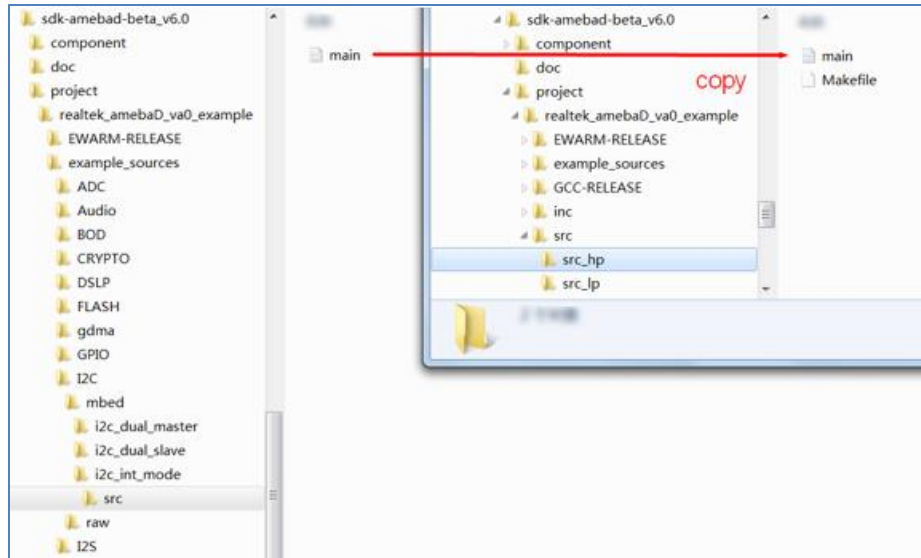
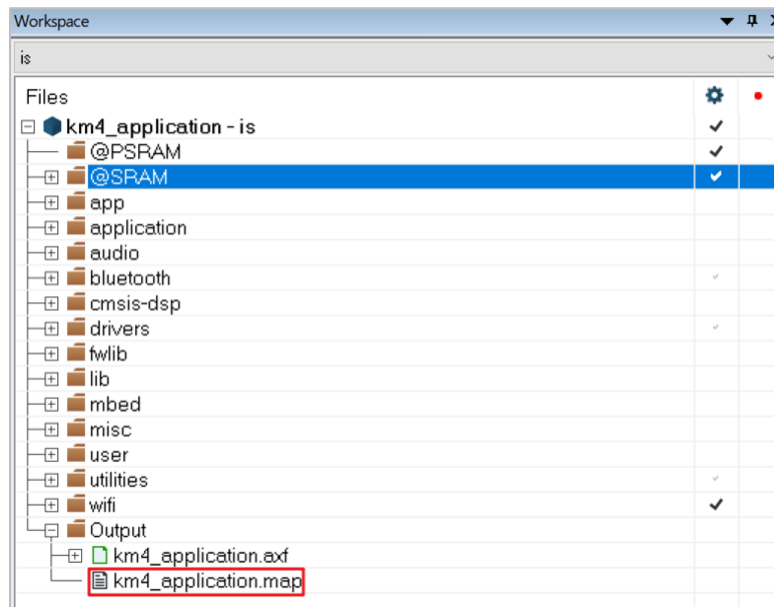


Fig 6-27 Building sample code

6.5 Used Memory Size Calculation

This section explains how to calculate used memory size by users in IAR project, whose version is IAR 8.30.1 or higher. You can refer to “km4_application.map” to observe them after project build. This file can be found in the following folders:

- Project folder: project\realtek_amebaD_va0_example\EWARM-RELEASE\Debug\List\km4_application
- IAR GUI's folder: Output



6.5.1 Memory Section

- A7 (PSRAM): This section is read-write data in PSRAM (0x2000000 to 0x23FFFFFF).
- BTTRACE: This section is reserved for BTTRACE.
- A5 (XIP): This section is read-only data in XIP.

- A4 (SRAM): This section is read-write data in SRAM.
- P1 (SRAM): This is BSS section in SRAM.

6.5.2 Memory Size

6.5.2.1 Memory Size in SRAM

There are two sections resident in SRAM, which are A4 and P1. As we can see from the map file, for standard SDK, these two sections use almost all of the memory space from SRAM (476KB).

- A4 has size 0x17ad7.

"A4":		0x17ad7	
IMAGE2	0x1000'5000	0x17ad7	<Block>
.ram image2.entry	0x1000'5000	0x20	<Block>
.image2.entry.data	rw data 0x1000'5000	0xc	rt18721dhp app start.o [1]

- P1 has size 0x5dfc8.

"P1":		0x5dfc8	
.ram heap.data	0x1001'cae0	0x50000	<Block>
.ram image2.bfsram.data			
	0x1001'cae0	0x50000	<Block>
.bfsram.data	uninit 0x1001'cae0	0x50000	freertos heap5 config.o [1]

So totally 0x17ad7 + 0x5dfc8 = 470K bytes memory in SRAM is used.

The total SRAM space is defined in project\realtek_amebaD_va0_example\EWARM-RELEASE\rt18721d_memory_layout_is.icf.

```
define symbol __ICFEDIT_region_HS_BD_RAM_NS_start__ = 0x10005000;
define symbol __ICFEDIT_region_HS_BD_RAM_NS_end__   = 0x1007C000-1;
```

For this case, the total size of SRAM is 0x1007C000 – 0x10005000 = 0x77000 = 476K bytes, there is still 6K (= 476K – 470K) bytes free SRAM space.

6.5.2.2 Memory Size in PSRAM

There is one section in PSRAM called A7, the memory space from PSRAM (4MB).

A7 has size 0x54800.

"A7":		0x54800	
IMG2 PSRAM	0x200'0000	0x0	<Block>
.psram ns.bss	0x200'0000	0x54800	<Block>
.psram.bss	0x200'0000	0x54800	<Block>
.psram.bss	uninit 0x200'0000	0x54800	rtw opt skbbuf.o [1]

So totally 0x54800 = 338K bytes memory in PSRAM is used.

6.5.2.3 Memory Size in XIP

XIP can only place text section, so there is only one section called A5.

A5 has size 0x8afe8.

```
"A5":
.xip image2.text          0xe00'0020  0x8afe8  <Block>
.text                    ro code  0xe00'0020    0xfc  app task.o [1]
.text.os msg queue create intern
                           ro code  0xe00'011c    0x40  os msg.o [2]
```

So totally 0x8afe8 = 555K bytes memory in XIP is used.

6.5.2.4 Available Heap Size

When calculating total used memory size, available heap size after WLAN association and BT connection needs to be considered. In the above case, it is 56096 bytes.

File	Edit	Setup	Control	Window	Help
UpperStac	B	5	588	23	
trace_tas	B	3	76	22	
ble_centr	B	1	220	25	
cmd_threa	B	6	750	20	
rtu_inter	B	6	208	19	
rtu_recy_	B	5	743	17	
LOGUART_T	B	5	2004	2	
rtu_xmit_	B	5	216	18	
(MEM) After	do cmd, available heap	56096			

Finally, you can use total SRAM 476K bytes to subtract these sections of memory to obtain totally free global memory and free heap in SRAM.

Formula is as below:

- SRAM free global memory: $476K - "A4" - "P1" = 476 * 1024 - 96983 - 384968 = 5473$ (bytes)
- Available heap: 56096 bytes.
- Totally free SRAM memory and heap: $5473 + 56096 = 61569$ (bytes)
- Totally free PSRAM memory: $4 * 1024 * 1024 - 338 * 1024 (A7) = 3848192$ (bytes)

7 Demo Board

7.1 PCB Layout Overview

The PCB layout of 2D and 3D are shown in Fig 7-1 and Fig 7-2.

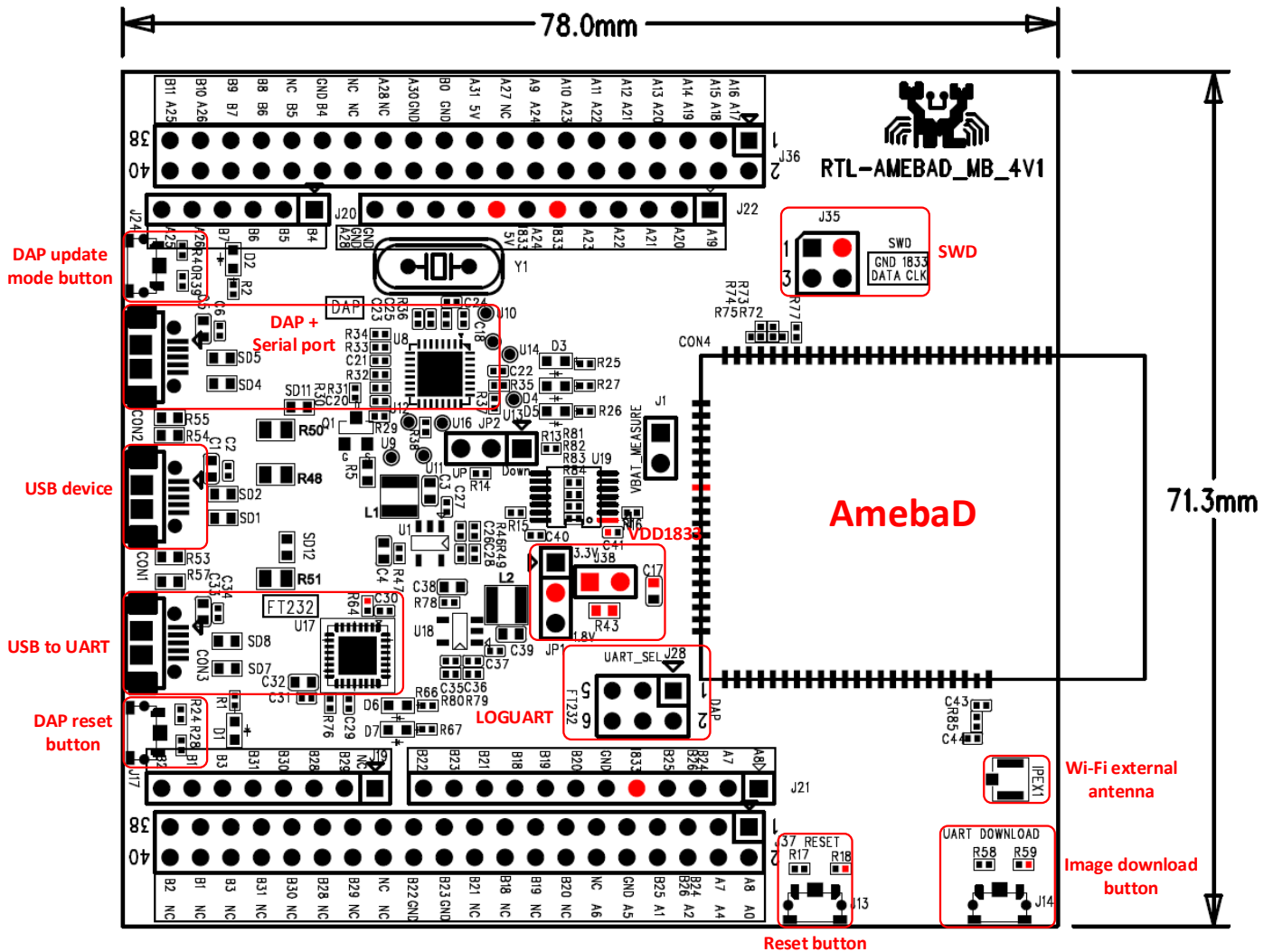


Fig 7-1 Demo board – PCB layout (2D)

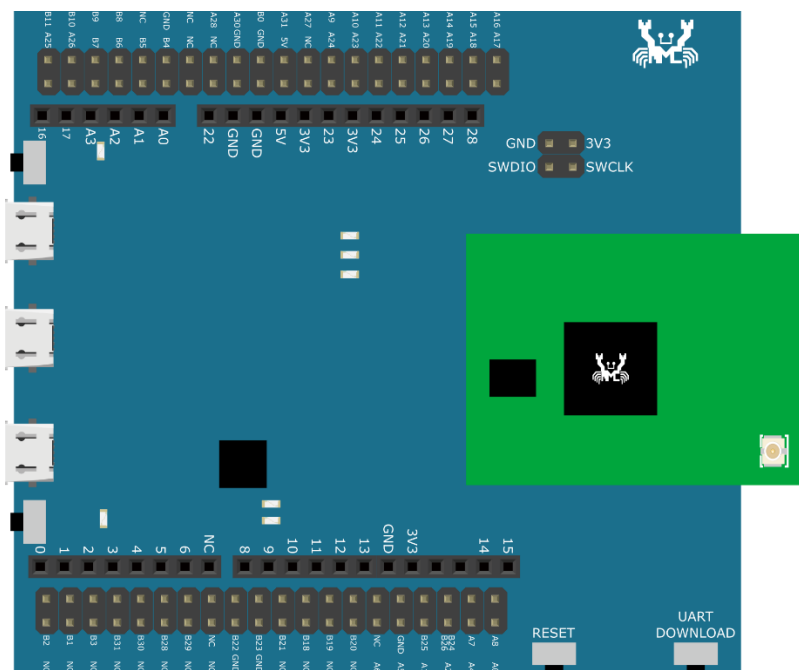


Fig 7-2 Demo board – PCB layout (3D)

7.2 Pin Out

The pin out board is shown in Fig 7-3.

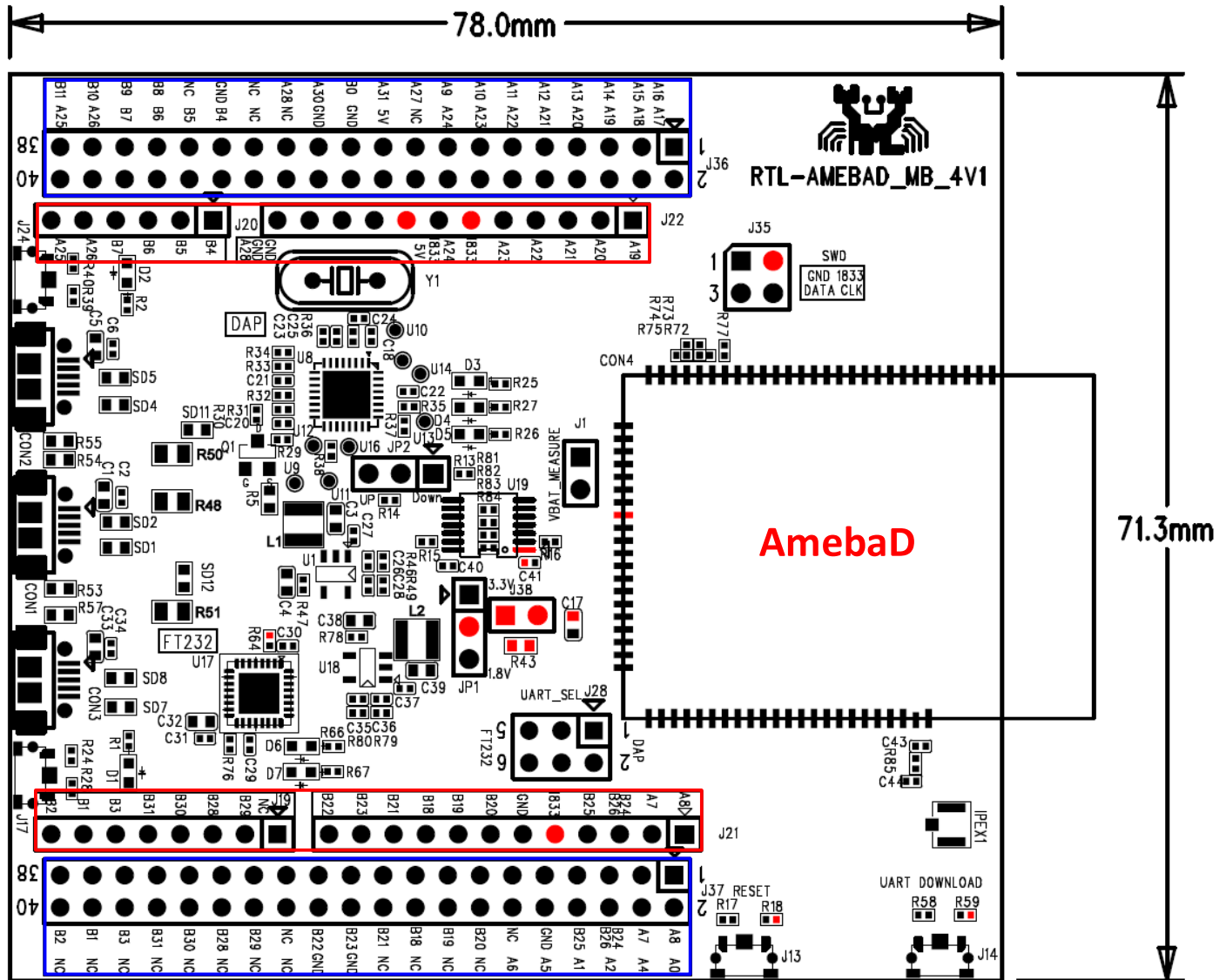


Fig 7-3 Demo board – pin out

There are four rows of pins on the board.

- The pins in the red box are used for Arduino REF.
- The pins in the blue box are all the GPIO pins.

7.3 DC Power Supply

The 3.3V/1.8V power supply board is shown in Fig 7-4.

- Jump JP1 is used to select 3.3V or 1.8V power supply
- Jump J38 is for current test. You can test the current power after taking off the R43.

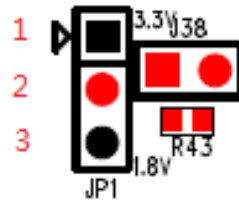


Fig 7-4 Demo board – 3.3V/1.8V power supply

When you select power supply, refer to Table 7-1.

Table 7-1 3.3V/1.8V power supply selection

Power Supply Select	JP1
3.3V	1-2 connected
1.8V	2-3 connected

7.4 USB Interface Configuration

The USB interface configuration board is shown in Fig 7-5 and Fig 7-6.



Fig 7-5 Mother board – USB interface configuration

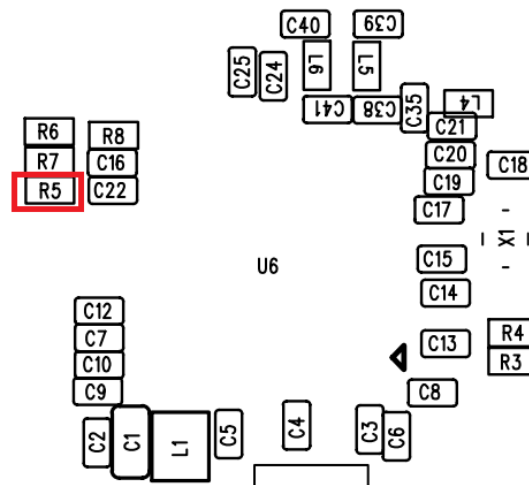


Fig 7-6 Module board – USB interface configuration

For normal GPIO usage by default, R72/R75/R77 on mother board will part on with 0 Ohm resistors, R5 on module board needs to take off. For USB usage, you need to take off R77, part on R73&R74 with 0 Ohm resistors on mother board and part on R5 on module board with a 12K Ohm 1% precision resistor.

7.5 LOGUART

The LOGUART board is shown in Fig 7-7. When you select LOGUART, please refer to Table 7-2.

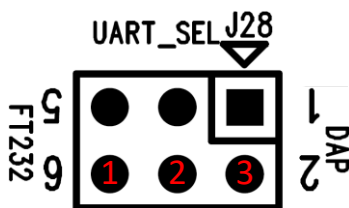


Fig 7-7 Demo board – LOGUART

Table 7-2 LOGUART selection

LOGUART Select	JP1
FT232	1-2 connected
DAP	2-3 connected

7.6 SWD

The SWD board is shown in Fig 7-8.

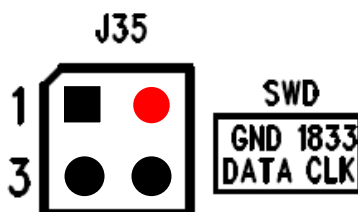


Fig 7-8 Demo board – SWD

Note: For 1V0 board, there is a bug, you should use CLK as DATA, and use DATA as CLK.

7.7 VBAT ADC

The VBAT ADC board is shown in Fig 7-9. J1 is used to test VBAT ADC.



Fig 7-9 Demo board – VBAT ADC

8 ImageTool

8.1 Introduction

This chapter introduces how to use ImageTool to encrypt, generate and download images. As show in Fig 8-1, ImageTool has four tabpages.

- Download: used as image download server to transmit images to Ameba through UART.
- Generate: contact individual images and generate a composite image.
- Encrypt: encrypt images which are used for firmware protection.
- Security: generate key pair and signature for secure bootloader and save as new image.

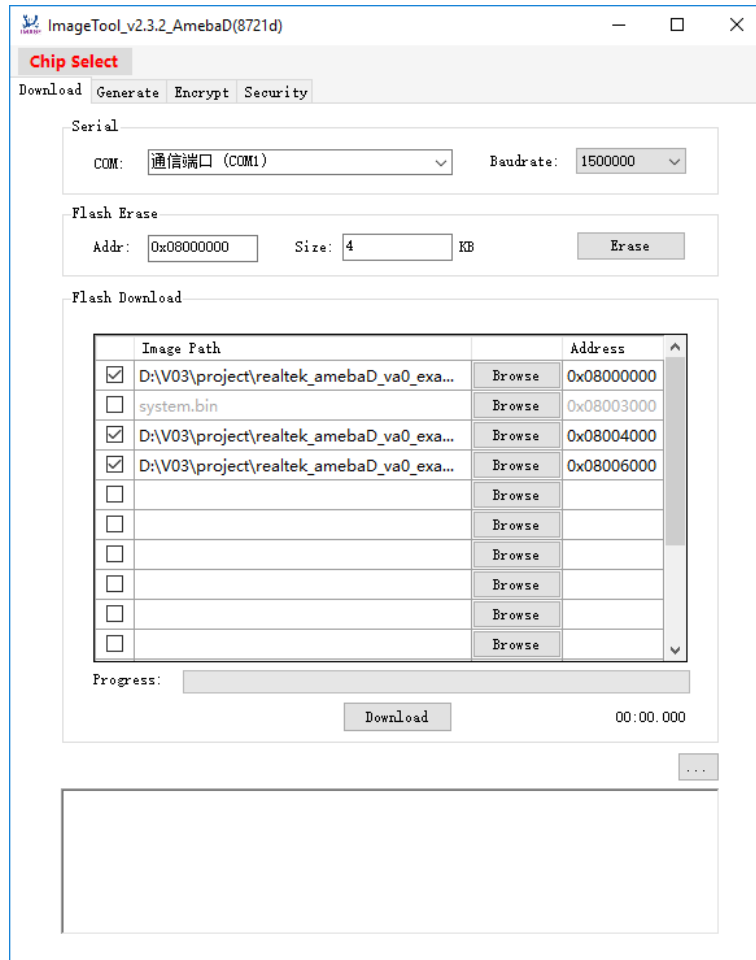


Fig 8-1 ImageTool UI

8.2 Environment Setup

8.2.1 Hardware Setup

The hardware setup is shown in Fig 8-2.

Note: If using external UART to download images, FT232 USB to UART dongle must be used.

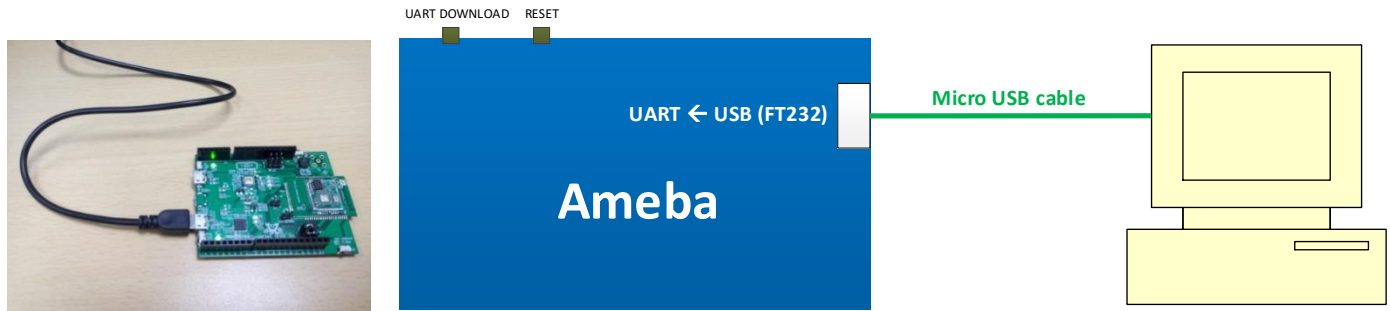


Fig 8-2 Hardware setup

8.2.2 Software Setup

- Environment Requirements: EX. WinXP, Win 7 Above, Microsoft .NET Framework 3.5
- ImageTool.exe Location: `\tools\AmebaD\Image_Tool\ImageTool.exe`

Name	Date modified	Type	Size
ChangeLog.txt	7/29/2019 11:52 AM	Text Document	4 KB
Download.ini	11/4/2019 5:44 PM	Configuration sett...	2 KB
Encrypt.ini	11/4/2019 5:44 PM	Configuration sett...	1 KB
ImageTool.exe	7/29/2019 11:52 AM	Application	282 KB
ImageTool.pdb	7/29/2019 11:52 AM	VisualStudio.pdb....	178 KB
ImageTool.vshost.exe	8/20/2018 1:41 PM	Application	14 KB
ImageTool.vshost.exe.manifest	8/20/2018 1:41 PM	MANIFEST File	1 KB
imgtool_flashloader_amebad.bin	6/6/2019 3:15 PM	BIN File	5 KB
imgtool_flashloader_amebaz.bin	6/6/2019 3:15 PM	BIN File	6 KB
SB.exe	8/20/2018 1:41 PM	Application	189 KB
system.bin	8/6/2019 9:53 AM	BIN File	4 KB
TestListView.dll	8/20/2018 1:41 PM	Application extens...	5 KB
TestListView.pdb	8/20/2018 1:41 PM	VisualStudio.pdb....	14 KB

8.3 Download

8.3.1 Image Download

Assuming that the ImageTool on PC is a server, it sends images files to Ameba (client) through UART. There are two ways to download images to board.

8.3.1.1 Based on Hardware Reset

The way based on hardware reset is a manual method to download images, and it is the primary and recommended method.

- (1) Enter into UART_DOWNLOAD mode.
 - a) Push the **UART DOWNLOAD** button and keep it pressed.
 - b) Re-power on the board or press the **Reset** button.
 - c) Release the **UART DOWNLOAD** button.

Now, Ameba board gets into UART_DOWNLOAD mode and is ready to receive data.
- (2) Click **Chip Select** (in red) on UI and select chip (AmebaD or AmebaZ).
- (3) Select the corresponding serial port and transmission baud rate. The default baud rate is 1.5Mbps (recommended).
- (4) Click the **Browse** button to select the images (km0_boot_all.bin/km4_boot_all.bin/km0_km4_image2.bin) to be programmed and input addresses.
 - The image path is located in `{path}\project\realtek_amebaD_va0_example\GCC-RELEASE\project_hp\asdk\image` and `{path}\project\realtek_amebaD_va0_example\GCC-RELEASE\project_hp\asdk\image`, where `{path}` is the location of the project on your own computer.

- The default target address is the SDK default image address, you can use it directly.
- (5) Click **Download** button to start. The progress bar will show the transmit progress of each image. You can also get the message of operation successfully or errors from the log window.

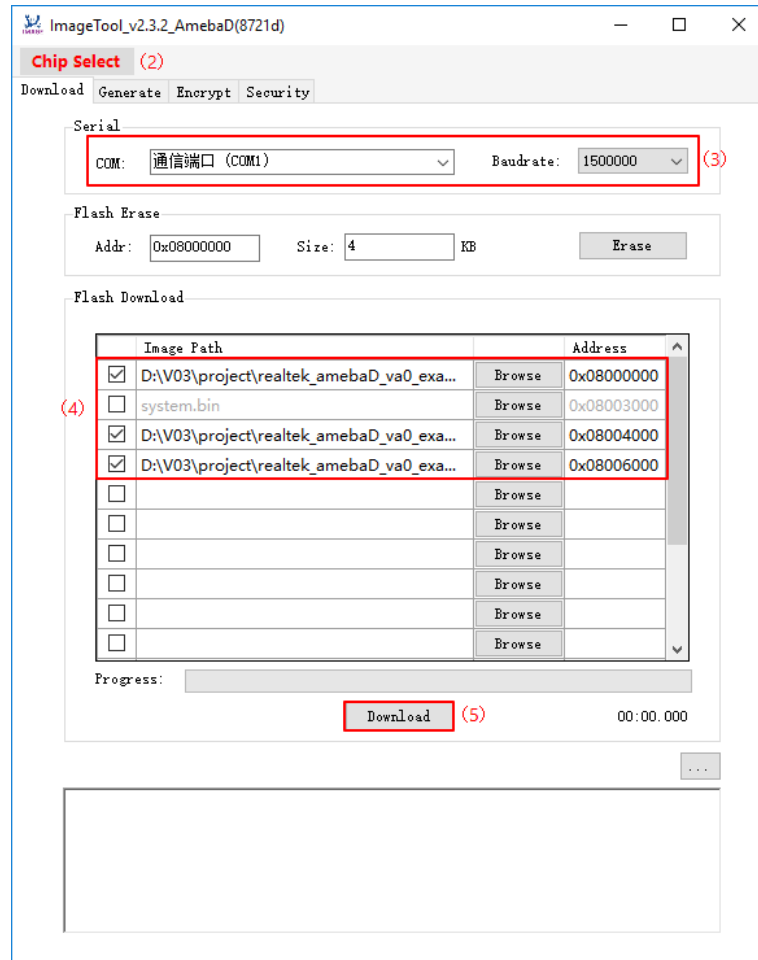


Fig 8-3 ImageTool 'Download' tabpage setting

8.3.1.2 Based on Command

The way based on command is the auto-download and alternate method to download images. We suggest you adopt the method "Based on Hardware Reset" because the auto-download method is dependent on the command in SDK.

Steps: Execute step (2) ~ (5) mentioned in section "Based on Hardware Reset".

After that, Ameba board will enter into UART_DOWNLOAD mode, and automatically response to the command "reboot uartburn" if this command is not removed from SDK and KM4 is running normally.

Note: The command "reboot uartburn" in SDK must not be removed anyhow if you want to download images in this way. Otherwise, download images according to the steps listed in section "Based on Hardware Reset".

8.3.2 Flash Erase

Steps to erase Flash are as follows:

- (1) Ameba enters into UART_DOWNLOAD mode as introduced in section 8.3.1.

- (2) Select the corresponding serial port and baud rate.
- (3) Input erase start address which has to be 4-byte aligned.
- (4) Input erase size which would be cast to a multiple of 4KB.
- (5) Click **Erase** button, and erase operation begins. You would get the operation result from the log window.

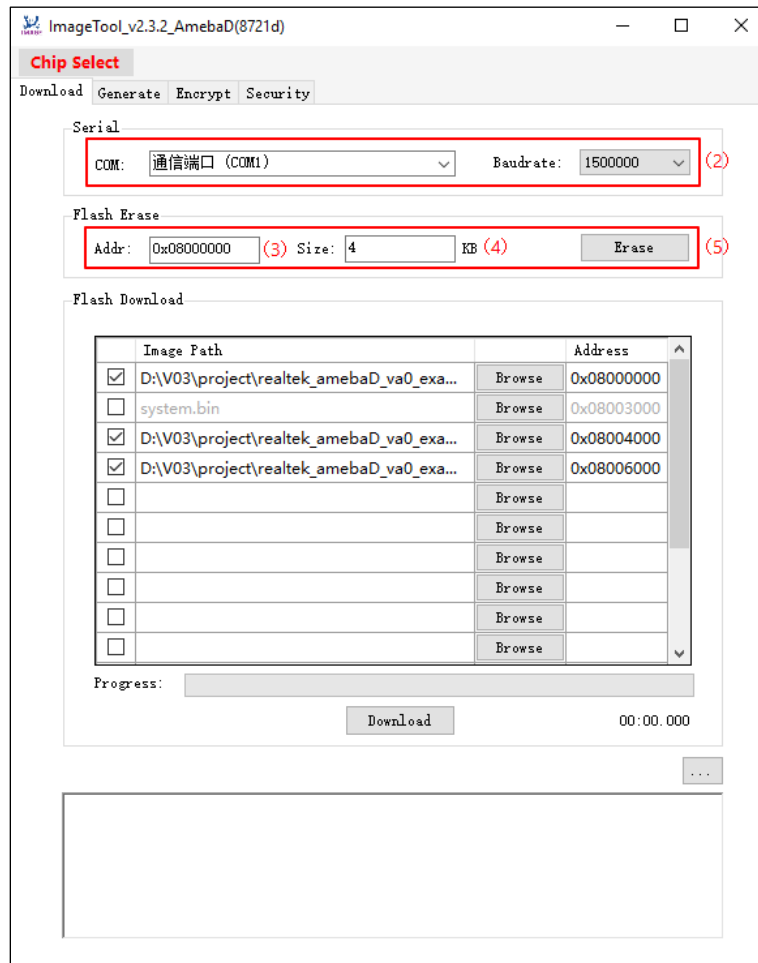


Fig 8-4 Flash erase operation

Note: Users don't need to erase Flash manually before download images into Flash. Image Download operation would erase Flash automatically according to the image and address to be programmed.

8.3.3 Flash Status Operation

After some abnormal operations such as power lost when Flash is programming, the Flash would enter into Write Protection (WP) to protect some area against program and erase.

To release Flash from WP state, the BPs in status register should be cleared. ImageTool provides the function of setting and getting Flash status register value. To use it, the following steps are necessary.

- (1) Check the Read/Write Status Register Command and Sequence according to the Flash datasheet. Notice that different Flash IC would have different sequence.
- (2) Get Ameba into UART_DOWNLOAD mode as introduced in section 8.3.1.
- (3) Select the corresponding serial port and baud rate.
- (4) Click **Advanced Settings** button in Fig 8-5, and the **Advanced Settings** window shown in Fig 8-6 popups.

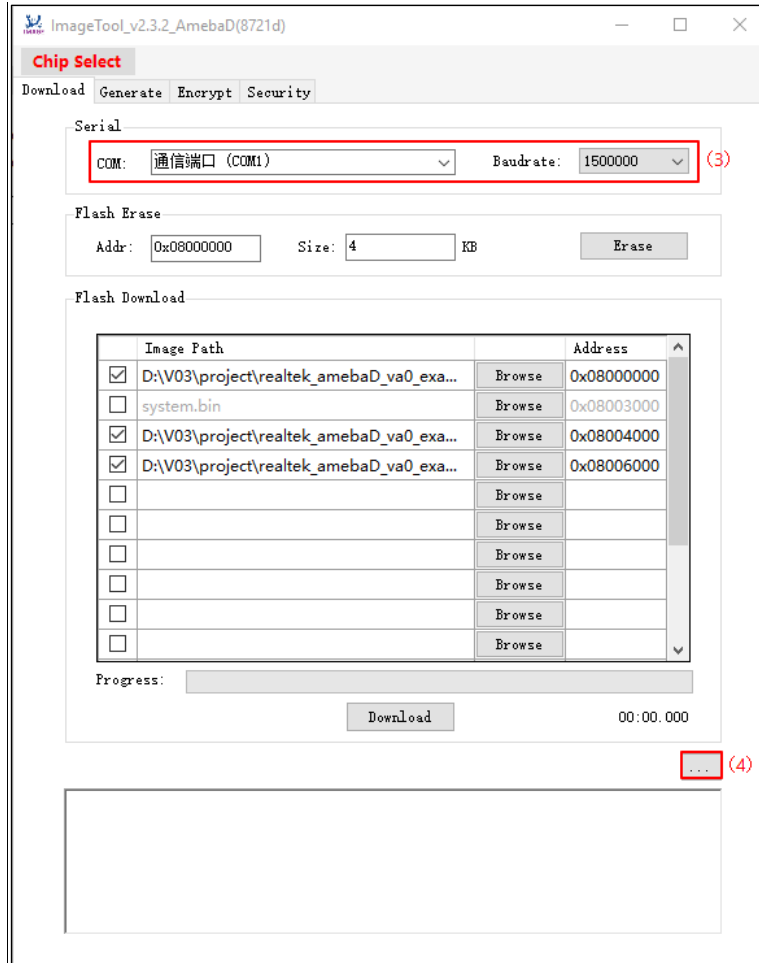


Fig 8-5 'Advanced Settings' window

- (5) Select Read/Write Command and Length according to Flash datasheet. If writing status to Flash, remember to input value.
- (6) Click **Read** button to read status register value, or **Write** button to write status value to Flash.

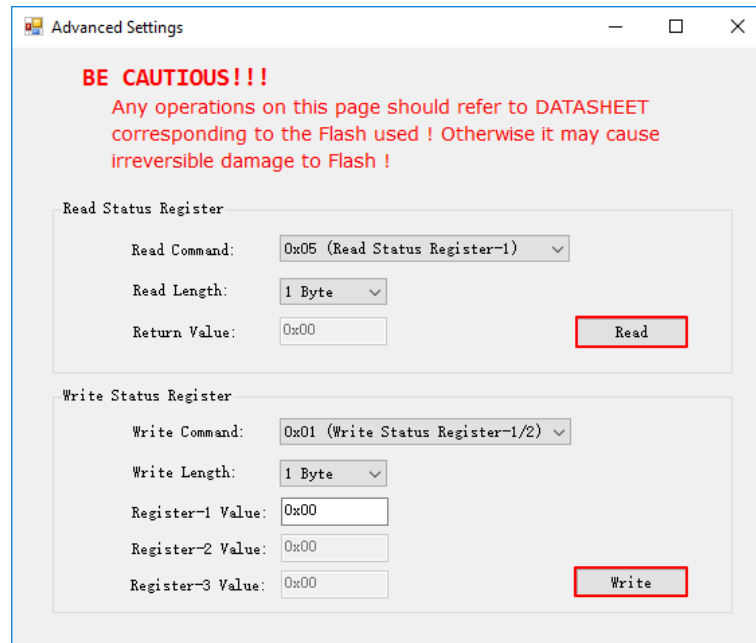


Fig 8-6 Flash status operation

In case of MXIC Flash chip, the status register is shown in Table 8-1.

- To clear BP bits, read status register configuration is as below:
 - Read Command: 0x05
 - Read Length: 1 Byte
- Write status register configuration is as below:
 - Write Command: 0x01
 - Write Length: 1 Byte
 - Register-1 Value: 0x40

Table 8-1 MXIC Flash status register

Bit	Name	Description	Volatile Bit
7	SRWD	Status register write protect. ● 1: Status register write disable ● 0: Status register write enable	No
6	QE	Quad enable. ● 1: Quad enable ● 0: Not Quad enable	No
5	BP3	Level of protected block.	No
4	BP2	Level of protected block.	No
3	BP1	Level of protected block.	No
2	BP0	Level of protected block.	No
1	WEL	Write enable latch. ● 1: Write enable ● 0: Not write enable	Yes
0	WIP	Write in progress bit. ● 1: In write operation ● 0: Not in write operation	Yes

Note: For other Flash IC, please refer to corresponding datasheet. Otherwise, wrong operation would cause irreversible damage to Flash.

8.4 Generate

The 'Generate' tabpage has two functions:

- Merge individual images and generate a composite image named **Image_All.bin**
- Generate Cloud OTA image named **OTA_All.bin**

8.4.1 Merge Images

Steps to generate a composite image are as follows:

- (1) Select **Image_All** as Generate Target type (in red).
- (2) Click **Browse** button to select images to be contacted and input corresponding relative address. The **Memory Layout** bar will show the relative positions of the selected images. If the contiguous images overlap, the overlapped area is in red color for warning.
- (3) Click **Generate** button. Specify the output file name and path yourself in the popup 'Save As' window, the final image (**Image_All.bin**) is generated.

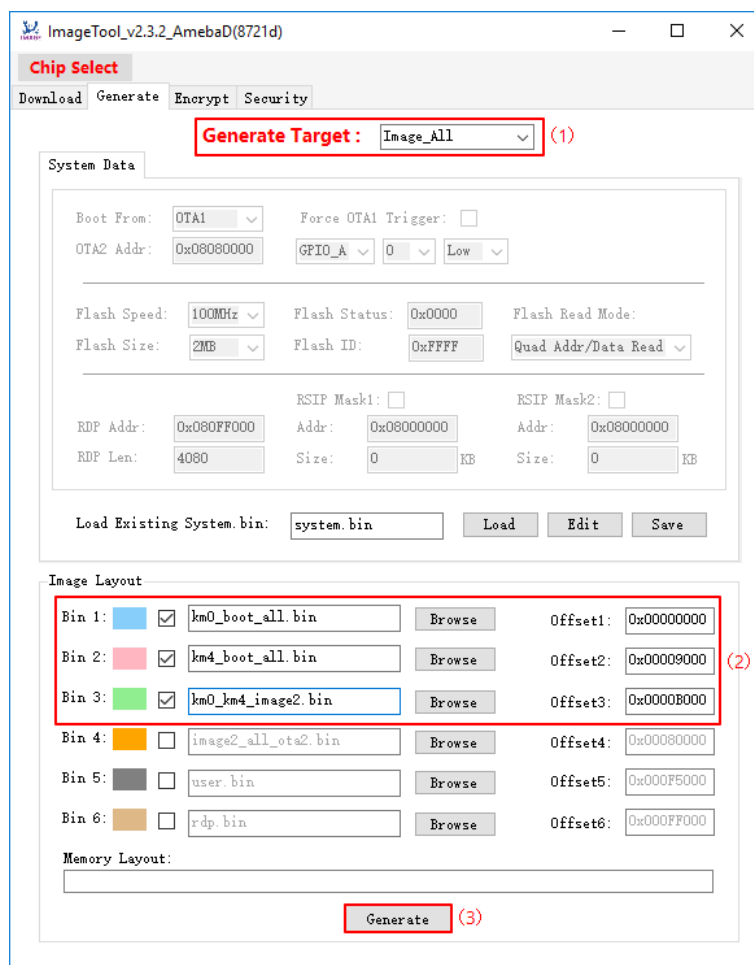


Fig 8-7 Image_All.bin generation

8.4.2 Generate Cloud OTA Image

Steps to generate a Cloud OTA image are as follows:

- (1) Select **OTA_All** as Generate Target type (in red).
- (2) Input Image Version, the default value is 0xFFFFFFFF.

- (3) Click **Browse** button to select images. The address can be ignored. The **Memory Layout** bar will show the relative positions of the two images. If they overlap, the overlapped area is in red color for warning.
- (4) Click **Generate** button to specify the name and path of the output file. After the operation is done, the cloud image (**OTA_All.bin**) is generated.

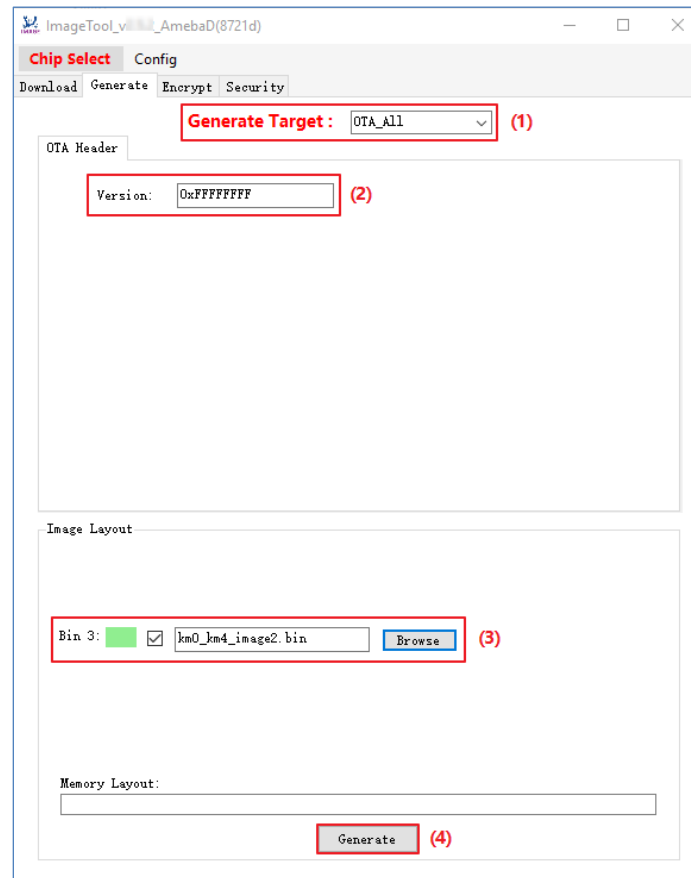


Fig 8-8 OTA_All generation

9 Memory Layout

9.1 Flash Program Layout

The default Flash layout used in SDK is illustrated in Fig 9-1 and Table 9-1. Because bootloader will be called by ROM code, the address of bootloader cannot be changed. Other addresses except bootloader can all be changed by users, but before changing these addresses, users must make sure that they are familiar with Ameba-D system and SDK.

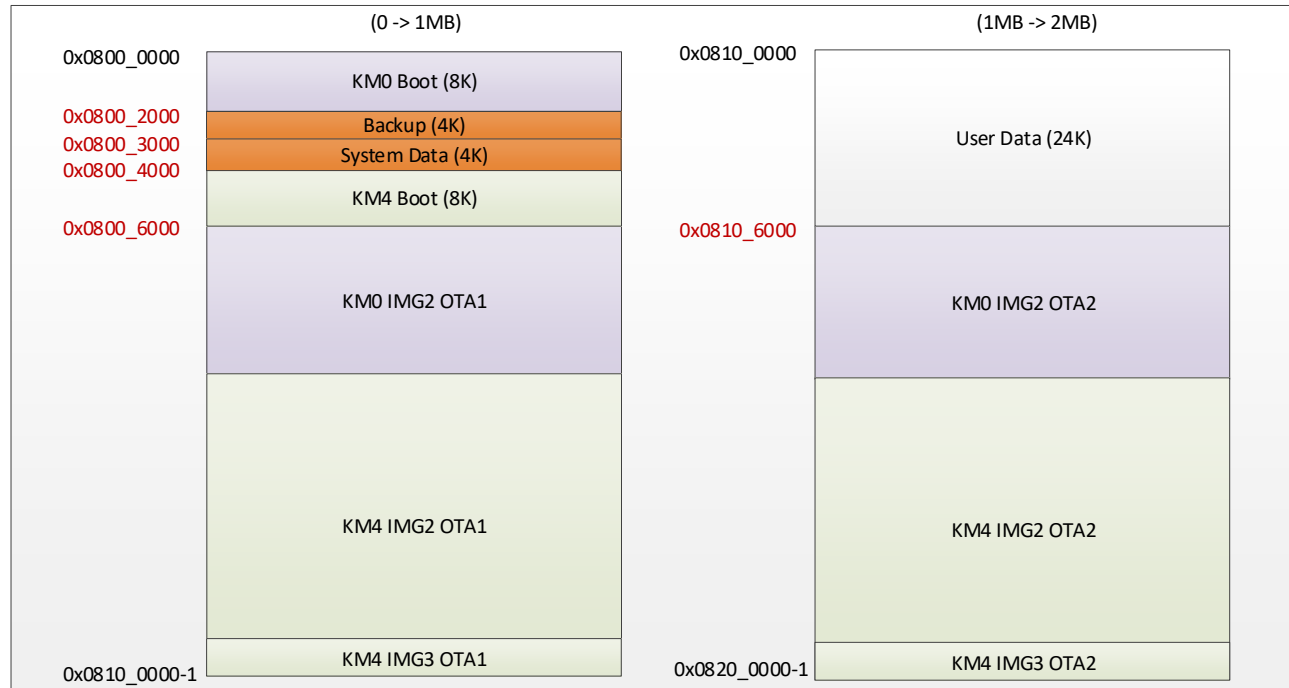


Fig 9-1 1M/2M Flash program layout

Table 9-1 1M/2M Flash program layout

Items	Start Address	Limited Address	Size	Description	Mandatory
KM0 Boot	0x0800_0000	0x0800_2000-1	8K	KM0 bootloader code/data	Yes
Backup	0x0800_2000	0x0800_3000-1	4K	Backup Flash area, reserved for system use. When program system data, this area will be used as backup area.	Yes
System Data	0x0800_3000	0x0800_4000-1	4K	System Data, user should program this area carefully.	Yes
KM4 Boot	0x0800_4000	0x0800_6000-1	8K	KM4 bootloader code/data	Yes
KM0 IMG2 OTA1	0x0800_6000	0x0810_0000-1	1000K	KM0 image2 OTA1 code/data	No
KM4 IMG2 OTA1				KM4 image2 OTA1 code/data	No
KM4 IMG3 OTA1				RDP image OTA1 code/data	No
User Data	0x0810_0000	0x0810_2000-1	8K	Reserved for user data	No
	0x0810_2000	0x0810_5000-1	12K	BT FTL physical map	No
	0x0810_5000	0x0810_6000-1	4K	WIFI Fast Connect profile	No
KM0 IMG2 OTA2	0x0810_6000	0x0820_0000-1	1000K	KM0 image2 OTA2 code/data	No
KM4 IMG2 OTA2				KM4 image2 OTA2 code/data	No
KM4 IMG3 OTA2				RDP image OTA2 code/data	No

There is an image header for every image; it helps the bootloader to load code or data to the correct destination address, or it's used to realize ECC secure boot.

9.1.1 Image Header

Normal bootloader doesn't include secure boot signature. It's just used for loading code or data to the correct destination, as shown in see Fig 9-2 and Table 9-2.

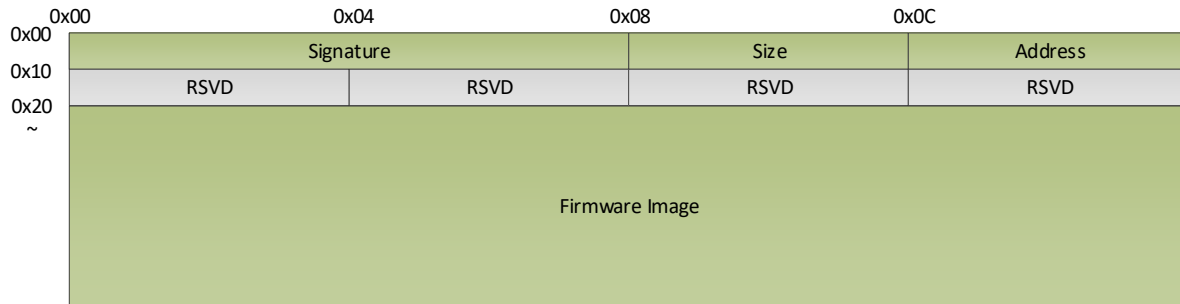


Fig 9-2 Normal image header

Table 9-2 Image header without secure boot

Items	Comment
Signature (8 bytes)	Bootloader Signature for Flash calibration
Size (4 bytes)	The size of Firmware Image
Address (4 bytes)	Code executes start address after boot

Image header with secure includes secure boot signature. It is used to realize ECC secure boot. For more information, refer to AN0410.

9.1.2 System Data (4K)

The System data layout is illustrated in Fig 9-3 and Table 9-3.

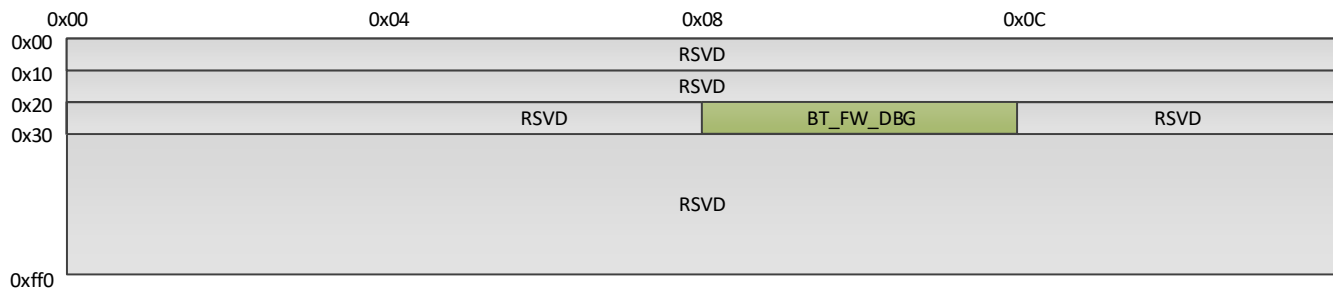


Fig 9-3 System data layout

Table 9-3 System Data Layout

Items	Default	Description
BT_FW_DBG	0xFFFFFFFF	Reserved for FW debug.

9.1.3 User Data

There are totally 24KB for user data. Parts of this region are defined by default SDK.

Table 9-4 User data layout

Items	Range	Description
Reserved	0x0810_0000~0x0810_2000-1	Reserved for user data
BT FTL	0x0810_2000~0x0810_5000-1	Bluetooth FTL physical map. The start address is defined by <code>ftl_phy_page_start_addr</code> variable (<code>rtl8721dhp_intfcfg.c</code>). If Bluetooth is not enabled in your system, this can be used by users.

Wi-Fi Fast Connect Profile	0x0810_5000~0x0810_6000-1	Used to store Wi-Fi fast connect profile. Defined by FAST_RECONNECT_DATA macro (platform_opts.h). If Wi-Fi fast connect function is not enabled in your system, this area can be used by users.
----------------------------	---------------------------	--

Warning: If Flash memory layout is modified and the 0x0800_2000~0x0800_5FFF is covered, it is important to modify the FTL physical map or Wi-Fi fast connect area address.

9.1.4 4M Flash Usage Example

For 4M Flash, an example of Flash layout is illustrated in Fig 9-4. In this example, 2024K Flash size can be used for KM0 & KM2 image, and you can modify the layout according to your actual conditions. Because bootloader will be called by ROM code, the address of bootloader cannot be changed. Other addresses except bootloader can all be changed by user, but before you change these addresses, make sure that you are familiar with Ameba-D system and SDK.

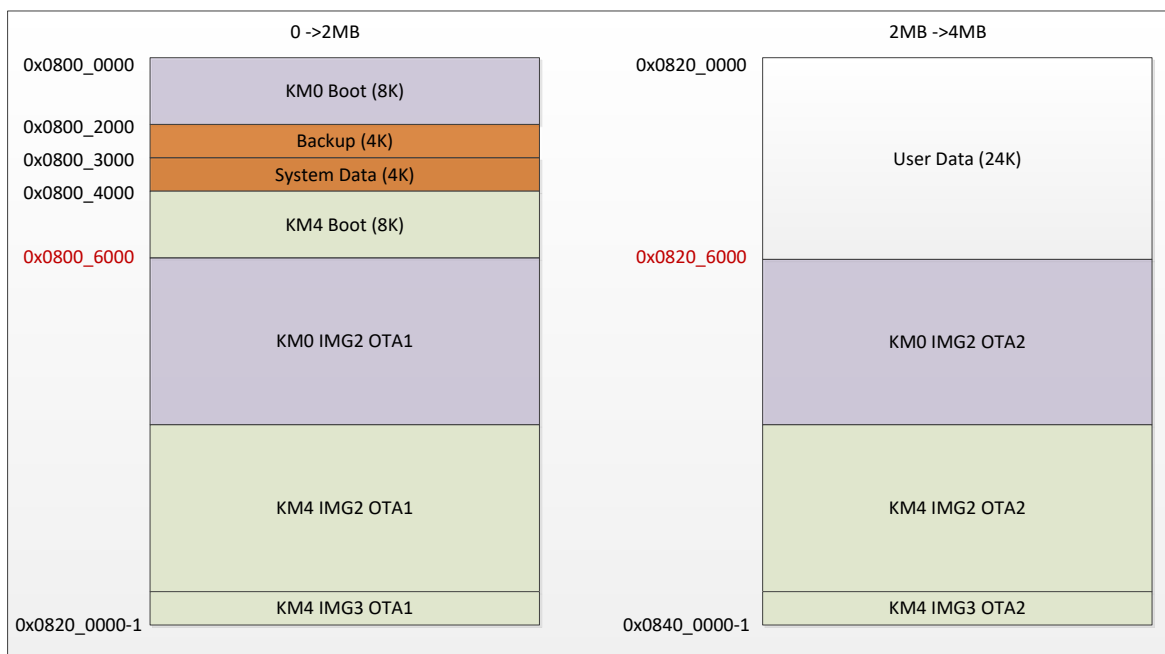


Fig 9-4 4M Flash program layout example

Table 9-5 4M Flash program layout example

Items	Start Address	Limited Address	Size	Description	Mandatory
KM0 Boot	0x0800_0000	0x0800_2000-1	8K	KM0 bootloader code/data	Yes
Backup	0x0800_2000	0x0800_3000-1	4K	Backup Flash area, reserved for system use. When program system data, this area will be used as backup area.	Yes
System Data	0x0800_3000	0x0800_4000-1	4K	System Data, user should program this area carefully.	Yes
KM4 Boot	0x0800_4000	0x0800_6000-1	8K	KM4 bootloader code/data	Yes
KM0 IMG2 OTA1	0x0800_6000	0x0820_0000-1	2024K	KM0 image2 OTA1 code/data	No
KM4 IMG2 OTA1				KM4 image2 OTA1 code/data	No
KM4 IMG3 OTA1				RDP image OTA1 code/data	No
User Data	0x0820_0000	0x0820_2000-1	8K	Reserved for user data	No
	0x0820_2000	0x0820_5000-1	12K	BT FTL physical map	No
	0x0820_5000	0x0820_6000-1	4K	Wi-Fi Fast Connect profile	No
KM0 IMG2 OTA2	0x0820_6000	0x0840_0000-1	2024K	KM0 image2 OTA2 code/data	No

KM4 IMG2 OTA2				KM4 image2 OTA2 code/data	No
KM4 IMG3 OTA2				RDP image OTA2 code/data	No

For 4M Flash, some configurations need to be modified in default SDK.

- OTA start address

The default OTA start address in SDK is set for 2M Flash, thus OTA address in `rtl8721d_bootcfg.h` and `rtl8721d_ota.h` needs to be modified when using 4M Flash.

- Modify **OTA_Region** in `rtl8721d_bootcfg.h`, change OTA2 region start address according to your actual Flash layout. When using 4M Flash program layout example as shown in Fig 9-4, OTA region should be modified as below:

```
/*
 * @brief OTA start address. Because KM0 & KM4 IMG2 are combined, users only need to set the start address
 * of KM0 IMG2.
 */
BOOT_RAM_DATA_SECTION
u32 OTA_Region[2] = {
    0x08006000, /* OTA1 region start address */
    0x08206000, /* OTA2 region start address */
};
```

- Modify OTA address definition in `rtl8721d_ota.h`, redefine **LS_IMG2_OTA2_ADDR** macro according to your actual Flash layout. When using 4M Flash program layout example as shown in Fig 9-4, OTA address should be modified as below:

```
#define LS_IMG2_OTA1_ADDR 0x08006000 /* KM0 OTA1 start address*/
#define LS_IMG2_OTA2_ADDR 0x08206000 /* KM0 OTA2 start address*/
```

- User data Flash address

In default SDK, 24KB user data is defined in Flash address `0x0800_2000~0x0800_5FFF`, which is now used for BT FTL and Wi-Fi fast connect profile. For 4M Flash, if Bluetooth and Wi-Fi fast connect function are enabled in your system, and Flash address `0x0800_2000~0x0800_5FFF` is covered, you need to modify BT FTL address and Wi-Fi fast connect profile address according to your actual Flash layout.

- For BT FTL start address, re-define **ftl_phy_page_start_addr** variable in `rtl8721dhp_intfcfg.c`, note that SPI FLASH address base `0x0800_0000` is subtracted when calculating `ftl_phy_page_start_addr` variable, and `ftl_phy_page` will consume $3*4=12$ KB Flash starting from `ftl_phy_page_start_addr`. When using 4M Flash program layout example as shown in Fig 9-4, BT FTL start address should be modified as below:

```
const u32 ftl_phy_page_start_addr = 0x00202000;
```

- For Wi-Fi fast connect profile address, redefine **FAST_RECONNECT_DATA** macro in `platform_opts.h`, and SPI FLASH address base `0x0800_0000` is subtracted when calculating `FAST_RECONNECT_DATA` macro. Note that Wi-Fi fast connect profile address should not recover the 12KB Flash starting from `ftl_phy_page_start_addr`, as this region is reserved for BT FTL. When using 4M Flash program layout example as shown in Fig 9-4, Wi-Fi fast connect profile address should be set to `0x202000+0x3000=0x205000`.

```
#define FAST_RECONNECT_DATA 0x205000
```

After the above modification, rebuild KM0 project and KM4 project, then download `km0_boot_all.bin`, `km4_boot_all.bin` and `km0_km4_image2.bin`.

9.2 SRAM Layout

9.2.1 KM4 SRAM Layout

The start address of KM4 SRAM is `0x1000_0000`. The size is 512KB. The memory layout is illustrated in Fig 9-5 and Table 9-6.

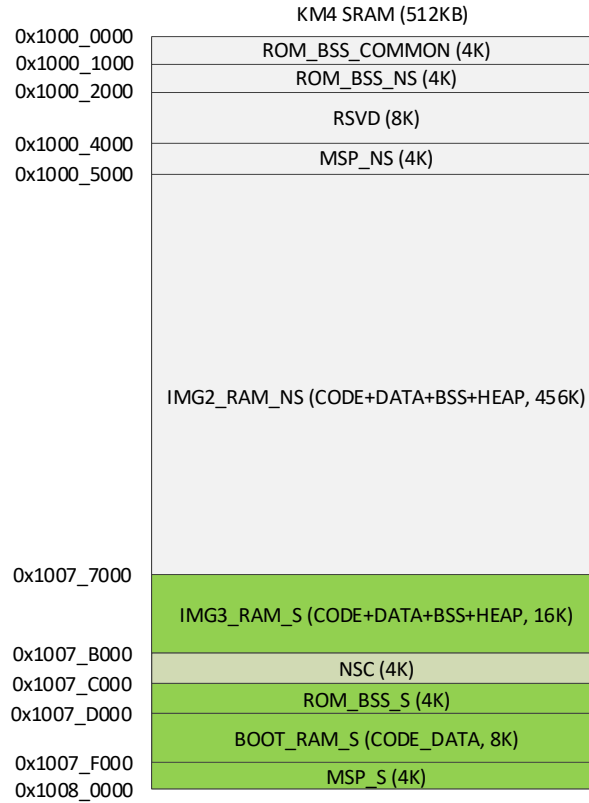


Fig 9-5 KM4 RAM program layout

Table 9-6 KM4 RAM program layout

Items	Start Address	End Address	Size	Description	Mandatory
ROM_BSS_COMMON	0x1000_0000	0x1000_1000-1	4K	ROM Secure and Non-secure common .bss	Yes
ROM_BSS_NS	0x1000_1000	0x1000_2000-1	4K	ROM Non-secure common .bss	Yes
RSVD	0x1000_2000	0x1000_4000-1	8K	Reserved for Non-secure MSP & Non-secure ROM .bss	No
MSP_NS	0x1000_4000	0x1000_5000-1	4K	Non-secure Main Stack Pointer	Yes
IMG2_RAM_NS	0x1000_5000	0x1007_7000-1	456K	Image2 Non-secure RAM includes CODE, DATA, BSS and HEAP	No
IMG3_RAM_S	0x1007_7000	0x1007_B000-1	16K	Image3 Secure RAM includes CODE, DATA, BSS and HEAP	No
NSC	0x1007_B000	0x1007_C000-1	4K	Images Secure function call entries	No
ROM_BSS_S	0x1007_C000	0x1007_D000-1	4K	ROM Secure only .bss	Yes
BOOT_RAM_S	0x1007_D000	0x1007_F000-1	8K	KM4 Secure bootloader, includes CODE and DATA	Yes
MSP_S	0x1007_F000	0x1008_0000-1	4K	Secure Main Stack Pointer	Yes

Note: If RDP is not enabled, the IMG3_RAM_S and NSC are non-secure and would be merged into IMG2_RAM_NS. So the IMG2_RAM_NS region would [0x1000_5000, 0x1007_C000-1].

9.2.2 KM0 SRAM Layout

The start address of KM0 SRAM is 0x0008_0000. The size is 64KB. The memory layout is illustrated in Fig 9-6 and Table 9-7.

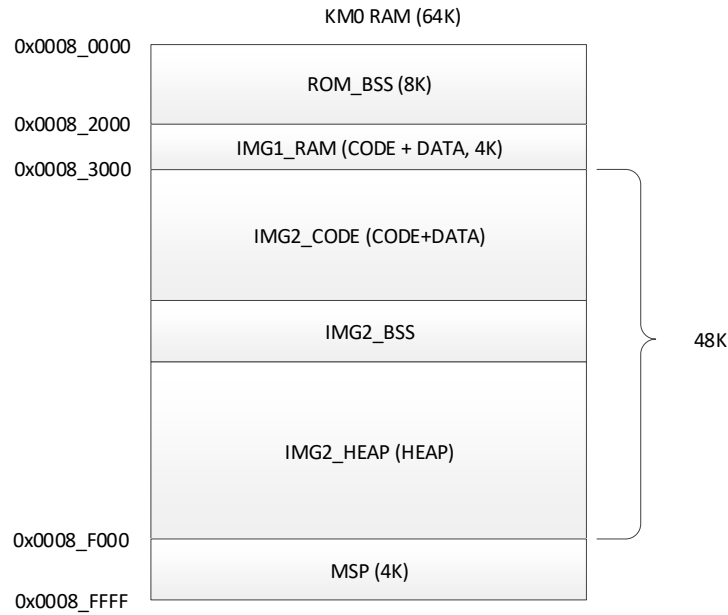


Fig 9-6 KM0 RAM program layout

Table 9-7 KM0 RAM program layout

Items	Start Address	End Address	Size	Description	Mandatory
ROM_BSS	0x0008_0000	0x0008_2000-1	8K	ROM .bss	Yes
IMG1_RAM	0x0008_2000	0x0008_3000-1	4K	KM0 bootloader SRAM, including CODE and DATA	Yes
IMG2_RAM	0x0008_3000	0x0008_F000-1	48K	KM0 image2 SRAM, including CODE, DATA, BSS and HEAP	No
MSP	0x0008_F000	0x0009_0000-1	4K	Main Stack Pointer	Yes

9.3 PSRAM Layout

The start address of KM4 PSRAM is 0x0200_0000. The total size is 4MB. The memory layout is illustrated in Fig 9-7 and Table 9-8.

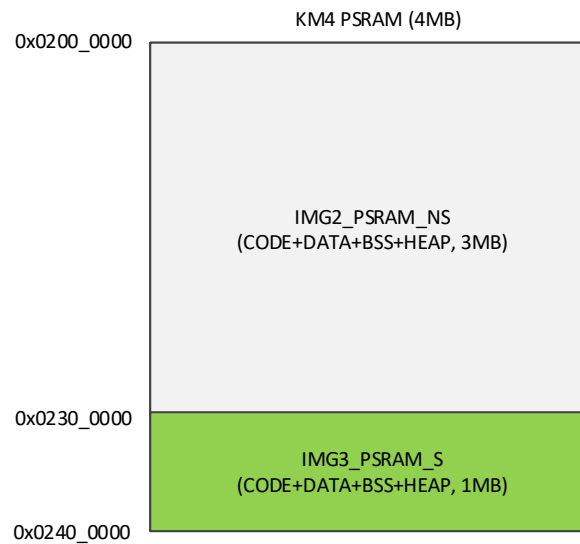


Fig 9-7 KM4 PSRAM program layout

Table 9-8 KM4 PSRAM program layout

Items	Start Address	End Address	Size	Description	Mandatory
IMG2_PSRAM_NS	0x0200_0000	0x0230_0000-1	3MB	Image2 Non-secure PSRAM includes CODE, DATA, BSS and HEAP	No
IMG3_PSRAM_S	0x0230_0000	0x0240_0000-1	1MB	Image3 Secure PSRAM includes CODE, DATA, BSS and HEAP	No

Note: If Trust-Zone is not enabled, the whole PSRAM would be non-secure. So the IMG2_PSRAM_NS region would be [0x0200_0000, 0x0240_0000-1].

9.4 Retention SRAM

Retention SRAM would not be power off even when system enters into deepsleep mode. As a result, it is designed to retain data or code which would be lost in SRAM for some cases, such as deepsleep.

9.4.1 Retention SRAM Layout

The start address of Retention SRAM is 0x000C_0000. The size is 1KB. The memory layout is illustrated in Table 9-9.

Table 9-9 Retention SRAM layout

Items	Start Address	End Address	Size	Description	Mandatory
System Area	0x000C_0000	0x000C_0080-1	128B	Used by Realtek. The size is indicated by the second dword (0x000C_0004).	Yes
User Area	0x000C_0080	0x000C_0130-1	176B	Refer to RRAM_TypeDef, and RRAM_USER_RSVD [user_size] is reserved for customer use.	No
Wi-Fi Area	0x000C_0130	0x000C_0400-1	720B	Realtek Wi-Fi FW use	Yes

9.4.2 User Area

RRAM_USER_RSVD is reserved for customer use. The size for user area is 124B now, it may be changed latter.

```

/*****
 * @defgroup AMEBAD_RRAM
 * @{
 * @brief AMEBAD_RRAM Declaration
 * @ the Max space for RRAM_TypeDef is 0xB0 user can alloc space from RRAM_USER_RSVD
 *****/
typedef struct {
    uint8_t RRAM_WIFI_STATUS; /* RETENTION_RAM_SYS_OFFSET 0x80 */
    uint8_t RRAM_WIFI_P_SECURITY;
    uint8_t RRAM_WIFI_G_SECURITY;
    uint8_t RRAM_RSVD1;

    uint32_t RRAM_NET_IP;
    uint32_t RRAM_NET_GW;
    uint32_t RRAM_NET_GW_MASK;

    uint32_t FLASH_ID2; /*offset 0x90*/
    uint32_t FLASH_cur_cmd;
    uint32_t FLASH_quad_valid_cmd;
    uint32_t FLASH_dual_valid_cmd;
    uint32_t FLASH_QuadEn_bit; /*offset 0xA0*/
    uint32_t FLASH_StructInit;

    uint8_t FLASH_phase_shift_idx;
    uint8_t FLASH_rd_sample_phase_cal;
    uint8_t FLASH_class;
    uint8_t FLASH_cur_bitmode;

    uint8_t FLASH_ClockDiv;
    uint8_t FLASH_ReadMode;
    uint8_t FLASH_pseudo_prm_en;
    uint8_t FLASH_addr_phase_len;

    uint8_t FLASH_cmd_wr_status2; /*offset 0xB0*/
    uint8_t FLASH_rd_dummy_cycle0;
    uint8_t FLASH_rd_dummy_cycle1;
    uint8_t FLASH_rd_dummy_cycle2;

    uint8_t RRAM_USER_RSVD[124]; /*usr can alloc from this RSVD space*/
} RRAM_TypeDef;
/****

```

9.5 OTA Layout

9.5.1 OTA Program Layout

The OTA program layout is shown in Fig 9-8.

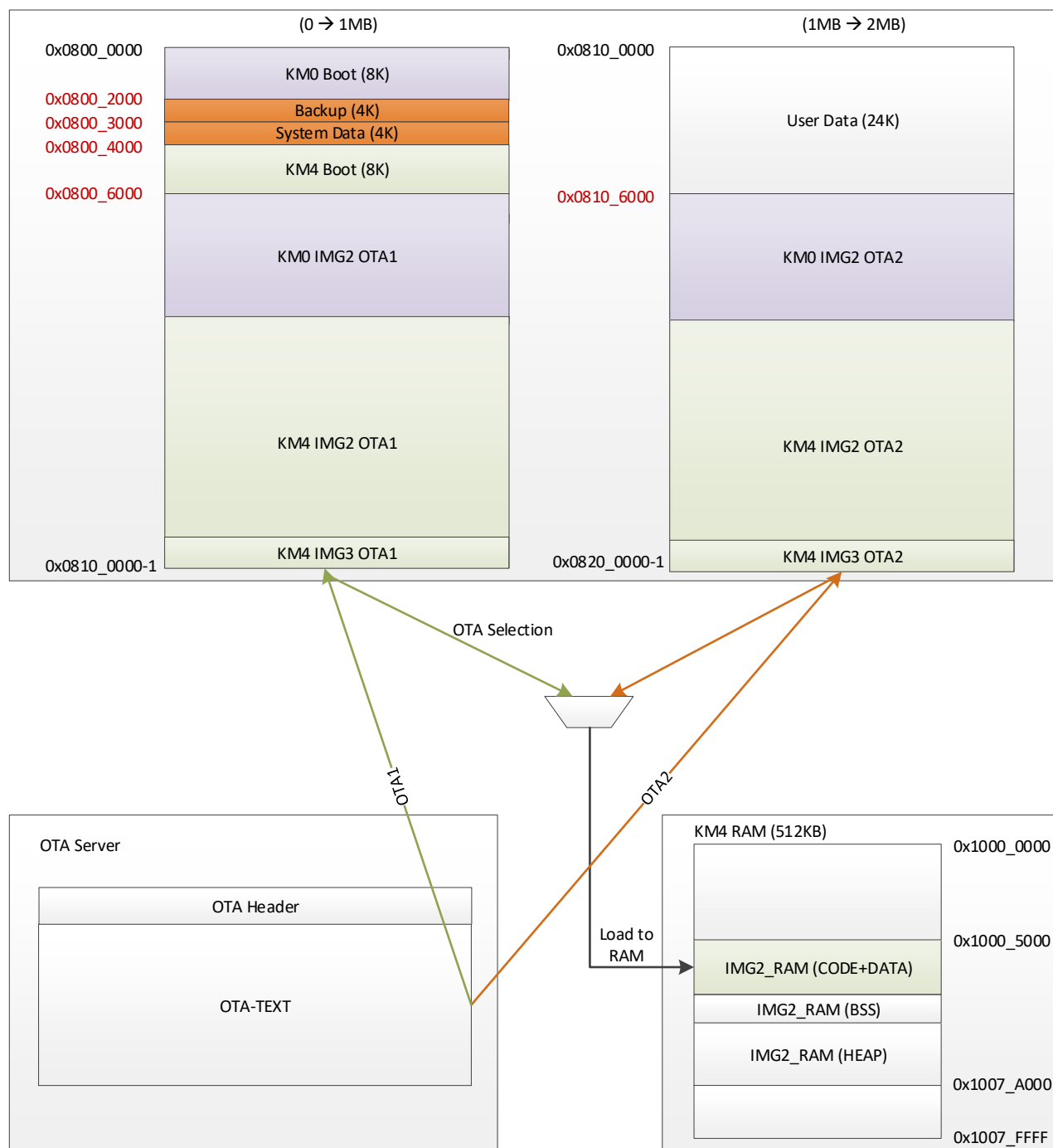


Fig 9-8 OTA program layout

9.5.2 OTA Header Layout

The OTA header layout is given in Fig 9-9 and Table 9-10.

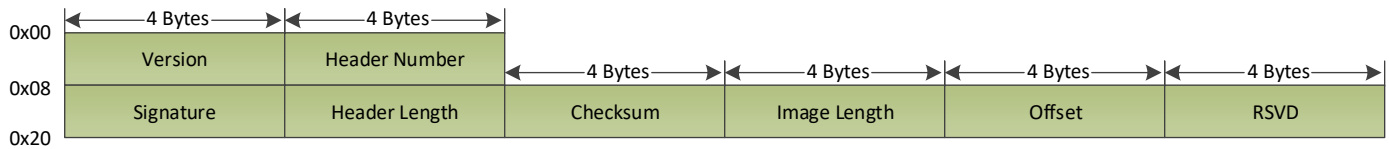


Fig 9-9 OTA header layout

Table 9-10 OTA header layout

Items	Address Offset	Size	Description
Version	0x00	4 bytes	The version of OTA Header, default 0xFFFFFFFF
Header Number	0x04	4 bytes	The number of OTA Header. For Ameba-D, this value is 0x01
Signature	0x08	4 bytes	OTA Signature is string. For Ameba-D, this value is "OTA"
Header Length	0x0C	4 bytes	The length of OTA header. For Ameba-D, this value is 0x18
Checksum	0x10	4 bytes	The checksum of OTA image
Image Length	0x14	4 bytes	The size of OTA image
Offset	0x18	4 bytes	The start position of OTA image in current image
RSVD	0x1C	4 bytes	Reserved

10 File System

10.1 Introduction

This chapter introduces how to run FAT file system (FatFs) example on Ameba-D, including FatFs examples of Flash and SD card. The example source code of FatFs is located in **component\common\example\fatfs**.

10.2 FatFs on Flash

10.2.1 Software Setup

- (1) Change some configurations in **project\realtek_amebaD_cm4_gcc_verification\inc\platform_opts.h** as follows:
 - a) Set the parameter **CONFIG_EXAMPLE_FATFS** to 1.
 - b) Set **FATFS_DISK_FLASH** to 1 while **FATFS_DISK_USB** and **FATFS_DISK_SD** both to 0.

```

/* For FATFS example*/
#define CONFIG_EXAMPLE_FATFS 1
#if CONFIG_EXAMPLE_FATFS
#define CONFIG_FATFS_EN 1
#if CONFIG_FATFS_EN
// fatfs version
#define FATFS_R_10C
// fatfs disk interface
#define FATFS_DISK_USB 0
#define FATFS_DISK_SD 0
#define FATFS_DISK_FLASH 1
#endif
#endif

```

- (2) Change some parameters in **component\common\file_system\fatfs\r0.10c\include\ffconf.h** as follows:
 - a) Set **_MAX_SS** to 4096 to define the maximum sector size supported.
 - b) Set **_USE_MKFS** to 1 to enable **f_mkfs()** function which creates FatFs volume on Flash.

```
#define _MIN_SS 512
#define _MAX_SS 4096//512
#define _USE_MKFS 1 /* 0:Disable or 1:Enable */
```

- (3) Change Flash memory base in **component\common\file_system\fatfs\disk_if\src\flash_fatfs.c** according to user's requirements.

```
#define FLASH_APP_BASE 0x100000 // 0x440000
```

- (4) Check the stack size that needs to be at least 4096 in **component\common\example\fatfs\example_fatfs.c**.

```
#define STACK_SIZE 4096
```

- (5) Rebuild the project and download image files to Ameba-D.

10.2.2 FatFs Bin File Generation

The commands to generate FatFs bin file are as follows:

- (1) root@ubuntu # dd if=/dev/zero of=test.bin count=64 bs=1KB
- (2) root@ubuntu # fdisk test.bin
- (3) root@ubuntu # mkfs.msdos -S 4096 -F 12 test.bin
- (4) root@ubuntu # mcopy -i test.bin ./fs/hello.txt ::hello.txt
- (5) root@ubuntu # sudo mount test.bin ./fs
- (6) sudo umount ./fs

- In step (1), test.bin is created which has 64 blocks and each block is 1KB.
- In step (2), test.bin is partitioned. Some additional options have to be chosen in this step in order to create a new partition table.
- In step (3), a MS-DOS file system is built.
- In step (4), files that users want to store are copied into test.bin. In this step, hello.txt is stored in test.bin.
- In step (5), test.bin is mounted to file folder fs.
- In step (6), after unmounting test.bin, the FatFs file is generated.

Users should find other related information from the internet, and copy test.bin into user data area of Flash finally.

10.3 FatFs on SD Card

10.3.1 Hardware Setup

Connect your Ameba-D EVB board to PC with micro-USB cable, and then plug in a compatible SD card to the card connector. Note that the terminal of the middle of J24 need to be connected to ground. Connection of J29-J33 is shown as Fig 10-1.



Fig 10-1 Hardware setup

10.3.2 Software Setup

- (1) Change some configurations in **project\realtek_amebaD_va0_example\inc\inc_hp\platform_opts.h** as follows:
 - a) Set the parameter CONFIG_EXAMPLE_FATFS to 1.
 - b) Set FATFS_DISK_SD to 1 while FATFS_DISK_USB and FATFS_DISK_FLASH both to 0.

```
/* For FATFS example*/
#define CONFIG_EXAMPLE_FATFS 1
#if CONFIG_EXAMPLE_FATFS
#define CONFIG_FATFS_EN 1
#endif
// fatfs version
#define FATFS_R_10C
// fatfs disk interface
#define FATFS_DISK_USB 0
#define FATFS_DISK_SD 1
#define FATFS_DISK_FLASH 0
#endif
#endif
```

- (2) Change some parameters in **component\common\file_system\fatfs\r0.10c\include\ffconf.h** as follows:
 - a) Set _MAX_SS to 512 to define the maximum sector size supported.
 - b) Set _USE_MKFS to 1.

```
#define _MIN_SS 512
#define _MAX_SS 512
#define USE_MKFS 1 /* 0:Disable or 1:Enable */
```

- (3) Change some parameters in **component\soc\realtek\amebad\fwlib\usrcfg\rtl8721dhp_intfcfg.c** as follows:

```
SDIOHCFG_TypeDef sdioh_config = {
    .sdioh_bus_speed = SD_SPEED_HS,
    .sdioh_bus_width = SDIOH_BUS_WIDTH_4BIT,
    .sdioh_cd_pin = _PA_6,
    .sdioh_wp_pin = _PNC,
};
```

- (4) Rebuild the project and download image files to Ameba-D. FAT file of SD card can be generated and copied directly by PC.

11 Inter Processor Communication (IPC)

11.1 Introduction

Inter-Processor Communication (IPC) hardware is designed for the purpose of making two CPUs communicate with each other. The system architecture is shown in Fig 11-1.

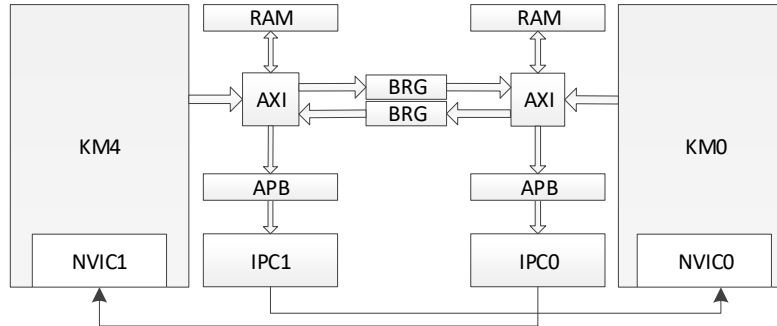


Fig 11-1 IPC system architecture

If you want KM0 to execute its IPC handler, IPC interrupt of KM4 should be enable and the status should be set, vice versa.

11.2 How to Use IPC?

It is complex because both KM0 and KM4 should be set to realize IPC. SDK has provided API and configuration file to simplify the procedure. Take KM4 requests an interrupt into direction KM0 for example.

- (1) Add new item in KM0 core section of `ipc_init_config[]` which is in `rtl8721d_ipccfg.c` for selected channel. IRQ handler and data should be set and defined. If message exchange is needed, you should specify the message type: data or point. Note that the message type for a specific channel should be the same on both ARM_CORE_CM4 side and ARM_CORE_CM0 side. In the following figure, although IRQFUNC of channel 3 on ARM_CORE_CM4 side is not used, its message type matches the type on ARM_CORE_CM0 side.

```
const IPC_INIT_TABLE ipc_init_config[] =
{
    #if defined (ARM_CORE_CM4)
        //USER_MSG_TYPE    IRQFUNC                                IRQDATA
        {IPC_USER_DATA,    (VOID*) shell_switch_ipc_int,        (VOID*) NULL}, // channel 0: IPC_INT_CHAN_SHELL_SWITCH
        {IPC_USER_DATA,    (VOID*) NULL,                        (VOID*) NULL}, // channel 1: IPC_INT_CHAN_WIFI_FW
        {IPC_USER_DATA,    (VOID*) FLASH_Write_IPC_Int,         (VOID*) NULL}, // channel 2: IPC_INT_CHAN_FLASHPG_REQ
        {IPC_USER_POINT,   (VOID*) NULL,                        (VOID*) NULL}, // channel 3: IPC_INT_KM4_TICKLESS_INDICATION
    #else
        //USER_MSG_TYPE    IRQFUNC                                IRQDATA
        {IPC_USER_DATA,    (VOID*) shell_switch_ipc_int,        (VOID*) NULL}, // channel 0: IPC_INT_CHAN_SHELL_SWITCH
        {IPC_USER_DATA,    (VOID*) driver_fw_flow_ipc_int,      (VOID*) IPCM4_DEV}, // channel 1: IPC_INT_CHAN_WIFI_FW
        {IPC_USER_DATA,    (VOID*) FLASH_Write_IPC_Int,         (VOID*) NULL}, // channel 2: IPC_INT_CHAN_FLASHPG_REQ
        {IPC_USER_POINT,   (VOID*) km4_tickless_ipc_int,        (VOID*) NULL}, // channel 3: IPC_INT_KM4_TICKLESS_INDICATION
    #endif
    {0xFFFFFFFF,          OFF,                                    OFF}, /* Table end */
};
```

- (2) SDK will enable the IPC interrupt of KM4 for the channel according to `ipc_init_config[]`, and register the corresponding KM0 IRQ handler and data for the channel.
- (3) When KM4 wants to request an interrupt into direction KM0, it should call `ipc_send_message()` and specify the channel number and message.
- (4) Corresponding KM0 IRQ handler will be executed, call `ipc_get_message()` to get message if need.

Note: Use IPC channel 16~31 and IPC semaphore index 8~15, Channel 0~15 and semaphore index 0~7 are reserved for Realtek use.

11.3 IPC Program APIs

11.3.1 ipc_table_init

Items	Description
Introduction	Initialize IPC channels according to <code>ipc_init_config[]</code>
Parameters	N/A
Return	N/A

11.3.2 ipc_send_message

Items	Description
Introduction	Request interrupt to target CPU by specified channel and exchange messages between KM0 and KM4
Parameters	- IPC_ChNum: the IPC channel number - Message: the message to be exchanged, and should not be stored in stack
Return	N/A

Note:

- The message supports two types: data or point. The point is used to point to complex message structure.
- The message is shared between two CPUs, so the point can't be stored in stack.

11.3.3 ipc_get_message

Items	Description
Introduction	Get IPC message from specified channel.
Parameters	IPC_ChNum: the IPC channel number
Return	Message: the message to be exchanged

Note: Cache in the system is write-through type, so the corresponding data cache must be invalidated before data fetch.

11.4 IPC Driver Code

11.4.1 IPC_INTConfig

Items	Description
Introduction	Enable or disable the specified IPC channel interrupts.
Parameters	- IPCx: - IPCM0_DEV for KM0 - IPCM4_DEV for KM4 - IPC_ChNum: the channel that want to be set. This parameter must be set to a value of 0 ~ 31. - NewState: - ENABLE - DISABLE
Return	N/A

11.4.2 IPC_IERSet

Items	Description
Introduction	Set IER of specified IPC channel interrupts

Parameters	● IPCx: ■ IPCM0_DEV for KM0 ■ IPCM4_DEV for KM4 ● IPC_Ch: the channel that want to be enable
Return	N/A

11.4.3 IPC_IERGet

Items	Description
Introduction	Get IER of specified IPCx
Parameters	IPCx: ● IPCM0_DEV for KM0 ● IPCM4_DEV for KM4
Return	The IER of specified IPCx

11.4.4 IPC_INTRequest

Items	Description
Introduction	Request a core-to-core interrupt
Parameters	● IPCx: ■ IPCM0_DEV for KM0 ■ IPCM4_DEV for KM4 ● IPC_ChNum: the channel that want to request interrupt. This parameter must be set to a value of 0 ~ 31.
Return	N/A

11.4.5 IPC_INTClear

Items	Description
Introduction	Clear a core-to-core interrupt
Parameters	● IPCx: ■ IPCM0_DEV for KM0 ■ IPCM4_DEV for KM4 ● IPC_ChNum: the channel that want to clear interrupt. This parameter must be set to a value of 0 ~ 31.
Return	N/A

11.4.6 IPC_INTGet

Items	Description
Introduction	Get a core-to-core interrupt status
Parameters	IPCx: ● IPCM0_DEV for KM0 ● IPCM4_DEV for KM4
Return	The ISR of specified IPCx

11.4.7 IPC_CPUID

Items	Description
Introduction	Get the current CPU ID
Parameters	N/A
Return	CPU ID: ● 0: KM0

	1: KM4
--	--

11.4.8 IPC_SEMGet

Items	Description
Introduction	Get a core-to-core hardware semaphore
Parameters	SEM_Idx: the semaphore index that want to get. This parameter must be set to a value of 0 ~ 15. 0 ~ 7: Reserved for Realtek use 8 ~ 15: Reserved for Customer use
Return	Result: - TRUE - FALSE

11.4.9 IPC_SEMFree

Items	Description
Introduction	Free a core-to-core hardware semaphore
Parameters	SEM_Idx: the semaphore index that want to free. This parameter must be set to a value of 0 ~ 15. 0 ~ 7: Reserved for Realtek use 8 ~ 15: Reserved for Customer use
Return	Result: - TRUE - FALSE

11.4.10 IPC_INTHandler

Items	Description
Introduction	The common IPC interrupt handler
Parameters	Data: the data passed to the IRQ Handler
Return	0

Note: Before entering the specified IRQ handler, the common IPC interrupt handler clears the pending interrupt first. There is no need to clear the pending interrupt in user defined IRQ handler.

11.4.11 IPC_INTUserHandler

Items	Description
Introduction	Register a user interrupt handler for a specified IPC channel
Parameters	- IPC_ChNum: the channel that want to register IRQ handler. This parameter must be set to a value of 0 ~ 31. - IrqHandler: the IRQ handler to be assigned to the specified IPC channel - IrqData: the pointer will be passed to the IRQ handler
Return	N/A

12 OTA Firmware Update

12.1 Introduction

Over-the-air (OTA) programming provides a methodology of updating device firmware remotely via TCP/IP network.

RTL8721d provides solutions to implement OTA firmware upgrade from local server or cloud. Next we will introduce the design principles and usage of OTA from local server. It has well-transportability to porting to OTA applications from cloud.

12.2 RSIP-MMU

The Flash MMU diagram is shown in Fig 12-1.

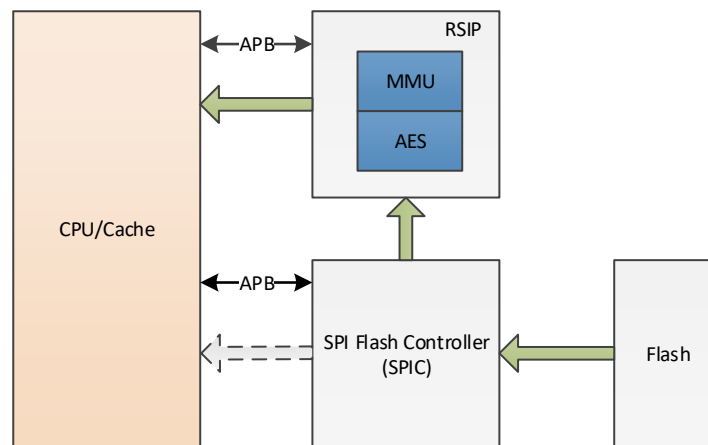


Fig 12-1 Flash MMU

The RSIP-MMU can perform virtual-to-physical memory address translation. This can be used to map an external Flash memory to a certain virtual memory area. For example, If you want to access physical page 4 ~ 7 through virtual page 0 ~ 3, you can use a MMU entry to map virtual page 0 ~ 3 to physical page 4 ~ 7 like Fig 12-2.

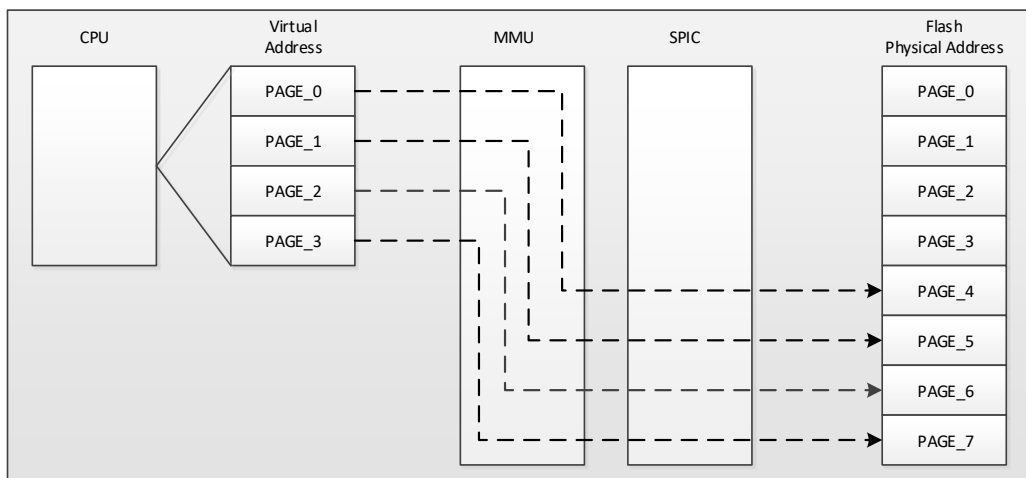


Fig 12-2 MMU virtual address to physical address

Ameba-D provides 8 MMU Entries. If virtual address is not included in the MMU entry, use virtual address as physical address. If virtual address is included in the MMU entry, physical address should be $VAddress \pm MMU_ENTRYx_OFFSET$.

MMU is implemented to facilitate OTA update procedure as Fig 12-3 shows. Fig 12-3 is an example for 2M Flash OTA, we need 2 MMU entries in this example.

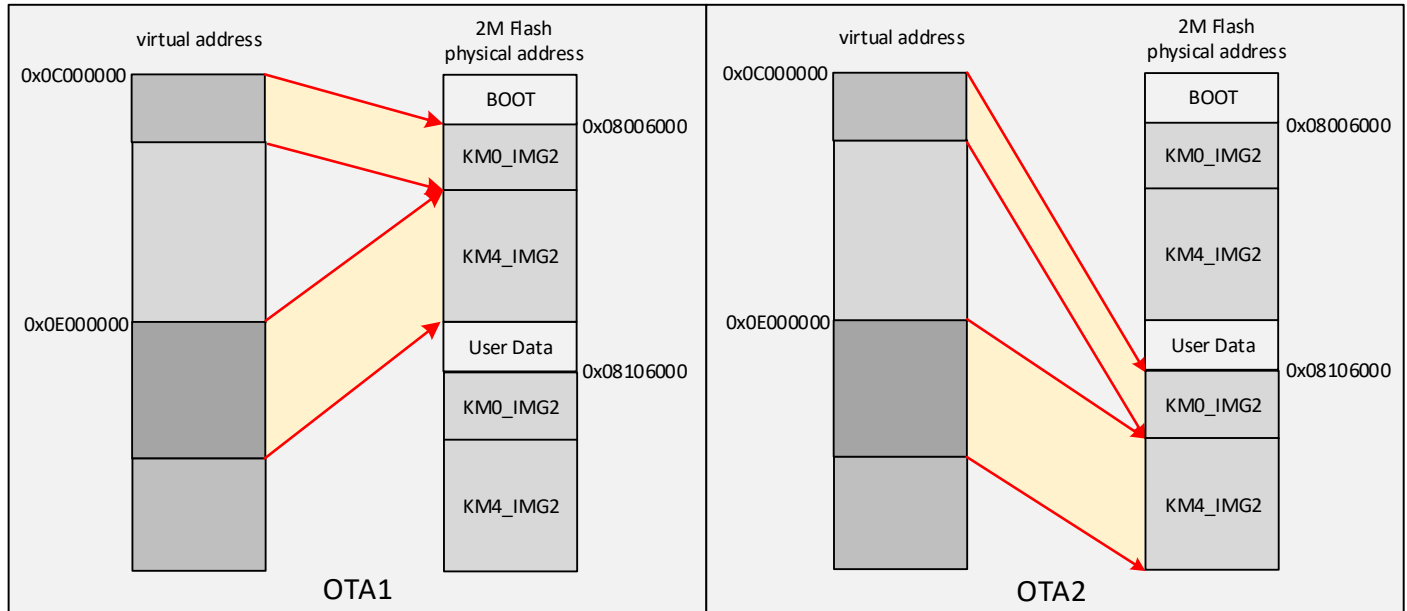


Fig 12-3 2M Flash OTA MMU

MMU entry should be set as Table 12-1.

Table 12-1 OTA under MMU

Register Type	Register	OTA1	OTA2
MMU Entry0	MMU_ENTRY0_STRADDR	0x0C00_0000	0x0C00_0000
	MMU_ENTRY0_ENDADDR	0x0C00_0000 + KM0 IMG2 size	0x0C00_0000 + KM0 IMG2 size
	MMU_ENTRY0_OFFSET	0x0C00_0000 – 0x0800_6000	0x0C00_0000 – 0x0810_6000
	+/-	-	-
MMU Entry1	MMU_ENTRY1_STRADDR	0x0E00_0000	0x0E00_0000
	MMU_ENTRY1_ENDADDR	0x0E00_0000 + KM4 IMG2 size	0x0E00_0000 + KM4 IMG2 size
	MMU_ENTRY1_OFFSET	0x0E00_0000 – KM4 start physical address	0x0E00_0000 – KM4 start physical address
	+/-	-	-

Note: Considering KM4 IMG2 is appended to the tail of KM0 IMG2, the KM4 start physical address is determined by KM0 start physical address and KM0 IMG2 size.

12.3 OTA Upgrade from Local Server

The OTA from local server shows how device updates image from a local download server. The local download server sends image to device based on network socket, as Fig 12-4 shows.

Make sure both device and PC are connecting to the same local network.

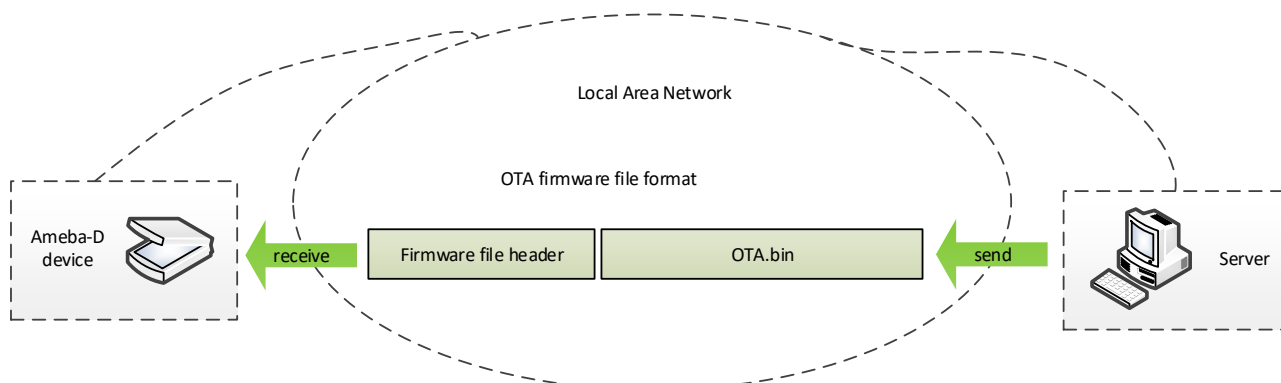


Fig 12-4 OTA update diagram

12.3.1 Firmware Format

The firmware format is illustrated in Fig 12-5.

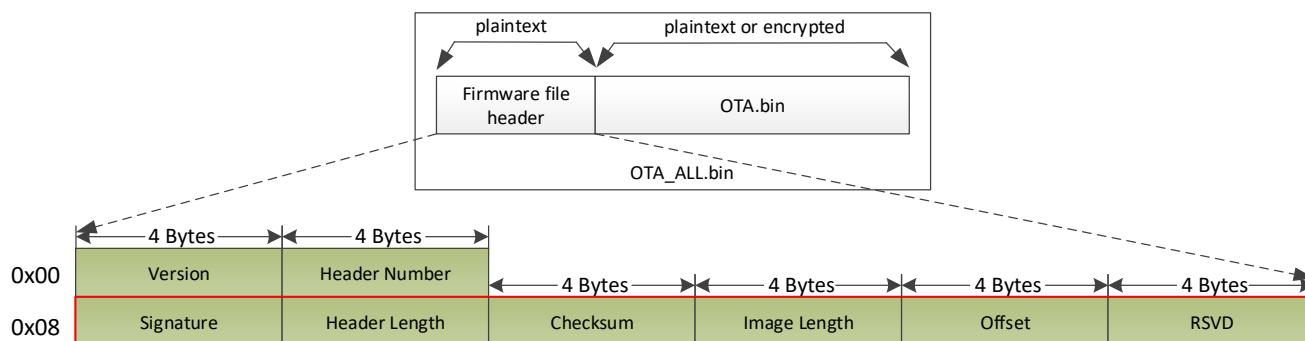


Fig 12-5 Firmware format

Table 12-2 Firmware header

Items	Address Offset	Size	Description
Version	0x00	4 bytes	The version of OTA Header, default 0xFFFFFFFF
Header Number	0x04	4 bytes	The number of OTA Header. For Ameba-D, this value is 0x01
Signature	0x08	4 bytes	OTA Signature is string. For Ameba-D, this value is "OTA"
Header Length	0x0C	4 bytes	The length of OTA header. For Ameba-D, this value is 0x18
Checksum	0x10	4 bytes	The checksum of OTA image
Image Length	0x14	4 bytes	The size of OTA image
Offset	0x18	4 bytes	The start position of OTA image in current image
RSVD	0x1C	4 bytes	Reserved

12.3.2 OTA Flow

The OTA demo is provided in `rtl8721d_ota.c`. The image upgrading is implemented in the following steps:

- (1) Connect to local server with socket. The IP address and port are needed.
- (2) Acquire the older firmware address to be upgraded according to MMU setting. If address is re-mapping to OTA1 space by MMU, the OTA2 address would be selected to upgrade. Otherwise OTA1 address would be selected.
- (3) Receive firmware file header to get the target OTA image information, such as image length and destination address.
- (4) Erase Flash space for new firmware

- (5) Download new firmware from server and write it to Flash
- (6) Verify checksum. If checksum error, OTA fail.
- (7) If checksum is ok, write signature to the upgraded firmware region and change another signature to all zero to indicate boot from new firmware next time.
- (8) OTA finish and reset the device. Then it would boot from new firmware.

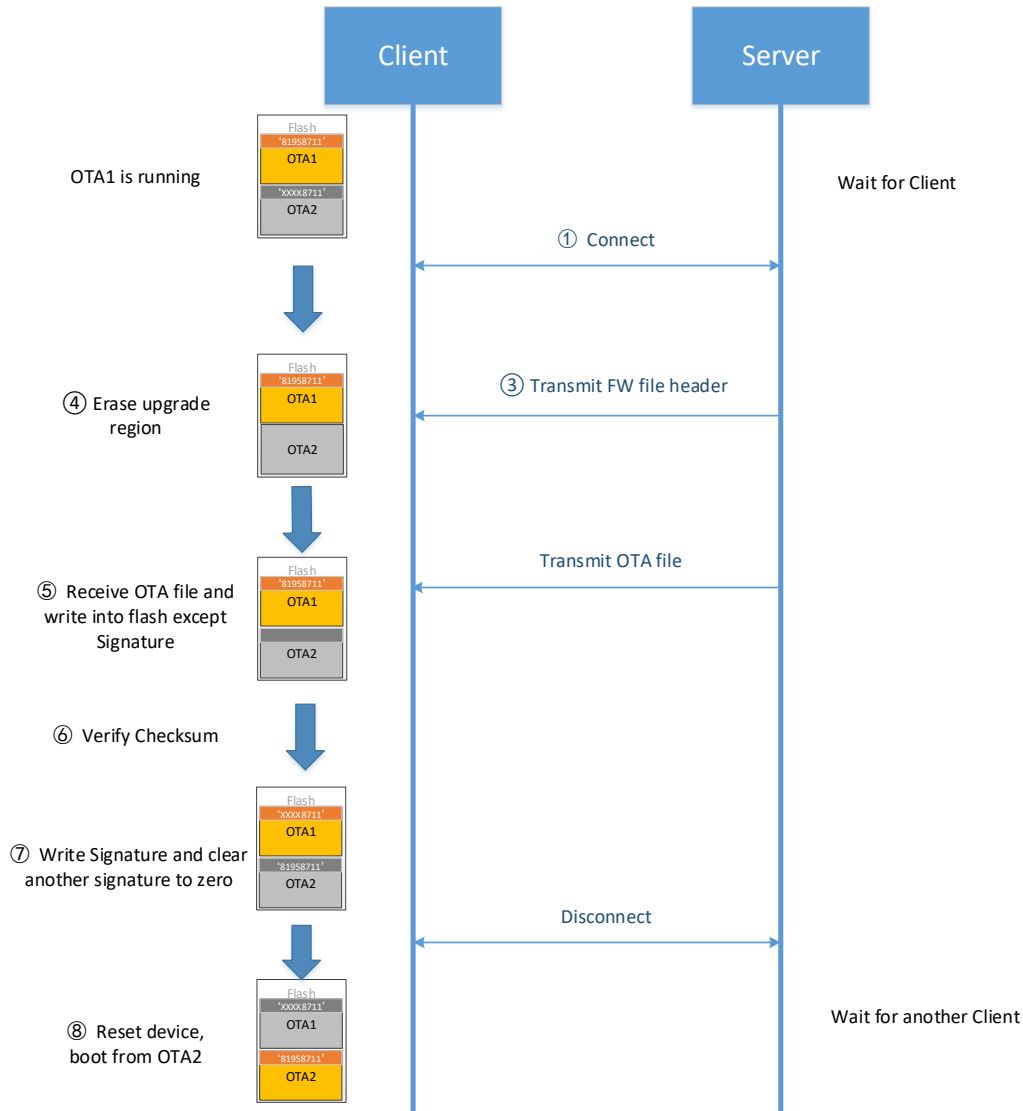


Fig 12-6 OTA operation flow

12.3.3 Boot Option

When reboot after OTA finished, device would check firmware to determine to boot from OTA1 or OTA2. The following items must be checked for each firmware:

- Signature. If it is "81958711", the signature is valid. Otherwise, the signature is invalid.
- Hash/checksum if needed. Users can define FwCheckCallback in rtl8721d_bootcfg.

The boot flow is as follows:

- (1) Check Signature and hash (if need) of OTA1 and OTA2 firmware.
- (2) If both images are invalid, boot fail
- (3) If only one image is valid, boot from this image

- (4) If both images are valid, boot from OTA2

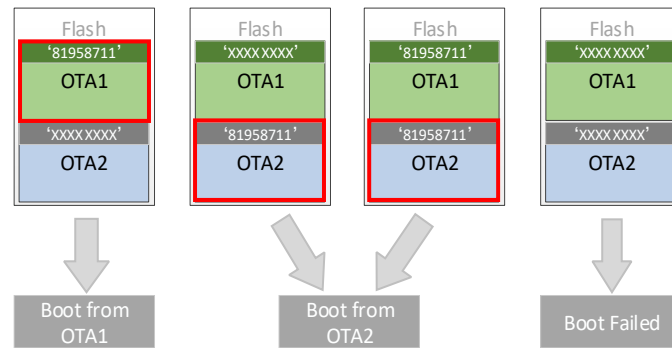


Fig 12-7 OTA select diagram

12.3.4 Address Remapping

In bootloader, MMU entry0 and entry1 would be set as Table 12-1 to remap memory. Here, the OTA space includes these images: KM0 IMG2, KM4 IMG2, KM4 IMG3 if need. These images all boot from OTA1 or OTA2.

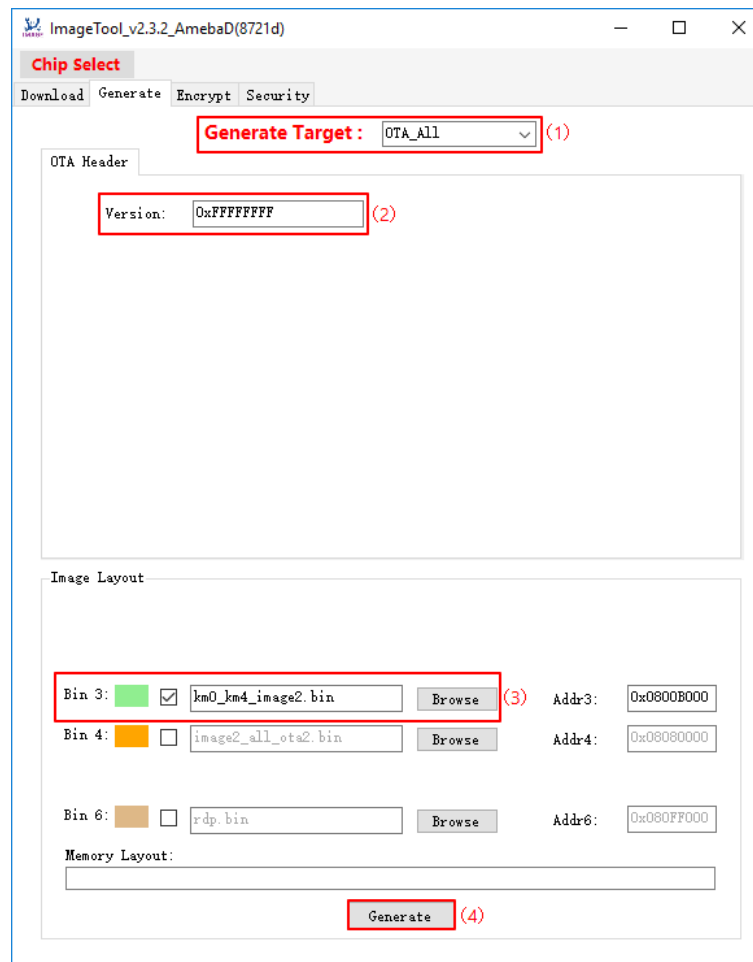
As Fig 12-3 illustrates, when boot from OTA, CPU may access different physical address.

- When boot from OTA1, MMU entry0 and entry1 would be set to remap virtual memory to OTA1 address space. That is, if CPU wants to read code or data from address 0x0C00_0000, it actually accesses physical address 0x0800_6000.
- When boot from OTA2, MMU entry0 and entry1 would be set to remap virtual memory to OTA2 address space. That is, if CPU wants to read code or data from address 0x0C00_0000, it actually accesses physical address 0x0810_6000.

12.3.5 How to Use OTA Demo?

These steps can be followed to run the OTA demo:

- Define CONFIG_OTA_UPDATE macro to 1 in **platform_opts.h** to enable OTA function. Rebuild the project.
- Download images to device.
- Generate OTA upgrade image file, named **OTA_All.bin**, with Image Tool and new firmware. This operation prepends firmware header information to new firmware, which is necessary for OTA.



- (4) Copy **OTA_All.bin** into the **DownloadServer** folder.
Tool path in SDK: tools\DownloadServer

DownloadServer.exe	2016/11/7 16:12	34 KB
OTA_All.bin	2016/11/7 15:21	657 KB
start.bat	2016/11/6 11:43	1 KB

- (5) Edit tools\DownloadServer\start.bat.
- Port = 8082
 - File name =OTA_All.bin

```
@echo off
DownloadServer 8082 OTA All.bin
set /p DUMMY=Press Enter to Continue ...
```

- (6) Click the **start.bat**, start to download server program.



- (7) Reboot the DUT and connect the device to the AP which the OTA Server in.
- (8) Enter command: **ATW0=IP[PORT]**.
 - IP: IP of the OTA Server.
 - Port: 8082, the same with start.bat

```
# ATW0=192.168.0.20[8082]  
[ATW0]: _AT_WLAN_OTA_UPDATE_  
[MEM] After do cmd, available heap 28400  
#  
[ota_update_local_task] Update task start
```

OTA upgrade procedure is started between DUT and server. Here is the local download server success message.

[illegible]

- (9) Reboot DUT to execute the new firmware after finishing image download.

12.3.6 OTA Firmware Swap

Fig 12-8 shows the firmware swap procedure after OTA upgrade.

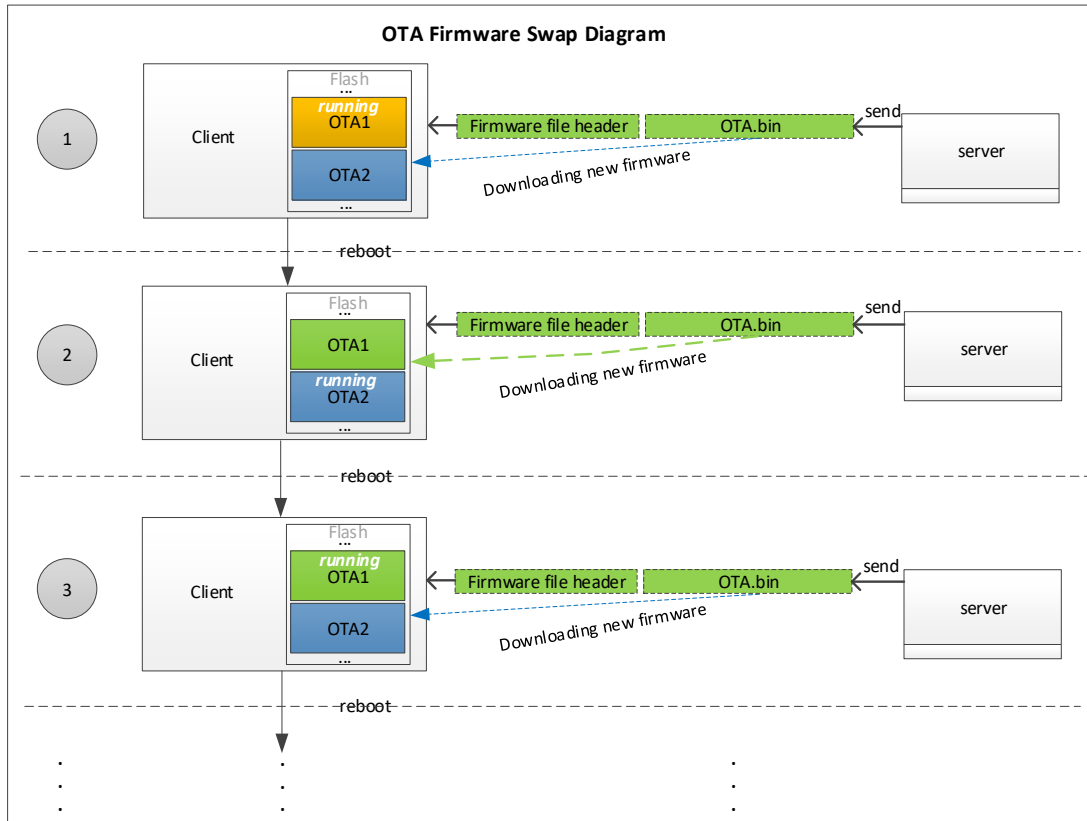


Fig 12-8 OTA firmware swap procedure

12.4 User Configuration

Users can find the following configure items in `rtl8721d_bootcfg.c`.

- OTA start address

```
/*
 * @brief OTA start address. Because KM0 & KM4 IMG2 are combined, users only need to set the start address
 * of KM0 IMG2.
 */
BOOT_RAM_DATA_SECTION
u32 OTA_Region[2] = {
    0x08006000, /* OTA1 region start address */
    0x08106000, /* OTA2 region start address */
};
```

- User-defined MMU configure

```

/*
 * @brif MMU Configuration.
 *      There are 8 MMU entries totally. Entry 0 & Entry 1 are already used by OTA, Entry 2~7 can be used by Users.
 */
BOOT_RAM_DATA_SECTION
MMU_ConfDef Flash_MMU_Config[] = {
    /*VAddrStart,  VAddrEnd,          PAddrStart,          PAddrEnd*/
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 2
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 3
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 4
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 5
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 6
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF}, //Entry 7
    {0xFFFFFFFF,  0xFFFFFFFF,        0xFFFFFFFF,          0xFFFFFFFF},
};

```

- Firmware check handler configure

```

/**
 * @brif Firmware verify callback handler.
 *      If users need to verify checksum/hash for image2, implement the function and assign the address
 *      to this function pointer.
 */
BOOT_RAM_DATA_SECTION
FwCheckFunc FwCheckCallback = NULL;

```

13 eFuse

13.1 Introduction

eFuse belongs to One Time Programmable (OTP) technology, its default value is '1', and can only be changed from '1' to '0'. eFuse can be used to hold the individual and stable data such as key, calibration data, MAC address and specific setting.

The total size of physical eFuse is 512 bytes, and divided into two parts by software. As Fig 13-1 shows, the first 288 bytes of physical eFuse are used for logical mapping, which can be mapped to logical eFuse by some algorithm and can be programmed multi-times. The left 224 bytes of physical eFuse are defined by Realtek.

Logical eFuse program will program header and package, and the package is in word, so it takes at least three bytes a time. Software reserves one byte for isolation, so when the space of eFuse for logical mapping is less than four bytes, it cannot be programmed in any mode.

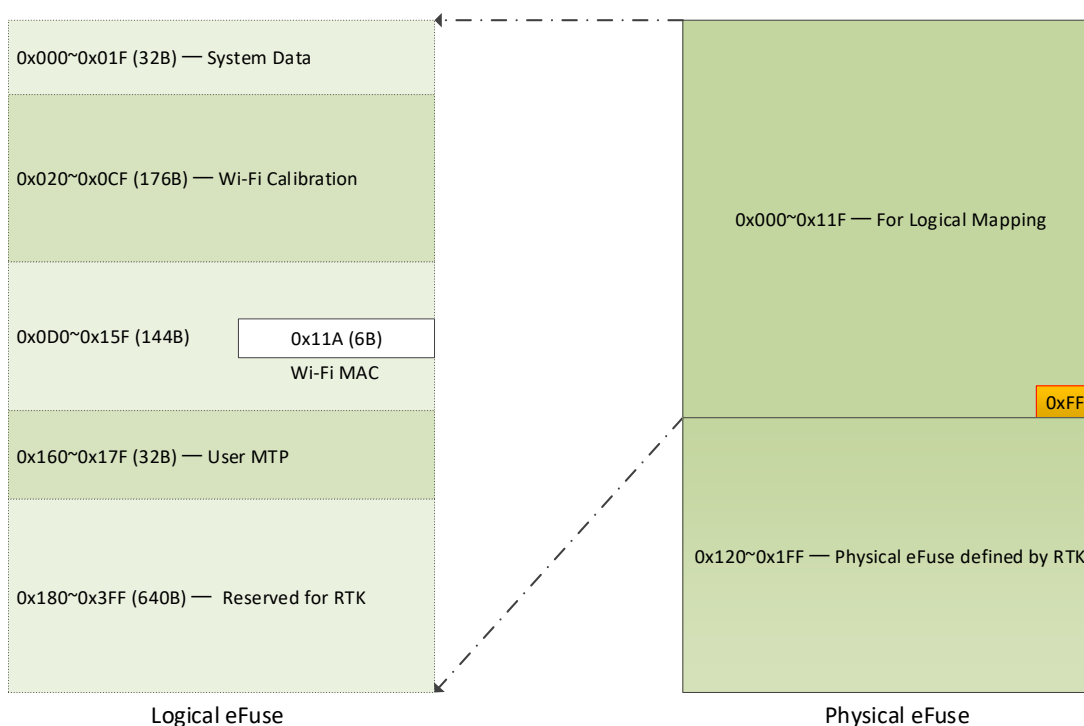


Fig 13-1 eFuse diagram

13.2 Power Requirement

The power requirement of eFuse operation is listed in Table 13-1.

Table 13-1 Operation under different voltage

Supply Voltage (V)	Operation
1.8	Only read operation is allowed.
3.3	Read & Write

Note: If you want to program eFuse, you must switch the power supply to 3.3V.

13.3 eFuse Auto-load

eFuse can auto-load part of setting to control the circuit. eFuse auto-load is changed in word (two bytes). Under default condition, value of all the System Data bytes is 0xFF, and the System Configure Registers have its default values.

The way to change the value of System Configure Registers is as follows:

- Make sure that the first two bytes of System Data area are written to 0x21, 0x87 correctly.
- Program the corresponding bytes in word. If just programming in byte, another byte will load the eFuse default value 0xFF, which may make mistakes.
- Reboot the chip, and the System Configure Registers will load the new values.

13.4 Physical eFuse

For detailed information about physical eFuse of Ameba-D, refer to AN0411.

13.5 Logical eFuse

13.5.1 Logical eFuse Layout

There are 1024 bytes logical eFuse in Ameba-D, as Table 13-2 shows. Most of the logical eFuse space are reserved for extension, so the 1024 bytes of logical eFuse can be easily mapped to 287 bytes of physical eFuse.

Table 13-2 Logical eFuse layout

Start Address	End Address	Description
0x000	0x01F	System Data to be auto-loaded
0x020	0x15F	Wi-Fi calibration data
0x160	0x17F	User MTP
0x180	0x183	Cap-Touch
0x184	0x18F	Reserved
0x190	0x1A4	BT parameters
0x1A5	0x1BF	Reserved
0x1C0	0x1CF	HCI USB
0x1D0	0x2FF	Reserved

20~25	2.4G CCK Index		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
26~2A	2.4G BW40 Index	000	21	87	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
2B	2.4G Difference	010	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
32~3F	5G BW40 Index	020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
40	5G Difference	030	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
C8	channel plan	040	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
C9	Crystal Calibration	050	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
CA	Thermal meter	060	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
11A~11F	MAC Address	070	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		080	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		090	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		0F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		100	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		110	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		120	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		130	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		140	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		150	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		160	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		170	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		180	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		190	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
		1F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

For Wi-Fi:
Users only need to fill on these
eFuse locations for MP (Mass
Production) calibration.

Fig 13-2 Wi-Fi eFuse data

13.5.2 Wi-Fi 2.4G Power Index

The address and specification of 2.4G power index is shown in Fig 13-3 and Fig 13-4.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-3 2.4G Wi-Fi power index address

offset	name	comment
20~25	2.4G CCK Index	Path A CCK Power Index for Ch 1,2, Range 0~127,0.25dbm/step.
		Path A CCK Power Index for Ch 3, 4, 5, Range 0~127,0.25dbm/step.
		Path A CCK Power Index for Ch 6, 7, 8, Range 0~127,0.25dbm/step.
		Path A CCK Power Index for Ch 9, 10, 11, Range 0~127,0.25dbm/step.
		Path A CCK Power Index for Ch 12, 13, Range 0~127,0.25dbm/step.
		Path A CCK Power Index for Ch 14, Range 0~127,0.25dbm/step.
26~2A	2.4G BW40 Index	Path A 2G BW40-1S Power Index for Ch 1, 2, Range 0~127,0.25dbm/step.
		Path A 2G BW40-1S Power Index for Ch 3, 4, 5, Range 0~127,0.25dbm/step.
		Path A 2G BW40-1S Power Index for Ch 6, 7, 8, Range 0~127,0.25dbm/step.
		Path A 2G BW40-1S Power Index for Ch 9, 10, 11, Range 0~127,0.25dbm/step.
		Path A 2G BW40-1S Power Index for Ch 12, 13, 14 Range 0~127,0.25dbm/step.
2B	2.4G Difference	Power Index Difference between BW20-1S and BW40-1S.
		Bit[7:4]: Path A 2G Offset, Range -8~7,0.5dbm/step.
		Power Index Difference between OFDM-1Tx and BW40-1S.
		Bit[3:0]: Path A 2G Offset, Range -8~7,0.5dbm/step.

Fig 13-4 2.4G Wi-Fi power index specification

13.5.3 Wi-Fi 5G Power Index

The address and specification of 5G power index is shown in Fig 13-5 and Fig 13-6.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
030	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
040	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-5 5G Wi-Fi power index address

32~3F	5G BW40 Index	Path A 5G BW40-1S Power Index for Ch 36,38,40, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 44,46,48, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 52,54,56, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 60,62,64, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 100,102,104, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 108,110,112, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 116,118,120, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 124,126,128, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 132,134,136, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 140,142,144, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 149,151,153, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 157,159,161, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 165,167,169, Range 0~127,0.25dbm/step.
		Path A 5G BW40-1S Power Index for Ch 173,175,177, Range 0~127,0.25dbm/step.
40	5G Difference	Power Index Difference between BW20-1S and BW40-1S.
		Bit[7:4]: Path A 5G Offset, Range -8~7,0.5dbm/step.
		Power Index Difference between OFDM-1Tx and BW40-1S.
		Bit[3:0]: Path A 5G Offset, Range -8~7,0.5dbm/step.

Fig 13-6 5G Wi-Fi power index specification

13.5.4 Wi-Fi Channel Plan

The address of Wi-Fi channel plan is shown in Fig 13-7.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-7 Channel plan address

eFuse	Name	Bit	Default	Comment
C8	Channel Plan	[7]	0h	Software configure mode 0h: Enable software configure (refer to Channel Plane Domain Code) 1h: Disable software configure (can't change Channel Plan Setting)
		[6:0]	7Fh	Channel Plan 0x20: 2G Worldwide 13, Active scan Ch1 ~ Ch11, Passive scan Ch12, Ch13 0x7F: 2G Worldwide 13, Active scan Ch1 ~ Ch11, Passive scan Ch12, Ch13 5G Worldwide, all bands are Passive scan.

Frequently-used country codes and the corresponding channel plans are listed in Table 13-3, Table 13-4 and Table 13-5.

Table 13-3 Relationship between country code and channel plan

Country	Abbreviation	Enumeration Variable Name	Channel Plan
America	US	RTW_COUNTRY_US	0x76
Australia	AS	RTW_COUNTRY_AS	0x2A
Chile	CL	RTW_COUNTRY_CL	0x2D
China	CN	RTW_COUNTRY_CN	0x48
Europe	EU	RTW_COUNTRY_UA	0x26
Japan	JP	RTW_COUNTRY_JP	0x27
Mexico	MX	RTW_COUNTRY_MX	0x4D
Ukraine	UA	RTW_COUNTRY_UA	0x35
Default	-	RT_CHANNEL_DOMAIN_REALTEK_DEFINE	0x7F

Table 13-4 2.4G channel map of the above channel plan

Channel Plan	2.4G Channel Definition	Channels	Passive Scan
0x76	2G_02	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	-
0x2A	2G_02	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	-
0x2D	2G_01	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	12,13
0x48	2G_01	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	12,13
0x26	2G_01	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	12,13
0x27	2G_04	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14	-
0x4D	2G_02	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	-
0x35	2G_01	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	12,13
0x7F	2G_01	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13	12,13

Note: Active scan channels are channels listed in table except the passive scan channels.

Table 13-5 5G channel map of the above channel plan

Channel Plan	5G Channel Definition	Channels	Passive Scan	DFS Channels
0x76	5G_22	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144 149, 153, 157, 161, 165	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144
0x2A	5G_00	-	-	-
0x2D	5G_22	36, 40, 44, 48	52, 56, 60, 64	52, 56, 60, 64

		52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144 149, 153, 157, 161, 165	100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144	100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144
0x48	5G_07	36, 40, 44, 48 52, 56, 60, 64 149, 153, 157, 161, 165	52, 56, 60, 64	52, 56, 60, 64
0x26	5G_02	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140
0x27	5G_02	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140
0x4D	5G_01	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 132, 136, 140 149, 153, 157, 161, 165	52, 56, 60, 64 100, 104, 108, 112, 116, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 132, 136, 140
0x35	5G_03	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140 149, 153, 157, 161, 165	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140
0x7F	5G_36	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144 149, 153, 157, 161, 165	36, 40, 44, 48 52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144 149, 153, 157, 161, 165	52, 56, 60, 64 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144

Note: Active scan channels are channels listed in table except the passive scan channels.

13.5.5 Wi-Fi Crystal Calibration

The address of Wi-Fi crystal calibration is shown in Fig 13-8.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-8 Crystal Calibration address

eFuse	Name	Bit	Default	Comment
C9	Crystal Calibration	[7]	0h	Reserved
		[6:0]	40h	XTAL_K Value Xi=Xo, range: 0~7Fh FFh = 00h

13.5.6 Wi-Fi Thermal Meter

The address of Wi-Fi thermal meter is shown in Fig 13-9.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-9 Thermal Meter address

eFuse	Name	Bit	Default	Comment
CA	Thermal Meter	[7:0]	16h	Thermal Meter Value System maker will calibrate a value and save it in EEPROM. 0xFF: Disable Tx power tracking function

13.5.7 Wi-Fi MAC Address

The Wi-Fi MAC address is shown in Fig 13-10.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
110	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

Fig 13-10 MAC Address

Address Offset	Bit	Default	Comment
11A-11F	[47:0]	00E04C872101	After the auto-load command or hardware reset, Ameba-D loads MAC addresses to MACID of the I/O registers.

13.5.8 Cap-Touch

Address Offset	Bit	Default	Description
180h	[7:0]	0xFF	Touch key channel 0 threshold[7:0]
181h	[6:0]	0xFF	Touch key channel 1 threshold diff[6:0]
	[7]	0xFF	Touch key channel 0 threshold[8]
182h	[6:0]	0xFF	Touch key channel 2 threshold diff[6:0]
	[7]	0xFF	Touch key channel 0 threshold[9]
183h	[6:0]	0xFF	Touch key channel 3 threshold diff[6:0]
	[7]	0xFF	Touch key channel 0 threshold[10]

13.5.9 BLE

Address Offset	Bit	Default	Description
190 ~ 195h	[47:0]	0xFFFFFFFFFFFF	BT address
196h	[7:0]	0x08	- Bit[0]: Fixed to 0 - Bit[1]: Tx gain K valid bit - Bit[2]: Flatness K valid bit - Bit[3]: Fixed to 1 - Bit[7:4]: Fixed to 0 Note: After Tx gain K flow, remember to enable Tx gain K valid bit, then 0x196 will be 0x0A.
197h	[7:0]	0xFF	Tx gain K Note: Users need to write Tx gain K value if Tx gain K valid bit is enabled.
198 ~ 199h	[15:0]	0xFFFF	Flatness K
19A ~ 19Bh	[15:0]	0xFFFF	Reserved for Realtek
19Ch	[7:0]	0x23	Max. Tx gain LE1M iqm_max_txgain_LE1M
19Dh	[7:0]	0x23	Max. Tx gain LE2M iqm_max_txgain_LE2M
19E ~ 19Fh	[15:0]	0xFFFF	Reserved for Realtek

1A0h	[7:0]	0xFF	BT thermal meter tmeterx4_txgaink_module
1A1h	[7:0]	0xFF	Logic map IQK/KOK valid bit 0xFF: Disable logic map IQK/KOK values 0xFE: Enable logic map IQK/KOK values
1A6h ~ 1A9h	[31:0]	0xFFFFFFFF	Logic map IQK values
1AAh ~ 1ABh	[15:0]	0xFFFF	Logic map KOK values

13.5.10 HCI USB

Address Offset	Bit	Name
1C0h	[7:0]	VID[7:0]
1C1h	[7:0]	VID[15:8]
1C2h	[7:0]	PID[7:0]
1C3h	[7:0]	PID[15:8]
1C4h	[7:0]	USB device type
1C5 ~ 1CFh		RSVD

13.6 eFuse PG APIs

Items	API	Comment
Low Level API	EFUSE_PMAP_WRITE8	Physical map writes one byte
	EFUSE_PMAP_READ8	Physical map reads one byte
	EFUSE_LMAP_WRITE	Write eFuse logical address by length
	EFUSE_LMAP_READ	Read total eFuse logical map
mbed API	efuse_otp_write	Write content to OTP eFuse by length
	efuse_otp_read	Read eFuse OTP content by length
	efuse_mtp_write	Write user's content to USER MTP Space (0x160~0x17F)
	efuse_mtp_read	Read eFuse content from address (0x160 ~ 0x17F)

13.6.1 Low Level APIs

13.6.1.1 EFUSE_PMAP_WRITE8

Items	Description
Introduction	Physical map writes one byte.
Parameters	- CtrlSetting: eFuse control setting read from REG_LP_EFUSE_CTRL1 (suggest 0) - Addr: eFuse physical address - Data: 1 byte data to write - L25OutVoltage: L25EOUTVOLTAGE.
Return	Status value: _TRUE: Write is ok. _FALSE: Write is failed.

13.6.1.2 EFUSE_PMAP_READ8

Items	Description
Introduction	Physical map reads one byte.
Parameters	- CtrlSetting: eFuse control setting read from REG_LP_EFUSE_CTRL1 (suggest 0) - Addr: eFuse physical address - Data: 1 byte data buffer for eFuse data read

	● L25OutVoltage: L25EOUTVOLTAGE.
Return	Status value: ● _TRUE: Read is ok. ● _FALSE: Read is failed.

13.6.1.3 EFUSE_LMAP_WRITE

Items	Description
Introduction	Write eFuse logical address by length
Parameters	● addr: logical address, should be word-aligned ● cnts: byte number, should be even ● data: data buffer to be write
Return	Status value: ● _SUCCESS: Write is ok. ● _FAIL: Write is failed.

13.6.1.4 EFUSE_LMAP_READ

Items	Description
Introduction	Read total eFuse logical map
Parameters	pbuf: 1024 bytes length buffer used for eFuse Logical map
Return	Status value: ● _SUCCESS: Read is ok. ● _FALSE: Read is failed.

13.6.2 Mbed APIs

13.6.2.1 efuse_otp_write

Items	Description
Introduction	Write content to OTP eFuse by length (mbed API)
Parameters	● address: Specifies the offset of the programmed OTP. ● len: Specifies the data length of programmed data. ● buf: Specified the data to be programmed.
Return	Status value: ● 0: Success ● -1: Failure

13.6.2.2 efuse_otp_read

Items	Description
Introduction	Read eFuse OTP content by length (mbed API)
Parameters	● address: Specifies the offset of the OTP. ● len: Specifies the length of readback data. ● buf: Specified the address to save the readback data.
Return	Status value: ● 0: Success ● -1: Failure

13.6.2.3 efuse_mtp_write

Items	Description
-------	-------------

Introduction	Write user's content to USER MTP Space (0x160~0x17F)
Parameters	- data: Specified the data to write - len: Specified the data length to write
Return	Status value: 0 ~ 32: Success 0 or -1: Failure

13.6.2.4 efuse_mtp_read

Items	Description
Introduction	Read eFuse content from address (0x160 ~ 0x17F)
Parameters	Data: Specified the address to save the read back data
Return	N/A

13.7 eFuse PG Command

Items	Offset	Command
Tx Power Index	0x20~0x2B	iwpriv config_set wmap, 020, 202020202020202020202002
Channel plan, XTAL & Thermal & RTK reserved	0xC8~0xCF	iwpriv config_set wmap, 0C8, 20201A05000000FF ¹
MAC Address	0x11A~0x11F	iwpriv config_set wmap, 11a, 00e04c870102
Realtek RSVD	0x130~0x139	iwpriv config_set wmap, 130, FF01001000FF00FF1000
Get all eFuse map		iwpriv config_get realmap

- Channel plan does not affect the results of RF verification, so you can choose whether to write the correct channel plan here.

14 Power Save

14.1 Power Save Mode

14.1.1 Summary

Ameba-D supports two low power modes which are deepsleep mode and sleep mode. Deepsleep mode turns off more power domain than sleep mode so it has lower power consumption. Tickless is a FreeRTOS low power feature, which just halt CPU (no clock or power be turned off) when it has nothing to do. Table 14-1 explains power save related terms.

Table 14-1 Power save mode

Name	Domain	Description
Tickless	Software	FreeRTOS low power feature
Sleep mode	Chip-level	A power save mode on chip-level
Deepsleep mode	Chip-level	A more power save mode on chip-level

NOTE

configUSE_TICKLESS_IDLE must be enabled in sleep mode because sleep mode is based on tickless.

There are various of sources, and all wake-up source is to wake KM0. Then KM0 wakes KM4 as appropriate. Refer to Fig 14-1 of document UM0400 which show power of peripheral is on or off when they are in sleep mode or deepsleep mode by different colors.

Fig 14-1 shows all the wake-up sources. HS and AON are special because they are mater switch that manages some wake-up sources. HS UART is peripheral belongs to KM4. Cap-Touch, RTC, etc. belong to AON domain.

Peripherals in AON domain can wake system both in sleep mode and deepsleep mode.

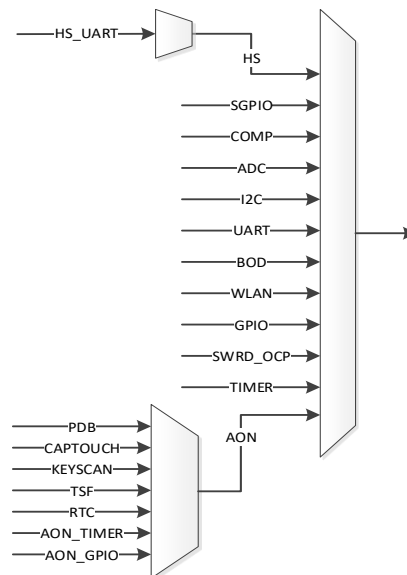


Fig 14-1 Wake sources

14.1.2 Sleep Mode

14.1.2.1 Wakelock

In some situations, system needs to keep awake to receive certain events. Otherwise, event may be missed when the system is under sleep. An idea of wakelock is introduced that system cannot sleep if some module is holding wakelock.

A wakelock bit map is used to store the wakelock status. Each module has its own bit in wakelock bit map (see enum. PMU_DEVICE). Users can also add wakelock in the enum. PMU_DEVICE. If the wakelock bit map equals zero, it means that there is no module holding wakelock. If the wakelock bit map is larger than zero, it means that there is some module holding wakelock.

Wakelock is a judging condition in function `freertos_ready_to_sleep()`. When system boots on, KM4 holds wakelock PMU_OS and KM0 holds PMU_KM4_RUN and PMU_OS wakelock. Only if all wakelocks are released, KM0 or KM4 are permitted to enter sleep mode. Fig 14-2 shows function `freertos_ready_to_sleep()` and it will judge the value of wakelock.

```
int freertos_ready_to_sleep(void)
{
    u32 current_tick = xTaskGetTickCount();

    /* timeout */
    if (current_tick < sleepwakelock_timeout) {
        return FALSE;
    }

    if (wakelock == 0) {           // return TRUE only if all wakelock release
#ifdef (ARM_CORE_CM0)
        /* timeout */
        if (current_tick >= km4_sleep_timeout) {
            return FALSE;
        } else {
            if (km4_sleep_timeout != 0xffffffff) {
                SOCPS_AONTimer(km4_sleep_timeout - current_tick);
                SOCPS_SetWakeEventAON(BIT_AON_WAKE_TIMO_MSK, ENABLE);
                SOCPS_AONTimerCmd(ENABLE);
            }
        }
    }
#endif
    return TRUE;
}
else
    return FALSE;
} ? end freertos_ready_to_sleep ?
```

Fig 14-2 Function `freertos_ready_to_sleep()`

It is recommended to enter sleep mode by release wakelock. After PMU_OS wakelock of KM4 release, KM4 will enter sleep mode in an idle task and send IPC to KM0. KM0 will power gate or clock gate KM4 and then release PMU_OS and PMU_KM4_RUN. The judgment of wakelock in function `freertos_ready_to_sleep()` will be the value TRUE, so KM0 can clock gate itself in its idle task.

When the system wakeup, it will enter sleep mode again quickly unless it acquires wakelock.

The definition of related functions is shown in section 14.2.

14.1.2.2 Suspend and Resume Function

It is a good way to use suspend and resume function If there is something to do before chip sleep or after chip wake up. The suspend and resume functions are executed respectively before CPU enters sleep mode and after CPU wakes up in an idle task.

A typical application of resume function is to acquire the wakelock to prevent chip sleep again. Also, If KM4 chooses PG, some peripheral will lose power so they need to reinitialize. It can be implemented in the resume function.

A typical application of suspend function is WoWLAN. Refer to section Wi-Fi Power Save to get more information.

The definition of related functions is shown in section 14.2.

14.1.2.3 Wi-Fi Power Save

In IEEE 802.11 power save management, it allows the station to enter its own sleep state. It defines that station needs to keep awake in a certain timestamp and enters sleep state otherwise.

WLAN driver acquires wakelock to avoid the system enter sleep mode when WLAN needs to keep awake. And it releases wakelock when it is permitted to enter sleep state.

IEEE 802.11 power management allows station entering power save mode. Station cannot receive any frame during power saving. Thus AP needs to buffer these frames and requires station periodically wakeup to check beacon which has the information of buffered frames.

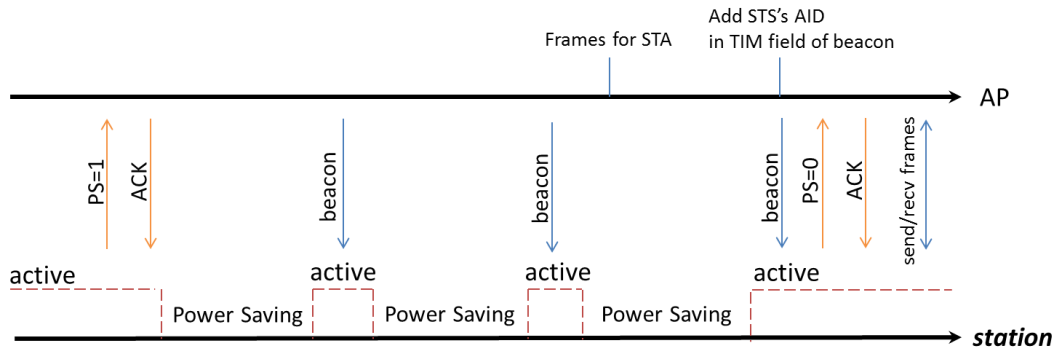


Fig 14-3 Timeline of power save

In SDK IEEE 802.11 power management is called LPS, and if KM4 enters sleep mode when Wi-Fi is in LPS mode, we call it WoWLAN mode.

In WoWLAN mode, a timer with a period of about 102ms will be set in the suspend function, and KM0 will wake up every 102ms to receive the beacon to maintain the connection.

Except LPS and WoWLAN we also have IPS, which can be used when Wi-Fi is not connected. Table 14-2 lists all three power save mode for Wi-Fi.

Table 14-2 Wi-Fi power save mode

Items	Wi-Fi Status	Comment
IPS	Not associated	Driver automatically turns off Wi-Fi to save power.
LPS	Associated	LPS is used to implement IEEE 802.11 power management. KM0 will control RF ON/OFF based on TSF and TIM IE in beacon.
WoWLAN	Associated	LPS + Tickless KM0 will wakeup KM4 when receiving data packet.

14.1.2.4 Wakeup Time

CPU can execute IRQ handler considering that system is wakeup. It takes about 2.8ms to wake KM0 only and 3.5ms to wake both KM0 and KM4 in KM0 CG + KM4 PG mode. For KM0, 1.6ms to initialize hardware (mainly XTAL), 1.1ms to execute critical code, and 200-300us to enter IRQ handler.

14.1.3 Sleep Mode Configuration

14.1.3.1 Wakeup Source Setup

Table 14-3 Sleep wakeup source

Wakeup source	Description	Default status
BIT_LP_WEVT_HS_MSK	1: enable HS wakeup event	OFF

	0: disable the event to wake up the system	
BIT_LP_WEVT_AON_MSK	1: enable AON wakeup event 0: disable the event to wake up the system i NOTE <i>It is a master switch for all AON wakeup sources. Ameba-D has 6 AON wakeup sources, listed in Table 14-4.</i>	ON
BIT_LP_WEVT_SGPIO_MSK	1: enable SGPIO wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_COMP_MSK	1: enable comparator wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_ADC_MSK	1: enable ADC wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_I2C_MSK	1: enable I2C wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_UART_MSK	1: enable UART wakeup event 0: disable the event to wake up the system	ON
BIT_LP_WEVT_BOD_MSK	1: enable BOD wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_WLAN_MSK	1: enable WLAN wakeup event 0: disable the event to wake up the system	ON
BIT_LP_WEVT_GPIO_MSK	1: enable GPIO wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_SWRD_OCP_MSK	1: enable DCORE SWR OCP wakeup event 0: disable the event to wake up the system	OFF
BIT_LP_WEVT_TIMER_MSK	1: enable timer wakeup LP event 0: disable the event to wake up the system	OFF

Table 14-4 Sleep AON wakeup sources for BIT_LP_WEVT_AON_MSK

Wakeup source	Description	Default status
BIT_CHIP_PDB_MSK	1: Enable chip power-down wake	ON
BIT_CAPTOUCH_WAKE_MSK	1: Enable Cap-Touch wake	ON
BIT_KEYSCAN_WAKE_MSK	1: enable Key-Scan wake	ON
BIT_DLPS_TSF_WAKE_MSK	1: enable TSF wake under deep-lps mode	OFF
BIT_RTC_WAKE_MSK	1: enable RTC wake	OFF
BIT_AON_WAKE_TIMO_MSK	1: enable AON timer wake	ON
BIT_GPIO_WAKE_MSK	1: enable GPIO wake	ON

To enable a specific wakeup source, the corresponding status in array sleep_wevent_config[] in rtl8721dnp_sleepcfg.c should be set firstly, as shown in Fig 14-4.

```

/* if X can wakeup dsleep, it can wakeup dstandby & sleep */
/* if X can wakeup dstandby, it can wakeup sleep */
FWRCFG_TypeDef sleep_wevent_config[]=
{
    //      Module                      Status
    {BIT_LP_WEVT_HS_MSK,                OFF},
    {BIT_LP_WEVT_AON_MSK,                ON},
    {BIT_LP_WEVT_SGPIO_MSK,              OFF},
    {BIT_LP_WEVT_COMP_MSK,               OFF},
    {BIT_LP_WEVT_ADC_MSK,                OFF},
    {BIT_LP_WEVT_I2C_MSK,                OFF},
    {BIT_LP_WEVT_UART_MSK,               ON},
    {BIT_LP_WEVT_BOD_MSK,                OFF},
    {BIT_LP_WEVT_WLAN_MSK,               ON},
    {BIT_LP_WEVT_GPIO_MSK,               OFF},
    {BIT_LP_WEVT_SWRD_OCP_MSK,           OFF},
    {BIT_LP_WEVT_TIMER_MSK,              OFF},
    {0xFFFFFFFF,                        OFF}, /* Table end */
};

```

Fig 14-4 Sleep wake source setup

If the AON wakeup event is chosen as the wakeup source, a specific AON wakeup source should be set in array `sleep_aon_wevent_config[]` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-5.

```
FWRCFG_TypeDef sleep_aon_wevent_config[] =
{
    //      Module      Status
    {BIT_CHIP_PDB_MSK,    ON},    /* [7] R/W 0    1: Indicate chip power-down */
    {BIT_CAPTOUCH_WAKE_MSK, ON},    /* [6] R/W 0    1: Indicate captouch wake event */
    {BIT_KEYSCAN_WAKE_MSK, ON},    /* [4] R/W 0    1: Indicate keyscan wake */
    {BIT_DLPS_TSF_WAKE_MSK, OFF},    /* [3] R/W 0    1: Indicate tsf wake under deep-lps mode */
    {BIT_RTC_WAKE_MSK,    OFF},    /* [2] R/W 0    1: Indicate RTC wake */
    {BIT_AON_WAKE_TIMO_MSK, ON},    /* [1] R/W 0    1: Indicate AON timer wake */
    {BIT_GPIO_WAKE_MSK,   ON},    /* [0] R/W 0    1: Indicate GPIO wake, see aon_wakepin & dsleep_wakepin_config */

    {0xFFFFFFFF,          OFF},    /* Table end */
};
```

Fig 14-5 Sleep AON wake source setup

The wakeup sources mentioned above are for KM0. You can choose if waking KM4 system up at the same time according to Table 14-5.

Table 14-5 Wakeup sources for KM4

Wakeup source	Description	Default status
BIT_LP_WEVT_HS_STS	1: Indicates HS wakeup event	ON
BIT_LP_WEVT_AON_STS	1: Indicates AON wakeup event	OFF
BIT_LP_WEVT_SGPIO_STS	1: Indicates SGPIO wakeup event	OFF
BIT_LP_WEVT_COMP_STS	1: Indicates Comparator wakeup event	OFF
BIT_LP_WEVT_ADC_STS	1: Indicates ADC wakeup event	OFF
BIT_LP_WEVT_I2C_STS	1: Indicates I2C wakeup event	OFF
BIT_LP_WEVT_UART_STS	1: Indicates UART wakeup event	OFF
BIT_LP_WEVT_BOD_STS	1: Indicates BOD wakeup event	OFF
BIT_LP_WEVT_WLAN_STS	1: Indicates WLAN wakeup event	OFF
BIT_LP_WEVT_GPIO_STS	1: Indicates GPIO wakeup event	OFF
BIT_LP_WEVT_SWRD_OCP_STS	1: Indicates DCORE SWR OCP wakeup event	OFF
BIT_LP_WEVT_TIMER_STS	1: Indicates timer wakeup LP event	OFF
BIT_CHIP_PDB_STS	1: Indicates chip power-down	OFF
BIT_CAPTOUCH_WAKE_STS	1: Indicates Cap-Touch wake event	ON
BIT_KEYSCAN_WAKE_STS	1: Indicates Key-Scan wake	ON
BIT_DLPS_TSF_WAKE_STS	1: Indicates TSF wake for Wi-Fi FW	OFF
BIT_RTC_WAKE_STS	1: Indicates RTC wake	ON
BIT_AON_WAKE_TIMO_STS	1: Indicates AON Timer wake	OFF
BIT_GPIO_WAKE_STS	1: Indicates AON wakepin wake	ON

To enable a specific wakeup source for KM4, corresponding status in array `hs_wakeevent_config []` in `rtl8721dlp_sleepcfg.c` should be set firstly, as shown in Fig 14-6.

```

HSWAKEEVENT_TypeDef hs_wakeevent_config[] =
{
    //
    // Module          Event          Status
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_HS_STS, ON), /* [30] 1: Indicate HS Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_AON_STS, OFF), /* [29] 1: Indicate AON Wakeup event (0x128) */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_SGPIO_STS, OFF), /* [28] 1: Indicate SGPIO Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_COMP_STS, OFF), /* [27] 1: Indicate Comparator Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_ADC_STS, OFF), /* [26] 1: Indicate ADC Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_I2C_STS, OFF), /* [24] 1: Indicate I2C Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_UART_STS, OFF), /* [20] 1: Indicate UART Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_BOD_STS, OFF), /* [6] 1: Indicate BOD Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_WLAN_STS, OFF), /* [5] 1: Indicate WLAN Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_GPIO_STS, OFF), /* [4] 1: Indicate GPIO Wakeup event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_SWRD_OCP_STS, OFF), /* [2] 1: Indicate DCORE SWR OCP event */
    (REG_LP_SLP_WAKE_EVENT_STATUS0, BIT_LP_WEVT_TIMER_STS, OFF), /* [1] 1: Indicate GTimer Wakeup system event; */

    (REG_AON_WAKE_OPT_STS, BIT_CHIP_PDB_STS, OFF), /* [7] 1: Indicate chip power-down */
    (REG_AON_WAKE_OPT_STS, BIT_CAPTOUCH_WAKE_STS, ON), /* [6] 1: Indicate captouch wake event */
    (REG_AON_WAKE_OPT_STS, BIT_KEYSCAN_WAKE_STS, ON), /* [4] 1: Indicate keyscan wake */
    (REG_AON_WAKE_OPT_STS, BIT_DLPS_TSF_WAKE_STS, OFF), /* [3] 1: Indicate tsf wake under deep-lps mode */
    (REG_AON_WAKE_OPT_STS, BIT_RTC_WAKE_STS, ON), /* [2] 1: Indicate RTC wake */
    (REG_AON_WAKE_OPT_STS, BIT_AON_WAKE_TIMER_STS, OFF), /* [1] 1: Indicate AON timer wake */
    (REG_AON_WAKE_OPT_STS, BIT_GPIO_WAKE_STS, ON), /* [0] 1: Indicate GPIO wake, see aon_wakepin & dsleep_wakepin_config */

    {0xFFFFFFFF, OFF, 0}, /* Table end */
};

```

Fig 14-6 Sleep wake source setup for KM4

NOTE

The power of KM4 is controlled by KM0, KM0 will decide whether to wakeup KM4 according to the wakeup source and the status of the wakeup source in array `hs_wakeevent_config[]`.

Some peripherals like HS UART need ANA4M, and it can be enabled in `km0_pwrmtg_config[]` in `rtl8721dip_sleepcfg.c`, as shown in Fig 14-7.

```

PWRCFG_TypeDef km0_pwrmtg_config[] =
{
    //
    // Module          Status
    {BIT_LSYS_PST_SLEP_EACK, OFF}, /*BIT1: 1 Enable ANA4M CLK when PMC enter into sleep mode
    {BIT_LSYS_PST_SLEP_EBUS, OFF}, /*BIT2: 1 Enable platform clock when PMC entro into sleep mode
    {BIT_LSYS_PST_SLEP_EMPM, OFF}, /*BIT3: 1 means RAM can not enter low power mode, 0 can enter,
    {BIT_LSYS_PST_SLEP_LDLM, OFF}, /*BIT4 0 LPSDO enter sleep mode
    {BIT_LSYS_PST_SLEP_ERCK, OFF}, /*BIT5 0 gate ls system clock
    {BIT_LSYS_PST_SLEP_DPSW, ON}, /*BIT6: 1: disable power switch and use SWR mode, 0: enable po
    {BIT_LSYS_PST_SLEP_EPWM, OFF}, /*BIT7: 1 LDO mode: 1.1V or SWR PWM mode, 0 LDO mode 0.9v or S
    {BIT_LSYS_PST_SLEP_ESWR, ON}, /*BIT8: 0 is disable SWR when enter sleep
    {BIT_LSYS_PST_SLEP_EXTL, OFF}, /*BIT9 0 disable XTAL when sleep, should conflict with BIT_LS
    {BIT_LSYS_PST_SLEP_XACT, ON}, /*BIT10 based on BIT_LSYS_PST_SLEP_EXTL

    {0xFFFFFFFF, OFF}, /* Table end */
};

```

Fig 14-7 Power management configuration

Some peripherals also need clock OSC2M in sleep mode. Whether to close clock OSC2M when the system is in sleep mode is determined by `km0_osc2m_close` in structure `ps_config` in `rtl8721dip_sleepcfg.c`, as shown in Fig 14-8. Value TRUE means it will close clock OSC2M in sleep mode.

```

PSCFG_TypeDef ps_config = {
    .km0_config_wifi_enable = TRUE,
    .km0_enable_key_touch = FALSE, /*BIT_KEY_ENABLE | BIT_CAPTOUCH_ENABLE,
    .km0_tickles_debug = FALSE, /* if open WIFI FW, should close it, or beacon will lost in WOWLAN */
    .km0_osc2m_close = TRUE,
    .km0_pg_enable = FALSE,
    .km0_rtc_calibration = FALSE,
    .km0_audio_pad_enable = TRUE,
};

```

Fig 14-8 Control of clock OSC2M in sleep mode

14.1.3.1.1 HS Wakeup Event (HS UART)

HS wakeup event includes wakeup sources HS UART (UART0). When using HS wakeup event as the wakeup source, clock ANA4M should not be closed. So `BIT_LSYS_PST_SLEP_EACK` should set to ON as shown in Fig 14-7.

When using UART0 as wakeup source, clock OSC2M. So `km0_osc2m_close` in Fig 14-8 should equal FALSE.

The following steps show how to configure HS UART in sleep mode.

- (1) Set `km0_osc2m_close = FALSE` in structure `ps_config` as shown in Fig 14-8.
- (2) Set `BIT_LSYS_PST_SLEEP_EACK` to ON in structure `km0_pwrmtg_config` as shown in Fig 14-7.
- (3) Initialize UART and enable its interrupt.
- (4) Switch UART to low power Rx mode and switch clock source to OSC2M. If using UART mbed API (`serial_api.c`), Set corresponding `LOW_POWER_RX_ENABLE` item to `ENABLE` in structure `uart_config[]` as shown in Fig 14-9 and then it will switch automatically in function `serial_baud()`. If using low level API, please do it manually as shown in Fig 14-10.

```
UARTCFG_TypeDef uart_config[4]=
{
    /* HS UART--> UART0 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE, /*to enable low power RX*/
    },
    /* BT UART--> UART1 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE,
    },
    /* Log UART--> UART2 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE,
    },
    /* LPUART-->UART3 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE,
    },
};
```

Fig 14-9 Configure low power RX mode for UART mbed API

```
UART_MonitorParaConfig(UART0_DEV, 100, ENABLE);
UART_RxMonitorCmd(UART0_DEV, ENABLE); //low power rx monitor
RCC_PeriphClockSource_UART(UART0_DEV, UART_RX_CLK_OSC_LP); //switch clock
UART_LPRxBaudSet(UART0_DEV, baudrate, 2000000); //low power rx mode
UART_RxCmd(UART0_DEV, ENABLE);
```

Fig 14-10 Configure low power RX mode manually for UART low level API

- (5) Set `BIT_LP_WEVT_HS_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (6) Set `BIT_LP_WEVT_HS_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.
- (7) Set HS UART wake event on KM4 platform before sleep by using "`SOCPS_SetWakeEvent_HP(BIT_HS_WEVT_UART_MSK, 1);`"

14.1.3.1.2 AON

AON wakeup event includes PDB, Cap-Touch, Key-Scan, RTC, AON timer and GPIO.

PDB

- (1) Change PMC power down direction to SW by register `AON_PM_OPT`.
- (2) Enable the interrupt of PDB by register `AON_CHIP_PWR_DOWN_MSK`.
- (3) Set `BIT_LP_WEVT_AON_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4 and set `BIT_CAPTOUCH_WAKE_MSK` status ON in array `sleep_aon_wevent_config[]` as shown in Fig 14-5.
- (4) Set `BIT_LP_WEVT_AON_STS` and `BIT_CHIP_PDB_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wakeup KM4 at the same time.

Cap-Touch

- (1) Initialize Cap-Touch and enable its interrupts.
- (2) Set `BIT_LP_WEVT_AON_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4 and set `BIT_CHIP_PDB_MSK` status ON in array `sleep_aon_wevent_config[]` as shown in Fig 14-5.
- (3) Set `BIT_LP_WEVT_AON_STS` and `BIT_CAPTOUCH_WAKE_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.

NOTE

Refer to code '`touch_key.c`' for Cap-Touch initialization.

Key-Scan

- (1) Initialize Key-Scan and enable its interrupts.
- (2) Set BIT_LP_WEVT_AON_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4 and set BIT_KEYSCAN_WAKE_MSK status ON in array sleep_aon_wevent_config[] as shown in Fig 14-5.
- (3) Set BIT_LP_WEVT_AON_STS and BIT_KEYSCAN_WAKE_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

NOTE

Refer to code 'touch_key.c' for Key-Scan initialization.

RTC

- (1) Initialize RTC and enable its interrupts.
- (2) Set BIT_LP_WEVT_AON_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4 and set BIT_RTC_WAKE_MSK status ON in array sleep_aon_wevent_config[] as shown in Fig 14-5.
- (3) Set BIT_LP_WEVT_AON_STS and BIT_RTC_WAKE_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

AON Timer

- (1) Enable AON Timer and set the time to wake up system.
- (2) Set BIT_LP_WEVT_AON_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4 and set BIT_AON_WAKE_TIM0_MSK status ON in array sleep_aon_wevent_config[] as shown in Fig 14-5.
- (3) Set BIT_LP_WEVT_AON_STS and BIT_AON_WAKE_TIM0_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

NOTE

The AON Timer is in imprecise milliseconds. Its maximum value is 32760000ms (546mins).

GPIO (Wakepin)

- (1) The setup of GPIO is shown in 14.1.5.1.5.
- (2) Set BIT_LP_WEVT_AON_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4 and set BIT_GPIO_WAKE_MSK status ON in array sleep_aon_wevent_config[] as shown in Fig 14-5.
- (3) Set BIT_LP_WEVT_AON_STS and BIT_GPIO_WAKE_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.3 SGPIO

When using SGPIO as wakeup source, clock OSC2M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by km0_osc2m_close in structure ps_config in rtl8721dlp_sleepcfg.c, as shown in Fig 14-8.

- (1) Set km0_osc2m_close = FALSE in structure ps_config as shown in Fig 14-8.
- (2) Initialize SGPIO and enable its interrupt.
- (3) Set BIT_LP_WEVT_SGPIO_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4.
- (4) Set BIT_LP_WEVT_SGPIO_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.
- (5) Switch SGPIO clock to OSC2M by register REG_SYS_CLK_CTRL1 before entering sleep mode.

14.1.3.1.4 Comparator

When using Comparator as wakeup source, clock OSC2M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by km0_osc2m_close in structure ps_config in rtl8721dlp_sleepcfg.c, as shown in Fig 14-8.

- (1) Set km0_osc2m_close = FALSE in structure ps_config as shown in Fig 14-8.
- (2) Initialize ADC and comparator, and enable corresponding interrupt.
- (3) Set BIT_LP_WEVT_COMP_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4.
- (4) Set BIT_LP_WEVT_COMP_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.
- (5) Switch ADC clock to OSC2M by register REG_SYS_CLK_CTRL1 before entering sleep mode.

14.1.3.1.5 ADC

When using ADC as wakeup source, clock OSC2M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by `km0_osc2m_close` in structure `ps_config` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-8.

- (1) Set `km0_osc2m_close = FALSE` in structure `ps_config` as shown in Fig 14-8.
- (2) Initialize ADC and enable its interrupt.
- (3) Set `BIT_LP_WEVT_ADC_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (4) Set `BIT_LP_WEVT_ADC_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.
- (5) Switch ADC clock to OSC2M by register `REG_SYS_CLK_CTRL1` before entering sleep mode.

14.1.3.1.6 I2C

When using I2C as wakeup source, clock OSC2M and ANA4M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by `km0_osc2m_close` in structure `ps_config` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-8, while clock ANA4M can be configured in `km0_pwrmtg_config[]` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-7.

- (1) Set `km0_osc2m_close = FALSE` in structure `ps_config` as shown in Fig 14-8, and set `BIT_LSYS_PST_SLEP_EACK` status ON in array `km0_pwrmtg_config[]`, as shown in Fig 14-7.
- (2) Initialize I2C and enable its interrupt.
- (3) Set `BIT_LP_WEVT_I2C_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (4) Set `BIT_LP_WEVT_I2C_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.
- (5) Switch I2C clock to OSC2M by calling `RCC_PeriphClockSource_I2C()` before entering sleep mode.

14.1.3.1.7 LP UART

When using LP UART (UART3) as wakeup source, clock OSC2M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by `km0_osc2m_close` in structure `ps_config` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-8.

- (1) Set `km0_osc2m_close = FALSE` in structure `ps_config` as shown in Fig 14-8.
- (2) Initialize UART and enable its interrupt.
- (3) Switch UART to low power Rx mode and switch clock source to OSC2M. If using UART mbed API (`serial_api.c`), Set corresponding `LOW_POWER_RX_ENABLE` item to `ENABLE` in structure `uart_config[]` as shown in Fig 14-9 and then it will switch automatically in function `serial_baud()`. If using low level API, please do it manually as shown in Fig 14-10.
- (4) Set `BIT_LP_WEVT_UART_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (5) Set `BIT_LP_WEVT_UART_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.8 Log UART

When using Log UART (UART2) as wakeup source, clock OSC2M shouldn't be closed when system is in sleep mode. Whether to close clock OSC2M when system is in sleep mode is decided by `km0_osc2m_close` in structure `ps_config` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-8.

- (1) Set `km0_osc2m_close = FALSE` in structure `ps_config` as shown in Fig 14-8.
- (2) Initialize UART and enable its interrupt.
- (3) Set `BIT_LP_WEVT_UART_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (4) Set `BIT_LP_WEVT_UART_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.9 BOD

- (1) Set BOD threshold and enable its interrupt.
- (2) Set `BIT_LP_WEVT_BOD_MSK` status ON in array `sleep_wevent_config[]` as shown in Fig 14-4.
- (3) Set `BIT_LP_WEVT_BOD_STS` status ON in array `hs_wakeevent_config []` as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.10 WLAN

- (1) Set BIT_LP_WEVT_WLAN_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4.
- (2) Set BIT_LP_WEVT_WLAN_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.11 GPIO

- (1) Initialize a GPIO and enable its interrupt.
- (2) Set BIT_LP_WEVT_GPIO_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4.
- (3) Set BIT_LP_WEVT_GPIO_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

14.1.3.1.12 LP Timer

When using timer in KM0 as wakeup source, clock ANA4M shouldn't be closed when system is in sleep mode. Whether to close clock ANA4M can be configured in km0_pwrmtg_config[] in rtl8721dlp_sleepcfg.c, as shown in Fig 14-7.

- (1) Set BIT_LSYS_PST_SLEP_EACK status ON in array km0_pwrmtg_config[], as shown in Fig 14-7.
- (2) Initialize the timer and enable its interrupt in KM0 platform.
- (3) Set BIT_LP_WEVT_TIMER_MSK status ON in array sleep_wevent_config[] as shown in Fig 14-4.
- (4) Set BIT_LP_WEVT_TIMER_STS status ON in array hs_wakeevent_config [] as shown in Fig 14-6 if you want to wake up KM4 at the same time.

NOTE

LP Timer (TIMM00-05) belongs to KM0, and only TIM00-03 can be wakeup source. Please distinguish them with HS timer (TIM0-5) which can not be wakeup source.

14.1.3.2 How to Enter Sleep Mode

Function pmu_release_wakelock(PMU_OS) can be called in KM4 if the system wants to enter into sleep mode. The global parameter wakelock will be checked in idle task, once wakelock equals zero and there is nothing to do, the system will enter into sleep mode.

14.1.3.3 How to Wake Up from Sleep Mode

14.1.3.3.1 HS Wakeup Event

The chip will wake up from sleep mode when the specific interrupt that enabled is triggered.

NOTE

Switch the clock back after the system wakes up from sleep mode if the clock of the source has been changed before entering sleep mode.

14.1.3.3.2 AON

PDB

The chip will wake up from sleep mode when the interrupt of PDB that enabled is triggered.

Cap-Touch

The chip will wake up from sleep mode when the interrupt of Cap-Touch that enabled is triggered.

Key-Scan

The chip will wake up from sleep mode when the interrupt of Key-Scan that enabled is triggered.

RTC

The chip will wake up from sleep mode when the interrupt of RTC that enabled is triggered.

AON Timer

The chip will wake up from sleep mode when the system has already slept for 'sleep time'.

GPIO (Wakepin)

The chip will wake up from sleep mode when there is a corresponding edge of AON wakepin(s) that setup in section 14.1.5.1.5.

14.1.3.3.3 SGPIO

The chip will wake up from sleep mode when the interrupt of SGPIO that enabled is triggered.

NOTE

Switch SGPIO clock back to LS APB Clock by register REG_SYS_CLK_CTRL1 after the system wakes up from sleep mode.

14.1.3.3.4 Comparator

The chip will wake up from sleep mode when the interrupt of comparator that enabled is triggered.

NOTE

Switch comparator clock back to LS APB Clock by register REG_SYS_CLK_CTRL1 after the system wakes up from sleep mode.

14.1.3.3.5 ADC

The chip will wake up from sleep mode when the interrupt of ADC that enabled is triggered.

NOTE

Switch ADC clock back to LS APB Clock by register REG_SYS_CLK_CTRL1 after the system wakes up from sleep mode.

14.1.3.3.6 I2C

The chip will wake up from sleep mode when the interrupt of I2C that enabled is triggered.

NOTE

Switch I2C clock back to LS APB Clock by calling RCC_PeriphClockSource_I2C() after the system wakes up from sleep mode .

14.1.3.3.7 LP UART

The chip will wake up from sleep mode when the interrupt of UART that enabled is triggered.

NOTE

Switch UART clock back to XTAL by calling RCC_PeriphClockSource_UART() after the system wakes up from sleep mode if need high baud rate.

14.1.3.3.8 Log UART

The chip will wake up from sleep mode when the interrupt of UART that enabled is triggered.

14.1.3.3.9 BOD

The chip will wake up from sleep mode when the interrupt of BOD that enabled is triggered.

14.1.3.3.10 WLAN

The chip will wake up from sleep mode every beacon interval, and the default interval is 102.4ms.

14.1.3.3.11 GPIO

The chip will wake from sleep mode when the interrupt of GPIO that enabled is triggered.

i NOTE

Switch GPIO clock back to LS APB by register REG_SYS_CLK_CTRL1 after the system wakes up from sleep mode.

14.1.3.3.12 Timer

The chip will wake up from sleep mode when the system has already slept for 'sleep time'.

14.1.3.4 How to Get Wakeup Information

Table 14-6 Sleep wakeup information

API	Introduction	Parameters
WakeEvent	Gets sleep wake reason It can only be accessed in KM0	Bit[1]: GTimer Bit[2]: DCORE SWR OCP Bit[4]: GPIO Bit[5]: WLAN Bit[6]: BOD Bit[20]: UART Bit[24]: I2C Bit[26]: ADC Bit[27]: Comparator Bit[28]: SGPIO Bit[29]: AON Bit[30]: HS
int SOCPS_AONWakeReason(void)	Gets AON wake reason It can be accessed in KM0 and KM4	- Parameter: None - Retval status value: - Bit[0]: AON wakepin - Bit[1]: AON Timer - Bit[2]: RTC - Bit[3]: TSF Timer - Bit[4]: Key-Scan - Bit[6]: Cap-Touch
int SOCPS_WakePinCheck(void)	Gets AON Wakepin index It can be accessed in KM0 and KM4	- Parameter: None - Retval status value: - Bit[0]: wakepin0 - Bit[1]: wakepin1 - Bit[2]: wakepin2 - Bit[3]: wakepin3

14.1.4 Deepsleep Mode

Deepsleep mode is has a lower power consumption as only AON domain is power on. So only peripheral in AON domain can wakeup chip. There are 2 ways to enter deepsleep mode. One is to release deepwakelock, the other is KM4 sends related IPC to KM0 directly. So If using way2, tickless is not necessary. Once KM0 receive IPC message, KM0 makes KM0 and KM4 to enter deep sleep mode in IPC interrupt.

The wake process of deep sleep is the chip boot on process. Fig 14-11 shows deepsleep mode flow.

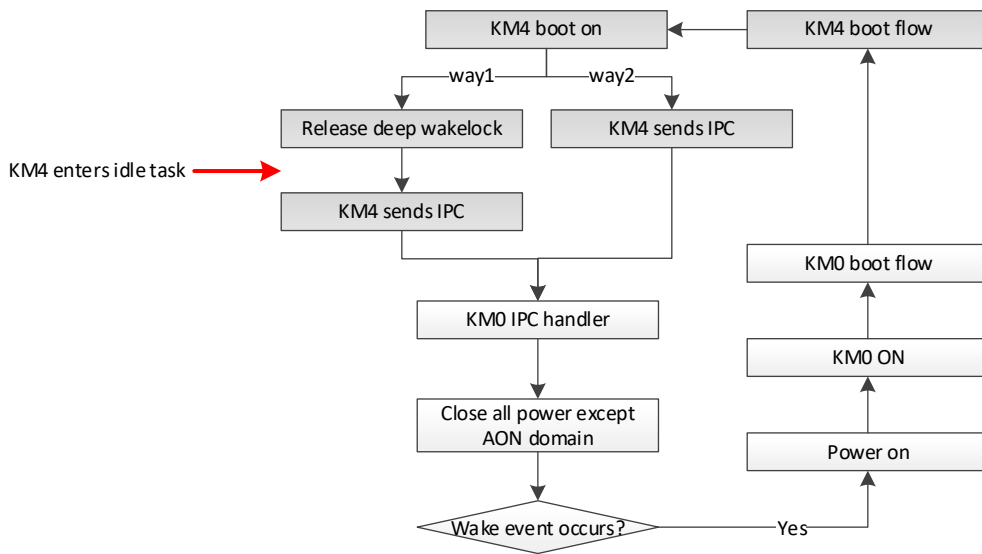


Fig 14-11 Deepsleep mode flow

14.1.5 Deepsleep Mode Configuration

14.1.5.1 Wakeup Source Setup

Table 14-7 Deepsleep wakeup source

Wakeup source	Description	Default status
BIT_CAPTOUCH_WAKE_STS	Indicates Cap-Touch wake event	ON
BIT_KEYSCAN_WAKE_STS	Indicates Key-Scan wake	ON
BIT_DLPS_TSF_WAKE_STS	Indicates TSF wake for Wi-Fi FW	ON
BIT_RTC_WAKE_STS	Indicates RTC wake	OFF
BIT_AON_WAKE_TIM0_STS	Indicates AON Timer wake	ON
BIT_GPIO_WAKE_STS	Indicates AON wakepin wake	ON

To set corresponding deepsleep source, firstly you need to set corresponding wakeup source Status ON in array `dsleep_aon_wevent_config[]` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-12.

```

PWRCFG_TypeDef dsleep_aon_wevent_config[] =
{
    /* Module */
    { BIT_CAPTOUCH_WAKE_STS, ON, /* [6] R/W 0 1: Indicate captouch wake event */ },
    { BIT_KEYSCAN_WAKE_STS, ON, /* [4] R/W 0 1: Indicate keyscan wake */ },
    { BIT_DLPS_TSF_WAKE_STS, ON, /* [3] R/W 0 1: Indicate tsf wake under deep-lps mode */ },
    { BIT_RTC_WAKE_STS, OFF, /* [2] R/W 0 1: Indicate RTC wake */ },
    { BIT_AON_WAKE_TIM0_STS, ON, /* [1] R/W 0 1: Indicate AON timer wake */ },
    { BIT_GPIO_WAKE_STS, ON, /* [0] R/W 0 1: Indicate GPIO wake, see aon_wakepin & dsleep_wakepin_config */ },
    { 0xFFFFFFFF, OFF, /* Table end */ }
};
    
```

Fig 14-12 Deepsleep wake source setup

When KM4 wants to enter deep sleep mode, it will construct a global variable in structure `KM4SLEEP_ParamDef` as shown in Fig 14-13. Then transmit it to KM0 to inform KM0 execute corresponding operation. The variable can be local variable, or if KM4 releases the local variable, KM0 may get wrong information.

NOTE

As `rtl8721dlp_sleepcfg.c` is the code under KM0, if you modify it, you need to rebuild KM0 project, and if you are using IAR, KM4 project also needs to be rebuilt to sync the KM0 image.

```
typedef struct
{
    u8 dlps_enable;
    u8 sleep_type;
    u32 sleep_time;
} KM4SLEEP_ParamDef;
```

Fig 14-13 KM4 sleep parameter structure

Table 14-8 Structure KM4SLEEP_ParamDef member description

Member	Description
dlps_enable	- Enable deep sleep mode: - TRUE: enable deep sleep - FALSE: disable deep sleep
sleep_type	- Just for sleep mode usage - SLEEP_PG: KM4 enter power gate mode - SLEEP_CG: KM4 enter clock gate mode
sleep_time	If using AON Timer to wakeup chip, you should set the variable to control the time to sleep.

14.1.5.1.1 Cap-Touch

- Initialize Cap-Touch and enable its interrupts that you want to wakeup chip in deepsleep mode. As RAM and CPU are shut down in deepsleep mode while the Cap-Touch register keep alive, the interrupt handler and interrupt enable of CPU should be reinitialized, then Cap-Touch interrupts can be handled.
- Set BIT_CAPTOUCH_WAKE_STS Status ON in array dsleep_aon_wevent_config[] as shown in Fig 14-12.

NOTE

Refer to code 'touch_key.c' for Cap-Touch initialization.

14.1.5.1.2 Key-Scan

- Initialize Key-Scan and enable its interrupts that you want to wakeup chip in deepsleep mode. As RAM and CPU are shut down in deepsleep mode while the Key-Scan register keep alive, the interrupt handler and interrupt enable of CPU should be reinitialized, then Key-Scan interrupts can be handled.
- Set BIT_KEYSCAN_WAKE_STS Status ON in array dsleep_aon_wevent_config[] as shown in Fig 14-12.

NOTE

Refer to code 'touch_key.c' for Key-Scan initialization.

14.1.5.1.3 RTC

- Initialize RTC and enable its interrupts that you want to wakeup chip in deepsleep mode. As RAM and CPU are shut down in deepsleep mode while the RTC register keep alive, the interrupt handler and interrupt enable of CPU should be reinitialized, then RTC interrupts can be handled.
- Set BIT_RTC_WAKE_STS Status ON in array dsleep_aon_wevent_config[] as shown in Fig 14-12.

14.1.5.1.4 AON Timer

- Set AON Timer and transmit it to KM0 by ipc_send_message(), the detailed steps is shown in Section 14.1.5.2.
- Set BIT_AON_WAKE_TIMO_STS Status ON in array dsleep_aon_wevent_config[] as shown in Fig 14-12.

NOTE

The AON Timer is in imprecise milliseconds. Its maximum value is 32760000ms (546mins).

14.1.5.1.5 GPIO

The GPIOs that can wake up chip in deepsleep is not all the common GPIOs, only 12 GPIOs can wake up the chip, as shown in Fig 14-14, we call these special GPIOs ‘AON wakepin’ in the following text. Only four AON wakepins can be set at the same time. As the note in Fig 14-14, there are four AON wakepins (0, 1, 2, 3), each AON wakepin has three PINMUX (S0, S1, S2).

For each AON wakepin, you can only select one PINMUX at a time by the Module column with the Status column ON in array `dsleep_aon_wevent_config[]` in `rtl8721dlp_sleepcfg.c`, as shown in Fig 14-15. The Polarity column is used to choose wakeup edge, Ameba-D only supports AON wakepin wake up the chip by edges. The value 0 means Falling Edge wakeup and value 1 means Rising Edge wakeup.

For AON wakepins, if setting Falling Edge wakeup, SDK will automatically set the pin internal pull up. And if setting Rising Edge wakeup, SDK will automatically set the pin internal pull down.

```
u8 aon_wakepin[4][3] = {
    { /* wakepin 0 */
        _PA_12, /* PINMUX_S0 */
        _PA_16, /* PINMUX_S1 */
        _PA_20, /* PINMUX_S2 */
    },
    { /* wakepin 1 */
        _PA_13, /* PINMUX_S0 */
        _PA_17, /* PINMUX_S1 */
        _PA_21, /* PINMUX_S2 */
    },
    { /* wakepin 2 */
        _PA_14, /* PINMUX_S0 */
        _PA_18, /* PINMUX_S1 */
        _PA_25, /* PINMUX_S2 */
    },
    { /* wakepin 3 */
        _PA_15, /* PINMUX_S0 */
        _PA_19, /* PINMUX_S1 */
        _PA_26, /* PINMUX_S2 */
    },
};
```

Fig 14-14 AON wakepins

```
WAKEPIN_TypeDef sleep_wakepin_config[] =
{
    // Module          Status      Polarity
    { PINMUX_S0,       OFF,        1 }, /* wakeup_0 config */
    { PINMUX_S0,       OFF,        1 }, /* wakeup_1 config */
    { PINMUX_S0,       OFF,        1 }, /* wakeup_2 config */
    { PINMUX_S0,       OFF,        1 }, /* wakeup_3 config */
    { 0xFFFFFFFF,     OFF,        0 }, /* Table end */
};
```

Fig 14-15 AON wakepins setup

AON wakepin setup low:

- (1) Set BIT_GPIO_WAKE_STS Status ON in array `dsleep_aon_wevent_config[]` as shown in Fig 14-12.
- (2) Set AON Wakepin and expected edge to wakeup chip in array `sleep_wakepin_config[]` as shown in Fig 14-15.

Example:

If want a failing edge of `_PA_18` to wakeup chip, the AON wakepin setting is as shown in Fig 14-16. `_PA_18` is PINMUX_S1 of wakepin 2.

```
WAKEPIN_TypeDef sleep_wakepin_config [] =
{
// Module          Status      Polarity
{ PINMUX_S0,      OFF,        1 }, /* wakeup_0 config */
{ PINMUX_S0,      OFF,        1 }, /* wakeup_1 config */
{ PINMUX_S1,      ON,         0 }, /* wakeup_2 config */
{ PINMUX_S0,      OFF,        1 }, /* wakeup_3 config */
|
{ 0xFFFFFFFF,    OFF,        0 }, /* Table end */
};
```

Fig 14-16 AON Wakepins setup example

14.1.5.2 How to Enter Deepsleep Mode

To enter deepsleep mode, you should execute the following steps:

- (1) Construct a **global variable** with the structure **KM4SLEEP_ParamDef**
- (2) Set member 'dlps_enable = TRUE' of the **global variable** to enable deep sleep mode
- (3) Set member 'sleep_time' if you want to wakeup chip at specified time by AON Timer wakeup source
- (4) Call 'ipc_send_message (IPC_INT_KM4_TICKLESS_INDICATION, (u32)& **global variable**)' to tell KM0 execute enter deep sleep operation

NOTE

Refer to example '\project\realtek_amebaD_va0_example\example_sources\PMC\raw\pm_dslp'.

Release deepwakeuplock is also a way to enter deepsleep mode but not recommend.

14.1.5.3 How to Wake Up from Deepsleep Mode

When the chip wakes up from deep sleep, it will do the boot process.

14.1.5.3.1 Cap-Touch

The interrupt of Cap-Touch that enabled happens will wake up the chip.

14.1.5.3.2 Key-Scan

The interrupt of Key-Scan that enabled happens will wake up the chip.

14.1.5.3.3 RTC

The interrupt of RTC that enabled happens will wake up the chip.

14.1.5.3.4 AON Timer

The 'sleep_time' reach will wake up the chip.

14.1.5.3.5 GPIO

The corresponding edge of AON wakepin(s) that setup in section 14.1.5.1.5 happens will wake up the chip.

14.1.5.4 How to Get Wakeup Information

Table 14-9 Deepsleep wakeup information API

API	Introduction	Parameters
u32 OCPS_DsleepWakeStatusGet(void)	Gets deepsleep wake status	- Parameter: None - Retval status value:

		■ TRUE: boot from deepsleep ■ FALSE: boot from power on
int SOCPS_AONWakeReason(void)	Gets deepsleep wake reason	● Parameter: None ● Retval status value: ■ Bit[0]: AON wakepin ■ Bit[1]: AON Timer ■ Bit[2]: RTC ■ Bit[3]: TSF Timer ■ Bit[4]: Key-Scan ■ Bit[6]: Cap-Touch
int SOCPS_WakePinCheck(void)	Gets AON wakepin index	● Parameter: None ● Retval status value: ■ Bit[0]: wakepin0 ■ Bit[1]: wakepin1 ■ Bit[2]: wakepin2 ■ Bit[3]: wakepin3

14.1.5.5 How to Maintain Information in Deepsleep Mode

The memory of Retention RAM can be kept when the chip enters deepsleep mode, so we can use it to keep some necessary information. RRAM_USER_RSVD is reserved for customer usage as shown in Fig 14-17.

```

/*****
 * @defgroup AMEBAD_RRAM
 * @{
 * @brief AMEBAD_RRAM Declaration
 * @the Max space for RRAM_TypeDef is 0xB0, user can alloc space from RRAM_USER_RSVD
 *****/
typedef struct {
    >> uint8_t RRAM_WIFI_STATUS; /* RETENTION_RAM_SYS_OFFSET 0x80 */
    >> uint8_t RRAM_WIFI_P_SECURITY;
    >> uint8_t RRAM_WIFI_G_SECURITY;
    >> uint8_t RRAM_RSVD1;
    >>
    >> uint32_t RRAM_NET_IP;
    >> uint32_t RRAM_NET_GW;
    >> uint32_t RRAM_NET_GW_MASK;
    >>
    >> uint32_t FLASH_ID2; >>> >> /* offset 0x90 */
    >> uint32_t FLASH_cur_cmd;
    >> uint32_t FLASH_quad_valid_cmd;
    >> uint32_t FLASH_dual_valid_cmd;
    >> uint32_t FLASH_QuadEn_bit; >> /* offset 0xA0 */
    >> uint32_t FLASH_StructInit;
    >>
    >> uint8_t FLASH_phase_shift_idx;
    >> uint8_t FLASH_rd_sample_phase_cal;
    >> uint8_t FLASH_class;
    >> uint8_t FLASH_cur_bitmode;
    >>
    >> uint8_t FLASH_ClockDiv;
    >> uint8_t FLASH_ReadMode;
    >> uint8_t FLASH_pseudo_prm_en;
    >> uint8_t FLASH_addr_phase_len;
    >>
    >> uint8_t FLASH_cmd_wr_status2; /* offset 0xB0 */
    >> uint8_t FLASH_rd_dummy_cyle0;
    >> uint8_t FLASH_rd_dummy_cyle1;
    >> uint8_t FLASH_rd_dummy_cyle2;
    >>
    >> uint8_t RRAM_USER_RSVD[124]; >>> /*usr can alloc from this RSVD space*/
    >>
} __packed __attribute__((aligned(4))) RRAM_TypeDef;
/*****/

```

Fig 14-17 Retention RAM usage

14.1.6 GPIO Pull Control

When entering sleep and deepsleep mode, I/O pull control is needed. Otherwise, it results in power leakage. For more information, refer to AN0400 (section: User Configuration→pinmapcfg).

14.2 Power Save Related APIs

API	Introduction
<pmu_register_sleep_callback>	Register suspend/resume call back function for one module
<pmu_unregister_sleep_callback>	Unregister suspend/resume call back function
<pmu_acquire_wakelock>	Acquire wakelock for one module
<pmu_release_wakelock>	Release wakelock
<pmu_set_sysactive_time>	Acquire wakelock, and release wakelock automatically after timeout
<pmu_set_max_sleep_time>	Set max. sleep time

14.2.1 pmu_register_sleep_callback

Register suspend/resume call back function for <nDeviceId>. Suspend callback function will be called by PMU before system enters sleep mode, and resume callback function will be called after system resumes.

NOTE

- Yield OS is not permitted in suspend/resume callback function like taskYIELD, vTaskDelay, mutex, sema and so on.
- pmu_set_sysactive_time is not permitted in suspend callback function, while permitted in resume call back function.

Parameter	Type	Introduction
<nDeviceId>	uint32_t	Device ID need suspend/resume callback typedef enum { PMU_OS = 0, ... PMU_MAX = 31 } PMU_DEVICE;
<sleep_hook_fun>	PSM_HOOK_FUN	Suspend call back function
<sleep_param_ptr>	void*	Suspend call back function parameter
<wakeup_hook_fun>	PSM_HOOK_FUN	Resume call back function
<wakeup_param_ptr>	void*	Resume call back function parameter

14.2.2 pmu_unregister_sleep_callback

Unregister suspend/resume call back function for <nDeviceId>.

Parameter	Type	Introduction
<nDeviceId>	uint32_t	The same as pmu_register_sleep_callback

14.2.3 pmu_acquire_wakelock

Wakelock is a 32-bit map. Each module owns 1 bit in this bit map. FreeRTOS tickless references the wakelock and decides that whether it can enter sleep state.

If any module acquires and holds a bit in wakelock, then the whole system won't enter sleep state.

Parameter	Type	Introduction
<nDeviceId>	uint32_t	The same as pmu_register_sleep_callback

14.2.4 pmu_release_wakelock

Release bit[nDeviceId] of wakelock bit map. If wakelock equals to 0, then the system may enter sleep state after system idle.

Parameter	Type	Introduction
<nDeviceId>	uint32_t	The same as pmu_register_sleep_callback

14.2.5 pmu_set_sysactive_time

Set system active time. System cannot sleep before timeout.

i NOTE

pmu_set_sysactive_time is not permitted in suspend callback function as it is ineffective, while permitted in resume call back function.

Parameter	Type	Introduction
<timeout>	uint32_t	System cannot sleep before timeout. Unit is ms.

14.2.6 pmu_set_max_sleep_time

Set system max. sleep time.

i NOTE

- System may be waked by other wake events before this timer is timeout.
- Before entering deepsleep mode, disable the AON timer by SOCPS_AONTimerCmd(DISABLE).
- This setting only works once. The timer will be cleared after system wakeup.

Parameter	Type	Introduction
<timeout>	uint32_t	System max. sleep timeout. Unit is ms.

14.3 Power Consumption Measurement

14.3.1 Test Command

We provide command in KM4 for tickless testing. Below is the description.

Items	Comment
tickps r debug	Release OS wakelock to open tickless function, KM4 will enter sleep mode when idle. In program you can use pmu_release_wakelock (PMU_OS) to open tickless function.
tickps a	Get OS wakelock to close tickless function.
tickps get	Get current wakelock. It prints current wakelock bit map. You can use this command to debug why system doesn't enter sleep.

14.3.2 Hardware Preparation

In Ameba-D reference board, there are other components that consume power. For example, DAP, LEDs, FT232, and capacitances. To measure power consumptions only for Ameba-D, you need to remove resistance at R43 as Fig 14-18.

You can use micro-USB to supply power for the board, and link current meter use J38 as Fig 14-19.

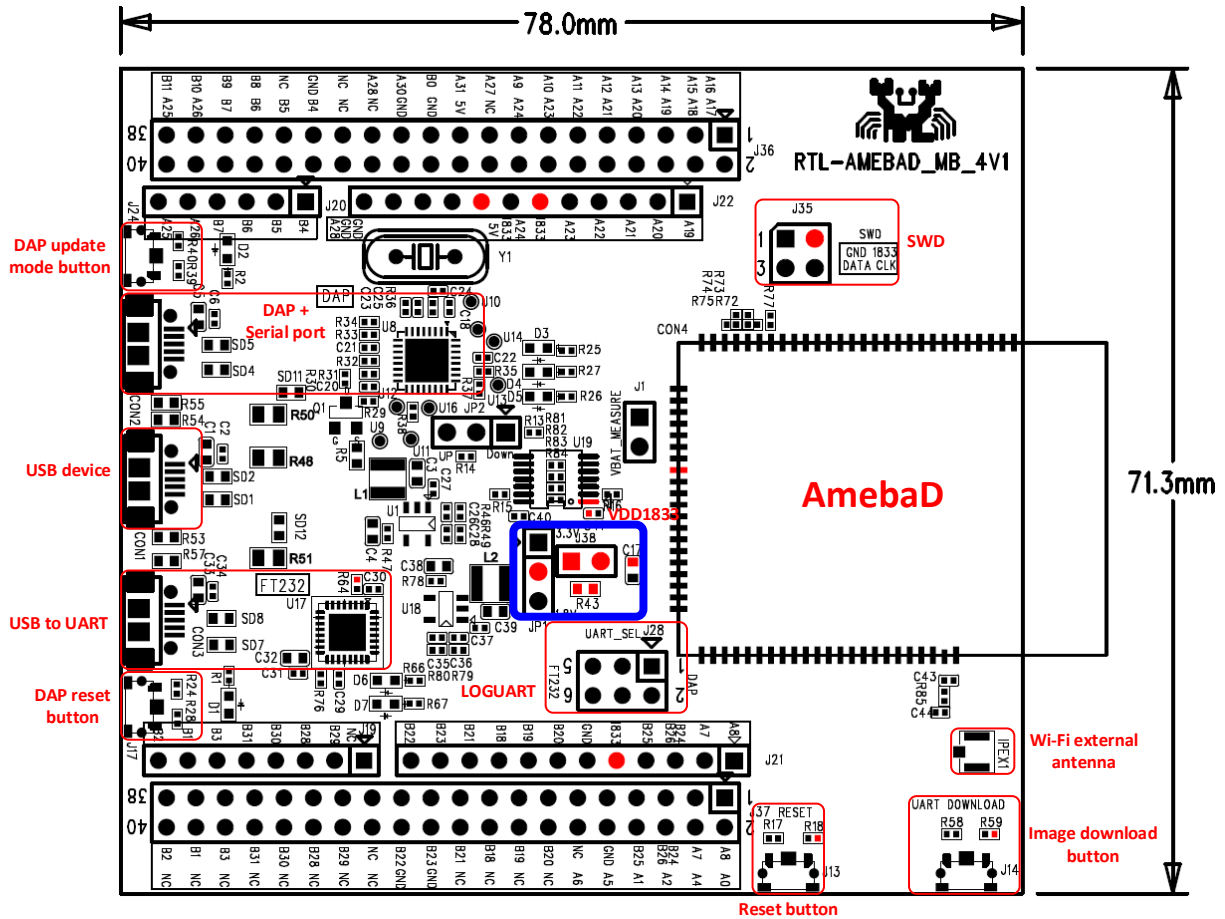


Fig 14-18 Power consumption measurement

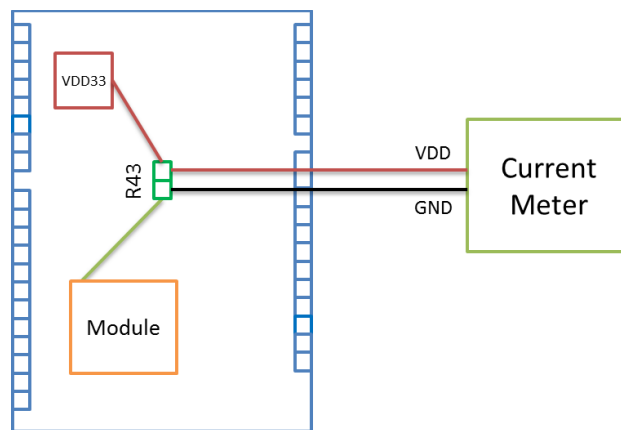


Fig 14-19 Measuring power consumption from micro-USB

15 Hardware Crypto Engine

15.1 Introduction

Hardware Crypto Engine is used to authenticate, encrypt and decrypt packets. It can support the following Hash and Cipher functions:

- Hash (include HMAC): MD5/SHA1/SHA2 (224, 256), Poly1305
- Cipher: AES (ECB/CBC/CFB/OFB/CTR/GMAC/GHASH/GCM), 3DES (ECB/CBC/CFB/OFB/CTR), DES(ECB/CBC/CFB/OFB/CTR), ChaCha20, Chacha20_poly1305

Hardware Crypto Engine also support sequential hash for long plaintext length.

The following sections illustrate the Hardware Crypto Engine APIs and how to use these APIs.

15.2 Hardware Crypto Engine APIs

15.2.1 Crypto Engine Initialization API

15.2.1.1 rtl_cryptoEngine_init

Items	Description
Introduction	Hardware crypto engine initialization
Parameters	N/A
Return	Status value: ● SUCCESS: initialization is ok.

15.2.2 Hash APIs

15.2.2.1 rtl_crypto_md5

Items	Description
Introduction	MD5 calculation digest
Parameters	● message: Plaintext ● msglen: Plaintext length ● pDigest: Result of MD5 function
Return	Status value: ● SUCCESS: MD5 is ok. ● Others: fail, refer to ERRNO.

15.2.2.2 rtl_crypto_md5_init

Items	Description
Introduction	MD5 initialization (used for sequential hash)
Parameters	N/A
Return	Status value: ● SUCCESS: MD5 initialization is ok. ● Others: fail, refer to ERRNO.

15.2.2.3 rtl_crypto_md5_update

Items	Description
Introduction	MD5 block calculation (used for sequential hash)
Parameters	● message: Plaintext ● msglen: Plaintext length
Return	Status value: ● SUCCESS: MD5 update is ok. ● Others: fail, refer to ERRNO.

15.2.2.4 rtl_crypto_md5_final

Items	Description
Introduction	MD5 last block calculation (used for sequential hash)
Parameters	pDigest: Result of MD5 function
Return	Status value: ● SUCCESS: MD5 final is ok. ● Others: fail, refer to ERRNO.

15.2.2.5 rtl_crypto_sha1

Items	Description
Introduction	SHA1 calculation digest
Parameters	● message: Plaintext ● msglen: Plaintext length ● pDigest: Result of SHA1 function
Return	Status value: ● SUCCESS: SHA1 is ok. ● Others: fail, refer to ERRNO.

15.2.2.6 rtl_crypto_sha1_init

Items	Description
Introduction	SHA1 initialization (used for sequential hash)
Parameters	N/A
Return	Status value: ● SUCCESS: SHA1 initialization is ok. ● Others: fail, refer to ERRNO.

15.2.2.7 rtl_crypto_sha1_update

Items	Description
Introduction	SHA1 block calculation (used for sequential hash)
Parameters	● message: Plaintext ● msglen: Plaintext length
Return	Status value: ● SUCCESS: SHA1 update is ok. ● Others: fail, refer to ERRNO.

15.2.2.8 rtl_crypto_sha1_final

Items	Description
-------	-------------

Introduction	SHA1 last block calculation (used for sequential hash)
Parameters	pDigest: Result of SHA1 function
Return	Status value: ● SUCCESS: SHA1 final is ok. ● Others: fail, refer to ERRNO.

15.2.2.9 rtl_crypto_sha2

Items	Description
Introduction	SHA2 calculation digest
Parameters	● sha2type: SHA2 type, SHA2_224 or SHA2_256 ● message: Plaintext ● msglen: Plaintext length ● pDigest: Result of SHA2 function
Return	Status value: ● SUCCESS: SHA2 is ok. ● Others: fail, refer to ERRNO.

15.2.2.10 rtl_crypto_sha2_init

Items	Description
Introduction	SHA2 initialization (used for sequential hash)
Parameters	sha2type: SHA2 type, SHA2_224 or SHA2_256
Return	Status value: ● SUCCESS: SHA2 initialization is ok. ● Others: fail, refer to ERRNO.

15.2.2.11 rtl_crypto_sha2_update

Items	Description
Introduction	SHA2 block calculation (used for sequential hash)
Parameters	● message: Plaintext ● msglen: Plaintext length
Return	Status value: ● SUCCESS: SHA2 update is ok. ● Others: fail, refer to ERRNO.

15.2.2.12 rtl_crypto_sha2_final

Items	Description
Introduction	SHA2 last block calculation (used for sequential hash)
Parameters	pDigest: Result of SHA2 function
Return	Status value: ● SUCCESS: SHA2 final is ok. ● Others: fail, refer to ERRNO.

15.2.2.13 rtl_crypto_hmac_md5

Items	Description
Introduction	HMAC-MD5 calculation digest
Parameters	● key: HMAC-MD5 key, need to be 4-byte aligned ● keylen: Key length ● message: Plaintext

	- msglen: Plaintext length - pDigest: Result of HMAC-MD5 function
Return	Status value: - SUCCESS: HMAC-MD5 is ok. - Others: fail, refer to ERRNO.

15.2.2.14 rtl_crypto_hmac_md5_init

Items	Description
Introduction	HMAC-MD5 initialization (used for sequential hash)
Parameters	- key: HMAC-MD5 key, need to be 4-byte aligned - keylen: Key length
Return	Status value: - SUCCESS: HMAC-MD5 initialization ok - Others: fail, refer to ERRNO

15.2.2.15 rtl_crypto_hmac_md5_update

Items	Description
Introduction	HMAC-MD5 block calculation (used for sequential hash)
Parameters	- message: Plaintext - msglen: Plaintext length
Return	retval status value: - SUCCESS: HMAC-MD5 update is ok. - Others: fail, refer to ERRNO.

15.2.2.16 rtl_crypto_hmac_md5_final

Items	Description
Introduction	HMAC-MD5 last block calculation (used for sequential hash)
Parameters	pDigest: Result of HMAC-MD5 function
Return	Status value: - SUCCESS: HMAC-MD5 final is ok. - Others: fail, refer to ERRNO.

15.2.2.17 rtl_crypto_hmac_sha1

Items	Description
Introduction	HMAC-SHA1 calculation digest
Parameters	- key: HMAC-SHA1 key, need to be 4-byte aligned - keylen: key length - message: Plaintext - msglen: Plaintext length - pDigest: Result of HMAC-SHA1 function
Return	Status value: - SUCCESS: HMAC-SHA1 is ok. - Others: fail, refer to ERRNO.

15.2.2.18 rtl_crypto_hmac_sha1_init

Items	Description
Introduction	HMAC-SHA1 initialization (used for sequential hash)
Parameters	key: HMAC-SHA1 key, need to be 4-byte aligned

	keylen: key length
Return	Status value: - SUCCESS: HMAC-SHA1 initialization is ok. - Others: fail, refer to ERRNO.

15.2.2.19 rtl_crypto_hmac_sha1_update

Items	Description
Introduction	HMAC-SHA1 block calculation (used for sequential hash)
Parameters	- message: Plaintext - msglen: Plaintext length
Return	Status value: - SUCCESS: HMAC-SHA1 update is ok. - Others: fail, refer to ERRNO.

15.2.2.20 rtl_crypto_hmac_sha1_final

Items	Description
Introduction	HMAC-SHA1 last block calculation (used for sequential hash)
Parameters	pDigest: Result of HMAC-SHA1 function
Return	Status value: - SUCCESS: HMAC-SHA1 final is ok. - Others: fail, refer to ERRNO.

15.2.2.21 rtl_crypto_hmac_sha2

Items	Description
Introduction	HMAC-SHA2 calculation digest
Parameters	- sha2type: SHA2 type, SHA2_224 or SHA2_256 - key: HMAC-SHA2 key, need to be 4-byte aligned - keylen: Key length - message: Plaintext - msglen: Plaintext length - pDigest: Result of HMAC-SHA2 function
Return	Status value: - SUCCESS: HMAC-SHA2 is ok. - Others: fail, refer to ERRNO.

15.2.2.22 rtl_crypto_hmac_sha2_init

Items	Description
Introduction	HMAC-SHA2 initialization (used for sequential hash)
Parameters	- sha2type: SHA2 type, SHA2_224 or SHA2_256 - key: HMAC-SHA2 key, need to be 4-byte aligned - keylen: Key length
Return	Status value: - SUCCESS: HMAC-SHA2 initialization is ok. - Others: fail, refer to ERRNO.

15.2.2.23 rtl_crypto_hmac_sha2_update

Items	Description
Introduction	HMAC-SHA2 block calculation (used for sequential hash)

Parameters	● message: Plaintext ● msglen: Plaintext length
Return	Status value: ● SUCCESS: HMAC-SHA2 update is ok. ● Others: fail, refer to ERRNO.

15.2.2.24 rtl_crypto_hmac_sha2_final

Items	Description
Introduction	HMAC-SHA2 last block calculation (used for sequential hash)
Parameters	pDigest: Result of HMAC-SHA2 function
Return	Status value: ● SUCCESS: HMAC-SHA2 final is ok. ● Others: fail, refer to ERRNO.

15.2.2.25 rtl_crypto_poly1305

Items	Description
Introduction	poly1305 calculation digest
Parameters	● key: poly1305 key, need to be 4-byte aligned ● keylen: Key length ● message: Plaintext ● msglen: Plaintext length ● pDigest: Result of poly1305 function
Return	Status value: ● SUCCESS: poly1305 is ok. ● Others: fail, refer to ERRNO.

15.2.3 Cipher APIs

15.2.3.1 rtl_crypto_xxx_xxx_init

Items	Description
Introduction	xxx initialization xxx can be aes_cbc/aes_ecb/aes_ctr/aes_cfb/aes_ofb/aes_gcm/3des_cbc/3des_ecb/3des_ctr/3des_cfb/3des_ofb/ des_cbc/des_ecb/des_ctr/des_cfb/des_ofb
Parameters	● key: Cipher key. ● keylen: Key length
Return	status value: ● SUCCESS: initialization is ok. ● Others: fail, refer to ERRNO.

15.2.3.2 rtl_crypto_xxx_xxx_encrypt

Items	Description
Introduction	xxx encryption xxx can be aes_cbc/aes_ecb/aes_ctr/aes_cfb/aes_ofb/aes_gcm/3des_cbc/3des_ecb/3des_ctr/3des_cfb/3des_ofb/ des_cbc/des_ecb/des_ctr/des_cfb/des_ofb
Parameters	● message: Point to source message ● msglen: Message length ● iv: Initial vector ● ivlen: iv length ● pResult: Point to cipher result

Return	Status value: ● SUCCESS: encryption is ok. ● Others: fail, refer to ERRNO.
--------	---

15.2.3.3 rtl_crypto_xxx_xxx_decrypt

Items	Description
Introduction	xxx decryption xxx can be aes_cbc/aes_ecb/aes_ctr/aes_cfb/aes_ofb/aes_gcm/3des_cbc/3des_ecb/3des_ctr/3des_cfb/3des_ofb/ des_cbc/ des_ecb/des_ctr/des_cfb/des_ofb
Parameters	● message: Point to source message ● msglen: Message length ● iv: Initial vector ● ivlen: IV (initialization vector) length ● pResult: Point to cipher result
Return	Status value: ● SUCCESS: decryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.4 rtl_crypto_aes_gcm_init

Items	Description
Introduction	AES-GCM initialization
Parameters	● key: Cipher key ● keylen: Key length
Return	Status value: ● SUCCESS: initialization is ok. ● Others: fail, refer to ERRNO.

15.2.3.5 rtl_crypto_aes_gcm_encrypt

Items	Description
Introduction	AES-GCM encryption
Parameters	● message: Point to source message ● msglen: Message length ● iv: Initial vector ● aad: Point to AAD (additional authentication data) ● aadlen: AAD length ● pResult: Point to cipher result ● pTag: Point to MAC (Message Authentication Code)
Return	Status value: ● SUCCESS: encryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.6 rtl_crypto_aes_gcm_decrypt

Items	Description
Introduction	AES-GCM decryption
Parameters	● message: Point to source message ● msglen: Message length ● iv: Initial vector ● aad: Point to AAD (additional authentication data) ● aadlen: AAD length ● pResult: Point to cipher result

	● pTag: Point to MAC (Message Authentication Code)
Return	Status value: ● SUCCESS: decryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.7 rtl_crypto_chacha_init

Items	Description
Introduction	ChaCha20 initialization
Parameters	key: Cipher key.
Return	Status value: ● SUCCESS: initialization OK ● Others: fail, refer to ERRNO

15.2.3.8 rtl_crypto_chacha_encrypt

Items	Description
Introduction	ChaCha20 encryption
Parameters	● message: Point to source message ● msglen: Message length ● iv: Point to IV (initialization vector) ● count: Counter value. ● pResult: Point to cipher result
Return	Status value: ● SUCCESS: encryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.9 rtl_crypto_chacha_decrypt

Items	Description
Introduction	ChaCha20 decryption
Parameters	● message: Point to source message ● msglen: Message length ● iv: Point to IV (initialization vector) ● count: Counter value ● pResult: Point to cipher result
Return	Status value: ● SUCCESS: decryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.10 rtl_crypto_chacha_poly1305_init

Items	Description
Introduction	ChaCha Poly1305 initialization
Parameters	key: Cipher key
Return	Status value: ● SUCCESS: initialization is ok. ● Others: fail, refer to ERRNO.

15.2.3.11 rtl_crypto_chacha_poly1305_encrypt

Items	Description
Introduction	ChaCha Poly1305 encryption

Parameters	● message: Point to source message ● msglen: Message length ● nonce: Random value ● aad: Point to AAD (additional authentication data) ● aadlen: AAD length ● pResult: Point to cipher result ● pTag: Point to MAC (Message Authentication Code)
Return	Status value: ● SUCCESS: encryption is ok. ● Others: fail, refer to ERRNO.

15.2.3.12 rtl_crypto_chacha_poly1305_decrypt

Items	Description
Introduction	ChaCha Poly1305 decryption
Parameters	● message: Point to source message ● msglen: Message length ● nonce: Random value ● aad: Point to AAD (additional authentication data) ● aadlen: AAD length ● pResult: Point to cipher result ● pTag: Point to MAC (Message Authentication Code)
Return	Status value: ● SUCCESS: decryption is ok. ● Others: fail, refer to ERRNO.

15.3 Hardware Crypto Engine APIs Usage

15.3.1 Starting Hardware Crypto Engine

Before using hardware crypto engine, you need to initialize it first. You can use Crypto Engine Initialization API to do this.

15.3.2 Starting Crypto Engine Calculation

Choose a hash or cipher algorithm, and call the following APIs to calculate hash digest or ciphertext.

15.3.2.1 Hash Algorithm

If a hash algorithm is selected, then Hash APIs (*rtl_crypto_xxx*) can be used to calculate digest.

Sequential hash is needed when source message length beyond hardware support. Sequential hash breaks a whole long message into several piece of message payload, then calculate the payload in sequence, until the last message payload.

When use sequential hash, you can follow the steps below:

- (1) Initialize sequential hash: use Hash APIs (*rtl_crypto_xxx_init*) to initialize.
- (2) Handle each piece of message payload: call Hash APIs (*rtl_crypto_xxx_update*) once when one piece message payload.
- (3) Handle the last piece of message payload: call Hash APIs (*rtl_crypto_xxx_final*) once when one piece message payload.

Note: Considering that the Crypto Engine moves data through DMA and bypasses the D-Cache, if the destination array is placed in stack and the start address is not 32-byte aligned, the cache line would be dirty during function call in some cases. To avoid reading wrong digest back, users should choose one of the following methods:

- The digest array is a global variable.
- Or the start address of digest array should be 32-byte aligned if it's a local variable.
- Or call the following line to restore data from memory before reading digest when calculation is finished if it's a local variable.

```
DCache_Invalidate(((u32)digest & CACHE_LINE_ADDR_MSK), (sizeof(digest) + CACHE_LINE_SIZE));
```

15.3.2.2 Cipher Algorithm

Steps to encrypt or decrypt message are as following:

- (1) Initialize
Use Cipher APIs (*rtl_crypto_XXX_init*) to initialize.
- (2) Encrypt or decrypt message
Call Cipher APIs (*rtl_crypto_XXX_encrypt*) to encrypt source message.
Call Cipher APIs (*rtl_crypto_XXX_decrypt*) to decrypt source message.

15.4 Demo Code Path

Hardware Crypto Engine demo code locates in folder: **project/realtek_amebaD_va0_example/example_sources/CRYPTO**

16 User Configuration

Some functions have complex parameters, it is hard to give standard APIs in SDK. For example, different users have different parameters for BOOT. So we provide some user configuration files which will be called by SDK, and SDK will execute different activities based on different user configurations.

These configuration files should be maintained by users.

16.1 Configuration File List

The user configuration files are listed in Table 16-1.

Table 16-1 User configuration file

Configuration file	Configuration item	Image
rtl8721dlp_flashcfg.c	Used to configure Flash settings: - Flash_Speed - Flash_ReadMode - Flash_AVL You can also configure your Flash use flash_init_userdef, if your Flash is not in Flash_AVL.	KM0 image2
Rtl8721dlp_pinmapcfg.c	Used to reduce I/O power leakage: - Per pin function configuration - Per pin function pull up & pull down - Per pin sleep pull up & pull down	KM0 image2
Rtl8721dlp_sleepcfg.c	- Sleep/deepsleep power management - Sleep/deepsleep wakeup event - Sleep/deepsleep wake pin	KM0 image2
rtl8721d_bootcfg.c	Used to configure bootloader: - Flash_MMU_Config - RSIP_Mask_Config - Force_OTA1_GPIO - Boot_Log_En	KM0 & KM4 bootloader
rtl8721d_ipccfg.c	Used to define your IPC channels	KM0 & KM4 image2
rtl8721dlp_intfcfg.c	Enable/disable low power Rx	KM0 image2
rtl8721dhp_intfcfg.c	- PSRAM configuration: - Enable PSRAM - Enable PSRAM calibration function - Enable PSRAM retention - SDIO host configuration: - SDIO Host bus speed - SDIO Host bus width - Card Detect pin - Write Protection pin - FTL parameter configuration: - The number of physical map pages - The offset of Flash sectors which is allocated to FTL physical map	KM4 image2

16.2 flashcfg.c

The Flash configuration items for users are listed in Table 16-2.

Table 16-2 User Flash configuration

Item	Comment	Default
Flash_Speed	Indicates the Flash baud rate. It can be one of the following value:	0xFFFF

	● 0xFFFF: 80MHz ● 0x7FFF: 100MHz ● 0x3FFF: 67MHz ● 0x1FFF: 57MHz ● 0x0FFF: 50MHz	
Flash_ReadMode	Indicates the Flash read I/O mode. It can be one of the following value: ● 0xFFFF: Read quad I/O, Address & Data 4 bits mode ● 0x7FFF: Read quad O, just data 4 bits mode ● 0x3FFF: Read dual I/O, Address & Data 2 bits mode ● 0x1FFF: Read dual O, just data 2 bits mode ● 0x0FFF: 1 bit mode If the configured read mode is not supported, other modes would be searched until find the appropriate mode.	0xFFFF
Flash_AVL	Maintains the Flash IC supported by SDK, refer to Fig 16-1 for detailed information.	
flash_init_userdef	Initializes the parameters in this function in order to adopt user-defined Flash on Ameba-D.	

16.2.1 Flash Classification

Ameba-D support Flash chips of multi-vendor, such Winbond, MXIC, Gigadevice, ESMT etc. The Flash chips which have been verified on Ameba platform can be found in *UM0400 Ameba Flash AVL.pdf*. We divide the supported Flash into 6 species as Table 16-3 shows according to their characteristics, and they can be used on Ameba-D directly.

Table 16-3 Flash species

Flash classification	Manufacture	Flash ID
FlashClass1	Winbond	0xEF
	FM	0xA1
	XTX	0x0B
	XTX (FT)	0x0E
FlashClass2	GD (GD normal)	0xC8
FlashClass3	MXIC (MXIC normal)	0xC2
	Hua Hong	0x68
	GD (GD MD serial)	0x51
FlashClass4	ESMT	0x1C
FlashClass5	Micron	0x20
FlashClass6	MXIC (MXIC wide-range VCC)	0x28C2

For a new Flash chip that is not available in AVL, users should follow these steps to check if it can be supported by Ameba-D.

(1) Review Flash datasheet to collect essential information

- Flash ID
- Number of Flash status register and the definition (QE, BUSY, WEL, BP etc.)
- Flash Commands (Write/Read status register, Read, Read Dual/Quad Output/IO, Page Program, Sector/Block/Chip Erase, Enter/Release from Power-down etc.)
- Dummy cycle number of Read Dual/Quad Output/IO mode

(2) Check if these parameters match with those of the existing species

- a) If the Flash belongs to one of the Realtek defined classes, it is already supported by SDK and you can use it directly.
- b) If all the parameters except Flash ID match with those of one species, user should add the Flash ID into Flash_AVL table in SDK before using it, as Fig 16-1 shows.
- c) If both the Flash parameters and Flash ID mismatch with the existing species, user should define the parameters in function flash_init_userdef.

```

const FlashInfo_TypeDef Flash_AVL[] = {
    /*flash_id,      flash_id_mask,  flash_class,      status_mask,      FlashInitHandler */
    /* case1: Realtek defined class, any modification is not allowed */
    {0xEF,          0x000000FF,      FlashClass1,      0x000043FC,      NULL}, /* Winbond: MANUFACTURER_ID_WINBOND */
    {0xA1,          0x000000FF,      FlashClass1,      0x0000FFFC,      NULL}, /* Fudan Micro: MANUFACTURER_ID_FM */
    {0x0B,          0x000000FF,      FlashClass1,      0x000043FC,      NULL}, /* XTX */
    {0x0E,          0x000000FF,      FlashClass1,      0x000043FC,      NULL}, /* XTX(FT) */
    {0xC8,          0x000000FF,      FlashClass2,      0x000043FC,      NULL}, /* GD normal: MANUFACTURER_ID_GD */
    {0x28C2,        0x0000FFFF,      FlashClass6,      0x000200FC,      NULL}, /* MXIC wide-range VCC: MANUFACTURER_ID_MXIC */
    {0xC2,          0x000000FF,      FlashClass3,      0x000000FC,      NULL}, /* MXIC normal: MANUFACTURER_ID_BOHONG */
    {0x68,          0x000000FF,      FlashClass3,      0x000000FC,      NULL}, /* Hua Hong */
    {0x51,          0x000000FF,      FlashClass3,      0x000000FC,      NULL}, /* GD MD serial */
    {0x1C,          0x000000FF,      FlashClass4,      0x000000FC,      NULL}, /* ESMT: MANUFACTURER_ID_EON */
    {0x20,          0x000000FF,      FlashClass5,      0x000000FC,      NULL}, /* Microm: MANUFACTURER_ID_MICRON */

    /* case2: new flash, ID is not included in case1 list, but specification is compatible with FlashClass1~FlashClass6 */
    /* {0XXX,          0x0000XXXX,      FlashClassX,      0x0000XXXX,      NULL},

    /* case3: new flash, ID is not included in case1 list, and specification is not compatible with FlashClass1~FlashClass6 */
    {0x00,          0x000000FF,      FlashClassUser,  0xFFFFFFFF,      &flash_init_userdef},

    /* End */
    {0xFF,          0xFFFFFFFF,      FlashClassNone,  0xFFFFFFFF,      NULL},
};

```

Fig 16-1 Flash_AVL

16.2.1.1 Dummy Cycles

When reading Flash, host sends command and address to Flash. After that host needs to send dummy cycles to Flash before it sends data back. The number of dummy cycles is various for different read modes.

The following section shows you how to find dummy cycles from Flash datasheet.

Taking Winbond W25Q16JV for an example:

- There are 6 dummy cycles in Read Quad I/O mode, as Fig 16-2 shows.
- There are 4 dummy cycles in Read Dual I/O mode, as Fig 16-3 shows.

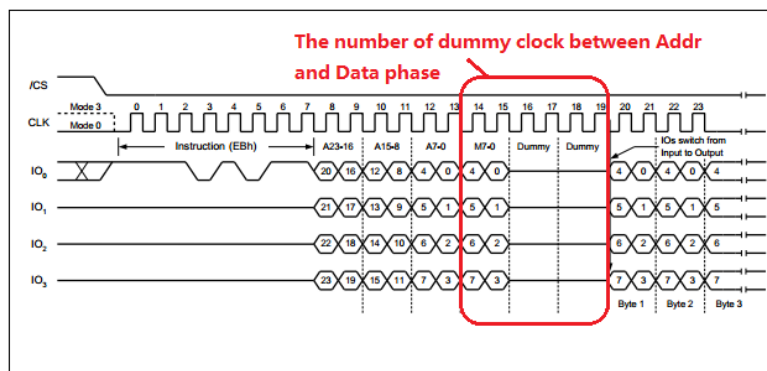


Fig 16-2 Fast read quad I/O instruction sequence

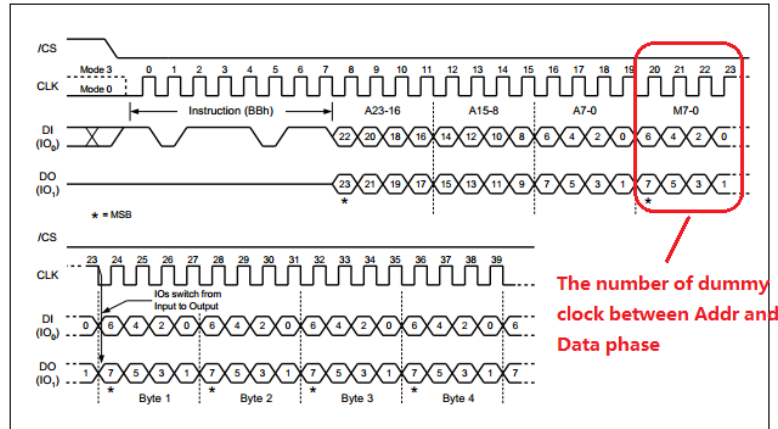


Fig 16-3 Fast read dual I/O instruction sequence

Users can find the dummy cycle of Read Quad Output and Read Dual Output mode in the same way.

16.2.1.2 Other Parameters

The parameters comparison of these species are listed in Table 16-4.

Table 16-4 Parameters comparison of different Flash species

Item	Parameter	FlashClass 1	FlashClass 2	FlashClass 3	FlashClass 4	FlashClass 5	FlashClass 6
Status Register	Status Register Number	2	2 or 3	1	1	1	1
	QE bit	Bit[9]	Bit[9]	Bit[6]	No exist	No exist	Bit[6]
	BUSY bit	Bit[0]					
	WEL bit	Bit[1]					
Dummy Cycle	READ	0	0	0	0	0	0
	DREAD (1I/2O Read)	8	8	8	8	3	8
	2READ (2*I/O Read)	4	4	4	4	5	4
	QREAD (1I/4O Read)	8	8	8	8	5	8
	4READ (4*I/O Read)	6	6	6	6	9	6
Command	Write Enable	0x06	0x06	0x06	0x06	0x06	0x06
	Read JEDEC ID	0x9F	0x9F	0x9F	0x9F	0x9F	0x9F
	Read Status Register 1	0x05	0x05	0x05	0x05	0x05	0x05
	Read Status Register 2	0x35	0x35	-	-	-	-
	Write Status Register 1	0x01	0x01	0x01	0x01	0x01	0x01
	Write Status Register 2 ¹	-	-/0x31	-	-	-	-
	Chip Erase	0x60	0x60	0x60	0x60	0xC7	0x60
	Block Erase	0xD8	0xD8	0xD8	0xD8	0xD8	0xD8
	Sector Erase	0x20	0x20	0x20	0x20	0x20	0x20
	Power-down	0xB9	0xB9	0xB9	0xB9	0xB9	0xB9
	Release from Power-down	0xAB	0xAB	0xAB	0xAB	0xAB	0xAB
	READ	0x03	0x03	0x03	0x03	0x03	0x03
	DREAD (1I/2O Read)	0x3B	0x3B	0x3B	0x3B	0x3B	0x3B
	2READ (2*I/O Read)	0xBB	0xBB	0xBB	0xBB	0xBB	0xBB
	QREAD (1I/4O Read)	0x6B	0x6B	0x6B	0x6B	0x6B	0x6B
	4READ (4*I/O Read)	0xEB	0xEB	0xEB	0xEB	0xEB	0xEB
	Read Configuration Register ²	-	-	-	-	-	0x15
	Write Configuration Register ³	-	-	-	-	-	-

Note 1:

- For FlashClass1, the Write Status Register 2 is written together with the Write Status Register 1 using 0x01, as Fig 16-4 shows.

- For FlashClass2, when GD density is less than 2MB, the Write Status Register 2 is written together with the Write Status Register 1 using 0x01, as Fig 16-4 shows. When GD density is larger than 2MB, the Write Status Register 2 is written individually using 0x31, as Fig 16-5 shows.

Note 2: For MXIC wide-range VCC, the Read Configuration Register would be checked whether the chip is in high performance mode by default once power-on, as Fig 16-6 shows. If not, we would switch it to high performance mode.

Note 3: For MXIC wide-range VCC, the Write Configuration Register is written together with Write Status Register using 0x01, as Fig 16-7 shows.

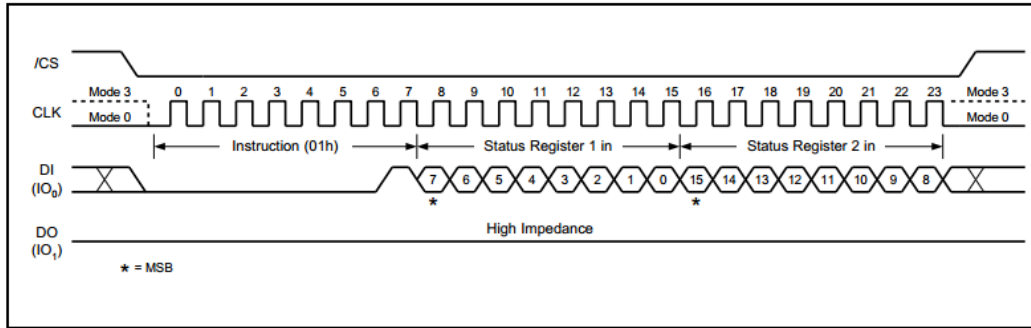


Fig 16-4 Write Status Register 1 & 2 sequence (FlashClass1, FlashClass2 when density < 2M)

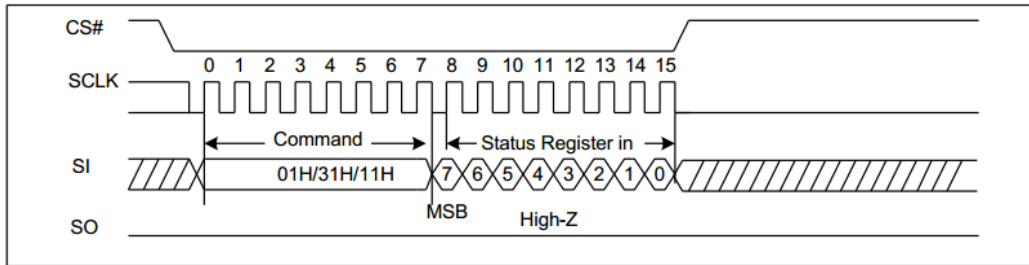


Fig 16-5 Write Status Register 1/2 sequence (FlashClass2 when density > 2MB)

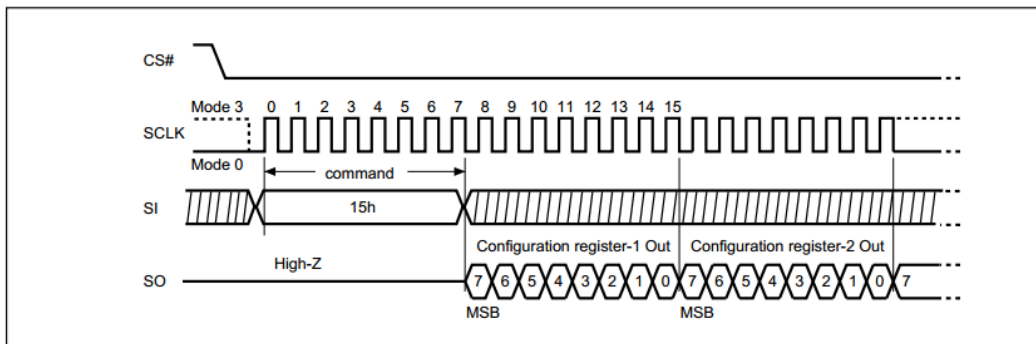


Fig 16-6 Read Configuration Register sequence (FlashClass6)

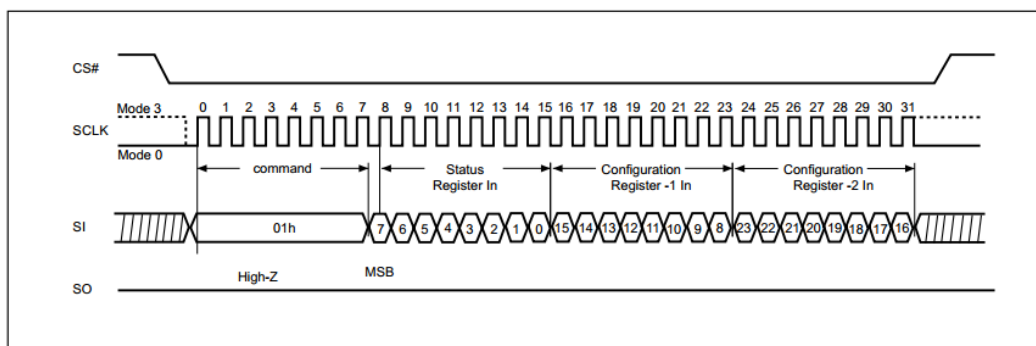


Fig 16-7 Write Status Register and Write Configuration Register sequence (FlashClass6)

16.2.2 FlashClass1 ~ FlashClass6

Table 16-5 lists the parameters for FlashClass1 to FlashClass6 which cannot be modified by users.

Table 16-5 Parameters of different Flash classification (FlashClass1 ~ FlashClass6)

Flash classification	Command list	Note
FlashClass1	<pre>FLASH_InitStruct->FLASH_Id = FLASH_ID_WINBOND; /*1.1 status bit define */ FLASH_InitStruct->FLASH_QuadEn_bit = BIT(9); FLASH_InitStruct->FLASH_Busy_bit = BIT(0); FLASH_InitStruct->FLASH_WLE_bit = BIT(1); FLASH_InitStruct->FLASH_Status2_exist = 1; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x04; FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x06; /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB; FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0x35; //FLASH_CMD_RDSR2 FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; FLASH_InitStruct->FLASH_cmd_chip_e = 0x60; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwdn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwdn = 0xB9;</pre>	
FlashClass2	<pre>FLASH_InitStruct->FLASH_Id = FLASH_ID_GD; /*1.1 status bit define */</pre>	FLASH_cmd_wr_status2 would be set when density is larger than 2MB in SDK.

	<pre> FLASH_InitStruct->FLASH_QuadEn_bit = BIT(9); FLASH_InitStruct->FLASH_Busy_bit = BIT(0); //WIP FLASH_InitStruct->FLASH_WLE_bit = BIT(1); //WLE FLASH_InitStruct->FLASH_Status2_exist = 1; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x04; //(M7-4)2 + 2 dummy FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x06; //(M7-0)2 + 4 dummy /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB; FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0x35; FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; FLASH_InitStruct->FLASH_cmd_chip_e = 0x60; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwdn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwdn = 0xB9; </pre>	
FlashClass3	<pre> FLASH_InitStruct->FLASH_Id = FLASH_ID_MXIC; /*1.1 status bit define */ FLASH_InitStruct->FLASH_QuadEn_bit = BIT(6); FLASH_InitStruct->FLASH_Busy_bit = BIT(0); //WIP FLASH_InitStruct->FLASH_WLE_bit = BIT(1); //WLE FLASH_InitStruct->FLASH_Status2_exist = 0; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x04; FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x06; /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB; FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0; FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; </pre>	

	FLASH_InitStruct->FLASH_cmd_chip_e = 0x60; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwdn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwdn = 0xB9;	
FlashClass4	FLASH_InitStruct->FLASH_Id = FLASH_ID_MXIC; /*1.1 status bit define */ FLASH_InitStruct->FLASH_QuadEn_bit = 0; FLASH_InitStruct->FLASH_Busy_bit = BIT(0); //WIP FLASH_InitStruct->FLASH_WLE_bit = BIT(1); //WLE FLASH_InitStruct->FLASH_Status2_exist = 0; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x04; FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x06; /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB; FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0; FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; FLASH_InitStruct->FLASH_cmd_chip_e = 0x60; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwdn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwdn = 0xB9;	
FlashClass5	FLASH_InitStruct->FLASH_Id = FLASH_ID_MICRON; /*1.1 status bit define */ FLASH_InitStruct->FLASH_QuadEn_bit = 0; FLASH_InitStruct->FLASH_Busy_bit = BIT(0); FLASH_InitStruct->FLASH_WLE_bit = BIT(1); FLASH_InitStruct->FLASH_Status2_exist = 0; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; /* set Micron rd_dummy_cyle based on baud rate, default 100MHz */ FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x05; FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x09; /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB;	

	<pre>FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0; FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; FLASH_InitStruct->FLASH_cmd_chip_e = 0xC7; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwn = 0xB9;</pre>	
FlashClass6	<pre>FLASH_InitStruct->FLASH_Id = FLASH_ID_MXIC; /*1.1 status bit define */ FLASH_InitStruct->FLASH_QuadEn_bit = BIT(6); FLASH_InitStruct->FLASH_Busy_bit = BIT(0); //WIP FLASH_InitStruct->FLASH_WLE_bit = BIT(1); //WLE FLASH_InitStruct->FLASH_Status2_exist = 0; /*1.2 dummy cycle */ FLASH_InitStruct->FLASH_rd_dummy_cyle[0] = 0; FLASH_InitStruct->FLASH_rd_dummy_cyle[1] = 0x04; FLASH_InitStruct->FLASH_rd_dummy_cyle[2] = 0x06; /*1.3 set 2 bits mode command */ FLASH_InitStruct->FLASH_rd_dual_io = 0xBB; FLASH_InitStruct->FLASH_rd_dual_o = 0x3B; /*1.4 set 4 bits mode command */ FLASH_InitStruct->FLASH_rd_quad_io = 0xEB; FLASH_InitStruct->FLASH_rd_quad_o = 0x6B; /*1.5 other Flash command set */ FLASH_InitStruct->FLASH_cmd_wr_en = 0x06; FLASH_InitStruct->FLASH_cmd_rd_id = 0x9F; FLASH_InitStruct->FLASH_cmd_rd_status = 0x05; FLASH_InitStruct->FLASH_cmd_rd_status2 = 0; FLASH_InitStruct->FLASH_cmd_wr_status = 0x01; FLASH_InitStruct->FLASH_cmd_wr_status2 = 0; FLASH_InitStruct->FLASH_cmd_chip_e = 0x60; FLASH_InitStruct->FLASH_cmd_block_e = 0xD8; FLASH_InitStruct->FLASH_cmd_sector_e = 0x20; FLASH_InitStruct->FLASH_cmd_pwn_release = 0xAB; FLASH_InitStruct->FLASH_cmd_pwn = 0xB9;</pre>	

16.3 pinmapcfg.c

When entering sleep and deepsleep mode, I/O pull control is needed. Otherwise it results in power leakage. For example, UART voltage level is high. If we pull down UART pin or not pull, then power leakage happens. So we must make sure that each pin has proper pull control.

In SDK, you should set GPIO function pull control and sleep pull control in pmap_func based on your PCB board, and SDK will set pull control based on your setting between suspend and resume.

```
const PMAP_TypeDef pmmap_func[] =
{
// Pin Name      Func PU/PD      Slp PU/PD      DSlp PU/PD      LowPowerPin
{ _PA_0,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_1,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_2,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_3,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_4,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_5,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_6,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_7,         GPIO_PuPd_KEE,     GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // UART_LOG_TXD
{ _PA_8,         GPIO_PuPd_KEE,     GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // UART_LOG_RXD
{ _PA_9,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_10,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_11,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_12,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_13,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_14,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_15,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_16,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_17,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_18,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_19,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_20,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_21,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_22,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_23,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_24,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_25,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_26,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   TRUE}, // keyscan
{ _PA_27,        GPIO_PuPd_KEE,     GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // SWD_DATA or normal_mode_sel
{ _PA_28,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_29,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_30,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PA_31,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_0,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_1,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_2,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_3,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // SWD_CLK
{ _PB_4,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // Touch0
{ _PB_5,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // Touch1
{ _PB_6,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // Touch2
// { _PB_6,       GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_7,         GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // Touch3
{ _PB_8,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_9,         GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_10,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_11,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_12,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // SPI_DATA3
{ _PB_13,        GPIO_PuPd_UP,      GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   FALSE}, // SPI_CLK
{ _PB_14,        GPIO_PuPd_UP,      GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   FALSE}, // SPI_DATA0
{ _PB_15,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // SPI_DATA2
{ _PB_16,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // SPI_CS
{ _PB_17,        GPIO_PuPd_UP,      GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   FALSE}, // SPI_DATA1
{ _PB_18,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_19,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_20,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_21,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_22,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // IR pin should no-pull when you use IR
// { _PB_22,      GPIO_PuPd_NOPULL,  GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_23,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_24,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_25,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_26,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // fix antenna diversity can not use issue
{ _PB_27,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_28,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_29,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_30,        GPIO_PuPd_UP,      GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PB_31,        GPIO_PuPd_DOWN,   GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, //
{ _PNC,          GPIO_PuPd_KEE,     GPIO_PuPd_KEE,   GPIO_PuPd_KEE,   FALSE}, // table end
};
```

16.4 sleepcfg.c

This file should be kept the default settings, you can only change it with Realtek RD's help.

16.5 bootcfg.c

Bootloader can be configured through this file, as Table 16-6 and Fig 16-8 show.

Table 16-6 User bootcfg

Configuration items	Comment
Flash_MMU_Config	Used for MMU configuration. OTA2 address mapping should be settled here.
OTA_Region	OTA start address
RSIP_Mask_Config	RSIP mask configure
Force_OTA1_GPIO	Force OTA1 GPIO configuration
Boot_Log_En	Used to disable Realtek boot log
FwCheckCallback	Firmware verify callback handler
OTASelectHook	Firmware select hook

```

/*
 * @brief MMU Configuration.
 * There are 8 MMU entries totally. Entry 0 & Entry 1 are already used by OTA, Entry 2~7 can be used by Users.
 */
BOOT_RAM_DATA_SECTION
MMU_ConfDefFlash_MMU_Config[] = {
    /*VAddrStart, VAddrEnd, PAddrStart, PAddrEnd*/
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 2
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 3
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 4
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 5
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 6
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}, //Entry 7
    {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF},
};

/*
 * @brief OTA start address. Because KM0 & KM4 IMG2 are combined, users only need to set the start address
 * of KM0 IMG2.
 */
BOOT_RAM_DATA_SECTION
u32 OTA_Region[2] = {
    0x08006000, /* OTA1 region start address */
    0x08106000, /* OTA2 region start address */
};

/*
 * @brief RSIP Mask Configuration.
 * There are 4 RSIP mask entries totally. Entry 0 is already used by System Data, Entry 3 is reserved by Realtek.
 * Only Entry 1 & Entry 2 can be used by Users.
 * MaskAddr: start address for RSIP Mask, should be 4KB aligned
 * MaskSize: size of the mask area, unit is 4KB
 */
BOOT_RAM_DATA_SECTION
RSIP_MaskDefRSIP_Mask_Config[] = {
    /*MaskAddr, MaskSize*/
    {0x08001000, 3}, //Entry 0: 4K system data & 4K backup & DPK

    /* customer can set here */
    {0xFFFFFFFF, 0xFFFF}, //Entry 1: can be used by users
    {0xFFFFFFFF, 0xFFFF}, //Entry 2: can be used by users
    {0xFFFFFFFF, 0xFFFF}, //Entry 3: Reserved by Realtek. If RDP is not used, this entry can be used by users.

    /* End */
    {0xFFFFFFFF, 0xFFFF},
};

/**
 * @brief GPIO force OTA1 as image2. 0xFF means force OTA1 trigger is disabled.
 * BIT[7]: active level, 0 is low level active, 1 is high level active,
 * BIT[6:5]: port, 0 is PORT_A, 1 is PORT_B
 * BIT[4:0]: pin num 0~31
 */
BOOT_RAM_DATA_SECTION
u8 Force_OTA1_GPIO = 0xFF;

/**
 * @brief boot log enable or disable.
 * FALSE: disable
 * TRUE: enable
 */
BOOT_RAM_DATA_SECTION
u8 Boot_Log_En = FALSE;

/**
 * @brief Firmware verify callback handler.
 * If users need to verify checksum/ hash for image2, implement the function and assign the address
 * to this function pointer.
 * @param None
 * @retval 1: Firmware verify successfully, 0: verify failed
 */
BOOT_RAM_DATA_SECTION
FuncPtr FwCheckCallback = NULL;

/**
 * @brief Firmware select hook.
 * If users need to implement own OTA select method, implement the function and assign the address
 * to this function pointer.
 * @param None
 * @retval 0: both firmwares are invalid, select none, 1: boot from OTA1, 2: boot from OTA2
 */
BOOT_RAM_DATA_SECTION
FuncPtr OTASelectHook = NULL;

```

Fig 16-8 User bootcfg

16.6 ipccfg.c

You can add IPC entry referring to Fig 16-9, so that KM4 & KM0 can send message to each other by the method you defined.

For USER_MSG_TYPE:

- IPC_USER_POINT: Message in IRQFUNC is a pointer (address).
- IPC_USER_DATA: Message in IRQFUNC is just a value.

```
#if defined (ARM_CORE_CM4)
const IPC_INIT_TABLE ipc_init_config[] =
{
    // USER_MSG_TYPE      IRQFUNC      IRQDATA
    {IPC_USER_DATA,      shell_switch_ipc_int, (VOID*) NULL}, //channel 0: IPC_INT_CHAN_SHELL_SWITCH
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 1: IPC_INT_CHAN_WIFI_FW
    {IPC_USER_DATA,      FLASH_Write_IPC_Int, (VOID*) NULL}, //channel 2: IPC_INT_CHAN_FLASHPG_REQ
    {IPC_USER_POINT, NULL, (VOID*) NULL}, //channel 3: IPC_INT_KM4_TICKLESS_INDICATION
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 4: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 5: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 6: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 7: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 8: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 9: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 10: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 11: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 12: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 13: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 14: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 15: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 16: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 17: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 18: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 19: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 20: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 21: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 22: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 23: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 24: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 25: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 26: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 27: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 28: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 29: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 30: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 31: Reserved for Customer use
},
{0xFFFFFFFF, OFF, OFF}, /* Table end */
};
#else
#if CONFIG_WIFI_FW_EN
extern void driver_fw_flow_ipc_int(VOID *Data, u32 IrqStatus, u32 ChanNum);
#define fw_flow_ipc_int driver_fw_flow_ipc_int
#else
#define fw_flow_ipc_int NULL
#endif
const IPC_INIT_TABLE ipc_init_config[] =
{
    // USER_MSG_TYPE      IRQFUNC      IRQDATA
    {IPC_USER_DATA,      shell_switch_ipc_int, (VOID*) NULL}, //channel 0: IPC_INT_CHAN_SHELL_SWITCH
    {IPC_USER_DATA,      fw_flow_ipc_int, (VOID*) IPCM4_DEV}, //channel 1: IPC_INT_CHAN_WIFI_FW
    {IPC_USER_DATA,      FLASH_Write_IPC_Int, (VOID*) NULL}, //channel 2: IPC_INT_CHAN_FLASHPG_REQ
    {IPC_USER_POINT, km4_tickless_ipc_int, (VOID*) NULL}, //channel 3: IPC_INT_KM4_TICKLESS_INDICATION
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 4: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 5: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 6: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 7: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 8: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 9: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 10: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 11: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 12: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 13: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 14: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 15: Reserved for Realtek use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 16: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 17: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 18: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 19: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 20: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 21: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 22: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 23: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 24: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 25: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 26: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 27: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 28: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 29: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 30: Reserved for Customer use
    {IPC_USER_DATA,      NULL, (VOID*) NULL}, //channel 31: Reserved for Customer use
},
{0xFFFFFFFF, OFF, OFF}, /* Table end */
};
#endif
```

Fig 16-9 User ipccfg

NOTE

Use IPC channel 16~31 and IPC semaphore index 8~15, Channel 0~15 and semaphore index 0~7 are reserved for Realtek use.

16.7 lp_intfcfg.c

UART low power Rx enable/disable can be configured through this file.

```
UARTCFG_TypeDef uart_config[2]=
{
    /* UART0 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE, /* Enable low power RX*/
    },
    /* UART1 */
    {
        .LOW_POWER_RX_ENABLE = DISABLE,
    },
};
```

16.8 hp_intfcfg.c

PSRAM parameters, SDIO host parameters and FTL parameters can be configured through this file.

```
#include "ameba_soc.h"
#include "autoconf.h"

PSRAMCFG_TypeDef psram_dev_config = {
    .psram_dev_enable = FALSE,           // enable psram
    .psram_dev_cal_enable = FALSE,       // enable psram calibration function
    .psram_dev_retention = FALSE,        // enable psram retention
};

SDIOHCFG_TypeDef sdioh_config = {
    .sdioh_bus_speed = SD_SPEED_HS,      // SD_SPEED_DS or SD_SPEED_HS
    .sdioh_bus_width = SDIOH_BUS_WIDTH_4BIT, // SDIOH_BUS_WIDTH_1BIT or SDIOH_BUS_WIDTH_4BIT
    .sdioh_cd_pin = _PB_25,              // _PB_25/_PA_6/_PNC
    .sdioh_wp_pin = _PNC,                // _PB_24/_PA_5/_PNC
};

#if defined(CONFIG_FTL_ENABLED)
#define FTL_MEM_CUSTEM 0
#elif FTL_MEM_CUSTEM == 0
#error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, Please refer to Application Note, FTL chapter."
#else
const u8 ftl_phy_page_num = 3;           /* The number of physical map pages, default is 3*/
const u32 ftl_phy_page_start_addr = 0x00102000; /* The start offset of flash pages which is allocated to FTL physical map.
Users should modify it according to their own memory layout! */
#endif
#endif
```

17 Flash Operation

17.1 Functional Description

Ameba-D uses SPI Controller (SPIC) to communicate with SPI NOR Flash.

There are two operation modes for SPIC: auto mode and user mode.

- User mode: A typical software flow to implement all serial transfer. User can transmit or receive data from SPI Flash through setting up SPIC registers.
- Auto mode: Compared to user mode, auto mode is a hardware control flow to execute. After initialize setting, user don't need to configure the related control register for each transfer operation. Auto mode is a convenient way to access SPI Flash just as access memory (SRAM or DRAM).

The SPIC is initialized automatically in boot sequence, and user must not initialize SPIC manually. The supported mode for different operations can be found in Table 17-1.

Table 17-1 SPIC operation mode

Flash Operation	SPIC Operation Mode
Read data	Auto mode/User mode
Program data	User mode
Erase	User mode
Receive/transmit command	User mode

Note: Auto mode and user mode can't be used at the same time.

17.2 Protection Method

Considering Ameba-D supports Flash executed in place (XIP), which enables code execute directly in Flash memory without copying to RAM. Both KM0 and KM4 CPU cores can issue request to SPIC in auto mode to read instructions from Flash to execute.

To prevent being interrupted by auto mode reading, the user mode operation should be protected until it is finished. Fig 17-1 shows the flow of KM0 manipulating Flash safely in user mode, which has been protected. The method of KM4 manipulating Flash is in the same way.

The protection methods are implemented in **FLASH_Write_Lock()** and **FLASH_Write_Unlock()** functions. The APIs provided by Realtek in 17.3 and 17.4 are already protected and can be called by users directly.

Note: If users need to implement user mode function by themselves, the code of manipulating SPIC should locate in RAM instead of Flash. They should also call **FLASH_Write_Lock()** before operation and call **FLASH_Write_Unlock()** to release from protection after operation.

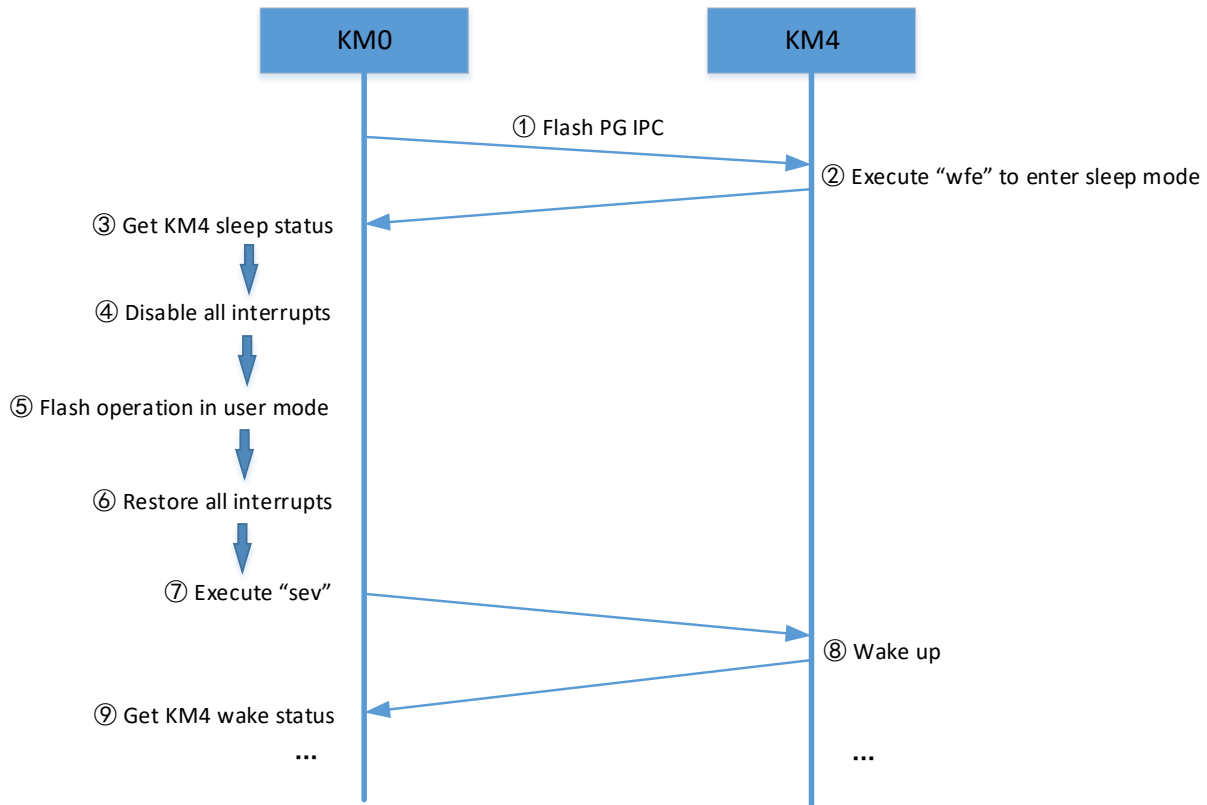


Fig 17-1 User mode operation flow

17.3 Flash Raw APIs

17.3.1 Read

17.3.1.1 HAL_READ8/HAL_READ16/HAL_READ32

Items	Description
Introduction	Read 1-byte/1-word/1-dword data from Flash in auto mode, just as access SRAM.
Parameters	- base: the base address of Flash memory. It should be SPI_FLASH_BASE which is 0x08000000. - addr: the offset address in Flash memory. It should be byte-aligned for HAL_READ8, word-aligned for HAL_READ16 and dword-aligned for HAL_READ32.
Return	The read data

17.3.1.2 FLASH_ReadStream

Items	Description
Introduction	Read a stream of data from specified offset address of Flash in auto mode.
Parameters	- address: specify the starting offset address to read from. It has no alignment restrictions. - len: specify the length of the data to read. - data: specify the buffer to save the read-back data.
Return	Status 1: success - Others: fail

17.3.2 Write

17.3.2.1 FLASH_TxData12BXIP

Items	Description
Introduction	Write data to flash in user mode. This function is already protected and can be called by user directly.
Parameters	- StartAddr: start offset address in Flash from which SPIC writes. - DataPhaseLen: the number of bytes that SPIC sends in Data Phase. It should not be more than 12 bytes. - pData: pointer to a byte array that is to be sent.
Return	N/A

17.3.2.2 FLASH_WriteStream

Items	Description
Introduction	Write a stream of data to specified address in user mode. This function is already protected and can be called by user directly.
Parameters	- address: specifies the starting offset to write to. - len: specifies the length of the data to write. It has no restrictions of less than 12 bytes. - data: pointer to a byte array that is to be written.
Return	Status 1: success - Others: fail

17.3.3 Erase

17.3.3.1 FLASH_EraseXIP

Items	Description
Introduction	Erase sector/block/chip for Flash in user mode. The function is already protected and can be called by users directly.
Parameters	- EraseType: can be one of the following value. - EraseChip: erase the whole chip. - EraseBlock: erase the specified block (64KB). - EraseSector: erase the specified sector (4KB). - Address: address should be 4-byte aligned. The block/sector which the address in will be erased.
Return	N/A

17.3.4 Receive/Transmit Command

17.3.4.1 FLASH_RxCmdXIP

Items	Description
Introduction	Send Rx command to Flash to get status register or Flash ID in user mode. This function is already protected and can be called by user directly.
Parameters	- cmd: command that need to be sent. - read_len: the number of bytes that will be read by SPIC after sending command. - read_data: pointer to a byte array which is used to save data received.
Return	N/A

17.3.4.2 FLASH_SetStatusXIP

Items	Description
-------	-------------

Introduction	Set Flash status register in user mode. The function is already protected and can be called by users directly.
Parameters	- cmd: command to be sent. - len: the number of bytes to be sent after sending command. - status: pointer to byte array to be sent.
Return	N/A

17.4 Flash Mbed APIs

The descriptions of Flash Mbed APIs can be found in *AN403 Peripheral Driver Mbed API.chm*. All these functions have already been protected and can be called directly.

17.5 User Configuration

Ameba-D provides `rtl8721dlp_flashcfg.c` to configure Flash speed and read mode.

```
/**
 * @brief Indicate the flash baudrate. It can be one of the following value:
 *      0xFFFF: 80MHz
 *      0x7FFF: 100MHz
 *      0x3FFF: 67MHz
 *      0x1FFF: 57MHz
 *      0x0FFF: 50MHz
 */
const u16 Flash_Speed = 0xFFFF;
```

Fig 17-2 Configuring Flash speed

```
/**
 * @brief Indicate the Flash read I/O mode. It can be one of the following value:
 *      0xFFFF: Read quad I/O, Address & Data 4 bits mode
 *      0x7FFF: Read quad 0, Just data 4 bits mode
 *      0x3FFF: Read dual I/O, Address & Data 2 bits mode
 *      0x1FFF: Read dual 0, Just data 2 bits mode
 *      0x0FFF: 1 bit mode
 * @note If the configured read mode is not supported, other modes would be searched until find the appropriate mode.
 */
const u16 Flash_ReadMode = 0xFFFF;
```

Fig 17-3 Configuring Flash read mode

Note: For some Flash chip which is not available in Flash AVL, the principle to determine whether it can be supported by SDK, and the method to add a new Flash can be found in User Configuration.

18 Battery Measurement

18.1 Functional Description

LP-ADC has a total of 11 channels, of which 8 are external channels and 3 are internal channels. 8 external battery measurement channels include 7 normal channels and 1 VBAT channel, as Table 18-1 shows. The VBAT channel measurable range is 0~5V.

Table 18-1 Channel description

Function	ADC Channel	Pin Name	Voltage Range
Normal channel	CH0 ~ CH6	PB[4] ~ PB[7] (CH0 ~ CH3)	0 ~ 3.3V (when power is 3.3V)
		PB[1] ~ PB[3] (CH4 ~ CH6)	0 ~ 1.8V (when power is 1.8V)
VBAT	CH7	VBAT_MEAS	0 ~ 5V

When one channel is added to channel switch list, LP-ADC would convert the voltage of corresponding pin to digital data. The resolution of digital sample data is 12-bit.

To obtain sample data, auto mode, timer-trigger mode and software-trigger mode can be adopted according to different usages. LP-ADC can also be configured to send wakeup and interrupt signal to system when the sample data matches criteria.

18.2 Calibration

The relationship between input voltage and sample data is almost linear. Fig 18-1 shows the conversion of sample data and input voltage of VBAT_MEAS.

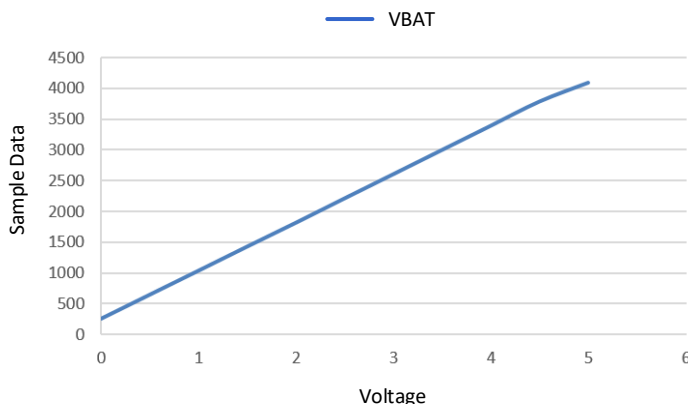


Fig 18-1 Relationship between sample data and input voltage

To obtain voltage from sample data, the following formula can be used:

$$\text{Voltage} = \frac{10 * \text{Data} - \text{OFFSET}}{\text{GAIN}} \text{ (V)}$$

Where

- Data: 12-bit sample data
- OFFSET: 10 times of sample data of 0.000V
- GAIN: 10 times of sample data increment when voltage rises 1V

To enhance conversion accuracy, ADC calibration is necessary to obtain precise OFFSET and GAIN parameters for channels.

The calibration operations can be done in FT test. The FT test would sample for different voltages and use Linear Fitting Method to calculate OFFSET and GAIN. Pay attention that the OFFSET and GAIN values gotten from FT test are 10 times of the actual values. When calculating voltage using the above two values, they must be divided by 10 first; that is,

$$\text{Voltage} = \frac{\text{Data} - \frac{\text{OFFSET}}{10}}{\frac{\text{GAIN}}{10}} = \frac{10 * \text{Data} - \text{OFFSET}}{\text{GAIN}} \text{ (V)}$$

As normal external channels use the same voltage divider circuit, they can use one set of calibration parameters. But VBAT channel need to be calibrated independently. These parameters would be programmed into eFuse after calibration, as Table 18-2 shows.

Table 18-2 Calibration parameters location

Channel	Input Mode	Parameter	Address
Normal (CH0 ~ CH6)	Single-ended	OFFSET	Physical map 0x1D0 ~ 0x1D1
		GAIN	Physical map 0x1D2 ~ 0x1D3
	Differential	OFFSET	Physical map 0x1D8 ~ 0x1D9
		GAIN	Physical map 0x1DA ~ 0x1DB
VBAT (CH7)	Single-ended	OFFSET	Physical map 0x1D4 ~ 0x1D5
		GAIN	Physical map 0x1D6 ~ 0x1D7

To calculate voltage from sample data, user only need to obtain calibration parameters from eFuse and call the voltage computational formula.

19 Wi-Fi

19.1 Wi-Fi Data Structures

Data Structures	Introduction
<_cus_ie>	The structure is used to set Wi-Fi custom IE list, and type match CUSTOM_IE_TYPE. The IE will be transmitted according to the type.
<rtw_ssid>	The structure is used to describe the SSID.
<rtw_mac>	The structure is used to describe the unique 6-byte MAC address.
<rtw_ap_info>	The structure is used to describe the setting about SSID, security type, password and default channel, used to start AP mode.
<rtw_network_info>	The structure is used to describe the station mode setting about SSID, security type and password, used when connecting to an AP.
<rtw_scan_result>	The structure is used to describe the scan result of the AP.
<rtw_scan_handler_result>	The structure is used to describe the data needed by scan result handler function.
<rtw_wifi_setting>	The structure is used to store the Wi-Fi setting gotten from Wi-Fi driver.
<rtw_wifi_config>	The structure is used to describe the setting when configure the network.
<rtw_maclist_t>	The structure is used to describe the maclist.
<rtw_bss_info_t>	The structure is used to describe the bss info of the network. It includes the version, BSSID, beacon_period, capability, SSID, channel, atm_window, dtim_period, RSSI etc.

19.2 Wi-Fi APIs

19.2.1 System APIs

API	Introduction
<wifi_on>	Enable Wi-Fi.
<wifi_off>	Disable Wi-Fi.
<wifi_is_up>	Check if the specified interface is up.
<wifi_is_ready_to_transceive>	Determine if a particular interface is ready to transceiver Ethernet packets.
<wifi_rf_on>	Enable Wi-Fi RF.
<wifi_rf_off>	Disable Wi-Fi RF.

19.2.1.1 wifi_on

Enable Wi-Fi: bring the wireless interface “Up”.

Parameter	Type	Introduction
<mode>	rtw_mode_t	Decide to enable Wi-Fi in which mode. The optional modes are enumerated in rtw_mode_t.

19.2.1.2 wifi_off

Disable Wi-Fi.

Parameter: None.

19.2.1.3 wifi_is_up

Check if the specified interface is up.

Parameter	Type	Introduction
<interface>	rtw_interface_t	The interface can be set as RTW_STA_INTERFACE or RTW_AP_INTERFACE (rtw_interface_t).

19.2.1.4 wifi_is_ready_to_transceive

Determine if a particular interface is ready to transceiver Ethernet packets.

Parameter	Type	Introduction
<interface>	rtw_interface_t	Interface to check ● RTW_STA_INTERFACE ● RTW_AP_INTERFACE

19.2.2 Scan APIs

API	Introduction
<wifi_scan>	Initiate a scan to search for 802.11 networks
<wifi_scan_networks>	Simplify the scan operation based on wifi_scan to search for 802.11 networks
<wifi_scan_networks_with_ssid>	Initiate a scan to search for 802.11 networks with specified SSID

19.2.2.1 wifi_scan

Initiate a scan to search for 802.11 networks.

Parameter	Type	Introduction
<scan_type>	rtw_scan_type_t	Specifies whether the scan should be active, passive or scan prohibited channels.
<bss_type>	rtw_bss_type_t	Specifies whether the scan should search for infrastructure networks (those using an AP), Ad-hoc networks, or both types.
<result_ptr>	void *	Scan specific ssid. The first 4 bytes is ssid length, and ssid name append after it. If no specific ssid need to scan, please clean result_ptr before pass it into parameter.
< result_ptr >	void *	[out] a pointer to a pointer to a result storage structure.

Note:

- The scan progressively accumulates results over time, and may take 1 ~ 6 seconds to complete. The results of the scan will be individually provided to the callback function.
- The callback function will be executed in the context of the RTW thread.
- When scanning specific channels, devices with a strong signal strength on nearby channels may be detected.

19.2.2.2 wifi_scan_networks

Initiate a scan to search for 802.11 networks, a higher level API based on wifi_scan to simplify the scan operation.

Parameter	Type	Introduction
<results_handler>	rtw_scan_result_handler_t	The callback function which will receive and process the result data.
<user_data>	void *	User specified data that will be passed directly to the callback function.

Note: Callback must not use blocking functions, since it is called from the context of the RTW thread. The callback, user_data variables will be referenced after the function returns. Those variables must remain valid until the scan is completed. The usage of this API can refer to ATWS in atcmd_wifi.c.

19.2.2.3 wifi_scan_networks_with_ssid

Initiate a scan to search for 802.11 networks with specified SSID.

Parameter	Type	Introduction
<results_handler>	int	The callback function which will receive and process the result data.
<user_data>	void *	User specified data that will be passed directly to the callback function.
<scan_bufalen>	int	The length of the result storage structure.
<ssid>	char *	The SSID of target network.
<ssid_len>	int	The length of the target network SSID.

Note: Callback must not use blocking functions, since it is called from the context of the RTW thread. The callback, user_data variables will be referenced after the function returns. Those variables must remain valid until the scan is completed.

19.2.3 Connection APIs

API	Introduction
<wifi_connect>	Join a Wi-Fi network with specified SSID.
<wifi_connect_bssid>	Join a Wi-Fi network with specified BSSID.
<wifi_disconnect>	Disassociates from current Wi-Fi network.
<wifi_is_connected_to_ap>	Check if Wi-Fi has connected to AP before dhcp.
<wifi_config_autoreconnect>	Set reconnection mode with configuration.
<wifi_set_autoreconnect>	Set reconnection mode with 3 retry limit and 5 seconds timeout as default.
<wifi_get_autoreconnect>	Get the result of setting reconnection mode.
<wifi_get_last_error>	Present the device disconnect reason while connecting.

19.2.3.1 wifi_connect

Join a Wi-Fi network with specified SSID. Scan for, associate and authenticate with a Wi-Fi network. On successful return, the system is ready to send data packets.

Parameter	Type	Introduction
<ssid>	char *	A null terminated string containing the SSID name of the network to join.
<security_type>	rtw_security_t	Authentication type: ● RTW_SECURITY_OPEN: open security ● RTW_SECURITY_WEP_PSK: WEP Security with open authentication ● RTW_SECURITY_WEP_SHARED: WEP Security with shared authentication ● RTW_SECURITY_WPA_TKIP_PSK: WPA Security ● RTW_SECURITY_WPA2_AES_PSK: WPA2 Security using AES cipher ● RTW_SECURITY_WPA2_TKIP_PSK: WPA2 Security using TKIP cipher ● RTW_SECURITY_WPA2_MIXED_PSK: WPA2 Security using AES and/or TKIP ciphers
<password>	char *	A byte array containing either the cleartext security key for WPA/WPA2 secured networks, or a pointer to an array of rtw_wep_key_t structures for WEP secured networks.
<ssid_len>	int	The length of the SSID in bytes.
<password_len>	int	The length of the security_key in bytes.
<key_id>	int	The index of the wep key (0, 1, 2, or 3). If not using it, leave it with value -1.
<semaphore>	void *	A user provided semaphore is flagged when the join is complete. If not using, leave it with NULL value.

Note: Make sure the Wi-Fi is enabled before invoking this function. (wifi_on())

19.2.3.2 wifi_connect_bssid

Join a Wi-Fi network with specified BSSID. Scan for, associate and authenticate with a Wi-Fi network. On successful return, the system is ready to send data packets.

Parameter	Type	Introduction
<bssid>	unsigned char	The specified BSSID to connect.

<ssid>	char *	A null terminated string containing the SSID name of the network to join.
<security_type>	rtw_security_t	Authentication type: ● RTW_SECURITY_OPEN: open security ● RTW_SECURITY_WEP_PSK: WEP Security with open authentication ● RTW_SECURITY_WEP_SHARED: WEP Security with shared authentication ● RTW_SECURITY_WPA_TKIP_PSK: WPA Security ● RTW_SECURITY_WPA2_AES_PSK: WPA2 Security using AES cipher ● RTW_SECURITY_WPA2_TKIP_PSK: WPA2 Security using TKIP cipher ● RTW_SECURITY_WPA2_MIXED_PSK: WPA2 Security using AES and/or TKIP ciphers
<password>	char *	A byte array containing either the cleartext security key for WPA/WPA2 secured networks, or a pointer to an array of rtw_wep_key_t structures for WEP secured networks.
<ssid_len>	int	The length of the SSID in bytes.
<password_len>	int	The length of the security_key in bytes.
<key_id>	int	The index of the wep key (0, 1, 2, or 3). If not using it, leave it with value -1.
<semaphore>	void *	A user provided semaphore is flagged when the join is complete. If not using, leave it with NULL value.

Note:

- Make sure the Wi-Fi is enabled before invoking this function. (wifi_on())
- The difference between wifi_connect_bssid() and wifi_connect() is that BSSID has higher priority as the basis of connection in wifi_connect_bssid().

19.2.3.3 wifi_disconnect

Disassociates from current Wi-Fi network.

Parameter: None.

19.2.3.4 wifi_is_connected_to_ap

Check if Wi-Fi has connected to AP before dhcp.

Parameter: None.

19.2.3.5 wifi_config_autoreconnect

Set reconnection mode with configuration.

Parameter	Type	Introduction
<mode>	_u8	Set 1/0 to enable/disable the reconnection mode.
<callback>	_u8	The number of retry limit.
<len_used>	_u16	The timeout value (in seconds).

Note: The difference between **wifi_config_autoreconnect()** and **wifi_set_autoreconnect()** is that user can specify the retry times and timeout value in **wifi_config_autoreconnect()**. But in **wifi_set_autoreconnect()** these values are set with 8 retry limit and 5 seconds timeout as default.

19.2.3.6 wifi_set_autoreconnect

Set reconnection mode with 8 retry limit and 5 seconds timeout as default.

Parameter	Type	Introduction
<mode>	_u8	Set 1/0 to enable/disable the reconnection mode.

Note: The difference between **wifi_config_autoreconnect()** and **wifi_set_autoreconnect()** is that user can specify the retry times and timeout value in **wifi_config_autoreconnect()**. But in **wifi_set_autoreconnect()** these values are set with 8 retry limit and 5 seconds timeout as default.

19.2.3.7 wifi_get_autoreconnect

Get the result of setting reconnection mode.

Parameter	Type	Introduction
<mode>	_u8	Pointer to the result of setting reconnection mode.

19.2.3.8 wifi_get_last_error

Present the device disconnect reason while connecting.

Parameter: None.

19.2.4 Channel APIs

API	Introduction
<wifi_set_channel>	Set the listening channel on STA interface.
<wifi_get_channel>	Get the current channel on STA interface.
<wifi_set_channel_plan>	Set channel plan into eFuse, must reboot after setting channel plan.
<wifi_get_channel_plan>	Get channel plan from eFuse.

19.2.4.1 wifi_set_channel

Set the listening channel on STA interface.

Parameter	Type	Introduction
<channel>	int	The desired channel.

Note: Do not need to call this function for STA mode Wi-Fi driver, since it will be determined by the channel from received beacon.

19.2.4.2 wifi_get_channel

Get the current channel on STA interface.

Parameter	Type	Introduction
<channel>	int *	A pointer to the variable where the channel value will be written.

19.2.4.3 wifi_set_channel_plan

Set channel plan into eFuse, must reboot after setting channel plan.

Parameter	Type	Introduction
<channel_plan>	uint8_t	The value of channel plan, define in wifi_constants.h

19.2.4.4 wifi_get_channel_plan

Get channel plan from eFuse.

Parameter	Type	Introduction
<channel_plan>	uint8_t	Point to the value of channel plan, define in wifi_constants.h

19.2.5 Power APIs

API	Introduction
<wifi_enable_powersave>	Enable Wi-Fi power save mode.
<wifi_disable_powersave>	Disable Wi-Fi power save mode.
<wifi_get_txpower>	Get the Tx power in index units.
<wifi_set_txpower>	Set the Tx power in index units.
<wifi_set_power_mode>	Set IPS/LPS mode.
<wifi_set_lps_dtim>	Set LPS DTIM.
<wifi_get_lps_dtim>	Get LPS DTIM.

19.2.5.1 wifi_enable_powersave

Enable Wi-Fi power save mode. When power save mode is enabled, RF will wake up only when there is data to be sent or beacon to be received. Otherwise, RF will be awake all the time.

Parameter: None.

19.2.5.2 wifi_disable_powersave

Disable Wi-Fi power save mode.

Parameter: None.

19.2.5.3 wifi_get_txpower

Get the Tx power in index units.

Parameter	Type	Introduction
<poweridx>	int *	The variable to receive the Tx power in index.

19.2.5.4 wifi_set_txpower

Set the Tx power in index units.

Parameter	Type	Introduction
<poweridx>	int	The desired Tx power in index.

19.2.5.5 wifi_set_power_mode

Set IPS/LPS mode.

Parameter	Type	Introduction
<ips_mode>	unsigned char	The desired IPS mode. It becomes effective when WLAN enters IPS. ips_mode is inactive power save mode. Wi-Fi automatically turns RF off if the device is not associated to AP. Set 1 to enable inactive power save mode.
<lps_mode>	unsigned char	The desired LPS mode. It becomes effective when WLAN enters LPS. lps_mode is leisure power save mode. Wi-Fi turns RF on to listen to beacon automatically and periodically during the association to AP. If traffic is not busy, it also automatically turns RF off. Set 1 to enable leisure power save mode.

19.2.5.6 wifi_set_lps_dtim

Set LPS DTIM.

Parameter	Type	Introduction
<dtim>	unsigned char	In LPS, the package can be buffered at AP side. STA leaves LPS until DTIM count of packages buffered at AP side.

Note: DTIM is the duration to listen beacon. The default DTIM is 1, which is 100ms. If DTIM is set bigger than 1, the advantage is that power can be saved and the disadvantage is that STA has the risk of losing packets.

19.2.5.7 wifi_get_lps_dtim

Get LPS DTIM.

Parameter	Type	Introduction
<dtim>	unsigned char *	In LPS, the package can be buffered at AP side. STA leaves LPS until DTIM count of packages buffered at AP side.

19.2.6 AP Mode APIs

API	Introduction
<wifi_start_ap>	Trigger Wi-Fi driver to start an infrastructure Wi-Fi network.
<wifi_start_ap_with_hidden_ssid>	Start an infrastructure Wi-Fi network with hidden SSID.
<wifi_restart_ap>	Trigger Wi-Fi driver to restart an infrastructure Wi-Fi network.
<wifi_get_associated_client_list>	Get the associated clients with SoftAP.
<wifi_enable_forwarding>	Enable packets forwarding in AP mode.
<wifi_disable_forwarding>	Disable packets forwarding in AP mode.

19.2.6.1 wifi_start_ap

Trigger Wi-Fi driver to start an infrastructure Wi-Fi network.

Parameter	Type	Introduction
<ssid>	char *	A null terminated string containing the SSID name of the network.
<security_type>	security_type	- RTW_SECURITY_OPEN: Open Security - RTW_SECURITY_WPA_TKIP_PSK: WPA Security - RTW_SECURITY_WPA2_AES_PSK: WPA2 Security using AES cipher - RTW_SECURITY_WPA2_MIXED_PSK: WPA2 Security using AES and/or TKIP ciphers WEP security is NOT IMPLEMENTED. It is NOT SECURE!
<password>	char *	A byte array containing the cleartext security key for the network.
<ssid_len>	int	The length of the SSID in bytes.
<password_len>	int	The length of the security_key in bytes.
<channel>	int	802.11 channel number.

Note:

- Make sure the Wi-Fi is enabled before invoking this function. (wifi_on())
- If a STA interface is active when this function is called, the softAP will start on the same channel as the STA. It will not use the channel provided.

19.2.6.2 wifi_start_ap_with_hidden_ssid

Start an infrastructure Wi-Fi network with hidden SSID.

Parameter	Type	Introduction
<ssid>	char *	A null terminated string containing the SSID name of the network.
<security_type>	security_type	● RTW_SECURITY_OPEN: Open Security ● RTW_SECURITY_WPA_TKIP_PSK: WPA Security ● RTW_SECURITY_WPA2_AES_PSK: WPA2 Security using AES cipher ● RTW_SECURITY_WPA2_MIXED_PSK: WPA2 Security using AES and/or TKIP ciphers WEP security is NOT IMPLEMENTED. It is NOT SECURE!
<password>	char *	A byte array containing the cleartext security key for the network.
<ssid_len>	int	The length of the SSID in bytes.
<password_len>	int	The length of the security_key in bytes.
<channel>	int	802.11 channel number.

Note: if a STA interface is active when this function is called, the softAP will start on the same channel as the STA. It will not use the channel provided.

19.2.6.3 wifi_restart_ap

Trigger Wi-Fi driver to restart an infrastructure Wi-Fi network.

Parameter	Type	Introduction
<ssid>	unsigned char *	A null terminated string containing the SSID name of the network.
<security_type>	rtw_security_t	● RTW_SECURITY_OPEN: Open Security ● RTW_SECURITY_WPA_TKIP_PSK: WPA Security ● RTW_SECURITY_WPA2_AES_PSK: WPA2 Security using AES cipher ● RTW_SECURITY_WPA2_MIXED_PSK: WPA2 Security using AES and/or TKIP ciphers WEP security is NOT IMPLEMENTED. It is NOT SECURE!
<password>	unsigned char *	A byte array containing the cleartext security key for the network.
<ssid_len>	int	The length of the SSID in bytes.
<password_len>	int	The length of the security_key in bytes.
<channel>	int	802.11 channel number.

Note:

- Make sure the Wi-Fi is enabled before invoking this function. (wifi_on())
- If a STA interface is active when this function is called, the softAP will start on the same channel as the STA. It will not use the channel provided.

19.2.6.4 wifi_get_associated_client_list

Get the associated clients with SoftAP.

Parameter	Type	Introduction
<client_list_buffer>	int *	The location where the client list will be stored.
<buffer_length>	unsigned short	The buffer length.

19.2.6.5 wifi_enable_forwarding

Enable packets forwarding in AP mode.

Parameter: None.

19.2.6.6 wifi_disable_forwarding

Disable packets forwarding in AP mode.

Parameter: None.

19.2.7 Custom IE APIs

API	Introduction
<wifi_add_custom_ie>	Setup custom IE list.
<wifi_update_custom_ie>	Update the item in Wi-Fi custom IE list.
<wifi_del_custom_ie>	Delete Wi-Fi custom IE list.

Note: These three APIs are only effective on beacon, probe request and probe response frames.

19.2.7.1 wifi_add_custom_ie

Setup custom IE list.

Parameter	Type	Introduction
<cus_ie>	void *	Pointer to Wi-Fi CUSTOM IE list.
<ie_num>	int	The number of Wi-Fi CUSTOM IE list.

Note: This API can't be executed twice before deleting the previous custom IE list.

19.2.7.2 wifi_update_custom_ie

Update the item in Wi-Fi custom IE list.

Parameter	Type	Introduction
<cus_ie>	void *	Pointer to Wi-Fi CUSTOM IE list.
<ie_index>	int	The number of Wi-Fi CUSTOM IE list.

19.2.7.3 wifi_del_custom_ie

Delete Wi-Fi custom IE list.

Parameter: None.

19.2.8 Wi-Fi Setting APIs

API	Introduction
<wifi_get_mac_address>	Retrieves the current Media Access Control (MAC) address (or Ethernet hardware address) of the 802.11 device.
<wifi_get_ap_bssid>	Get connected AP's BSSID.
<wifi_get_ap_info>	Get the SoftAP information.
<wifi_set_country>	Set the country code to driver to determine the channel set.
<wifi_get_sta_max_data_rate>	Retrieved STA mode max. data rate.
<wifi_get_rssi>	Retrieved the latest RSSI value.
<wifi_register_multicast_address>	Register interest in a multicast address. Once a multicast address has been registered, all packets detected on the medium destined for that address are forwarded to the host. Otherwise they are ignored.
<wifi_unregister_multicast_address>	Unregister interest in a multicast address. Once a multicast address has been unregistered, all packets detected on the medium destined for that address are ignored.

<wifi_set_mib>	Setup the adaptive mode. You can replace this weak function by the same name function to setup adaptive mode you can.
<wifi_set_country_code>	Setup country code. You can replace this weak function by the same name function to setup country code you want.
<wifi_set_tdma_param>	Set TDMA parameters.
<wifi_get_setting>	Get current Wi-Fi setting from driver.
<wifi_show_setting>	Show the network information stored in the rtw_wifi_setting_t structure.
<wifi_set_network_mode>	Set the network mode according to the data rate it supported.
<wifi_get_network_mode>	Get the network mode.
<wifi_get_antenna_info>	Get antenna information.
<wifi_set_ch_deauth>	Set flag for concurrent mode wlan1 issue_deauth when channel switched by wlan0.

19.2.8.1 wifi_get_mac_address

Retrieve the current Media Access Control (MAC) address (or Ethernet hardware address) of the 802.11 device.

Parameter	Type	Introduction
<mac>	char *	Point to the result of the mac address will be get.

19.2.8.2 wifi_get_ap_bssid

Get connected AP's BSSID.

Parameter	Type	Introduction
<bssid>	unsigned char *	The location where the AP BSSID will be stored.

19.2.8.3 wifi_get_ap_info

Get the SoftAP information.

Parameter	Type	Introduction
<ap_info>	rtw_bss_info_t *	The location where the AP info will be stored.
<security>	rtw_security_t *	The security type.

19.2.8.4 wifi_set_country

Set the country code to driver to determine the channel set.

Parameter	Type	Introduction
<country_code>	rtw_country_code_t	Specify the country code.

19.2.8.5 wifi_get_sta_max_data_rate

Retrieved STA mode max. data rate.

Parameter	Type	Introduction
<inidata_rate>	_u8 *	Max data rate.

19.2.8.6 wifi_get_rssi

Retrieved the latest RSSI value.

Parameter	Type	Introduction
<pRSSI>	int *	Points to the integer to store the RSSI value gotten from driver.

19.2.8.7 wifi_register_multicast_address

Register interest in a multicast address.

Once a multicast address has been registered, all packets detected on the medium destined for that address are forwarded to the host, otherwise they are ignored.

Parameter	Type	Introduction
<mac>	rtw_mac_t *	Ethernet MAC address.

19.2.8.8 wifi_unregister_multicast_address

Unregister interest in a multicast address.

Once a multicast address has been unregistered, all packets detected on the medium destined for that address are ignored.

Parameter	Type	Introduction
<mac>	rtw_mac_t *	Ethernet MAC address.

19.2.8.9 wifi_set_mib

Setup the adaptive mode. You can replace this weak function by the same name function to setup adaptive mode you can.

Parameter: None.

19.2.8.10 wifi_set_country_code

Setup country code. You can replace this weak function by the same name function to setup country code you want.

Parameter: None.

19.2.8.11 wifi_set_tdma_param

Set TDMA parameters.

Parameter	Type	Introduction
<slot_period>	unsigned char	We separate TBTT into 2 or 3 slots. If we separate TBTT into 2 slots, then slot_period should be larger or equal to 50ms. It means 2 slot period is slot_period, 100-slot_period If we separate TBTT into 3 slots, then slot_period should be less or equal to 33ms. It means 3 slot period is 100 - 2 * slot_period, slot_period, slot_period.
<rfof_period_len_1>	unsigned char	RF on period of slot 1.
<rfof_period_len_2>	unsigned char	RF on period of slot 2.
<rfof_period_len_3>	unsigned char	RF on period of slot 3.

19.2.8.12 wifi_get_setting

Get current Wi-Fi setting from driver.

Parameter	Type	Introduction
<ifname>	const char *	The WLAN interface name

		● WLAN0_NAME ● WLAN1_NAME
<pSetting>	rtw_wifi_setting_t *	Points to the rtw_wifi_setting_t structure to store the Wi-Fi setting gotten from driver.

19.2.8.13 wifi_show_setting

Show the network information stored in the rtw_wifi_setting_t structure.

Parameter	Type	Introduction
<ifname>	const char *	The WLAN interface name, can be WLAN0_NAME or WLAN1_NAME.
<pSetting>	rtw_wifi_setting_t *	Points to the rtw_wifi_setting_t structure which information is gotten by wifi_get_setting().

19.2.8.14 wifi_set_network_mode

Set the network mode according to the data rate it supported. Driver works in BGN mode in default after driver initialization. This function is used to change wireless network mode for station mode before connecting to AP.

Parameter	Type	Introduction
<mode>	rtw_network_mode_t	Network mode to set. ● RTW_NETWORK_B ● RTW_NETWORK_BG ● RTW_NETWORK_BGN

19.2.8.15 wifi_get_network_mode

Get the network mode. Driver works in BGN mode in default after driver initialization. This function is used to get the current wireless network mode for station mode.

Parameter	Type	Introduction
<pmode>	rtw_network_mode_t *	Network mode to get.

19.2.8.16 wifi_get_antenna_info

Get antenna information.

Parameter	Type	Introduction
<antenna>	unsigned char *	Points to store the antenna value gotten from driver. ● 0: main ● 1: aux

19.2.8.17 wifi_set_ch_deauth

Set flag for concurrent mode wlan1 issue_deauth when channel switched by wlan0.

Parameter	Type	Introduction
<enable>	_u8	● 0: Disable ● 1: Enable

Usage: wifi_set_ch_deauth(0) -> wlan0 wifi_connect -> wifi_set_ch_deauth(1)

19.2.9 Wi-Fi Indication APIs

API	Introduction
<wifi_manager_init>	Initialize Realtek Wi-Fi API System.
<wifi_indication>	WLAN driver indicate event to upper layer through wifi_indication.
<init_event_callback_list>	Initialize the event callback list.
<wifi_reg_event_handler>	Register the event listener.
<wifi_unreg_event_handler>	Un-register the event listener.

19.2.9.1 wifi_manager_init

Initialize Realtek Wi-Fi API System.

- Initialize the required parts of the software platform, such as worker, event registering and semaphore.
- Initialize the RTW API thread which handles the asynchronous event.

Parameter: None.

19.2.9.2 wifi_indication

WLAN driver indicates event to upper layer through wifi_indication().

Parameter	Type	Introduction
<event>	rtw_event_indicate_t	An event reported from driver to upper layer application. Refer to rtw_event_indicate_t enum.
<buf>	char *	If it is not NUL, buf is a pointer to the buffer for message string.
<buf_len>	int	The length of the buffer.
<flags>	int	Indicate some extra information, sometimes it is 0.

Note:

- If upper layer application triggers additional operations on receiving of wext_wlan_indicate, please strictly check current stack size usage (by using uxTaskGetStackHighWaterMark()), and tries not to share the same stack with WLAN driver if remaining stack space is not available for the following operations.
- Using semaphore to notice another thread instead of handing event directly in **wifi_indication()**.

19.2.9.3 init_event_callback_list

Initialize the event callback list.

Parameter: None.

Note: Make sure this function has been invoked before using the event handler related mechanism.

19.2.9.4 wifi_reg_event_handler

Register the event listener.

Parameter	Type	Introduction
<event_cmds>	unsigned int	The event command number indicated.
<handler_func>	rtw_event_handler_t	The callback function which will receive and process the event.
<handler_user_data>	void *	User specific data that will be passed directly to the callback function.

Note: Set the same even_cmds with empty handler_func will unregister the event_cmds.

19.2.9.5 wifi_unreg_event_handler

Un-register the event listener.

Parameter	Type	Introduction
<event_cmds>	unsigned int	The event command number indicated.
<handler_func>	rtw_event_handler_t	The callback function which will receive and process the event.

19.2.10 eFuse Writing APIs

API	Introduction
<wifi_set_mac_address>	This function sets the current Media Access Control (MAC) address of the 802.11 device.

Note: eFuse writing related API can only be called when the voltage is 3.3V. eFuse writing with the voltage 1.8V may fail. What's more, eFuse writing can only be operated one time, so be careful to do eFuse writing.

19.2.10.1 wifi_set_mac_address

Sets the current Media Access Control (MAC) address of the 802.11 device.

Parameter	Type	Introduction
<mac>	char *	Wi-Fi MAC address.

19.3 Fast Connection

This section illustrates the principle of fast connection and how to implement user's own fast connection code.

Fast connection is used to reconnect with AP automatically after Wi-Fi initialized, the principle is to store the AP information in Flash and reconnect to AP after Wi-Fi initialized.

19.3.1 Implement

19.3.1.1 AP Information Storage

User should implement a function to write AP information to Flash, just like demo function **wlan_wrtie_reconnect_data_to_flash()** in example code. In this function, you should reserve some space for AP information, and write the AP information to the reserved space in a pre-defined data format. The address of the function must be assigned to the global variable **p_write_reconnect_ptr**. After Wi-Fi connection success, if **p_write_reconnect_ptr** points to a valid address, **wlan_wrtie_reconnect_data_to_flash()** will be called.

Note: The path of example source code is **SDK/component/common/example/example_wlan_fast_connect/example_wlan_fast_connect.c**.

19.3.1.2 Reconnection

User should implement his own function to read AP information from Flash and connect to AP, just like demo function **wlan_init_done_callback()** in example code. The address of the function must be assigned to the global variable **p_wlan_init_done_callback**. This global variable should be defined before Wi-Fi initializing. After Wi-Fi initialized, if **p_wlan_init_done_callback** points to a valid address, this function will be called.

19.3.1.3 Fast Connection Data Erase

User should implement his own function to erase fast connection data, just like demo function **Erase_Fastconnect_data()** in example code.

19.3.2 APIs

API	Introduction
<wlan_wrtie_reconnect_data_to_flash>	Wi-Fi connection indication trigger this function to save current Wi-Fi profile in Flash.
<wlan_init_done_callback>	After Wi-Fi initialization done, WLAN driver calls this function to check whether auto-connection is required. This function reads previous saved WLAN profile in Flash and execute connection.

19.3.2.1 wlan_wrtie_reconnect_data_to_flash

Wi-Fi connection indications trigger this function to save current Wi-Fi profile in Flash. Write AP information to Flash.

Parameter	Type	Introduction
<data>	u8 *	Data will be written to Flash
<len>	uint32_t	Length of data
return	int	Status value: 0: write is ok -1: write is failed

Note: This is only a demo API, user should define his own API.

19.4 WPS APIs

API	Introduction
<wps_start>	Start WPS enrollee process
<wps_stop>	Stop WPS enrollee process

19.4.1 wps_start

Start WPS enrollee process.

Parameter	Type	Introduction
<wps_config>	u16	WPS configure method: - WPS_CONFIG_DISPLAY - WPS_CONFIG_KEYPAD - WPS_CONFIG_PUSHBUTTON
<pin>	char *	PIN number. Can be set to NULL if using WPS_CONFIG_PUSHBUTTON.
<channel>	u8	Channel. Currently un-used, can be set to 0.
<ssid>	char *	Target network SSID. Can be set to NULL if no target network specified.

Note:

- Before invoking this function, the Wi-Fi should be enabled by calling **wifi_on()**.
- Make sure CONFIG_ENABLE_WPS is enabled in **platform_opts.h**. After calling wps_start(), the longest time of WPS is 120s. You can call wps_stop() to quit WPS.

19.4.2 wps_stop

Stop WPS enrollee process.

Parameter: None.

Note: Make sure CONFIG_ENABLE_WPS is enabled in **platform_opts.h**.

20 Liquid Crystal Display Controller (LCDC)

Ameba-D LCDC supports Thin Film Transistor (TFT) color display. It provides I8080 MCU interface (8-bit bus width) and RGB interface (6-bit bus width), and supports RGB565 data format

This chapter introduces how to use LCDC I/F to control LCD module.

20.1 Interface

The available LCDC interfaces for different IC packages are shown in Table 20-1.

Table 20-1 LCDC I/F supported for different packages

IC Package	MCU 8-bit	RGB 6-bit
RTL8722D/RTL8722CS	✓	✓
RTL8721D/RTL8721CS/RTL8720D/RTL8720CS	✗	✗

The data format supported by each interface is shown in Table 20-2. In order to display normally, the data format of LCM and LCDC interface should match.

Table 20-2 LCDC I/F data format

Interface	Data Format	Comment
MCU 8-bit	Count 0 1 2 3 4 ...	2 transfers/pixel, RGB565
	RS 0 1 1 1 1 ...	
	D7 C7 P1R4 P1G2 P2R4 P2G2 ...	
	D6 C6 P1R3 P1G1 P2R3 P2G1 ...	
	D5 C5 P1R2 P1G0 P2R2 P2G0 ...	
	D4 C4 P1R1 P1B4 P2R1 P2B4 ...	
	D3 C3 P1R0 P1B3 P2R0 P2B3 ...	
	D2 C2 P1G5 P1B2 P2G5 P2B2 ...	
	D1 C1 P1G4 P1B1 P2G4 P2B1 ...	
	D0 C0 P1G3 P1B0 P2G3 P2B0 ...	
	P1R4 : Pixel1/Red_bit4	
RGB 6-bit	Count 0 1 2 3 4 ...	3 transfers/pixel, RGB565
	D5 P1R4 P1G5 P1B4 P2R4 P2G5 ...	
	D4 P1R3 P1G4 P1B3 P2R3 P2G4 ...	
	D3 P1R2 P1G3 P1B2 P2R2 P2G3 ...	
	D2 P1R1 P1G2 P1B1 P2R1 P2G2 ...	
	D1 P1R0 P1G1 P1B0 P2R0 P2G1 ...	
	D0 P1G0 P2G0 ...	
	P1R4 : Pixel1/Red_bit4	

20.2 Resolution

The maximum resolution supported by each interface is listed in Table 20-3.

Table 20-3 Maximum supported resolution

Interface	Display Type	Max. Support Resolution
MCU 8-bit	Static Display	1024*1024
	Dynamic Display	645*645 (30fps)
RGB 6-bit	Dynamic Display	600*400 (60fps)

For RGB LCD, the typical frame rate is 60fps.

In order to support RGB-LCD with higher resolution than parameters in Table 20-1, users have to set the frame rate lower. The maximum frame rate for a RGB-LCD is calculated as follows:

- Max. dot clock: $\text{MAX_DOT_CLK} = \text{system_clock} / 2 = 100\text{M} / 2 = 50\text{MHz}$
- Width, height, HBP, HFP, VBP, VFP are specified by LCD datasheet.

For 6-bit I/F mode:

$$\text{image_size} = \text{MAX_DOT_CLK} / (3 * F) - (\text{width} + \text{HBP} + \text{HFP}) * (\text{VBP} + \text{VFP}) - \text{height} * (\text{HBP} + \text{HFP});$$

$$\rightarrow F = 50\text{M} / (\text{width} * \text{height} + (\text{width} + \text{HBP} + \text{HFP}) * (\text{VBP} + \text{VFP}) + \text{height} * (\text{HBP} + \text{HFP})) / 3;$$

When frame rate is lower than 30fps, the screen flickering may happen. Users should evaluate the visual artifacts when setting the frame rate lower.

20.3 Pinmux

The pin assignments of LCDC are listed in Table 20-4.

Table 20-4 LCDC pin assignments

Port Name	MCU 8-bit	RGB 6-bit	LED I/F
PB[19]	-	-	-
PB[18]	-	-	-
PB[11]	-	-	-
PB[10]	-	-	-
PB[9]	-	-	-
PB[8]	-	-	-
PA[25]	-	-	-
PA[26]	-	-	-
PA[28]	D[7]	-	-
PA[30]	D[6]	-	-
PB[0]	D[5]	D[5]	D[5]
PA[31]	D[4]	D[4]	D[4]
PA[24]	D[3]	D[3]	D[3]
PA[23]	D[2]	D[2]	D[2]
PA[20]	D[1]	D[1]	D[1]
PA[19]	D[0]	D[0]	D[0]
PB[20]	TE/VSYNC	VSYNC	-
PB[21]	RS	-	-
PB[22]	RD	HSYNC	LAT
PB[23]	WR	DCLK	DCLK
PB[28]	CS	ENABLE	OE

20.4 LCDC APIs

20.4.1 MCU Function

20.4.1.1 LCDC_MCUStructInit

Items	Description
Introduction	Initializes the parameters in the LCDC_MCUInitStruct with default values.
Parameters	LCDC_MCUInitStruct: pointer to an LCDC_MCUInitTypeDef structure which is initialized.
Return	N/A

Note: LCDC_MCUInitStruct contains the MCU configurable parameters for LCDC, which determine that the MCU I/F is 8-bit, IO mode or DMA mode, the plane size and some timing control. The parameters in this structure should be set according to LCD module datasheet.

20.4.1.2 LCDC_MCUInit

Items	Description
Introduction	Initializes the LCDC peripheral according to the specified parameters in LCDC_MCUInitStruct.
Parameters	● LCDCx: where LCDCx can be LCDC. ● LCDC_MCUInitStruct: pointer to a LCDC_MCUInitTypeDef structure that contains the configuration information.
Return	N/A

20.4.1.3 LCDC_MCUWriteCmd

Items	Description
Introduction	Writes command to LCD module via MCU I/F.
Parameters	● LCDCx: where LCDCx can be LCDC. ● Cmd: the command to transmit.
Return	N/A

20.4.1.4 LCDC_MCUWriteData

Items	Description
Introduction	Writes data to LCD module via MCU I/F.
Parameters	● LCDCx: where LCDCx can be LCDC. ● Data: the data to transmit.
Return	N/A

20.4.1.5 LCDC_MCUReadData

Items	Description
Introduction	Reads data from LCD module via MCU I/F.
Parameters	LCDCx: where LCDCx can be LCDC.
Return	The read value

20.4.1.6 LCDC_MCUDMATrigger

Items	Description
Introduction	Triggers to transfer data of one frame from DMA buffer to LCDC Transmit FIFO.
Parameters	LCDCx: where LCDCx can be LCDC.
Return	N/A

20.4.2 RGB Function

20.4.2.1 LCDC_RGBStructInit

Items	Description
Introduction	Initializes the parameters in the LCDC_RGBInitStruct with default values.
Parameters	LCDC_RGBInitStruct: pointer to an LCDC_RGBInitTypeDef structure which is initialized.
Return	N/A

Note: LCDC_RGBInitStruct contains the RGB configurable parameters for LCDC, which determine that the RGB I/F is 6-bit, DE or HV mode, the plane size, the refresh frequency and VSYNC & HSYNC control. The parameters in this structure should be set according to LCD module datasheet.

20.4.2.2 LCDC_RGBInit

Items	Description
Introduction	Initializes the LCDC peripheral according to the specified parameters in the LCDC_RGBInitStruct.
Parameters	- LCDCx: where LCDCx can be LCDC. - LCDC_RGBInitStruct: pointer to a LCDC_RGBInitTypeDef structure that contains the configuration information.
Return	N/A

20.4.3 LED Function

20.4.3.1 LCDC_LEDStructInit

Items	Description
Introduction	Initializes the parameters in the LCDC_LEDInitStruct with default values.
Parameters	LCDC_LEDInitStruct: pointer to an LCDC_LEDInitTypeDef structure which will be initialized.
Return	N/A

20.4.3.2 LCDC_LEDInit

Items	Description
Introduction	Initializes the LCDC peripheral according to the specified parameters in the LCDC_LEDInitStruct.
Parameters	- LCDCx: where LCDCx can be LCDC. - LCDC_LEDInitStruct: pointer to an LCDC_LEDInitTypeDef structure which will be initialized.
Return	N/A

20.4.4 Common Function

20.4.4.1 LCDC_DMAModeConfig

Items	Description
Introduction	Configures LCDC DMA burst size.
Parameters	- LCDCx: where LCDCx can be LCDC. - BurstSize: DMA burst size; unit is 64 bytes.
Return	N/A

Note:

- If BurstSize = 1, the actual burstsize = 1x64 bytes; if BurstSize = 2, the actual burstsize = 2x64 = 128 bytes; ...
- The parameter BurstSize is not more than 8. The recommended value is 2.

20.4.4.2 LCDC_DMAImageBaseAddrConfig

Items	Description
Introduction	Configures image base address.
Parameters	- LCDCx: where LCDCx can be LCDC. - ImgBaseAddr: the buffer address.
Return	N/A

20.4.4.3 LCDC_INTConfig

Items	Description
Introduction	Enables or disables the specified LCDC interrupts.
Parameters	● LCDCx: where LCDCx can be LCDC. ● LCDC_IT: specifies the LCDC interrupt sources to be enabled or disabled. This parameter can be any combination of the following values: ■ LCDC_IT_DMAUNDFW: DMA FIFO underflow interrupt ■ LCDC_IT_FRDN: LCD refresh done interrupt ■ LCDC_IT_LINE: line interrupt ■ LCDC_IT_IO_TIMEOUT: I/O write/read timeout interrupt ■ LCDC_IT_FRM_START: Frame Start interrupt ● NewState: new state of the specified LCDC interrupts. This parameter can be ENABLE or DISABLE.
Return	N/A

20.4.4.4 LCDC_GetINTStatus

Items	Description
Introduction	Gets LCDC interrupt status.
Parameters	LCDCx: where LCDCx can be LCDC.
Return	Interrupt status

20.4.4.5 LCDC_ClearINT

Items	Description
Introduction	Clears the LCDC's interrupt pending bits.
Parameters	● LCDC_IT: specifies the interrupt to be cleared. This parameter can be any combination of the following values: ■ LCDC_IT_LINE: line interrupt ■ LCDC_IT_FRDN: refresh frame done interrupt ■ LCDC_IT_DMAUNDFW: DMA FIFO under flow interrupt ■ LCDC_IT_IO_TIMEOUT: IO write/read timeout interrupt ■ LCDC_IT_FRM_START: Frame Start interrupt
Return	N/A

20.4.4.6 LCDC_Cmd

Items	Description
Introduction	Enables or disables the LCDC.
Parameters	● LCDCx: where LCDCx can be LCDC. ● NewState: new state of the LCDC. This parameter can be: ENABLE or DISABLE.
Return	N/A

Note: When NewState is DISABLE, during the period of valid line (VTIMING =valid data), the disabled action is performed after the last valid line has transferred. If you want to disable the LCDC instantly, use the API LCDC_InsDisable(). During the other periods, the disabled action is performed instantly.

20.4.4.7 LCDC_InsDisable

Items	Description
Introduction	Disables the LCDC instantly.
Parameters	LCDCx: where LCDCx can be LCDC.
Return	N/A

20.4.4.8 LCDC_DeInit

Items	Description
Introduction	De-initializes the LCDC.
Parameters	LCDCx: where LCDCx can be LCDC.
Return	N/A

Note: when disabling LCDC instantly, all interrupts are cleared and disabled.

20.5 How to Use LCDC

Table 20-5 lists the typical application scenarios of LCDC.

Table 20-5 Typical application scenario of LCDC

Interface	Data Mode	LCD GRAM	Ameba-D Frame Buffer	Application
MCU	I/O mode	✓	✗	Static Display
	DMA Trigger-mode	✓	✓	Static Display
	DMA Auto-mode (VSYNC/TE mode)	✓	✓	dynamic Display
RGB	DMA Auto-mode (DE/HV mode)	✗	✓	dynamic Display

20.5.1 MCU Interface

20.5.1.1 I/O Mode

To use the LCDC MCU interface I/O mode, the following steps are mandatory.

(1) Configure the LCDC pinmux according to Table 20-4.

For example, in order to use PA[19] as LCDC pin, call the following function. It is the same for other LCDC pins.

```
Pinmux_Config (_PA_19, PINMUX_FUNCTION_LCD);
```

(2) Initialize the LCDC_MCUInitStruct variable.

a) Use the following function to initialize LCDC_MCUInitStruct variable with default parameters.

```
LCDC_MCUInitStructInit (LCDC_MCUInitStructDef * LCDC_MCUInitStruct);
```

b) Change other parameters according to LCM datasheet, such as 8-bit interface mode, Data/WR/RD/CS/RS pulse polarity, WR/RD pulse width.

(3) Initialize the LCDC using the initialized structure in step (2).

```
LCDC_MCUInit (LCDC_TypeDef * LCDCx, LCDC_MCUInitStructDef * LCDC_MCUInitStruct);
```

(4) Enable the LCDC using the function LCDC_Cmd().

(5) Send commands and parameters to LCM using the function LCDC_MCUWriteCmd()/LCDC_MCUWriteData() to initialize the LCD module.

(6) After LCM is initialized, call LCDC_MCUWriteCmd()/LCDC_MCUWriteData()/LCDC_MCUReadData() to send commands/write data to LCM or read data from LCM to drive LCD displaying.

20.5.1.2 Trigger DMA Mode

To use the LCDC MCU interface trigger DMA mode, the following steps are mandatory.

(1) Configure the LCDC pinmux according to Table 20-4.

(2) Configure LCDC to work in MCU I/O mode, then send commands and parameters to LCM to initialize LCD. After that, users need to send a command to inform LCM to start receiving data before transferring frame data.

(3) Initialize the LCDC_MCUInitStruct variable.

a) Configure the LCDC_MCUInitStruct parameter to Trigger DMA mode.

```
LCDC_MCUInitStructInit (LCDC_MCUInitStructDef * LCDC_MCUInitStruct);
```

```
LCDC_MCUInitStruct->LCDC_MCU_Mode = LCDC_MCU_DMA_MODE;
```

```
LCDC_MCUInitStruct->LCDC_MCU_DMA_Mode = LCDC_TRIGGER_DMA_MODE;
```

b) Change other parameters according to LCM datasheet, such as LCD width/height, 8-bit I/F mode, Data/WR/RD/CS/RS pulse polarity, WR/RD pulse width.

- (4) Initialize the LCD using the initialized structure in step (3).
`LCD_MCUInit (LCD_TypeDef * LCDx, LCD_MCUInitTypeDef * LCD_MCUInitStruct);`
- (5) Configure the LCD DMA parameters.
 - a) Configure the LCD DMA burst size using `LCD_DMAModeConfig()`.
 - b) Allocate DMA buffer and assign the address to LCD using `LCD_DMAImageBaseAddrConfig()`.
- (6) Enable LCD interrupt using the function `LCD_INTConfig()`, if needed.
- (7) Enable the LCD using the function `LCD_Cmd()`.
- (8) Trigger one frame transfer using the function `LCD_MCU DMA Trigger()`, and update the frame buffer to change the display effect.

20.5.1.3 VSYNC Mode

To use the LCD MCU I/F VSYNC mode, the following steps are mandatory.

- (1) Configure the LCD pinmux according to Table 20-4.
- (2) Configure LCD to work in MCU I/O mode, then send commands and parameters to LCM to make LCD work in VSYNC mode.
- (3) Initialize the `LCD_MCUInitStruct` variable.
 - a) Configure the `LCD_MCUInitStruct` parameter corresponding to VSYNC mode.
`LCD_MCUStructInit (LCD_MCUInitTypeDef * LCD_MCUInitStruct);`
`LCD_MCUInitStruct->LCD_MCU_Mode = LCD_MCU_DMA_MODE;`
`LCD_MCUInitStruct->LCD_MCU DMA Mode = LCD_AUTO_DMA_MODE;`
`LCD_MCUInitStruct->LCD_MCU Sync Mode = LCD_MCU_SYNC_WITH_VSYNC;`
 - b) Change other parameters according to LCM datasheet, such as VSYNC pulse polarity/pulse width/idle period, LCD width/height.
- (4) Initialize the LCD using the initialized structure in step (3).
`LCD_MCUInit (LCD_TypeDef * LCDx, LCD_MCUInitTypeDef * LCD_MCUInitStruct);`
- (5) Configure the LCD DMA parameters.
 - a) Configure the LCD DMA burst size using the function `LCD_DMAModeConfig()`.
 - b) Allocate DMA buffer and assign the address to LCD using the function `LCD_DMAImageBaseAddrConfig()`.
- (6) Enable the specified LCD interrupt using the function `LCD_INTConfig()`, if needed.
- (7) Enable the LCD using the function `LCD_Cmd()`.
- (8) The LCD can transfer frame data to LCM automatically synchronized with the VSYNC signal to LCM, and you can update the frame buffer to change the display.

20.5.1.4 TE Mode

To use the LCD MCU Iinterface TE mode, the following steps are mandatory.

- (1) Configure the LCD pinmux according to Table 20-4.
- (2) Configure LCD to work in MCU I/O mode, then send commands and parameters to LCM to let LCD work in TE mode.
- (3) Initialize the `LCD_MCUInitStruct` variable.
 - a) Configure the `LCD_MCUInitStruct` parameter corresponding to VSYNC mode.
`LCD_MCUStructInit (LCD_MCUInitTypeDef * LCD_MCUInitStruct);`
`LCD_MCUInitStruct->LCD_MCU_Mode = LCD_MCU_DMA_MODE;`
`LCD_MCUInitStruct->LCD_MCU DMA Mode = LCD_AUTO_DMA_MODE;`
`LCD_MCUInitStruct->LCD_MCU Sync Mode = LCD_MCU_SYNC_WITH_TE;`
 - b) Change other parameters if needed, such as TE pulse polarity, TE Delay, LCD width/height.
- (4) Initialize the LCD using the initialized structure in step (3).
`LCD_MCUInit (LCD_TypeDef * LCDx, LCD_MCUInitTypeDef * LCD_MCUInitStruct);`
- (5) Configure the LCD DMA parameters.
 - a) Configure the LCD DMA burst size using the function `LCD_DMAModeConfig()`.
 - b) Allocate DMA buffer and assign the address to LCD using the function `LCD_DMAImageBaseAddrConfig()`.
- (6) Enable the specified LCD interrupt using the function `LCD_INTConfig()`, if needed.
- (7) Enable the LCD using the function `LCD_Cmd()`.
- (8) The LCD can transfer frame data to LCM automatically synchronized with the TE signal from LCM, and you can update the frame buffer to change the display.

20.5.2 RGB Interface

20.5.2.1 DE Mode

To use the LCDC RGB interface DE mode, the following steps are mandatory.

- (1) Configure the LCDC pinmux according to Table 20-4.
- (2) Configure LCM parameters through SPI or other interfaces if needed.
- (3) Initialize the LCDC_RGBInitStruct variable.
 - a) Configure the LCDC_RGBInitStruct parameter corresponding to DE mode.
`LCDC_RGBStructInit (LCDC_RGBInitTypeDef * LCDC_RGBInitStruct);`
`LCDC_RGBInitStruct->LCDC_RGBSyncMode = LCDC_RGB_DE_MODE;`
 - b) Change other parameters if needed, such as Date/ENABLE/VSYSN/HSYN pulse polarity, DCLK active edge, VFP, VBP, VSW, HBP, HFP, HSW, refresh frequency, LCD width/height.
- (4) Initialize the LCDC using the initialized structure in step (3).
`LCDC_RGBInit (LCDC_TypeDef* LCDCx, LCDC_RGBInitTypeDef* LCDC_RGBInitStruct);`
- (5) Configure the LCDC DMA parameters.
 - a) Set burst size using the function `LCDC_DMAModeConfig()`.
 - b) Set DMA FIFO under flow mode and error data using the functions `LCDC_DMAUnderFlowModeConfig()` and `LCDC_DMAUnderFlowModeConfig()`.
 - c) Allocate DMA buffer and assign the address to LCDC using the function `LCDC_DMAImageBaseAddrConfig()`.
- (6) Enable the specified LCDC interrupt using the function `LCDC_INTConfig()`, if needed.
- (7) Enable the LCDC using the function `LCDC_Cmd()`.
- (8) The LCDC can transfer frame data to LCM automatically according to the refresh frequency, and you can update the frame buffer to change the display.

20.5.2.2 HV Mode

To use the LCDC RGB I/F HV mode, the following steps are mandatory.

- (1) Configure the LCDC pinmux according to Table 20-4.
- (2) Configure LCM parameters through SPI or other interfaces if needed.
- (3) Initialize the LCDC_RGBInitStruct variable.
 - a) Configure the LCDC_RGBInitStruct parameter corresponding to HV mode.
`LCDC_RGBStructInit (LCDC_RGBInitTypeDef * LCDC_RGBInitStruct);`
 - b) Change other parameters if needed, Date/ENABLE/VSYSN/HSYN pulse polarity, DCLK active edge, VFP, VBP, VSW, HBP, HFP, HSW, refresh frequency, LCD width/height, 6-bit parallel I/F mode, refresh frequency.
- (4) Initialize the LCDC using the initialized structure in step (3).
`LCDC_RGBInit (LCDC_TypeDef* LCDCx, LCDC_RGBInitTypeDef* LCDC_RGBInitStruct)`
- (5) Configure the LCDC DMA parameters
 - a) Set burst size using the function `LCDC_DMAModeConfig()`.
 - b) Set DMA FIFO under flow mode and error data using the functions `LCDC_DMAUnderFlowModeConfig()` and `LCDC_DMAUnderFlowModeConfig()`.
 - c) Allocate DMA buffer and assign the address to LCDC using the function `LCDC_DMAImageBaseAddrConfig()`.
- (6) Enable the specified LCDC interrupt using the function `LCDC_INTConfig()`, if needed.
- (7) Enable the LCDC using the function `LCDC_Cmd()`.
- (8) The LCDC can transfer frame data to LCM automatically according to the refresh frequency, and you can update the frame buffer to change the display.

20.5.3 LED Interface

To use the LCDC LED interface mode, the following steps are mandatory.

- (1) Configure the LCDC pinmux according to Table 20-4.
- (2) Initialize the LCDC_RGBInitStruct variable.
 - a) Configure the LCDC_LEDInitStruct parameter corresponding to LED interface mode
`LCDC_LEDStructInit (LCDC_LEDInitTypeDef * LCDC_LEDInitStruct)`
 - b) Change other parameters if needed, such as color channel, color numbers, timing (latch start time, latch pulse width, OE active width), refresh frequency, LED width/height.

- (3) Initialize the LCDC using the initialized structure in step (2).
LCDC_LEDInit (LCDC_TypeDef* LCDCx, LCDC_LEDInitTypeDef * LCDC_LEDInitStruct)
- (4) Configure the LCDC DMA parameters
 - a) Set burst size using the function LCDC_DMAModeConfig().
 - b) Allocate DMA buffer and assign the address to LCDC using the function LCDC_DMAImageBaseAddrConfig().
- (5) Enable the specified LCDC interrupt using the function LCDC_INTConfig(), if needed.
- (6) Enable the LCDC using the function LCDC_Cmd().
- (7) The LCDC can transfer frame data to LED array board automatically according to the refresh frequency, and you can update the frame buffer to change the display.

20.6 GUI

emWin is the embedded GUI solution. It can be adapted to any size, either physical or virtual display, not dependent of the display controller. Making it a professional GUI for the embedded market, usable for multiple different scenarios.

Realtek has reached an emWin Pro Buyout Agreement with SEGGER. The emWin LICENSED SOFTWARE, which is located in component\common\ui\emwin, is available in object code form for Realtek customers who have been authorized.

20.6.1 Authorization

In order to be authorized to use emWin software, you need to follow these steps:

- (1) Read *emWin_Software_License_Agreement.pdf* carefully under the path **component\common\ui**. The document is a binding, legal agreement between Realtek and you (either an individual or a legal entity). It explains the terms and conditions that you should accept and agree.
- (2) If you do not accept and agree to this Agreement, do not unzip “emwin.zip” and do not use any of the LICENSED SOFTWARE. If you do accept and agree this Agreement, then you can unzip “emwin.zip” with the password, which can be found in *emWin_Software_License_Agreement.pdf*.

20.6.2 emWin Software

After the “emwin.zip” is extracted, you can see the following directories of emWin software.

Directory	Sub-directory	Description
emWinLibrary	/	Contains the basic contents in emWin PRO package
	Config	The files in this sub-directory are template configuration files for reference to adapt for different SoC and LCD module. You should customize GUIConf.c and LCDConf.c according to their devices. The details of these two files can be found in <i>Chapter 38 Configuration, UM03001_emWin5.pdf</i> . For Ameba-D, we provide porting samples of EVB LCM with MCU or RGB I/F. The samples are located in component\common\ui\emwin\Sample\rtl8721d\Config . It's much easier for users to modify these files to adapt to new LCM. The files in this sub-directory are used to adapter for different LCD module and different scenarios.
	Doc	<i>UM03001_emWin5.pdf</i> is the User Guide and Reference Manual of emWin.
	GUI_X	GUI_X_Freertos.c is the configuration of the timing routines, the debugging routines and the kernel interface routines.
	Include	The header files of emWin
	Lib	The object code of emWin
	Tool	The available tools which are useful during UI development, such as Font Converter, GUIBuilder.
Sample	/	These are porting samples of Ameba-D driving EVB LCM. ● MCU I/F LCM on EVB: ILI9488 TFT-LCD module (320*480, MCU 8-bit I/F) ● RGB I/F LCM on EVB: TCX043DTLN-04 TFT-LCD module (480*272, RGB 6-bit I/F)
Third_Party	/	It contains the TrueType Font for emWin, which should be used if fonts need to be scalable at run-time. This is the adapted version of FreeType font library from David Turner, Robert Wilhelm and Werner Lemberg.

20.6.3 How to Adapt to LCM

As mentioned above, we provide porting samples of Ameba-D driving EVB LCM in `component\common\ui\emwin\Sample\rtl8721d`. Table 20-6 demonstrate how to customize configuration files to adapt a new LCM.

Table 20-6 Customizing configuration files

Configuration file	Step	Description	Note
GUIConf.c	(1) Modify the value of macro GUI_NUMBYTES to define the size of memory block. (2) Define macro PSRAM_BUF_USED in LCDConf.h to place the memory block into PSRAM. Otherwise, undefine it to place the memory block into SRAM.	Provide emWin with the function GUI_X_Config(), which is responsible for assigning a memory block to the memory management system.	
LCDConf_MCU_8bit_eval.c	(1) Set XSIZE_PHYS and YSIZE_PHYS to the horizontal and vertical resolution of LCM. (2) Implement the functions illustrated in Table 20-7. - void lcd_mcu_write (int x0, int y0, int x1, int y1, void *buf, U32 color) - void lcd_mcu_read (int x0, int y0, int x1, int y1, void *buf) - void lcd_init (void)	Used for MCU I/F LCM. As an example, these functions are implemented in Sample\rtl8721d\hal\hal_mcu_lcd.c . You should modify this file according to the flow of Initialize, Read/Write pixel specified by datasheet of LCM.	The LCDConf_MCU_8bit_eval.c adopts GUI MCU driver porting by Realtek, and it applies to display controllers with indirect interface, such as MCU I/F LCM.
LCDConf_RGB_6bit_eval.c	(1) Set XSIZE_PHYS and YSIZE_PHYS to the horizontal and vertical resolution of LCM. (2) Define macro PSRAM_BUF_USED in LCDConf.h to place the VRAM buffer into PSRAM. Otherwise, undefine it to place the VRAM buffer into SRAM. (3) Implement the functions illustrated in Table 20-8. - void lcd_init (unsigned int width, unsigned int height, unsigned int bufAddr) - void lcd_set_dma_addr (unsigned int bufAddr)	Used for RGB I/F LCM. As an example, these functions are implemented in Sample\rtl8721d\hal\hal_rgb_lcd.c . You should modify this file according to the timing parameters specified by datasheet of LCM.	The LCDConf_RGB_6bit_eval.c adopts GUI LIN driver provided by emWin package, and it applies to display controllers with linear video memory accessible via direct interface, such as RGB I/F LCM.
LCDConf_SPI_eval.c	(1) Set XSIZE_PHYS and YSIZE_PHYS to the horizontal and vertical resolution of LCM. (2) Implement the functions illustrated in Table 20-9. - void lcd_spi_write (int x0, int y0, int x1, int y1, void *buf, U32 color) - void lcd_spi_read (int x0, int y0, int x1, int y1, void *buf) - void lcd_init (void)	Used for SPI I/F LCM. As an example, these functions are implemented in Sample\rtl8721d\hal\hal_spi_lcd.c . You should modify this file according to the flow of Initialize, Read/Write pixel specified by datasheet of LCM.	● lcd_spi_read function is not necessary. ● The LCDConf_SPI_eval.c adopts GUI SPI driver porting by Realtek, and it applies to display controllers with indirect interface, such as SPI I/F LCM.

Table 20-7 Functions for MCU I/F LCM

Function	lcd_mcu_write	lcd_mcu_read	lcd_init
Introduction	Writes pixels to LCD module.	Reads pixels from LCD module.	Initialize LCD controller and LCM.
Parameters	● x0: the start column of area to be updated ● y0: the start row of area to be updated ● x1: the end column of area to be updated ● y1: the end row of area to be updated ● buf: pointer to a buffer which is used to store pixel data of the area defined by (x0,y0,x1,y1)	● x0: the start column of area to be read ● y0: the start row of area to be read ● x1: the end column of area to be read ● y1: the end row of area to be read ● buf: pointer to a buffer which is used to store pixel data read from the area defined by (x0,y0,x1,y1)	N/A

	color: If buf is NULL, fill the area with the same color		
Return	N/A	N/A	N/A

Table 20-8 Functions for RGB I/F LCM

Function	lcd_init	lcd_set_dma_addr
Introduction	Initialize LCD controller	Set the address of VRAM. It is necessary when Multiple Buffering or Virtual Screen function is used.
Parameters	- width: horizontal resolution - height: vertical resolution - bufAddr: the start address of VRAM	bufAddr: the start address of VRAM
Return	N/A	N/A

Table 20-9 Functions for SPI I/F LCM

Function	lcd_spi_write	lcd_spi_read	lcd_init
Introduction	Writes pixels to LCD module.	Reads pixels from LCD module.	Initialize LCD controller and LCM.
Parameters	- x0: the start column of area to be updated - y0: the start row of area to be updated - x1: the end column of area to be updated - y1: the end row of area to be updated - buf: pointer to a buffer which is used to store pixel data of the area defined by (x0,y0,x1,y1) - color: If buf is NULL, fill the area with the same color	- x0: the start column of area to be read - y0: the start row of area to be read - x1: the end column of area to be read - y1: the end row of area to be read - buf: pointer to a buffer which is used to store pixel data read from the area defined by (x0,y0,x1,y1)	N/A
Return	N/A	N/A	N/A

Note:

To adapt the driver to a display controller not supported currently, user should

- Refer to Chapter 33.7.26 GUIDRV_Template in *UM03001_emWin5.pdf*
- Adapt the routines _SetPixelIndex() and _GetPixelIndex() in **component\common\ui\emwin\emWinLibrary\Config\GUIDRV_Template.c**
- Optimize some operation in **GUIDRV_Template.c** to improve drawing speed

20.6.4 How to Adapt to Touch Panel

We provide porting samples of Ameba-D driving EVB touch panel in **component\common\ui\emwin\Sample\rtl8721d**. The following steps demonstrate how to customize configuration files to adapt a new touch panel.

- Copy your touch driver to the folder **component\common\ui\emwin\Sample\rtl8721d\hal**
The touch driver needs to provide an initialization function and functions to get the horizontal and vertical coordinates of the touch point.
- Modify hal_touch.h/hal_touch.c
 - Define the macro **"TP_ID"** which means the touch panel you want to use in hal_touch.h
 - Implement the following functions in hal_touch.c
 - void hal_touch_init (void)
 - int hal_touch_measureX (void)
 - int hal_touch_measureY (void)

Function	hal_touch_init	hal_touch_measureX	hal_touch_measureY
Introduction	Initialize the touch panel.	Measure the x coordinate of touch point	Measure the y coordinate of touch point
Parameters	N/A	N/A	N/A
Return	N/A	X coordinate of touch point	Y coordinate of touch point

20.6.5 How to Use emWin in SDK

To build and run emWin demo in SDK, you need to follow these steps:

- (1) Enable emWin compile
KM4 make menuconfig > MENUCONFIG FOR CHIP CONFIG > GUI Config > Enable GUI, and select **emwin**.
- (2) Enable PSRAM if needed
 By default, the VRAM buffer, which is necessary for RGB I/F LCM, is located in PSRAM. So you should enable PSRAM by setting `psram_dev_config`, `psram_dev_enable` to TRUE in `rtl8721dhp_intfcfg.c`.
 If you want to put the VRAM buffer in RAM instead of PSRAM, you should modify `LCDConf.h` as follows:
`#undef PSRAM_BUF_USED`
- (3) By default, the configuration files of RGB I/F LCM (`hal_rgb_lcd.c`, `LCDConf_RGB_6bit_eval.c`) is compiled.
 To use MCU I/F LCM, you should modify **project\realtek_amebaD_va0_example\GCC-RELEASE\project_hp\asdk\make\ui\emwin\Sample\Makefile** to compile `hal_mcu_lcd.c` and `LCDConf_MCU_8bit_eval.c` instead.
- (4) Add your own code or run emWin demo code.
 The emWin demo code is located in **project\realtek_amebaD_va0_example\example_sources\LCDC\GUI_demo\emWin**. The `ReadMe.txt` in each demo folder demonstrates how to use it.

21 PSRAM

Pseudo Static Random Access Memory (PSRAM) is used for high speed transmission of data stream, and is suitable for audio codec. RTL8721D uses PSRAM controller to communicate with PSRAM. PSRAM is in KM4 platform, so only KM4 can access it.

The features of PSRAM are:

- Density: 32Mbit
- Address Mapping: 0x0200_0000 ~ 0x0240_0000
- Clock rate: 50MHz
- Double Data Rate (DDR)
- 16/32/64/128 bytes burst access
- Half sleep mode and deep power-down mode

21.1 Throughput

PSRAM supports direct access and DMA access.

Table 21-1 PSRAM throughput

Access mode		Write		Read	
		Theory	Test	Theory	Test
Direct Access	Cache off	200Mbps	< 160Mbps	177.78Mbps	$< (32)/(180\text{ns}+360\text{ns}) = 59.26 \text{ Mbps}$
	Cache on	200Mbps	160Mbps (CS is high for 40ns between two transmits)	556.52Mbps	$< (32*8)/(460\text{ns}+360\text{ns}) = 312\text{Mbps}$
DMA		731.43Mbps	711.11Mbps	721.13Mbps	589.18Mbps

Note:

- When Cache off:
 - Read or write operation only accesses 32 bits once with header and delay.
 - Instruction execution time also needs to take into consideration.
 - The 360ns in test is caused by hardware characteristic. Command from CPU to PSRAM controller needs 152ns to sync, PSRAM controller controlling PHY circuit to work needs 82us, PHY giving read data to CPU needs 126ns.
- When Cache on:
 - Read operation reads 32 bytes (cache line) once.
 - Write operation writes 32 bits once, but the interval between two write operations is greatly reduced.
 - The 360ns in test is caused by hardware characteristic. Command from CPU to PSRAM controller needs 152ns to sync, PSRAM controller controlling PHY circuit to work needs 82us, PHY giving read data to CPU needs 126ns.
- DMA sets burst length 128 bytes and disables cache.
 - For DMA write, DAM moving data to PSRAM FIFO and PSRAM controller writing FIFO data to PSRAM slave can work simultaneously, so the interval between two burst write operations is small.
 - For DMA read, similar operation as DMA write is not supported, so the interval between two burst read operations is large.
 - The test data above takes variable initial latency and 3 clocks initial latency for example.

Table 21-2 PSRAM throughput theoretical calculation

Item	Writing 32 bits	Reading 32 bits
Header + delay	$[3 + (3-1)] * 20\text{ns} = 100\text{ns}$	$[3 + (3-1)] * 20\text{ns} = 100\text{ns}$
Data transmit period	$2 * 20\text{ns} = 40\text{ns}$	$16 * 20\text{ns} = 320\text{ns}$
Hardware hold	$1 * 20\text{ns} = 20\text{ns}$	$2 * 20\text{ns} = 40\text{ns}$
Total without considering instruction execution time	$100\text{ns} + 40\text{ns} + 20\text{ns} = 160\text{ns}$	$100\text{ns} + 320\text{ns} + 40\text{ns} = 460\text{ns}$
Throughput theoretical value	$32/160\text{ns} = 200\text{Mbps}$	$(32*8)/460\text{ns} = 556.52\text{Mbps}$

21.2 Power Management

When entering KM4 powergate mode, PSRAM will be reset and its memory will be lost. If you want to retain the PSRAM, KM4 powergate mode is not supported to use. If you want to keep the PSRAM and save power, KM4 clockgate mode is supported.

Table 21-3 PSRAM power management

Retention	Options	Power Consumption (uA)	Comment
No	● KM4 PG ● PSRAM_LDO OFF	<30	PSRAM cannot keep, so memory retention applications are not supported.
Yes	● KM4 CG ● PSRAM_LDO ON ● PSRAM Half Sleep mode	About 325	

21.3 How to Use PSRAM?

21.3.1 Initializing PSRAM

Before accessing PSRAM, you should enable PSRAM power, initialize PSRAM controller and PSRAM slave to synchronize the related parameters.

In SDK, you should set `psram_dev_enable` in `rtl8721dhp_intfcfg.c`. If the chip works in the environment with large fluctuations in temperature, you should set `psram_dev_cal_enable` to enable calibration function. If you want to keep the PSRAM and save power when KM4 enters sleep mode, you should also set `psram_dev_retention` to enable PSRAM retention.

```
PSRAMCFG_TypeDef psram_dev_config = {
    .psram_dev_enable = TRUE,           //enable psram
    .psram_dev_cal_enable = TRUE,       //enable psram calibration function
    .psram_dev_retention = TRUE,        //enable psram retention
};
```

21.3.2 Adding BSS/TEXT/DATA Section into PSRAM Region

Table 21-4 lists how to add a BSS/TEXT/DATA section into PSRAM region.

Table 21-4 Add a BSS/TEXT/DATA section into PSRAM region

Section	Description	Operation
BSS	To put a BSS section in PSRAM, add PSRAM_BSS_SECTION before the buffer definition.	PSRAM_BSS_SECTION u32 PSRAM_Testbuf[1024];
TEXT	To put a TEXT section in PSRAM, add PSRAM_TEXT_SECTION before the function definition.	PSRAM_TEXT_SECTION VOID Test_Function(VOID) { u32 i = 0; for (i = 0; i < 10; i++) { DBG_8195A("test\r\n"); } }
DATA	To put a DATA section in PSRAM, add PSRAM_DATA_SECTION or PSRAM_RODATA_SECTION before the data definition.	PSRAM_DATA_SECTION u8 Test_Data = FALSE;

After the operation, rebuild KM4 project.

21.3.3 Allocating Heap from PSRAM

Before allocating heap, Initializing PSRAM must be completed, then follow the steps below.

- (1) Confirm the heap size you want in **Psram_reserve.c**.

```
#define configTOTAL_PSRAM_HEAP_SIZE (0x200000)
```

- (2) Use **void *Psram_reserve_malloc(int size)** to allocate a heap or use **void *Psram_reserve_calloc(int num, int size)** to allocate several consecutive spaces. Both functions return a pointer to the start address of the allocation. Use **void Psram_reserve_free(void *mem)** to free the allocation.

Note: The PSRAM_BSS_SECTION will be cleared only in the function **void app_init_psram(void)** in **rtl8721dhp_app_start.c**.

21.4 PSRAM Cache “Write Back” Policy Change Note

21.4.1 Cache Policy Change

To prevent PSRAM cell charge leak from frequent access, the cache policy is changed to “Write Back” instead. “Write back” policy also can enhance memory access performance and efficiency.

21.4.2 Scope

All of chips stacking PSRAM inside on Ameba-D series: RTL8721CSM, RTL8722CSM, RTL8721DM, and RTL8722DM.

21.4.3 Notice

On “Write Through” policy, the written data into cache are updated simultaneously to the corresponding PSRAM. However, on the “Write Back” policy, the synchronization operations need to be taken between cache and PSRAM to keep content consistency, especially for multiple access by different sources, e.g. CPU, serial ports and peripherals.

Follow the described instructions on the following paragraphs while manipulating PSRAM for DMA application and heap usage.

As cache line of Ameba-D cache is 32 bytes, and cache operations are all based on cache line. So the buffer size and buffer starting address should be 32 bytes alignment to avoid synchronization issues.

21.4.3.1 DMA Buffer Definition

- When using **Psram_reserve_malloc()** to allocate a space for DMA buffer, our API has realized 32 bytes of the header address of the memory allocated. (This approach is recommended to definite DMA Buffer)
- When allocating a space for DMA buffer from PSRAM by array, keyword “**__attribute__((aligned(32)))**” can be added to realize the starting address 32 bytes alignment.

```
char rx_buf[RX_BUF_SIZE] __attribute__((aligned(32)));
```

21.4.3.2 DMA Operation

The following steps should be added when executing DMA Rx/Tx.

Operation	Step
DMA Rx	(1) Prepare Rx Buffer (2) Do DCache_CleanInvalidate() to avoid cache data write back during DMA Rx (3) Do DMA Rx config (4) Trigger DMA Rx interrupt

	(5) Do DCache_Invalidate() in Rx Done Handler to clean old data (take example_sources/USI_UART/raw/usi_uart_stream_dma/src/main.c for example, uart_rcv_string_done() is DMA Rx Done Interrupt Handler) <pre> void uart_rcv_string_done(void) { DCache_Invalidate((u32)rx_buf, SRX_BUF_SZ); /*!!!To solve the cache consistency problem, DMA mode need it!!!*/ dma_free(); rx_done += 1; } </pre> (6) CPU reads Rx Buffer
DMA Tx	(1) CPU prepares Tx buffer data (2) Do DCache_CleanInvalidate() for Tx buffer to synchronize the data (3) Do DMA tx config (4) Trigger DMA tx interrupt

In SDK, only the example of one time xxx_GDMA_Init one time transmission is illustrated. The step (2) is included in xxx_GDMA_Init by default.

If you need multitime DMA TRx with only one time xxx_GDMA_Init, DCache_CleanInvalidate() should be called every time before DMA transmission starts.

<pre> BOOL USI_UARTTXGDMA_Init(u8 USIIndex, GDMA_InitTypeDef *GDMA_InitStruct, void *CallbackData, IRQ_FUN CallbackFunc, u8 *pTxBuf, int TxCount) { u8 GdmaChnl; assert_param(GDMA_InitStruct != NULL); DCache_CleanInvalidate((u32)pTxBuf, TxCount); </pre>	<pre> BOOL USI_UARTRXGDMA_Init(u8 USIIndex, GDMA_InitTypeDef *GDMA_InitStruct, void *CallbackData, IRQ_FUN CallbackFunc, u8 *pRxBuf, int RxCount) { u8 GdmaChnl; USI_TypeDef *pUSIx; assert_param(GDMA_InitStruct != NULL); DCache_CleanInvalidate((u32)pRxBuf, RxCount); </pre>
--	--

21.4.3.3 Heap Usage

If you want to allocate heap in PSRAM, **void *Psram_reserve_malloc(int size)** should be used to allocate a heap for the starting address 32 bytes alignment. It returns a pointer to the starting address of the allocation. **void Psram_reserve_free(void *mem)** is used to free the allocation.

22 MPU and Cache

22.1 Functional description

22.1.1 MPU

Memory Protection Unit (MPU) is used to provide Hardware Protection by Software definition. Our code provides **mpu_region_config** structure to include the region memory attribute. Default attribute of all KM0 & KM4 SRAM are write-through.

Table 22-1 shows member variables of the **mpu_region_config** structure.

Table 22-1 mpu_region_config structure

Member Variable Name	Type	Description
region_base	uint32_t	MPU region base, 32 bytes aligned
region_size	uint32_t	MPU region size, 32 bytes aligned
xn	uint8_t	Execute Never attribute ● MPU_EXEC_ALLOW: Allows program execution in this region ● MPU_EXEC_NEVER: Does not allow program execution in this region
ap	uint8_t	Access permissions ● MPU_PRIV_RW: Read/write by privileged code only ● MPU_UN_PRIV_RW: Read/write by any privilege level ● MPU_PRIV_R: Read only by privileged code only ● MPU_PRIV_W: Read only by any privilege level
sh	uint8_t	Share ability for Normal memory ● MPU_NON_SHAREABLE: Non-shareable ● MPU_OUT_SHAREABLE: Outer shareable ● MPU_INR_SHAREABLE: Inner shareable
attr_idx	uint8_t	Memory attribute indirect index This parameter can be a value of 0 ~ 7, the detailed attribute is defined in mpu_init() and is customized. The typical definition is as following: ● 0: MPU_MEM_ATTR_IDX_NC, defines memory attribute of Normal memory with non-cacheable ● 1: MPU_MEM_ATTR_IDX_WT_T_RA, defines memory attribute of Normal memory with write-through transient, read allocation ● 2: MPU_MEM_ATTR_IDX_WB_T_RWA, defines memory attribute of Normal memory with write-back transient, read and write allocation ● 3 ~ 7: MPU_MEM_ATTR_IDX_DEVICE, defines memory attribute of Device memory with non-gathering, non-recording, non-early Write Acknowledge

Table 22-2 shows how to set a MPU region.

Table 22-2 How to set a MPU region

Steps	Description
New variable and structure	● Variable to store MPU entry index ● Structure mpu_region_config to store the region memory attribute
Allocate a free MPU entry	Call mpu_entry_alloc()
Set region memory attribute	Set structure of region memory attribute
Configure MPU region memory attribute	Call mpu_region_cfg()

22.1.2 Cache

Cache is used to improve the CPU performance of data access. Ameba-D Cache supports Enable/Disable, Flush and Clean Operation, as Table 22-3 lists.

Table 22-3 Enable/Disable, Flush and Clean operation supported by Cache

Operation	Description	I-Cache	D-Cache
Enable/Disable	Enable or Disable Cache function	✓	✓
Flush (Invalidate)	● Flush Cache ● D-Cache can be flushed by address ● Can be used after DMA Rx, and CPU reads DMA data from DMA buffer for D-Cache	✓	✓
Clean	● Clean D-Cache ● D-Cache will be write back to memory ● D-Cache can be cleaned by address ● Can be used before DMA Tx, after CPU writes data to DMA buffer for D-Cache	✗	✓

22.2 MPU APIs

22.2.1 mpu_init

Items	Description
Introduction	Initialize MPU region memory attribute to typical value
Parameters	N/A
Return	N/A

22.2.2 mpu_set_mem_attr

Items	Description
Introduction	Change MPU region memory attribute
Parameters	● attr_idx: region memory attribute index, which can be 0 ~ 7. ● mem_attr: region memory attributes.
Return	N/A

22.2.3 mpu_region_cfg

Items	Description
Introduction	Configure MPU region memory attribute.
Parameters	● region_num: ■ KM0: 0 ~ 3 ■ KM4_NS: 0 ~ 7 ■ KM4_S: 0 ~ 3 ● pmpu_cfg: pointer to an mpu_region_config structure which has been configured
Return	N/A

22.2.4 mpu_entry_free

Items	Description
Introduction	Free MPU entry
Parameters	entry_index: ● KM0: 0 ~ 3 ● KM4_NS: 0 ~ 7 ● KM4_S: 0 ~ 3
Return	N/A

22.2.5 mpu_entry_alloc

Items	Description
Introduction	Allocate a free MPU entry
Parameters	N/A
Return	MPU entry index: ● KM0: 0 ~ 3 ● KM4_NS: 0 ~ 7 ● KM4_S: 0 ~ 3 ● Fail: -1

22.3 Cache APIs

22.3.1 ICache_Enable

Items	Description
Introduction	Enable I-Cache
Parameters	N/A
Return	N/A

22.3.2 ICache_Disable

Items	Description
Introduction	Disable I-Cache
Parameters	N/A
Return	N/A

22.3.3 ICache_Invalidate

Items	Description
Introduction	Invalidate I-Cache
Parameters	N/A
Return	N/A

22.3.4 DCache_IsEnabled

Items	Description
Introduction	Check D-Cache enabled or not
Parameters	N/A
Return	D-Cache enable status: ● 1: Enable ● 0: Disable

22.3.5 DCache_Enable

Items	Description
Introduction	Enable D-Cache
Parameters	N/A
Return	N/A

22.3.6 DCache_Disable

Items	Description
Introduction	Disable D-Cache
Parameters	N/A
Return	N/A

22.3.7 DCache_Invalidate

Items	Description
Introduction	Invalidate D-Cache by address
Parameters	- Address: Invalidated address (aligned to 32-byte boundary) - Bytes: Size of memory block (in number of bytes)
Return	N/A

22.3.8 DCache_Clean

Items	Description
Introduction	Clean D-Cache by address
Parameters	- Address: Clean address (aligned to 32-byte boundary). - Bytes: size of memory block (in number of bytes). Note: Address set 0xFFFFFFFF is used to clean all D-Cache.
Return	N/A

22.3.9 DCache_CleanInvalidate

Items	Description
Introduction	Clean and invalidate D-Cache by address
Parameters	- Address: Clean and invalidated address (aligned to 32-byte boundary) - Bytes: size of memory block (in number of bytes) Note: Address set 0xFFFFFFFF is used to clean and flush all D-Cache.
Return	N/A

22.4 How to Define a Non-cacheable Data Buffer

To define a data buffer with non-cacheable attribute, you should add **SRAM_NOCACHE_DATA_SECTION** before the buffer definition.

```
SRAM_NOCACHE_DATA_SECTION u8 noncache_buffer[DATA_BUFFER_SIZE];
```

23 Audio Codec Controller Guide

23.1 Audio Codec

Ameba-D audio codec (AC) is often used to play and record audio data. It is a stereo audio codec with stereo headphone amplifiers, as well as 2-way inputs and 1-way stereo/mono output that are programmable to single-ended or differential.

Ameba-D audio codec integrates anti-pop circuit for audible pop noise cancellation, MIC bias circuit for MIC power supply and programmable MIC boost ability.

Ameba-D audio codec transmits record data to or receives playback data from platform through SPORT interface. By means of specific serial interface (SI), Ameba-D platform configures and drives audio codec.

Ameba-D also provides external I²S interface for audio application extension, which supports the highest 384kHz sampling frequency.

23.1.1 Diagram

The diagram of AC is shown in Fig 23-1.

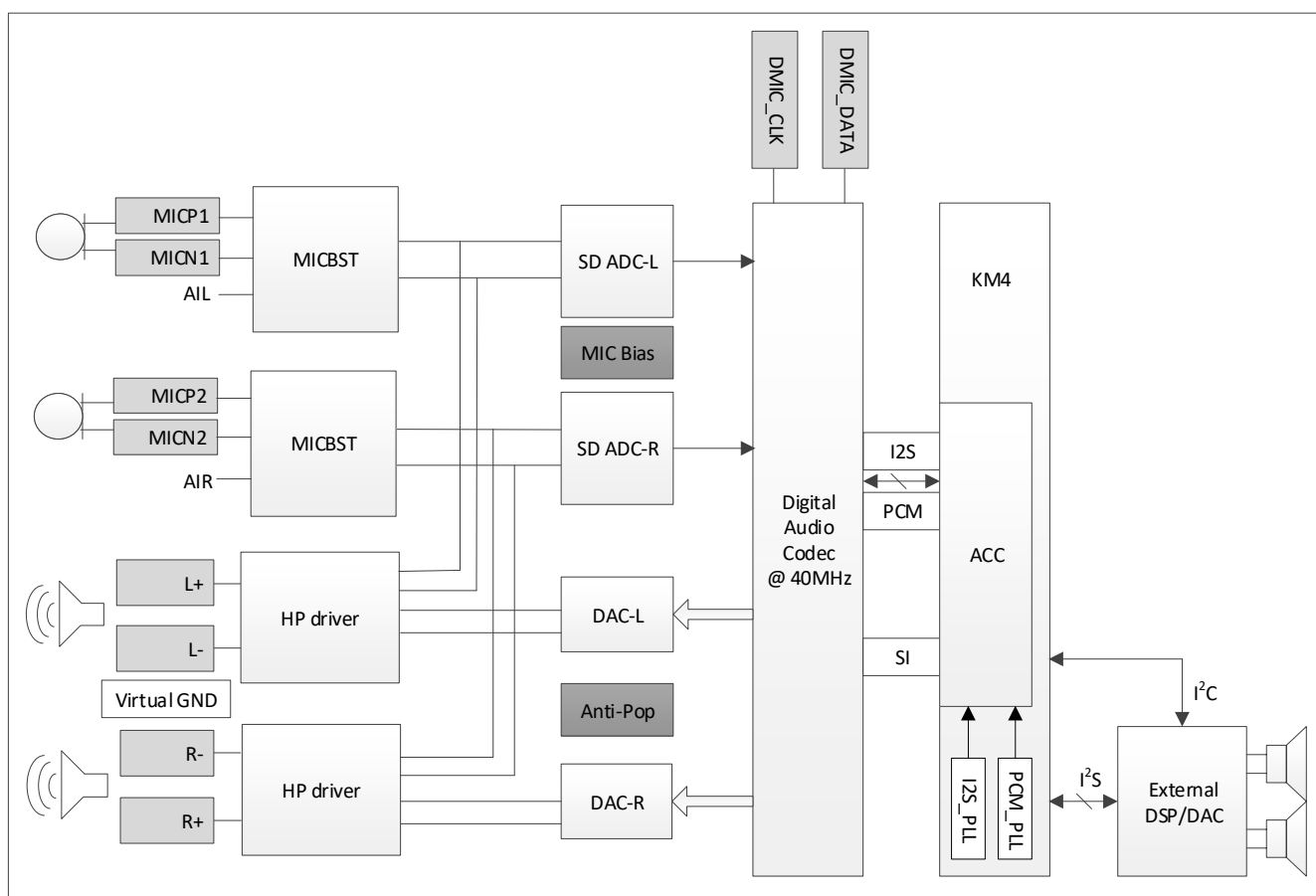


Fig 23-1 Audio codec diagram

23.1.2 Features

- Mono and stereo channel
- 8-bit, 16-bit and 24-bit sample bits
- 8k, 16k, 32k, 48k, 96k, 44.1k, 48k and 88.2k sample rate
- I2S, left justify, PCM mode A, PCM mode B, PCM mode A-N and PCM mode B-N data format
- Anti-pop function to reduce audible pop
- Programmable MIC boost gain
- Programmable gain in ADC and DAC path
- Three line-out output modes: cap-less, differential and single-end
- Three input ways: line-in, AMIC-in and DMIC-in

23.1.3 Application Mode

Audio codec supports three input ways: line-in, AMIC-in and DMIC-in, but only supports one output way: line-out.

23.1.3.1 Line-out

Line-out has no amplifier to amplify output voice. Line-out supports three output modes: cap-less, differential and single-ended. User can select the wanted mode by setting the related registers.

- Line-out cap-less mode

In this mode, the N-end of L/R channel outputs common-level voltage, while P-end drives the available analog audio signal. When earphone inserts into jack, the ground must short with N-end output for audio signal de-couple. That is why the ground is called virtual ground.

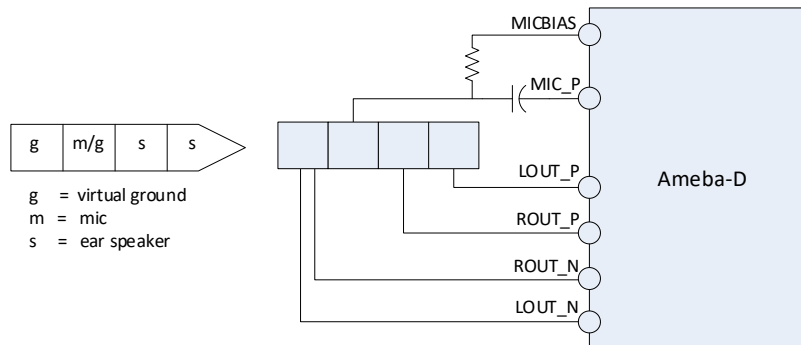


Fig 23-2 Cap-less mode connection with headphone jack

- Line-out differential mode

In this mode, both N-end and P-end drive the available analog audio signal. User should select the differential jack and earphone accordingly.

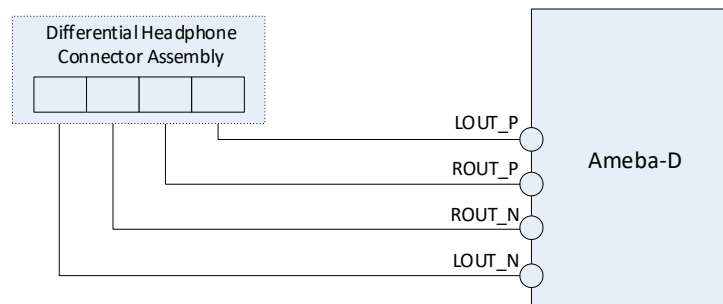


Fig 23-3 Differential mode connection with headphone jack

- Line-out single-ended mode

In this mode, board circuit designer needs to place a capacitor to the P-end output path for analog audio signal pick-up. No N-end output is required.

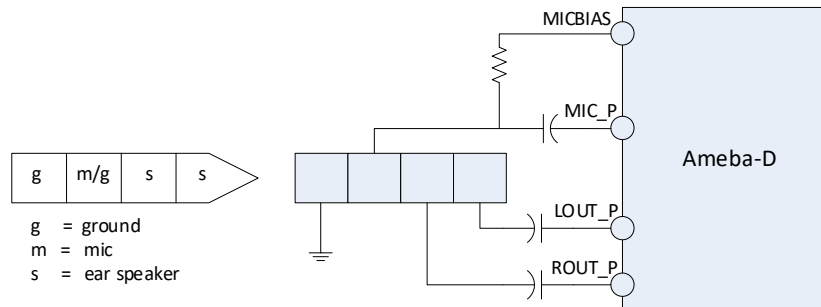


Fig 23-4 Single-ended mode connection with headphone jack

23.1.3.2 Line-in

Line-in has no preamplifier, its input signal often has a large output power. It often connects to the audio output of equipment such as electric guitar, electronic organ and synthesizer.

Connect the left channel of line-in signal to AUX_L, and the right channel to AUX_R accordingly.

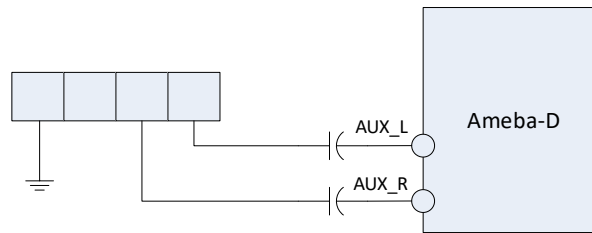


Fig 23-5 Line-in mode connection

23.1.3.3 AMIC-in

Analog microphone (AMIC) records audio data, it has preamplifier, its input signal often has a low output power. AMIC-in supports differential mode and single-ended mode.

- AMIC-in single-ended mode

Connect MIC_P with single-ended analog microphone, while MIC_BIAS provides the microphone bias voltage.

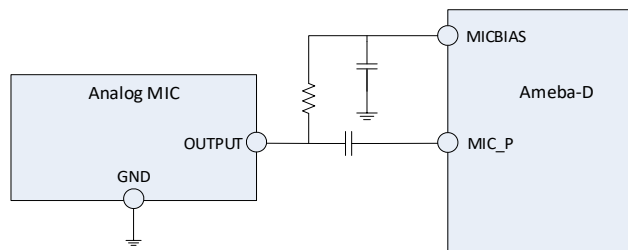


Fig 23-6 AMIC-in single-ended mode connection

- AMIC-in differential mode

Connect MIC_P/MIC_N with differential analog microphone, while MIC_BIAS provides the microphone bias voltage.

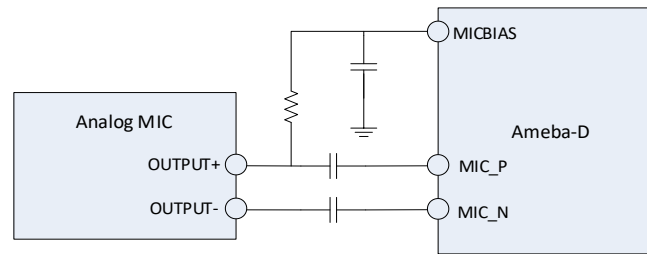


Fig 23-7 AMIC-in differential mode connection

23.1.3.4 DMIC-in

Digital microphone (DMIC) records audio data, it is integrated with ADC internal, and can directly output digital signal. DMIC-in supports mono mode and stereo mode.

- DMIC-in mono mode

Tie the L/R of digital microphone to ground or VDD if only one digital microphone is placed.

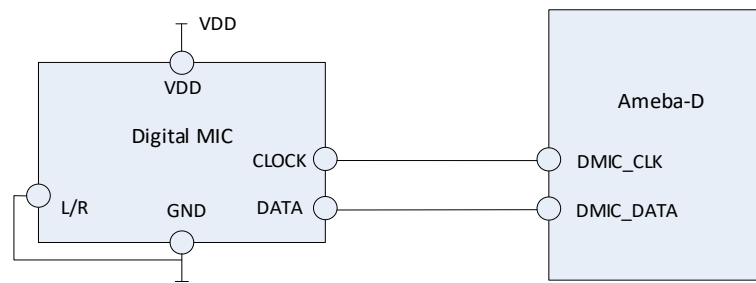


Fig 23-8 DMIC-in mono mode connection

- DMIC-in stereo mode

Tie the L/R of two digital microphones to ground and VDD respectively if stereo microphone is needed. The two microphones share the DMIC_DATA according to the rising/falling edge.

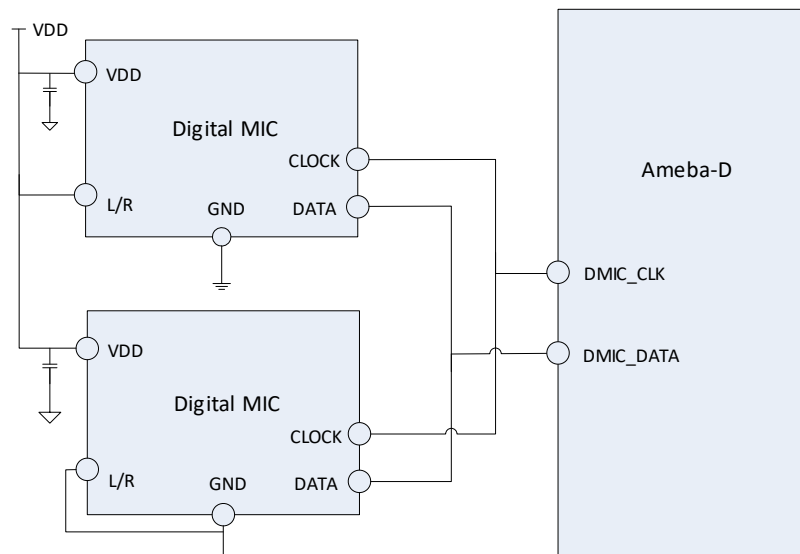


Fig 23-9 DMIC-in stereo mode connection

23.2 Audio Codec Controller

Ameba-D audio codec controller (ACC) is the bridge between host audio buffers and audio codec module. It is used for audio codec input/output control.

Ameba-D audio codec controller uses GDMA to move data, and transfers audio data to or from audio codec module via SPORT, and configures audio codec module via SI.

The diagram of ACC is shown in Fig 23-10.

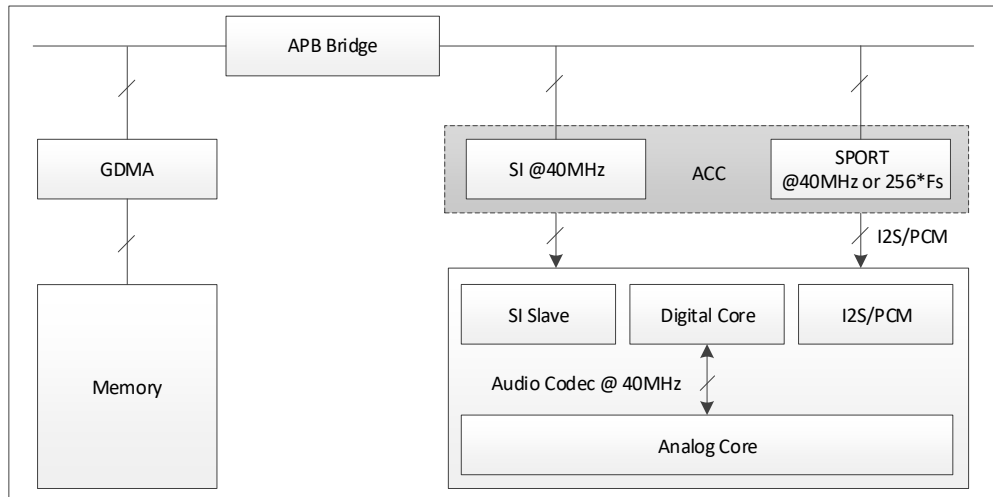


Fig 23-10 Audio codec controller diagram

23.2.1 Features

- Mono and stereo channel
- 8-bit, 16-bit and 24-bit sample bits
- I2S, left justify, PCM mode A, PCM mode B, PCM mode A-N and PCM mode B-N data format
- Mandatory or optional sample rate which audio codec module declares for support
- GDMA for data moving
- Data loopback between SDI and SDO

23.2.2 Control Interface

There are two control interfaces: SPORT interface and SI interface.

- SPORT interface

Audio codec transfers audio data via SPORT interface sequentially according to user's setting. It supports multiple data format, such as I2S, left justify, PCM mode A, PCM mode B, PCM mode A-N and PCM mode B-N.

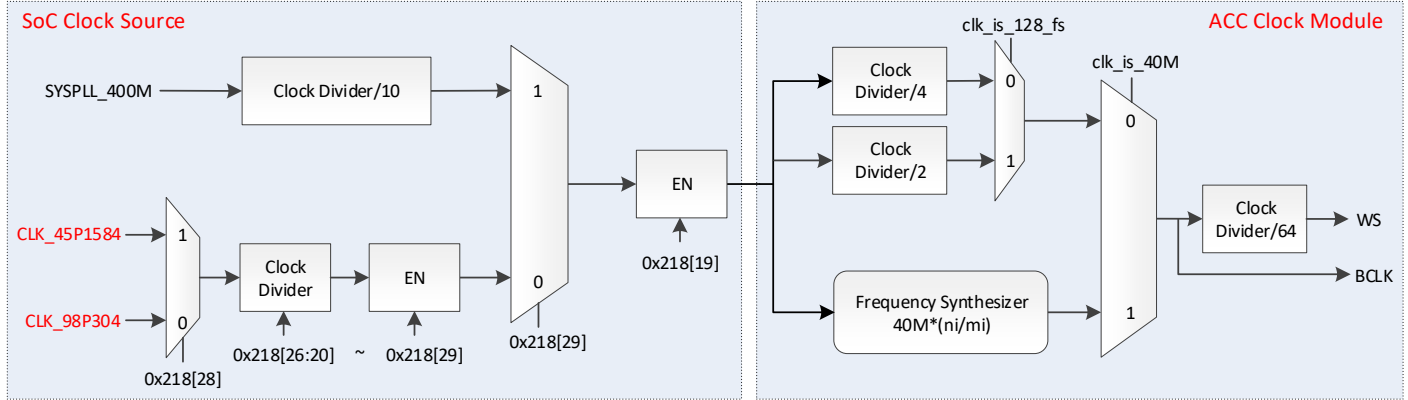
- SI interface

Audio codec needs to configure the related analog and digital parameters before transferring audio data. SI interface is used to configure codec parameters. It can read or write codec register.

For more details about AC and AAC, refer to UM0400 Ameba-D User Manual.

23.3 Audio PLL

The ACC clock architecture is shown in Fig 23-11.



Note: CLK_45P1584 is clock derived from PCM PLL internal, while CLK_98P304 from I²S PLL.

Fig 23-11 ACC clock architecture

23.3.1 Diagram

The ACC clock diagram is illustrated in Fig 23-12.

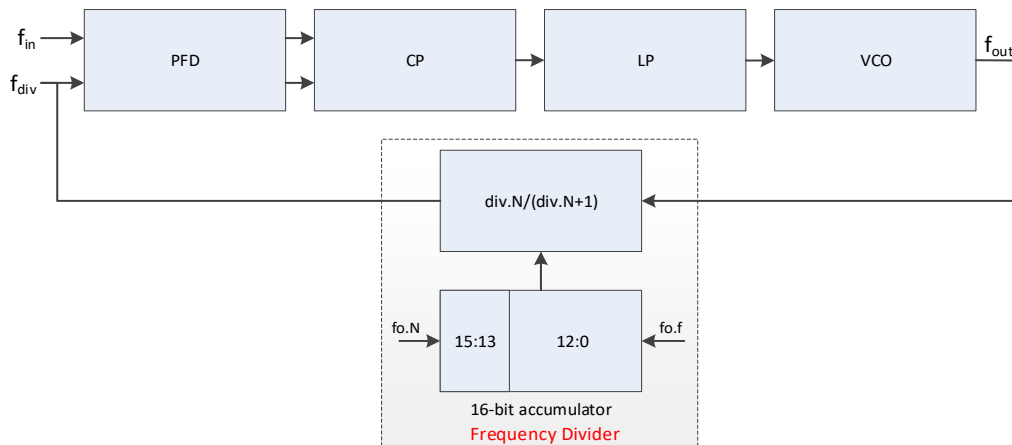


Fig 23-12 ACC clock diagram

$$\frac{f_{out}}{f_{in}} = \frac{\text{div.N} \times (2^{16} - (\text{fo.N} \times 2^{13} + \text{fo.f})) + (\text{div.N} + 1) \times (\text{fo.N} \times 2^{13} + \text{fo.f})}{2^{16}} = \text{div.N} + \frac{\text{fo.N}}{2^3} + \frac{\text{fo.f}}{2^{16}}$$

Where f_{in} is derived from the clock of crystal, while div.N , fo.N and fo.f from the register settings. Therefore, the resolution of f_{out} equals to $f_{in}/2^{16}$.

The relationship between crystal clock and f_{in} is listed in Table 23-1.

Table 23-1 Relationship between crystal clock and f_{in}

Crystal Clock (MHz)	f_{in} (MHz)	Divider
40	10	4
25	12.5	2
13	13	1
19.2	9.6	2
20	10	2
26	13	2
38.4	9.6	4

17.664	8.832	2
16	8	2
14.318	14.318	1
12	12	1
52	13	4
48	12	4
27	13.5	2
24	12	2

23.3.2 Operation Mode

23.3.2.1 Auto Mode

In auto mode, PLL circuit automatically sets the parameters (div.N, fo.N, fo.f, etc.) to output clock with the exact frequency, 196.608MHz (=98.304MHz x 2) is offered by I²S PLL while 180.6336MHz (=45.1584MHz x 4) by PCM PLL.

23.3.2.2 Manual Mode

If fine tuning PLL clock is required, users could switch the I²S or PCM PLL to manual mode. In this mode, users could control the output frequency of I²S PLL max ± 100 ppm around 196.608MHz or PCM PLL max ± 100 ppm around 180.6336MHz by calling the corresponding APIs. As the adjusted step of f_{out} equals to $f_{in}/2^{16}$, f_{in} decides the exact ppm per step ($(f_{in}/2^{16})/196.608$ or $(f_{in}/2^{16})/180.6336$).

The relationship between f_{in} and ppm per step is listed in Table 23-2.

Table 23-2 Relationship between f_{in} and ppm per step

f_{in} (MHz)	I ² S ppm per step	PCM ppm per step
10	0.78	0.84
12.5	0.97	1.06
13	1.01	1.10
9.6	0.75	0.81
10	0.78	0.84
13	1.01	1.10
9.6	0.75	0.81
8.832	0.69	0.75
8	0.62	0.68
14.318	1.11	1.21
12	0.93	1.01
13	1.01	1.10
12	0.93	1.01
13.5	1.05	1.14
12	0.93	1.01

23.4 Audio Codec APIs

23.4.1 PLL APIs

API	Introduction
<PLL_Div>	Divider to generate 256*fs or 128*fs clock
<PLL_Sel>	Selects I ² S PLL or PCM PLL
<PLL_I2S_Set>	Enables or disables I ² S PLL
<PLL_PCM_Set>	Enables or disables PCM PLL
<PLL_I2S_ClkTune>	Tunes I ² S PLL output faster or slower, or resets it to auto mode

<PLL_PCM_ClkTune>	Tunes PCM PLL output faster or slower, or resets it to auto mode
-------------------	--

23.4.1.1 PLL_Div

Parameter	Type	Introduction
<div>	u32	Divider to generate 256*fs or 128*fs clock.

23.4.1.2 PLL_Sel

Parameter	Type	Introduction
<sel>	u32	- Selects I²S PLL if fs=8/16/32/48/96kHz - Selects PCM PLL if fs=44.1/88.2kHz

23.4.1.3 PLL_I2S_Set

Parameter	Type	Introduction
<new_state>	u32	Enables or disables I ² S PLL

23.4.1.4 PLL_PCM_Set

Parameter	Type	Introduction
< new_state >	u32	Enables or disables PCM PLL

23.4.1.5 PLL_I2S_ClkTune

Parameter	Type	Introduction
<ppm>	u32	Required fine tuning ppm value
<action>	U32	Faster or slower, or reset to auto mode

23.4.1.6 PLL_PCM_ClkTune

Parameter	Type	Introduction
<ppm>	u32	Required fine tuning ppm value
<action>	U32	Faster or slower, or reset to auto mode

23.4.2 SPORT APIs

API	Introduction
<AUDIO_SP_StructInit>	Fills each SP_StructInit member with its default value
<AUDIO_SP_Init>	Initializes the audio SPORT registers according to the specified parameters in SP_InitStruct
<AUDIO_SP_TxStart>	Starts or stops SPORT Tx path
<AUDIO_SP_RxStart>	Starts or stops SPORT Rx path
<AUDIO_SP_TdmaCmd>	Enables or disables SPORT Tx DMA request
<AUDIO_SP_RdmaCmd>	Enables or disables SPORT Rx DMA request
<AUDIO_SP_SetWordLen>	Sets the AUDIO SPORT word length
<AUDIO_SP_GetWordLen>	Gets the AUDIO SPORT word length
<AUDIO_SP_SetMonoStereo>	Sets the AUDIO SPORT channel number
<AUDIO_SP_TXGDMA_Init>	Initializes GDMA peripheral for Tx data
<AUDIO_SP_RXGDMA_Init>	Initializes GDMA peripheral for Rx data

23.4.2.1 AUDIO_SP_StructInit

Fills each SP_StructInit member with its default value.

Parameter	Type	Introduction
<SP_InitStruct >	SP_InitTypeDef*	SP_InitTypeDef structure that contains the configuration information for the specified audio SPORT peripheral

23.4.2.2 AUDIO_SP_Init

Initializes the audio SPORT registers according to the specified parameters in SP_InitStruct.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<SP_InitStruct >	SP_InitTypeDef*	SP_InitTypeDef structure that contains the configuration information for the specified AUDIO SPORT peripheral

23.4.2.3 AUDIO_SP_TxStart

Starts or stops SPORT Tx path.

If playing with audio codec, it starts SPORT Tx path.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<NewState>	u32	State (enable or disable) of the SPORT Tx

23.4.2.4 AUDIO_SP_RxStart

Starts or stops SPORT Rx path.

If recording with audio codec, it starts SPORT Rx path.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<NewState>	u32	State (enable or disable) of the SPORT Rx

23.4.2.5 AUDIO_SP_TdmaCmd

Enables or disables SPORT Tx DMA request.

If Tx DMA request is not enabled, you should start Tx when GDMA completes every time.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<NewState>	u32	State (enable or disable) of the SPORT Tx DMA request

23.4.2.6 AUDIO_SP_RdmaCmd

Enables or disables SPORT Rx DMA request.

If Rx DMA request is not enabled, you should start Rx when GDMA completes every time.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<NewState>	u32	State (enable or disable) of the SPORT Rx DMA request

23.4.2.7 AUDIO_SP_SetWordLen

Sets the audio SPORT word length.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
<SP_WordLen>	u32	The value of word length

23.4.2.8 AUDIO_SP_GetWordLen

Gets the audio SPORT word length.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral

23.4.2.9 AUDIO_SP_SetMonoStereo

Sets the audio SPORT channel number.

SPORT only supports stereo channel and mono channel.

Parameter	Type	Introduction
<SPORTx>	AUDIO_SPORT_TypeDef*	The base address of audio SPORT peripheral
< SP_MonoStereo>	u32	Mono or stereo channel

23.4.2.10 AUDIO_SP_TXGDMA_Init

Initializes GDMA peripheral for sending data.

Parameter	Type	Introduction
<Index>	u32	GDMA index
<GDMA_InitStruct>	GDMA_InitTypeDef *	GDMA_InitTypeDef structure that contains the configuration information for the GDMA peripheral
<CallbackData>	void *	GDMA callback data
<CallbackFunc>	IRQ_FUN	GDMA callback function
<pTxData>	u8 *	Tx buffer that stores Tx data
<Length>	u32	Tx data length

23.4.2.11 AUDIO_SP_RXGDMA_Init

Initializes GDMA peripheral for receiving data.

Parameter	Type	Introduction
<Index>	u32	GDMA index
<GDMA_InitStruct>	GDMA_InitTypeDef *	GDMA_InitTypeDef structure that contains the configuration information for the GDMA peripheral
<CallbackData>	void *	GDMA callback data
<CallbackFunc>	IRQ_FUN	GDMA callback function
<pRxData>	u8 *	Rx buffer that stores the received data

<Length>	u32	Rx data length
----------	-----	----------------

23.4.3 SI APIs

SI APIs are used to read and write codec register.

API	Introduction
<AUDIO_SI_Cmd>	Enables or disables the specified audio SI peripheral
<AUDIO_SI_WriteReg>	SI writes codec register
<AUDIO_SI_ReadReg>	SI reads codec register
<AUDIO_SI_ClkCmd>	Turns on or turns off the clock of register bank of audio codec

23.4.3.1 AUDIO_SI_Cmd

Enables or disables the specified audio SI peripheral.

Parameter	Type	Introduction
<new_state>	u8	New state of the SI peripheral

23.4.3.2 AUDIO_SI_WriteReg

Uses SI interface to write codec register.

Parameter	Type	Introduction
<address>	u32	Codec register address
<data>	u32	Data value written to the register

23.4.3.3 AUDIO_SI_ReadReg

Uses SI interface to read codec register.

Parameter	Type	Introduction
<address>	u32	Codec register address

23.4.3.4 AUDIO_SI_ClkCmd

Turns on or turns off the clock of register bank of audio codec.

Parameter	Type	Introduction
<new_state>	u8	New state of the clock of register bank of audio codec

23.4.4 Codec APIs

API	Introduction
<CODEC_Init>	Initializes codec peripheral according to the application mode
<CODEC_SetVolume>	Sets codec volume by controlling mon DAC channel DVOL gain
<CODEC_GetVolume>	Gets codec mon DAC channel gain control
<CODEC_SetSr>	Sets codec ADC and DAC sample rate
<CODEC_SetAdcGain>	Sets codec ADC gain
<CODEC_SetAmicBst>	Set codec AMIC boost
<CODEC_SetDmicBst>	Set codec DMIC boost

<CODEC_SetMicBias>	Sets MIC_BIAS output voltage
<CODEC_MuteRecord>	Mutes or unmutes per AD channel
<CODEC_MutePlay>	Mutes or unmutes per DA channel.
<CODEC_Deinit>	De-initializes codec peripheral

23.4.4.1 CODEC_Init

Initializes codec peripheral according to the application mode.

Parameter	Type	Introduction
<sample_rate>	u32	Codec ADC and DAC sample rate
<word_len>	u32	Codec data sample bit
<mono_stereo>	u32	Codec mono channel or stereo channel
<application>	u32	Codec application mode, such as APP_AMIC_IN, APP_LINE_OUT

23.4.4.2 CODEC_SetVolume

Sets codec volume by controlling mon DAC channel DVOL gain.

Parameter	Type	Introduction
<vol_lch>	u8	Codec mon DAC left channel DVOL gain control (0.375dB/step) 8'hAF: 0dB 8'h00: -65.625dB
<vol_rch>	u8	Codec mon DAC right channel DVOL gain control (0.375dB/step) 8'hAF: 0dB 8'h00: -65.625dB

23.4.4.3 CODEC_GetVolume

Gets codec mon DAC channel gain control.

Parameter	Type	Introduction
<vol>	u16 *	Mon DAC channel DVOL gain (high 8 bits is RCH gain, low 8 bits are LCH gain)

23.4.4.4 CODEC_SetSr

Sets codec ADC and DAC sample rate.

Parameter	Type	Introduction
<sample_rate>	u32	Codec ADC and DAC sample rate

23.4.4.5 CODEC_SetAdcGain

Sets codec ADC gain.

Parameter	Type	Introduction
<ad_gain_left>	u32	ADC left channel digital volume gain
<ad_gain_right>	u32	ADC right channel digital volume gain

23.4.4.6 CODEC_SetAmicBst

Sets codec AMIC boost.

Parameter	Type	Introduction
<amic_bst_left>	u32	AMIC left channel boost gain
<amic_bst_right>	u32	AMIC right channel boost gain

23.4.4.7 CODEC_SetDmicBst

Sets codec DMIC boost.

Parameter	Type	Introduction
<dmic_bst_left>	u32	DMIC left channel boost gain
<dmic_bst_right>	u32	DMIC right channel boost gain

23.4.4.8 CODEC_SetMicBias

Sets MIC_BIAS output voltage.

Parameter	Type	Introduction
<mic_bias>	u8	Microphone bias voltage setting

23.4.4.9 CODEC_MuteRecord

Mutes or unmutes per AD channel.

Parameter	Type	Introduction
<mute_lch>	u32	Mutes for left AD channel
<mute_rch>	u32	Mutes for right AD channel

23.4.4.10 CODEC_MutePlay

Mutes or unmutes per DA channel.

Parameter	Type	Introduction
<mute_lch>	u32	Mutes for left DA channel
<mute_rch>	u32	Mutes for right DA channel

23.4.4.11 CODEC_DeInit

De-initializes codec peripheral.

Parameter	Type	Introduction
<application>	u32	Codec application mode

23.5 How to Use AC APIs?

23.5.1 Audio Play

To play the audio data through audio codec, follow the steps below:

- (1) Open audio codec clock and function
 - PLLx_Set (0, ENABLE); (x is 0 or 1)
 - RCC_PeriphClockCmd (APBPeriph_AUDIOC, APBPeriph_AUDIOC_CLOCK, ENABLE);

- RCC_PeriphClockCmd (APBPeriph_SPORT, APBPeriph_SPORT_CLOCK, ENABLE);
- (2) Enable pin for audio codec function
PAD_CMD (PinName, DISABLE);
- (3) Initialize codec with desired parameters
CODEC_Init (SampleRate, WordLen, MonoStereo, Application); (Application is APP_LINE_OUT)
- (4) If you need to change codec volume, use
CODEC_SetVolume (vol_lch, vol_rch);
- (5) If you want to adjust microphone bias output voltage, use
CODEC_SetMicBias (mic_bias);
- (6) Fill the SPORT desired parameters
AUDIO_SP_InitStruct (&SP_InitStruct);
- (7) Configure audio SPORT with the corresponding configuration
AUDIO_SP_Init (AUDIO_SP_DEV, &SP_InitStruct);
- (8) Start Tx path
AUDIO_SP_TdmaCmd (AUDIO_SPORT_DEV, ENABLE);
AUDIO_SP_TxStart (AUDIO_SPORT_DEV, ENABLE);
- (9) Activate GDMA to Tx data
AUDIO_SP_TXGDMA_Init (Index, &GDMA_InitStruct, *CallbackData, CallbackFunc, pTxData, Length);

23.5.2 Audio Record

To record the audio data through audio codec, follow the steps below:

- (1) Open audio codec clock and function
PLLx_Set (0, ENABLE); (x is 0 or 1)
RCC_PeriphClockCmd (APBPeriph_AUDIOIC, APBPeriph_AUDIOIC_CLOCK, ENABLE);
RCC_PeriphClockCmd (APBPeriph_SPORT, APBPeriph_SPORT_CLOCK, ENABLE);
- (2) Enable pin for audio codec function
PAD_CMD (PinName, DISABLE);
- (3) Initialize codec with desired parameters
CODEC_Init (SampleRate, WordLen, MonoStereo, Application);
Application can select APP_AMIC_IN for analog microphone, select APP_DMIC_IN for digital microphone, or select APP_LINE_IN.
- (4) If codec needs to change volume, use
CODEC_SetVolume (vol_lch, vol_rch);
- (5) If codec needs to change ADC gain, use
CODEC_SetAdcGain (ad_gain_left, ad_gain_right);
- (6) Fill the desired parameters
AUDIO_SP_InitStruct (&SP_InitStruct);
- (7) Configure audio SPORT with the corresponding configuration
AUDIO_SP_Init (AUDIO_SP_DEV, &SP_InitStruct);
- (8) Start Rx path
AUDIO_SP_RdmaCmd (AUDIO_SPORT_DEV, ENABLE);
AUDIO_SP_RxStart (AUDIO_SPORT_DEV, ENABLE);
- (9) Activate GDMA to Rx data
AUDIO_SP_RXGDMA_Init (Index, &GDMA_InitStruct, *CallbackData, CallbackFunc, pRxData, Length);

23.5.3 Example List

Example		Location
Playback		/project/realtek_amebaD_va0_example/example_sources/Audio/dac
Record and playback	AMIC	/project/realtek_amebaD_va0_example/example_sources/Audio/adac
	DMIC	/project/realtek_amebaD_va0_example/example_sources/Audio/dmic
Record and store	AMIC	/component/common/example/audio_sport/audio_recorder
Record and upload	DMIC	/component/common/example/audio_sport/audio_pcm_upload
Decode algorithm	AC3	/component/common/example/audio_sport/audio_ac3
	AMR	/component/common/example/audio_sport/audio_amr
	FLAC	/component/common/example/audio_sport/audio_flac

	Helix AAC	/component/common/example/audio_sport/audio_helix_aac
	Helix MP3	/component/common/example/audio_sport/audio_helix_mp3
	HLS	/component/common/example/audio_sport/audio_hls
	M4A	/component/common/example/audio_sport/audio_m4a
	M4A self-parse	/component/common/example/audio_sport/audio_m4a_selfparse
	MP3	/component/common/example/audio_sport/audio_mp3

Note: In the example code, we don't disable the GDMA even if no valid data are available. If you want to disable the GDMA, it should be done in a GDMA related interrupt routine. Disabling the GDMA outside of an interrupt routine may cause an exception if the GDMA is still transferring data.

23.6 Hardware Design Guide

23.6.1 Line-out

The line-out connection of audio codec is illustrated in Fig 23-13. The capacitors between 3.5mm jack and IC should be 47uF tantalum capacitors rather than ceramic capacitors. The reason is that capacitance value of ceramic capacitors may decrease when bias voltage is applied to them, which causes audio performance bad.

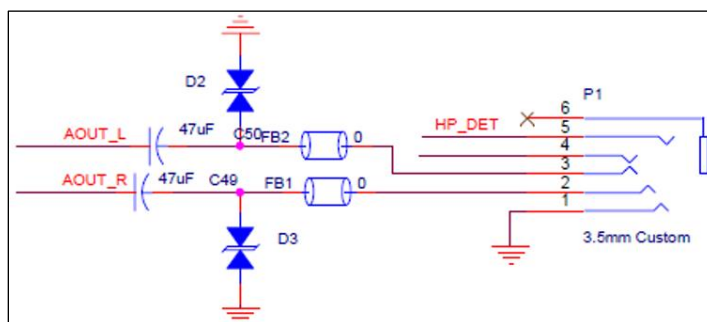


Fig 23-13 Line-out connection

23.6.2 AMIC-in

The AMIC-in connection of audio codec is illustrated in Fig 23-14. The capacitor between analog microphone and IC should be 1uF. Larger capacitance value makes longer period needed for capacitor charging.

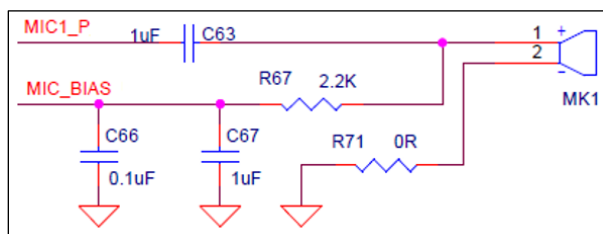


Fig 23-14 AMIC-in connection

MIC_BIAS connects to the positive side of microphone through a 2.2kohm resistor to offer bias voltage.

- Short the negative side of microphone to ground if working at single-ended mode, or connect to ground through a 2.2kohm resistor at differential mode.
- Connect the negative side of microphone to MIC_N through a 1uF capacitor at differential mode.

23.6.3 Power

The power connection of audio codec is illustrated in Fig 23-15.

- The capacitor between AUDIO_VREF and ground should be 4.7nF. Larger capacitance value makes longer period needed for AVCC's stabilization.
- The capacitor between AVCC or AVCC_DRIV and ground should be 1uF or a little larger to keep voltage stable.

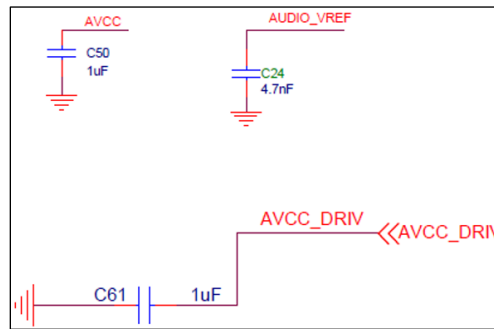


Fig 23-15 Power connection

23.7 Performance of Encoding & Decoding

The performance of decoding or encoding audio files of different formats are listed in Table 23-3 ~ Table 23-9. Data listed in these tables are obtained when the dynamic memory is allocated in SRAM. Data may be different if memory is allocated in PSRAM.

23.7.1 AC3 Format

The performance of decoding AC3 format audio data is listed in Table 23-3.

Table 23-3 Performance of decoding AC3 format audio data

AC Channel	Bit Rate (kHz)	Sample Rate (kHz)	Output Channel	Average MIPS Measured ¹	MIPS Simulated ²	
					Average	Maximum
5.1	640	48	1	50.5	53.7	54.5
5.1	448	48	1	72.2	70.8	76.8
5.1	448	48	1	69.9	68.2	90.8
5.1	448	48	1	68.7	65.9	92.3
2.1	192	48	1	54.8	50.6	52.2
2.0	192	48	1	TBD	50.7	49.6

1. The values are evaluated on the AmebaD_QFN88_EVB_V1.
2. The values are evaluated with the Keil simulator.

23.7.2 OPUS Format

The simulated CPU load of encoding and decoding of OPUS audio data is listed in Table 23-4. Keil simulator is used during the whole process. The decoding files used in the measurement process are created with opus-tools-0.2-opus-1.3.

Table 23-4 Simulated CPU load of encoding and decoding of OPUS audio data

Rate (kHz)	Channel	Bit Rate (kbps)	Complexity	MIPS		Measured	
				Decoding	Encoding	Decoding	Encoding
48	2	256	10	63	153.5	82.7	261
48	2	256	3	58	122.5	71.4	142.2
48	2	256	0	53	102	67.6	129.8

48	1	256	3	38	74.5	53.4	81.8
16	1	48	3	8.5	46.5	TBD	108.6
16	1	20	3	7	44.5	TBD	101.6

If users decide to use libopus library to deal with their data, the code size requirement is listed in Table 23-5. The version used here is libopus 1.1.4.

Table 23-5 Code size requirement using libopus

Type	Program Size	Data Size
SILK encoder	77.1k	8.5k
Original Source Code (Encode + Decode)	133k	23.6k

In Table 23-6, CPU load on using Silk encoder is listed.

Table 23-6 CPU load on using Silk encoder

Test Condition	Complexity	Average CPU Load (MHz)	Remarks
SILK encoder, 16k, 1ch, 20kbps	3	42	SILK VBR
	10	182	

23.7.3 FLAC Format

The simulated (with Keil simulator) and measured CPU load for decoding FLAC audio data is listed in Table 23-7. The tested file lasts for 279.64s, and the decoding time is 13.51s, which means 2.89s decoding time for 1-minute audio data.

Table 23-7 CPU load of decoding FLAC audio data

Rate (kHz)	Channel	Word Length	MIPS Simulated	MIPS Measured
44.1	2	16	19	9.66

23.7.4 AAC Format

The measured CPU load for decoding AAC audio data is listed in Table 23-8. The tested file lasts for 24.576s, and the decoding time is 3.55s, which means 8.68s decoding time for 1-minute audio data.

Table 23-8 CPU load of decoding AAC audio data

Rate (kHz)	Channel	Bit Rate (kbps)	MIPS
48	2	320	28.93

23.7.5 MP3 Format

The measured CPU load for decoding MP3 audio data is listed in Table 23-9. The tested file lasts for 5.58s, and the decoding time is 849ms, which means 9.13s decoding time for 1-minute audio data.

Table 23-9 CPU load of decoding MP3 audio data

Rate (kHz)	Channel	Bit Rate (kbps)	MIPS
32	2	320	30.43

23.8 Q & A

23.8.1 How to Connect the Output of DAC to A Power Amplifier?

In our SDK, the output of DAC is configured as single-ended by default. The N-end should be left alone. The P-end of L/R should be connected to the amplifier respectively. Choose either the P-end of L or R if only one channel is needed.

- If the power amplifier is an AB type amplifier, refer to the design shown in Fig 23-16 (power amplifier is LM4991).
- If the power amplifier is a D type amplifier, refer to the design shown in Fig 23-17 (power amplifier is ALC1003).

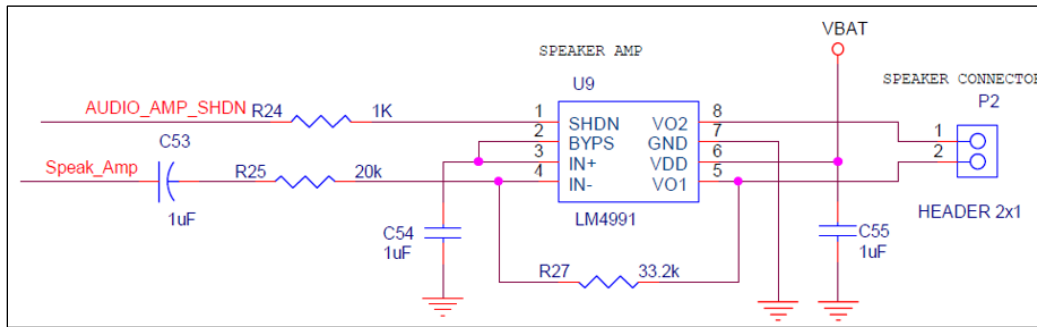


Fig 23-16 Reference design of using AB type power amplifier

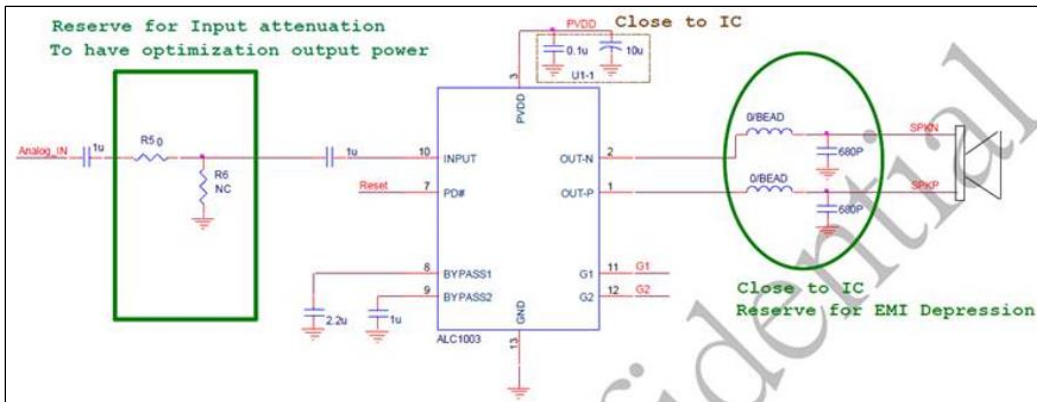


Fig 23-17 Reference design of using D type power amplifier

23.8.2 What's the Difference between Single-ended MIC Input and Differential MIC Input?

The SNR of differential mic input is better than single-ended input, but one more pin is needed. Ameba-D supports 2 mics. One supports both single-ended input and differential inputs, but another only supports single-ended input.

23.8.3 How to Play Local Audio Files?

If audio files are stored in SD card, you can use APIs related to SD card and file system to read and play audio files. If audio files are stored in flash, you need to copy the audio files to SRAM or PSRAM first, then call AUDIO_SP_TXGDMA_Init() to play them. Passing address directly to AUDIO_SP_TXGDMA_Init() doesn't work. The reason is that flash doesn't support GDMA transfer, but SRAM or PSRAM supports.

24 Cap-Touch

Ameba-D Cap-Touch provides 4 channels for capacitive sensing. The sensitivity and threshold for each channel are configurable. For different applications and surroundings, users should tune parameters to achieve the best performance.

This chapter introduces how to use Cap-Touch and design touch key.

24.1 Pinmux

The pin assignments of Cap-Touch channels are listed in Table 20-4.

Table 24-1 Cap-Touch pin assignments

Channel	Port Name	Pin Name	QFN48	QFN68	QFN88
0	PB[4]	TOUCH_KEY0	×	✓	✓
1	PB[5]	TOUCH_KEY1	×	✓	✓
2	PB[6]	TOUCH_KEY2	×	✓	✓
3	PB[7]	TOUCH_KEY3	×	✓	✓

24.2 APIs

24.2.1 CapTouch_StructInit

Items	Description
Introduction	Initializes the parameters in the CapTouch_InitStruct with default values.
Parameters	CapTouch_InitStruct: pointer to a CapTouch_InitTypeDef structure which is initialized.
Return	N/A

Note: CapTouch_InitStruct contains the configurable parameters for Cap-Touch and each channel, which includes the debounce control, ETC parameters, mbias current and threshold for channels.

24.2.2 CapTouch_Init

Items	Description
Introduction	Initializes the Cap-Touch peripheral according to the specified parameters in the CapTouch_InitStruct.
Parameters	- CapTouch: which should be CAPTOUCH_DEV. - CapTouch_InitStruct: pointer to a CapTouch_InitTypeDef structure that contains the configuration information for the specified Cap-Touch peripheral.
Return	N/A

24.2.3 CapTouch_Cmd

Items	Description
Introduction	Enables or disables the specified Cap-Touch peripheral.
Parameters	- CapTouch: which should be CAPTOUCH_DEV. - NewState: new state of the Cap-Touch peripheral. This parameter can be ENABLE or DISABLE.
Return	N/A

24.2.4 CapTouch_INTConfig

Items	Description
Introduction	Enables or disables the specified Cap-Touch interrupts.
Parameters	- CapTouch: which should be CAPTOUCH_DEV. - CapTouch_IT: specifies the Cap-Touch interrupt to be enabled or masked. This parameter can be one or combinations of the following values: - BIT_CT_OVER_N_NOISE_THRESHOLD_INT: Cap-Touch negative noise overflow interrupt - BIT_CT_FIFO_OVERFLOW_INT: Cap-Touch FIFO overflow interrupt - BIT_CT_OVER_P_NOISE_THRESHOLD_INT: Cap-Touch positive noise overflow interrupt - CT_CHX_PRESS_INT(x): Cap-Touch channel(x) press interrupt, where x can be 0~3 - CT_CHX_RELEASE_INT(x): Cap-Touch channel(x) release interrupt, where x can be 0~3 - NewState: new state of the specified Cap-Touch interrupts mask. This parameter can be ENABLE or DISABLE.
Return	N/A

24.2.5 CapTouch_GetISR

Items	Description
Introduction	Gets Cap-Touch interrupt status.
Parameters	CapTouch: which should be CAPTOUCH_DEV.
Return	Interrupt status

24.2.6 CapTouch_INTClearPendingBit

Items	Description
Introduction	Clears the specified Cap-Touch interrupt pending bit.
Parameters	- CapTouch: which should be CAPTOUCH_DEV. - CapTouch_IT: specifies the Cap-Touch interrupt to be cleared. This parameter can be one or combinations of the following values: - BIT_CT_N_NOISE_OVERFLOW_INT_CLR: Cap-Touch negative noise overflow interrupt - BIT_CT_FIFO_OVERFLOW_INT_CLR: Cap-Touch FIFO overflow interrupt - BIT_CT_P_NOISE_OVERFLOW_INT_CLR: Cap-Touch positive noise overflow interrupt - CT_CHX_PRESS_INT(x): Cap-Touch channel(x) press interrupt, where x can be 0~3 - CT_CHX_RELEASE_INT(x): Cap-Touch channel(x) release interrupt, where x can be 0~3
Return	N/A

24.3 How to Use CTC

24.3.1 CTC Initialization

The Cap-Touch initialization is implemented by the **app_captouch_init()** function in **main.c**. You can follow these steps to use it.

- Set the **km0_enable_key_touch** in **ps_config** (**Rtl8721dIp_sleepcfg.c**) to **BIT_CAPTOUCH_ENABLE** to enable Cap-Touch when KM0 boots.

```

PSCFG_TypeDef ps_config = {
    .km0_config_wifi_enable = TRUE,
    .km0_enable_key_touch = FALSE, //BIT_KEY_ENABLE | BIT_CAPTOUCH_ENABLE,
    .km0_tickles_debug = FALSE, /* if open WIFI FW, should close it, or beacon will lost in WOWLAN */
    .km0_os2m_close = TRUE,
    .km0_pg_enable = FALSE,
    .km0_rtc_calibration = FALSE,
    .km0_audio_pad_enable = TRUE,
};

```

- (2) Set configuration parameters in the following structure (**touch_key.c**), which includes touch threshold, noise threshold, mbias current, enable control for each channel. The method to tune parameters for each channel can be found in 24.3.2.

```
static CapTouch_CHInitTypeDef CTCChan[4] =
{
    /*DiffThreshold, MbiasCurrent, ETCNNoiseThr, ETCNNoiseThr, CHEnable*/
    {35,      0x16,      3,      18,      ENABLE}, /* Channel 0 */
    {500,     0x08,     250,     250,     DISABLE}, /* Channel 1 */
    {500,     0x08,     250,     250,     DISABLE}, /* Channel 2 */
    {500,     0x0b,     250,     250,     DISABLE}, /* Channel 3 */
};
```

- (3) Rebuild SDK and re-burn images, the Cap-Touch would work after boot up. When finger touch or proximity is detected, the Cap-Touch would send a “Key Press” interrupt to system.

24.3.2 CTC Calibration

To achieve the best performance (sensitivity, reliability or response time), users need to tune touch threshold, noise threshold and mbias current during development. The reference value of parameters is shown in Table 24-2. Of course, users can tune other parameters manually to fit special requirements.

The parameters for each channel should be tuned individually. To calibrate for channel 0, for example, users can follow these steps:

- (1) Initialize the parameter of CapTouch_InitStruct with default values using [CapTouch_StructInit\(\)](#) to enable channel 0.
 CapTouch_StructInit(&CapTouch_InitStruct);
 CapTouch_InitStruct.CT_Channel[0].CT_CHEnable = ENABLE;
- (2) Initialize the Cap-Touch peripheral according to CapTouch_InitStruct in step (1).
 CapTouch_Init(CAPTOUCH_DEV, &CapTouch_InitStruct);
- (3) Enable Cap-Touch.
 CapTouch_Cmd(CAPTOUCH_DEV, ENABLE);
- (4) Call CapTouch_GetChAveData() to read the sample data from channel 0 periodically.
 CapTouch_GetChAveData(CAPTOUCH_DEV, 0);

Table 24-2 Reference value of parameters

Parameters	Description	Reference Range	
		Finger Touch Detection	Proximity Detection
mbias	Set to a value which makes the baseline within the reference range.	Baseline: 2500~3500	Baseline: 3500~3800
Difference Threshold	Set to 80% of the touched difference value.	500~600	30~80
Noise Threshold	Set to no more than 50% of the difference threshold, and it can be adjusted according to the actual situation. Maximum value: 255	200~255	0~40

24.4 Cap-Touch Schematic Design and PCB Layout Guidelines

The CTC of Ameba-D has 4 Cap-Touch channels, and all the four channels support buttons and proximity sensors. Both the schematic design and PCB layout are important for Cap-Touch application. This chapter provides guidelines of Cap-Touch schematic design and PCB layout.

24.4.1 Cap-Touch Schematic Design

The CTC supports 4 channels, each channel supports button and proximity sensor, while only 2 channels are used. When designing the Cap-Touch schematic, you shouldn't select the sensor pins that are next to each other for better stability. Also, do not make the sensor trace run parallel to a clock signal or driven signal, which may result in cross-talk, and may cause the sensor next to those signals to make trigger false. Refer to PCB Layout in getting started with PCB layout design guidelines to avoid cross-talk.

All Cap-Touch channels must have a 47Ω series resistance (placed close to the chip) to improve noise immunity both for button and proximity sensor. If any Cap-Touch channel is not used, it is recommended to disable this channel and connect it to ground. Refer to How to Use CTC for details of disabling a channel.

For example, Fig 24-1 is a schematic in which only 4 channels are enabled.

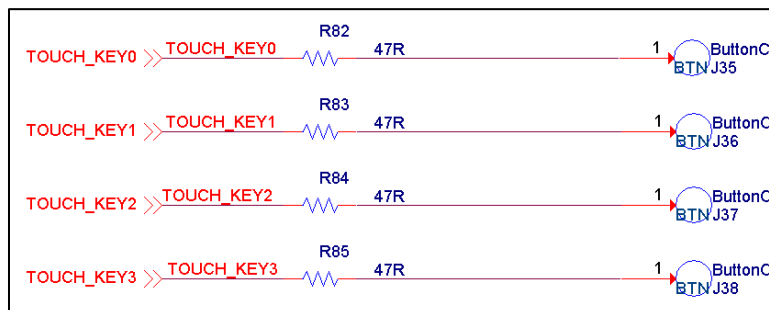


Fig 24-1 4 Channels enabled schematic

24.4.2 PCB Layout

In a typical Cap-Touch application, sensors are constructed with traces on a FR4/FR2 or flexible printed circuit (FPC). Cap-Touch layout design is an important step in the design phase, following the PCB layout design guidelines can help your design achieve higher noise immunity, lower parasitic capacitance (CP), and higher signal-to-noise ratio (SNR). The following factors must be considered during layout.

- Sensor parasitic capacitance: Sensor CP depends on sensor dimensions, PCB overlay and trace length. A high sensor CP makes it more difficult to sense small changes in sensor capacitance and thereby reduces sensitivity.
- Sensor dimensions: The sensor dimensions depend on the overlay thickness and suitable dimension for human touch. For touch convenience, a larger dimension is needed. A thicker overlay requires a larger sensor dimensions. But a larger dimension causes a higher parasitic capacitance.
- Sensor trace length: Longer trace lengths add the sensor CP and reduce the sensor sensitivity. Also, a long trace acts like an antenna and reduces the noise immunity of the sensor.
- Power consumption: The power consumption of the sensor depends on scan period and sensitivity (mbias), while the sensitivity depends on the CP of the sensor. Higher CP results in higher sensitivity; higher sensitivity and smaller scan period results in higher power consumption. To achieve a lower power consumption, reduce the sensor CP. To achieve a lower CP, higher SNR, and lower power consumption, follow the layout design guidelines.

For better performance, the follow guidelines provide recommendations of the sensor shape, minimum and maximum sensor dimensions, placement of the sensor, and routing traces on the PCB or FPC. Table 24-3 summarizes the sensor layout guidelines while designing a Cap-Touch application, both of button application and proximity application.

Table 24-3 Sensor layout recommendations

Category	Item	Min.	Max.	Description
Button	Button parasitic capacitance	4pF	50pF	Lower CP, lower sensitivity, lower power consumption
	Button shape	N/A	N/A	Round or rectangle with round corners
	Button size	5mm	15mm	Larger size causes more power consumption, the recommended size is 8mm~12mm for round shape.
	Button-Button spacing	Button-Ground clearance	N/A	Depending on button application, for better stability, the recommended spacing is larger than 8mm.
	Button-GND clearance	0.5mm	2.54mm	The recommended clearance is equal to the overlay thickness.
Proximity	Proximity parasitic capacitance	8pF	50pF	Proximity needs lower CP and higher mbias for better sensitivity, which causes more power consumption.
	Proximity sensor shape	N/A	N/A	● Circular or rectangular loop (with round corners) on PCB for large area. ● Circular or rectangular solid fill (with round corners) for small area. ● Line with recommended sensor trace width. ● A GND loop surrounding the sensor results in better noise immunity but worse sensitivity.
	Proximity sensor trace width	2mm		The recommended width is 2mm~3mm.
	Proximity sensor-GND loop clearance	1.5mm	2.5mm	The recommended width is 2mm.

	GND loop trace width	1.5mm		1.5mm
	Proximity sensor loop diameter			The recommended loop diameter needs to be larger than the proximity distance.

Table 24-4 summarizes the layout guidelines for placement of components, routing of sensor, sensor trace and GND layer while designing a Cap-Touch application, both button application and proximity application.

Table 24-4 Other layout recommendations

Category	Item	Min.	Max.	Description
Placement	2 layer PCB	N/A	N/A	- Top: Sensors, devices, and components; VDD and GND traces, other signal traces - Bottom: Sensor traces, devices and components; VDD and GND traces, other signal traces Note: - Avoid VDD traces, clock traces, and driven traces over sensor traces. - Avoid sensor trace run parallel to a clock signal or driven signal.
	4 layer PCB	N/A	N/A	- Top: Sensors, devices, and components - Second layer: Hatched ground - Third layer: Sensor traces, VDD layer - Bottom: Devices, and components Note: Avoid VDD traces, clock traces, and driven traces run parallel under or over sensor traces.
Routing	Sensor trace length			Not limited, make sure the trace capacitance and sensor capacitance is less than 50pF.
	Sensor trace width	6mil	9mil	The recommended value is 7mil (FR4/FR2/FPC).
	Sensor trace routing	N/A	N/A	The best choice for sensor is routed on the non-sensor side. Otherwise, make sure the sensor trace area is a touch forbidden area. Note: Avoid sensor trace run parallel to a clock signal or driven signal. If any non-sensor trace crosses the sensor trace, ensure that the intersection is orthogonal.
	Via position	N/A	N/A	Via should be placed near the edge.
	Via number	0	3	1 via is needed, more via results other parasitic capacitance.
	Via hole size	12mil/6mil	24mil/12mil	The recommended via hole size is 16mil/10mil.
	Trace series resistor	47	560	Series resistors close to the chip for better suppression and ESD.
	Trace-GND layer distance	10mil	20mil	20mil
GND	GND-Top layer			Hatched pattern 7mil trace and 35~45 mil grid
	GND-Bottom layer			Hatched pattern 7mil trace and 65~75 mil grid or no ground
Overlay	Overlay thickness	N/A	3mm	Depends on your product, 3mm for glass or plastic; higher thickness may decrease the sensitivity.
	Overlay material	N/A	N/A	Non-conductive material

For example, Fig 24-2 is a PCB design of button and proximity application.

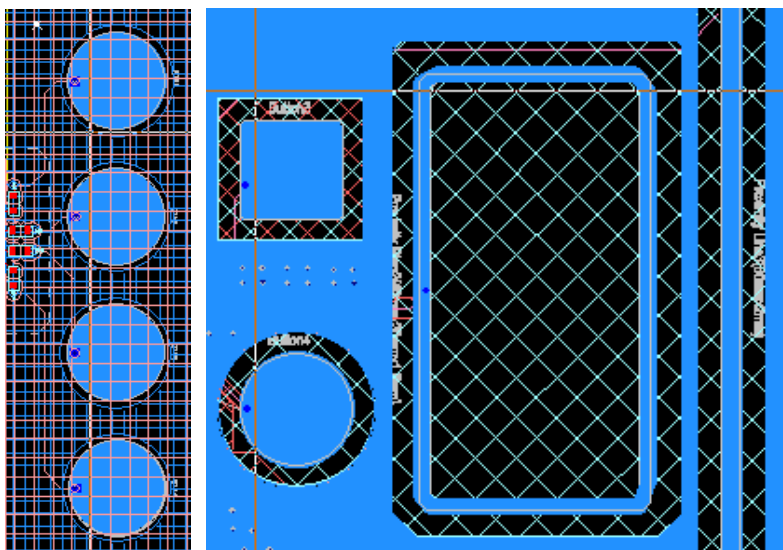


Fig 24-2 PCB design of button and proximity application

25 Infrared Radiation (IR)

Ameba-D Infrared Radiation (IR) provides hardware modulation for Infrared Radiation sending and hardware auto capture for receiving.

This chapter introduces how to use IR.

25.1 Pinmux

The pin assignments of IR are listed in Table 25-1.

Table 25-1 IR pin assignments

Port Name	Pin Name	QFN48	QFN68	QFN88
PA[25]	IR_TX	√	√	√
PA[26]	IR_RX	√	√	√
PB[23]	IR_TX	X	√	√
PB[22]	IR_RX	X	√	√
PB[31]	IR_TX	X	√	√
PB[29]	IR_RX	X	√	√

25.2 Data Format

This section introduces the data format of Tx FIFO and Rx FIFO.

25.2.1 IR Tx

To send IR data, you should write the data into IR Tx FIFO register, so it's important to understand the data format to be sent to Tx FIFO register. Before sending data, you should convert the data into the appropriate format that Tx FIFO register can recognized. The size of Tx FIFO register is 32 bits, where:

- Bit[31] indicates data type.
 - 0: inactive carrier
 - 1: active carrier
- Bit[30] is data end flag.
 - 0: normal packet
 - 1: last packet
- Bit[27:0], represented by cycle, stores the data to be sent and the data format is the number of carrier cycles. Let $f_{carrier}$ represents the carrier frequency (the unit of $f_{carrier}$ is kHz), $T_{duration}$ represents the duration of carrier or no carrier symbol (the unit of $T_{duration}$ is us). Then,

$$\text{bit}[27:0] = f_{carrier} * T_{duration} / 1000$$

Take NEC protocol for example, the carrier frequency is 38kHz. The format of NEC is shown in Fig 25-1 and the modulation of NEC is shown in Fig 25-2. The NEC format consists of 2 start symbols, 64 data symbols and 1 stop symbol.



Fig 25-1 NEC format

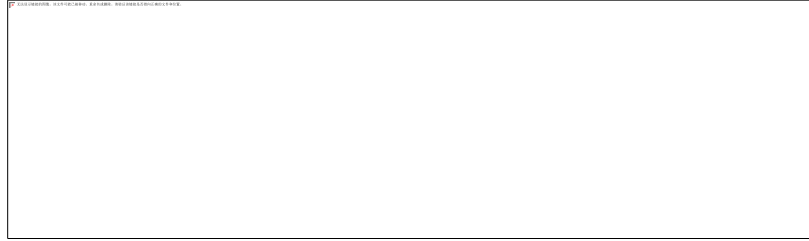


Fig 25-2 NEC modulation

- To send logical “1”, you should write two data into Tx FIFO register.
 - For the first data, bit[31] = 1, bit[30] = 0, bit[27:0] = $38 \times 560 / 1000 = 21$.
 - For the second data, bit[31] = 0, bit[30] = 0, bit[27:0] = $38 \times (2250 - 560) / 1000 = 64$.
- To send logical “0”, you should write two data into Tx FIFO register.
 - For the first data, bit[31] = 1, bit[30] = 0, bit[27:0] = $38 \times 560 / 1000 = 21$.
 - For the second data, bit[31] = 0, bit[30] = 0, bit[27:0] = $38 \times (1120 - 560) / 1000 = 21$.

At last, you need to send the stop symbol. In this symbol, set bit[30] = 1 because the stop symbol indicates the data end of the current transmission.

25.2.2 IR Rx

To receive IR data, you should read the data in Rx FIFO register, so it’s important to understand the data format to be received in Rx FIFO register. The size of Rx FIFO register is 32 bits, where:

- Bit[31] indicates data level
 - 0: high level
 - 1: low level
- Bit[30:0] stores the data to be received and the data format is cycle duration. Let $f_{sampling}$ represents the sampling frequency (the unit of $f_{sampling}$ is kHz), T_{level} represents the duration of each high/low level (the unit of T_{level} is us). Then,

$$T_{level} = \frac{bit[30:0]}{f_{sampling}} * 1000$$

With the use of T_{level} , you can do further processing to get the information you need.

25.3 APIs

25.3.1 IR Setting APIs

25.3.1.1 IR_StructInit

Items	Description
Introduction	Fills each IR_InitStruct member with its default value.
Parameters	IR_InitStruct: pointer to an IR_InitTypeDef structure which will be initialized.
Return	N/A

25.3.1.2 IR_Init

Items	Description
Introduction	Initializes the IR peripheral according to the specified parameters in the IR_InitStruct.
Parameters	● IRx: selected IR peripheral. ● IR_InitStruct: pointer to a IR_InitTypeDef structure that contains the configuration information for the specified IR peripheral.
Return	N/A

25.3.1.3 IR_Cmd

Items	Description
Introduction	Enables or disables the selected IR mode.
Parameters	● IRx: selected IR peripheral. ● mode: selected IR operation mode. This parameter can be the following values: ■ IR_MODE_TX: Transmission mode. ■ IR_MODE_RX: Receiving mode. ● NewState: new state of the operation mode. ■ ENABLE ■ DISABLE
Return	N/A

25.3.1.4 IR_INTConfig

Items	Description
Introduction	Enables or disables the specified IR interrupts.
Parameters	● IRx: selected IR peripheral. ● IR_INT: specifies the IR interrupt sources to be enabled or disabled. This parameter can be one or combinations of the following values: ■ IR_TX_FIFO_EMPTY_INT_EN: Tx FIFO empty interrupt. ■ IR_TX_FIFO_LEVEL_INT_EN: Tx FIFO threshold interrupt. ■ IR_TX_FIFO_OVER_INT_EN: Tx FIFO overflow interrupt. ■ IR_RX_FIFO_FULL_INT_EN: Rx FIFO full interrupt. ■ IR_RX_FIFO_LEVEL_INT_EN: Rx FIFO threshold interrupt. ■ IR_RX_CNT_OF_INT_EN: Rx counter overflow interrupt. ■ IR_RX_FIFO_OF_INT_EN: Rx FIFO overflow interrupt. ■ IR_RX_CNT_THR_INT_EN: Rx counter threshold interrupt. ■ IR_RX_FIFO_ERROR_INT_EN: Rx FIFO error read interrupt. Trigger when Rx FIFO empty and read Rx FIFO. ● NewState: new state of the specified IR interrupts. ■ ENABLE ■ DISABLE
Return	N/A

25.3.1.5 IR_MaskINTConfig

Items	Description
Introduction	Mask or unmask the specified IR interrupt.
Parameters	● IRx: selected IR peripheral. ● IR_INT: specifies the IR interrupt sources to be mask or unmask. This parameter can be one or combinations of the following values: ■ IR_TX_FIFO_EMPTY_INT_MASK: Tx FIFO empty interrupt mask. ■ IR_TX_FIFO_LEVEL_INT_MASK: Tx FIFO threshold interrupt mask. ■ IR_TX_FIFO_OVER_INT_MASK: Tx FIFO overflow interrupt mask. ■ IR_RX_FIFO_FULL_INT_Msk: Rx FIFO full interrupt mask. ■ IR_RX_FIFO_LEVEL_INT_Msk: Rx FIFO threshold interrupt mask. ■ IR_RX_CNT_OF_INT_Msk: Rx counter overflow interrupt mask. ■ IR_RX_FIFO_OF_INT_Msk: Rx FIFO overflow interrupt mask. ■ IR_RX_CNT_THR_INT_Msk: Rx counter threshold interrupt mask. ■ IR_RX_FIFO_ERROR_INT_Msk: Rx FIFO error read interrupt mask. Trigger when Rx FIFO empty and read Rx FIFO. ● NewState: new state of the specified IR interrupt mask. ■ ENABLE ■ DISABLE
Return	N/A

25.3.1.6 IR_GetINTStatus

Items	Description
Introduction	Get the specified IR interrupt status.
Parameters	IRx: selected IR peripheral.
Return	The new state of IR interrupt ● SET ● RESET

25.3.1.7 IR_GetIMR

Items	Description
Introduction	Get the specified IR interrupt mask status.
Parameters	IRx: selected IR peripheral.
Return	The new mask state of IR interrupt ● SET ● RESET

25.3.1.8 IR_FSMRunning

Items	Description
Introduction	Get the specified IR FSM status.
Parameters	IRx: selected IR peripheral.
Return	The new state of FSM: ● TRUE: RUN ● FALSE: IDLE

25.3.1.9 IR_ClearINTPendingBit

Items	Description
Introduction	Clears the IR interrupt pending bits.
Parameters	● IRx: selected IR peripheral. ● IR_CLEAR_INT: specifies the interrupt pending bit to clear. This parameter can be any combination of the following values: ■ IR_TX_FIFO_EMPTY_INT_CLR: Clear Tx FIFO empty interrupt. ■ IR_TX_FIFO_LEVEL_INT_CLR: Clear Tx FIFO threshold interrupt. ■ IR_TX_FIFO_OVER_INT_CLR: Clear Tx FIFO overflow interrupt. ■ IR_RX_FIFO_FULL_INT_CLR: Clear Rx FIFO full interrupt. ■ IR_RX_FIFO_LEVEL_INT_CLR: Clear Rx FIFO threshold interrupt. ■ IR_RX_CNT_OF_INT_CLR: Clear Rx counter overflow interrupt. ■ IR_RX_FIFO_OF_INT_CLR: Clear Rx FIFO overflow interrupt. ■ IR_RX_CNT_THR_INT_CLR: Clear Rx counter threshold interrupt. ■ IR_RX_FIFO_ERROR_INT_CLR: Clear Rx FIFO error read interrupt. Trigger when Rx FIFO empty and read Rx FIFO.
Return	N/A

25.3.2 IR Tx APIs

25.3.2.1 IR_SendBuf

Items	Description
Introduction	Send data buffer.
Parameters	● IRx: selected IR peripheral.

	● pBuf: data buffer to send. ● len: buffer length. ● IsLastPacket: this parameter can be the following values: ■ ENABLE: The last data in IR packet and there is no continuous data. In other words, an infrared data transmission is completed. ■ DISABLE: There is data to be transmitted continuously.
Return	N/A

Note: the difference between IR_SendBuf() and IR_SendData() is that IR_SendBuf() send a data buffer once and the number of sending data is the length of data buffer, while IR_SendData() only send one data once.

25.3.2.2 IR_SendData

Items	Description
Introduction	Send one data.
Parameters	● IRx: selected IR peripheral. ● data: data to send.
Return	N/A

25.3.2.3 IR_SetTxThreshold

Items	Description
Introduction	Set Tx threshold. When Tx FIFO depth <= threshold value, trigger interrupt.
Parameters	● IRx: selected IR peripheral. ● thd: Tx threshold.
Return	N/A

25.3.2.4 IR_GetTxFIFOFreeLen

Items	Description
Introduction	Get free size of Tx FIFO.
Parameters	IRx: selected IR peripheral.
Return	The free size of Tx FIFO.

25.3.2.5 IR_ClearTxFIFO

Items	Description
Introduction	Clear IR Tx FIFO.
Parameters	IRx: selected IR peripheral.
Return	N/A

25.3.3 IR Rx APIs

25.3.3.1 IR_ReceiveBuf

Items	Description
Introduction	Read data from Rx FIFO.
Parameters	● IRx: selected IR peripheral. ● pBuf: buffer address to receive data. ● len: read data length.
Return	N/A

Note: the difference between IR_ReceiveBuf() and IR_ReceiveData() is that IR_ReceiveBuf() read a data buffer once and the number of receiving data can be defined by users but shouldn't be larger than the data size in Rx FIFO, while IR_ReceiveData() only read one data once. The data size in Rx FIFO can be required by calling IR_GetRxDataLen().

25.3.3.2 IR_ReceiveData

Items	Description
Introduction	Read one data.
Parameters	IRx: selected IR peripheral.
Return	Data which read from Rx FIFO.

25.3.3.3 IR_SetRxThreshold

Items	Description
Introduction	Set Rx threshold. When Rx FIFO depth > threshold value, trigger interrupt
Parameters	- IRx: selected IR peripheral. - thd: Rx threshold.
Return	N/A

25.3.3.4 IR_GetRxDataLen

Items	Description
Introduction	Get data size in Rx FIFO.
Parameters	IRx: selected IR peripheral.
Return	Current data size in Rx FIFO.

25.3.3.5 IR_ClearRxFIFO

Items	Description
Introduction	Clear IR Rx FIFO.
Parameters	IRx: selected IR peripheral.
Return	N/A

25.3.3.6 IR_SetRxCounterThreshold

Items	Description
Introduction	Configure counter threshold value in receiving mode. You can use it to stop receiving IR data.
Parameters	- IRx: selected IR peripheral. - IR_RxCntThrType: This parameter can be the following values: - IR_RX_Count_Low_Level: Low level counter value >= IR_RxCntThr, trigger IR_INT_RX_CNT_THR interrupt. - IR_RX_Count_High_Level: High level counter value >= IR_RxCntThr, trigger IR_INT_RX_CNT_THR interrupt. - IR_RxCntThr: Configure IR Rx counter threshold value which can be 0 to 0x7ffffff.
Return	N/A

25.3.3.7 IR_StartManualRxTrigger

Items	Description
Introduction	Start trigger only in manual receive mode. Note: manual receive mode is seldom used.
Parameters	IRx: selected IR peripheral.
Return	N/A

25.4 IR Usage

25.4.1 Sending

25.4.1.1 Tx Polling Mode

To use IR sending function, the following steps are mandatory.

- (1) Configure the IR pinmux according to Table 25-1.

For example, in order to use PB[23] as IR Tx pin, call the following function. And it is the same for other IR pins.

```
Pinmux_Config(_PB_23, PINMUX_FUNCTION_IR);
```

- (2) Call IR_Cmd() to disable IR.
- (3) Set parameters, change some parameter if needed.
IR_StructInit(IR_InitTypeDef *IR_InitStruct);
- (4) Initialize hardware using the parameters in step (3).
IR_Init(IR_InitTypeDef *IR_InitStruct);
- (5) Write Tx data to FIFO using IR_SendBuf() or IR_SendData().
- (6) Call IR_Cmd() to enable IR to start transmission.
- (7) Write more data to FIFO if needed.

Note:

- In step (2) and step (6), It is suggested that disabling IR at first, and then enabling IR after writing data to FIFO.
- In step (5), pay attention to convert the data into the appropriate format that Tx FIFO register can recognized before writing data to FIFO. You can refer to section 25.2.1.

25.4.1.2 Special Notes

25.4.1.2.1 Tx FIFO Offset Issue

If you want to judge whether Tx data in FIFO has been sent completely or not, you'd better check Tx FIFO empty flag rather than TX_FIFO_OFFSET.

25.4.1.2.2 Tx Last Packet Cannot Let FSM Enter Idle Issue

If the last packet written to Tx FIFO cannot let Tx state machine enter idle, it is suggested that before enabling IR Tx, writing some data packets to Tx FIFO. You can refer to step (2) and step (6) in section 25.4.1.1.

25.4.2 Receiving

25.4.2.1 Rx Interrupt Mode

To use IR receiving function, the following steps are mandatory.

- (1) Configure the IR pinmux according to Table 25-1.

For example, in order to use PB[22] as IR Rx pin, call the following function. And it is the same for other IR pins.

```
Pinmux_Config(_PB_22, PINMUX_FUNCTION_IR);
```

- (2) Set parameters, such as sampling frequency, Rx FIFO threshold level, Rx counter threshold type, Rx counter threshold level, Rx trigger mode if needed.
IR_StructInit(IR_InitTypeDef *IR_InitStruct);
- (3) Initialize hardware using the parameters in step (2).
IR_Init(IR_InitTypeDef *IR_InitStruct);
- (4) Configure interrupt if needed and register interrupt callback function.
IR_INTConfig(IR_DEV, IR_RX_INT_ALL_EN, ENABLE);
InterruptRegister((IRQ_FUN) IR_irq_handler, IR_IRQ, (u32)NULL, 10);
InterruptEn(IR_IRQ, 10);
- (5) Call IR_Cmd() to enable IR.

- (6) Clear Rx FIFO by calling IR_ClearRxFIFO().
- (7) When Rx FIFO threshold interrupt triggers, read data from Rx FIFO with the use of IR_ReceiveBuf() and IR_ReceiveData(), and make further processing in interrupt handle function.

Note:

- In step (7), to decode the receiving data correctly, you should understand the data format in Rx FIFO register. You can refer to section 25.2.2.
- Waveform inverse issue: in Rx ending, if the waveform is inverse, you should #define INVERSE_DATA in Ir_nec_protocol.h and set IR_InitStruct.IR_RxCntThrType = IR_RX_COUNT_HIGH_LEVEL.

25.4.2.2 Rx Learning

The process of Rx learning is similar to common Rx introduced in section 25.4.1.2.1. As shown in Fig 25-3, the difference is that in interrupt handle function, Rx learning should store each pulse of the Rx waveform, while common Rx only needs to store the carrier or un-carrier duration.

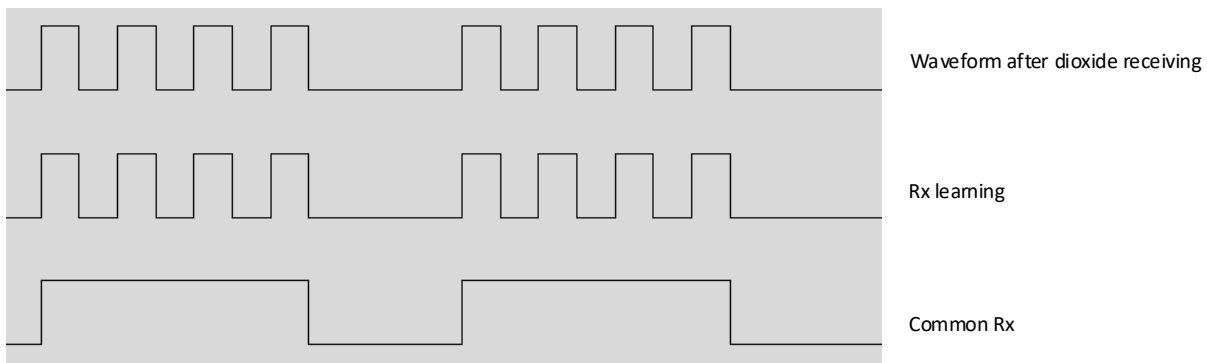


Fig 25-3 Difference of waveform between Rx learning and common Rx

Note:

- It is advised that putting the interrupt handle function code in RAM, and close other peripheral interrupt to avoid interfere.
- If the carrier frequency of learning waveform is larger than 400kHz, hardware may cannot respond to interrupt in time, which will result in decoding carrier frequency failed.

25.5 Tx Compensation Mechanism

25.5.1 Introduction

Tx waveforms are composed of some carrier symbols and no carrier symbols. Software calculates the duration of certain symbol by application specification (such as NEC). But no carrier symbols cannot be divisible by carrier frequency accurately. If there is no effective compensation mechanism, the error will increase. So Tx compensation is proposed and used in IR Tx to decrease the error.

In most scenarios, it is not necessary to adopt compensation mechanism. Therefore, if you're not interested in compensation mechanism, just skip this section. Next we would introduce the application of compensation mechanism.

25.5.2 Tx Compensation Mechanism Application

Take NEC application for example, the Tx NEC waveform shown in Fig 25-4 is with $f_{carrier} = 38\text{kHz}$, and duty = 1/3. The system clock is 100MHz.

For 38kHz carrier frequency:

$$T_{carrier} = \frac{1}{f_{carrier}} = \frac{1}{38K} = 26.31\mu s$$

$$T_{carrier_duty} = T_{carrier} * duty = 8.77\mu s$$

Compensation frequency f_{comp} is a dependent clock, you can set f_{comp} to any value you want. In this example, we set $f_{comp} = 1\text{MHz}$, so $T_{comp} = \frac{1}{f_{comp}} = 1\mu\text{s}$.

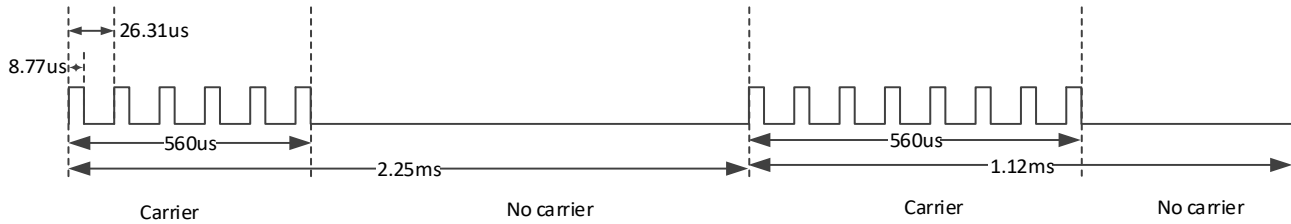


Fig 25-4 Tx NEC waveform

We divided this waveform into four symbols: 560us carrier symbol, 2250us ~ 560us no carrier symbol, 560us carrier symbol, and 1120us ~ 560us no carrier symbol.

- If no compensation mechanism is adopted ($\text{IR_TX_COMPENSATION} = 0$), the real wave can be calculated as follows.
 - The first carrier symbol: Referring to section 25.2.1, cycle = 21, $560\mu\text{s} \approx 21 \times 26.31 + 8.77 = 561.28\mu\text{s}$.
 - The second no carrier symbol: cycle = $(2250 - 560) \times 38 / 1000 = 64$, $(2250 - 560)\mu\text{s} = 1690\mu\text{s} \approx 64 \times 26.31 + (26.31 - 8.77) = 1701.38\mu\text{s}$.
 - The third carrier symbol is the same as the first one.
 - The forth no carrier symbol: cycle = 21, $560\mu\text{s} \approx 21 \times 26.31 + (26.31 - 8.77) = 570.05\mu\text{s}$.
- If compensation mechanism is adopted ($\text{IR_TX_COMPENSATION} = 3$), the real wave can be calculated as follows.
 - The first carrier symbol: Referring to section 25.2.1, cycle = 21, $560\mu\text{s} \approx 21 \times 26.31 + 8.77 = 561.28\mu\text{s}$.
 - The second no carrier symbol: cycle = $(2250 - 560) \times 1000 / 1000 = 1690$, $(2250 - 560)\mu\text{s} = 1690\mu\text{s} = 1690 \times 1\mu\text{s}$.
 - The third carrier symbol is the same as the first one.
 - The forth no carrier symbol: cycle = 560, $560\mu\text{s} = 560 \times 1\mu\text{s}$.

Compare the above two calculating methods, we can find that by using compensation mechanism, the accuracy can be improved.

Note:

- Compensation mechanism can only be used for no carrier symbol.
- $\text{IR_TX_COMPENSATION}$ is bit[28:29] of IR_TX_FIFO register. Compensation method 1 ($\text{IR_TX_COMPENSATION} = 1$) and method 2 ($\text{IR_TX_COMPENSATION} = 2$) are not recommended. If you want to use compensation mechanism, refer to method 3 ($\text{IR_TX_COMPENSATION} = 3$).

25.6 IR Schematic Design Guideline

25.6.1 Leakage

To avoid the leakage problem, we suggest using the IR circuit which is shown in Fig 25-5.

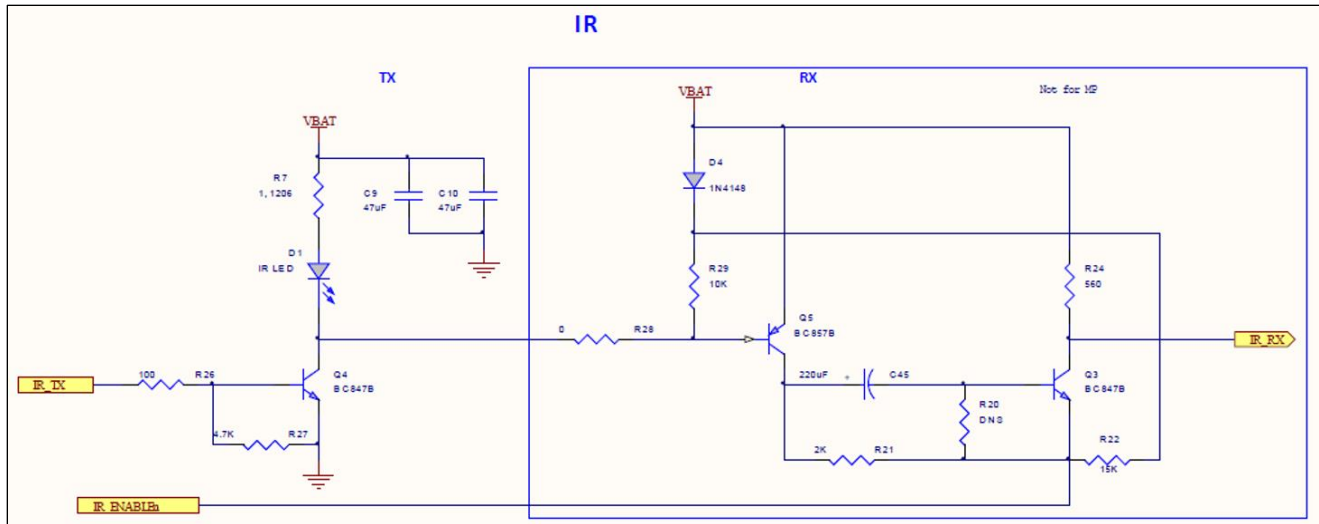


Fig 25-5 Circuit of IR

25.6.2 Carrier Problem in Rx Learning

Due to the characteristics of transistor, the response speed will slow down when the transistor works in deep saturation area. Therefore, when carrier frequency is set too large, the hardware has the risk of receiving carrier waveform failed in receiving end. In Rx learning, the maximum carrier frequency that can achieve is related to Rx circuit and the choice of transistor. The circuit we suggested in section 25.6.1 can support the maximum carrier frequency 70kHz, but you can choose the better transistor to acquire higher supported carrier frequency.

26 Brownout Detector (BOD)

BOD is mainly used to notify a user that the voltage level is low for the applications which use batteries. Ameba-D provides many thresholds to choose, the alternative levels and corresponding voltage value is shown in Table 26-1 and **Note:** The voltage value may have $\pm 5\%$ error. Table 26-2.

Table 26-1 High threshold of interrupt mode and reset mode (3.3V/1.8V)

Voltage	Value	Symbol	Reset	Interrupt
3.3V	001	BOR_TH_HIGH1	1.887	2.297
	010	BOR_TH_HIGH2	1.970	2.397
	011	BOR_TH_HIGH3	2.061	2.507
	100	BOR_TH_HIGH4	2.139	2.602
	101	BOR_TH_HIGH5	2.224	2.704
	110	BOR_TH_HIGH6	2.315	2.815
	111	BOR_TH_HIGH7	2.388	2.904
1.8V	001	BOR_TH_HIGH1	1.422	1.498
	010	BOR_TH_HIGH2	1.480	1.560
	011	BOR_TH_HIGH3	1.543	1.627
	100	BOR_TH_HIGH4	1.598	1.684
	101	BOR_TH_HIGH5	1.656	1.746
	110	BOR_TH_HIGH6	1.719	1.812
	111	BOR_TH_HIGH7	1.770	1.865

Note: The voltage value may have $\pm 5\%$ error.

Table 26-2 Low threshold of interrupt mode and reset mode (3.3V/1.8V)

Voltage	Value	Symbol	Reset	Interrupt
3.3V	001	BOR_TH_LOW1	1.784	2.194
	010	BOR_TH_LOW2	1.863	2.290
	011	BOR_TH_LOW3	1.949	2.395
	100	BOR_TH_LOW4	2.023	2.486
	101	BOR_TH_LOW5	2.103	2.584
	110	BOR_TH_LOW6	2.190	2.690
	111	BOR_TH_LOW7	2.260	2.775
1.8V	001	BOR_TH_LOW1	1.345	1.422
	010	BOR_TH_LOW2	1.400	1.480
	011	BOR_TH_LOW3	1.460	1.543
	100	BOR_TH_LOW4	1.512	1.598
	101	BOR_TH_LOW5	1.567	1.656
	110	BOR_TH_LOW6	1.627	1.719
	111	BOR_TH_LOW7	1.674	1.770

Note: The voltage value may have $\pm 5\%$ error.

26.1 Recommended Threshold Parameter

The recommended threshold parameters are shown in Table 26-3.

Table 26-3 Recommended Threshold Parameter

Work Voltage	Reset		Interrupt	
	High Threshold	Low Threshold	High Threshold	Low Threshold
3.3V	BOR_TH_HIGH7	BOR_TH_LOW6	BOR_TH_HIGH7	BOR_TH_LOW6
1.8V	BOR_TH_HIGH5	BOR_TH_LOW3	BOR_TH_HIGH5	BOR_TH_LOW3

Note: To avoid voltage fluctuation during work trigger BOD interrupt or reset by mistake, the difference between high and low threshold can be appropriately increased.

26.2 BOD APIs

26.2.1 BOR_ModeSet

Items	Description
Introduction	Choose BOD mode and enable BOD function
Parameters	- Option: - BOR_RESET: BOD reset mode - BOR_INTR: BOD interrupt mode - NewStatus: - ENABLE - DISABLE
Return	N/A

26.2.2 BOR_ThresholdSet

Items	Description
Introduction	Set BOD high and low thresholds
Parameters	- Thres_Low: BOD low threshold - BOR_TH_LOW1 - BOR_TH_LOW2 - BOR_TH_LOW3 - BOR_TH_LOW4 - BOR_TH_LOW5 - BOR_TH_LOW6 - BOR_TH_LOW7 - Thres_High: BOD high threshold - BOR_TH_HIGH1 - BOR_TH_HIGH2 - BOR_TH_HIGH3 - BOR_TH_HIGH4 - BOR_TH_HIGH5 - BOR_TH_HIGH6 - BOR_TH_HIGH7
Return	N/A

26.2.3 BOR_ClearINT

Items	Description
Introduction	Clear BOD interrupt.
Parameters	N/A
Return	N/A

26.2.4 BOR_DbncSet

Items	Description
Introduction	Set BOD interrupt mode debounce cycle
Parameters	- Option: - BOR_INTR: BOD interrupt mode

	Dbnc_Value: debounce cycle, in unit of ANA4M clock cycles.
Return	N/A

Note: Only BOD interrupt mode can set debounce cycle.

27 Flash Translation Layer (FTL)

27.1 Overview

NOR-Flash is comprised of blocks, which contains pages, and they contain individual cells of data. Flash read/write operations take place at page level. But erase operations take place at the block level. The flash needs to be erased before write. The memory portion for erasing differs in size from that for reading or writing, resulting in the major performance degradation of the overall flash memory system.

Therefore, a type of system software termed FTL has been introduced, which is provided to make the flash a friendly medium to store data. The architecture of Flash memory system is shown in Fig 27-1.

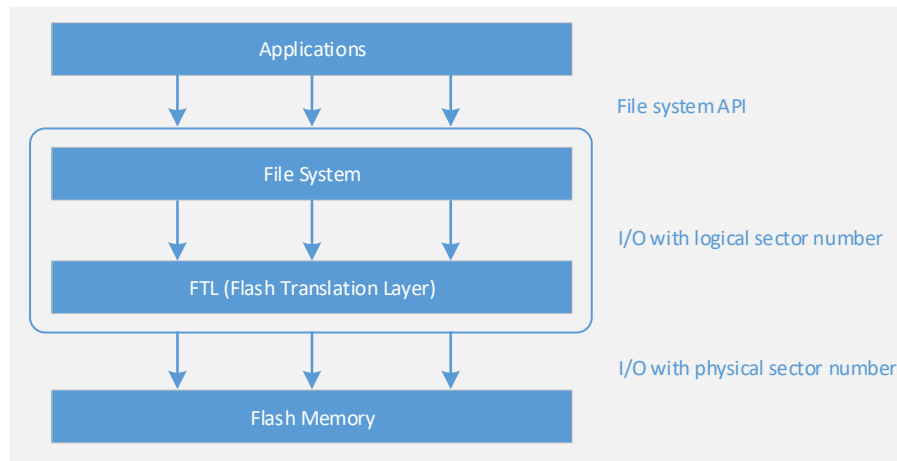


Fig 27-1 Architecture of Flash memory system

The FTL algorithm provides the following functionalities:

- **Logical-to-physical address mapping:** Convert logical addresses from the file system to physical addresses in flash memory.
- **Power-off recovery:** Even when a sudden power-off event occurs during FTL operations, FTL data structures should be preserved and data consistency should be guaranteed.
- **Wear-leveling:** Wear down memory blocks as evenly as possible.

As Fig 7-1 shows, to write data to logical map, FTL would generate a data packet in specified format and store it in flash. To modify data in logical map, a new packet would be generated and appended to the end of physical map. When the physical pages are nearly full, the garbage collection is triggered to recycle the old pages which would be erased.

To read data from logical map, FTL would search the physical map to find the newest packet, which contains the data of specified address.

When using FTL, users don't need to care about physical map, which is maintained automatically by FTL.

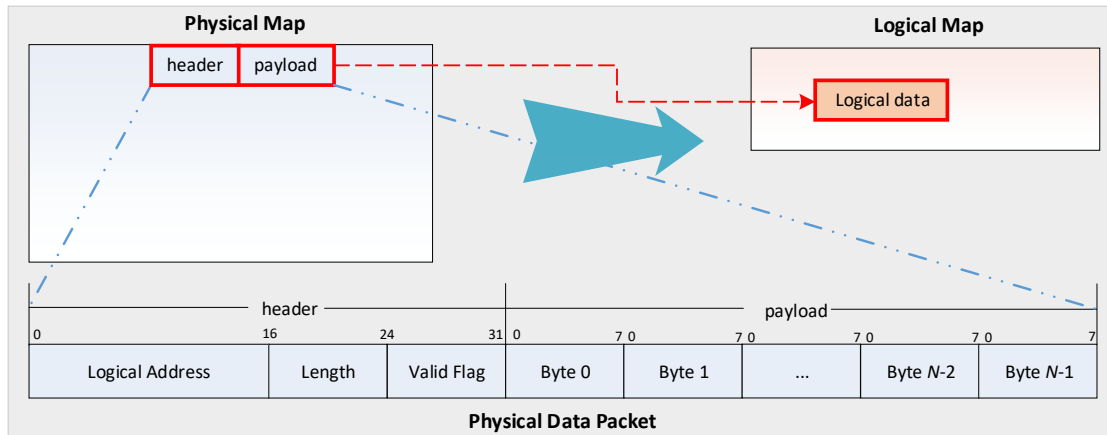


Fig 27-2 FTL overview

27.2 Features

- Physical Map
 - Physical page size: 4096 bytes
 - Configurable physical page number
 - Physical map size = 4096 * physical page number
- Logical Map: The maximum logical map size determined by physical map size is $((511 * (\text{physical page number} - 1)) - 1) * 4$.
- Auto Garbage Collection
- Abnormal Power-off Protection

27.3 FTL APIs

API	Introduction
<ftl_init>	Initializes FTL
<ftl_load_from_storage>	Gets specified length of data from logical map.
<ftl_save_to_storage>	Writes specified length of data to logical map.

27.3.1 ftl_init

Items	Description
Introduction	Initializes FTL
Parameters	- u32PageStartAddr: The start address of physical map - pagenum: The page number of physical map
Return	N/A

27.3.2 ftl_load_from_storage

Items	Description
Introduction	Gets specified length of data from logical map.
Parameters	- pdata_tmp: Pointer to a buffer to save logical data - offset: From which address read the logical map - size: The number of bytes to read
Return	0: Getting logical map values successfully - Others: Failed to get logical map value

27.3.3 ftl_save_to_storage

Items	Description
Introduction	Writes specified length of data to logical map.
Parameters	- pdata_tmp: Pointer to byte array of new logical data - offset: From which address to update the logical map - size: The number of bytes to update
Return	0: Writing logical map values successfully - Others: Failed to write logical map value

27.4 How to Use FTL

27.4.1 Precautions

When using FTL, you must know the precautions below.

- Theoretically, the logical map area can be extended to 64KB; but the maximum usable logical map area limited by the actual physical space is $((511 * (\text{physical page number} - 1)) - 1) * 4$.
- The default physical page number is 3, and it is not recommended to increase, because the default internal implement of FTL will malloc a buffer to cache the content of FTL.
 - For typical BLE application, each connection needs about 256 bytes FTL memory, and SDK will maintain 3 connections' information by default.
 - For BLE mesh, each node needs about 20 bytes FTL memory. The maximum node number can be set by dev_key_num.

Table 27-1 lists there typical scenarios of BLE, refer to it to set the appropriate offset of application according to your actual situation.

Table 27-1 Examples of recommended setting

Scenario	Calculation	Start address of application		Remark
		Configurable	Recommended	
3 connections - Less than 50 mesh nodes 1KB FTL space for application	$((3 * 256 + 20 * 50) + 1024)$ bytes (< 4084 bytes) are needed totally.	1768 ~ 3060	2500	- It is recommended to reserve some memory for system extension. The recommended offset has some reservation for both system and application. - For these two scenarios, 3 physical pages are enough.
5 connections 2KB FTL space for application	$(5 * 256 + 2048)$ bytes (< 4084 bytes) are needed totally.	1280 ~ 2036	1600	
5 connections - Less than 60 mesh nodes 6KB FTL space for application	$((5 * 256 + 20 * 60) + 6144)$ bytes are needed totally.	-	-	It is not recommended to store so much information in FTL. Try to find another solution, such as DCT.

27.4.2 General Steps

To use FTL in Ameba-D, the following steps are necessary.

- Define the macro CONFIG_FTL_ENABLED by yourself and set it to 1, the FTL initialization will be contained in main.

```
#define CONFIG_FTL_ENABLED 1
```

 If Bluetooth is enabled in your system, CONFIG_FTL_ENABLED is defined automatically after defining CONFIG_BT_EN.
- Define FTL_MEM_CUSTEM to 1 in rti8721dhp_intfcfg.c.

```

62: #if defined(CONFIG_FTL_ENABLED)
63: #define FTL_MEM_CUSTEM 1
64: #if FTL_MEM_CUSTEM == 0
65: #error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, Please refer to Application Note, FTL chapter."
66: #else
67: const u8 ftl_phy_page_num = 3; /* The number of physical map pages, default is 3*/
68: const u32 ftl_phy_page_start_addr = 0x00102000; /* The start offset of flash pages which is allocated to FTL physical map.
69:                                                    Users should modify it according to their own memory layout! */
70: #endif
71: #endif

```

Note: If the memory layout is modified and overwrites FTL area 0x0810_2000~0x0810_4FFF, you also need to modify the physical page start address. Otherwise, an error would be thrown out to remind you.

```
-o rt18721dhp_intfcfg.o
/cygdrive/e/AmebaD/AmebaD_svn/project/realtek_amebaD_cm4_gcc_verification/asdk/../../../../component/soc/realtek/amebaD/fwlib/usrcfg/rt18721dhp_intfcfg.c:64:2: error: #error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, Please refer to Application Note, FTL chapter."
#error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, Please refer to Application Note, FTL chapter."

make[4]: *** [/cygdrive/e/AmebaD/AmebaD_svn/project/realtek_amebaD_cm4_gcc_verification/asdk/Makefile.include:gen:449: rt18721dhp_intfcfg.o] Error 1
make[4]: Leaving directory '/cygdrive/e/AmebaD/AmebaD_svn/project/realtek_amebaD_cm4_gcc_verification/asdk/make/target/fwlib'
make[3]: *** [Makefile:19: all] Error 2
```

The default configurations for logical & physical map is as follows:

- 3 pages allocated for physical map: 0x0810_2000~0x0810_4FFF
- Logical map size: 4084 bytes

(3) Call ftl_load_from_storage/ftl_save_to_storage functions to read from/write to logical map.

28 Audio Signal Generation and Analysis

28.1 Introduction

When developing digital mic (DMIC) related applications, you may need to generate a signal of a certain frequency to test whether the DMIC works well or not. This chapter introduces how to generate a signal and how to analyze the frequency of a signal collected by the DMIC first. Then, digital analog-analog digital (DAAD) loopback is explained. DAAD loopback is helpful in determining which part (codec or DMIC) doesn't work well when problems occur.

28.1.1 Compilation

You need to add the **example_audio_signal_generate.c** to your project before you can use all the commands mentioned in this chapter. Related File used are placed under the path: **/component/common/example/audio_sport/audio_signal_generate**.

Follow two steps to use this function:

- (1) Set the macro named "AUDIO_SIGNAL_GENERATE" to 1 in **platform_opts.h** under the path: **/project/realtek_amebaD_va0_example/inc/inc_hp**.
- (2) In Cygwin terminal, change to the directory **/project/realtek_amebaD_va0_example/ GCC-RELEASE/project_hp**, type **make menuconfig**, and enable audio related configurations (MENUCONFIG FOR CHIP CONFIG > Audio Config > Enable Audio).

28.1.2 Generating a Signal of a Certain Frequency

Frequencies of signals supported by the program range from 20Hz ~ 20000Hz, which is the frequency range that people can hear.

Type command "Audio_generate tone spkr f " to generate a signal whose frequency is equal to f ($20 < f < 20000$).

28.1.3 Analyzing Signals Collected by DMIC

One way of testing a DMIC is to analyze the signals collected by the DMIC in frequency domain. If the detected frequency is the same as the frequency of the signal, the DMIC works well.

Type command "Audio_generate dmic fft 1" to analyze the signal collected by DMIC in frequency domain. The main frequency and its relative strength are printed out. Due to frequency leakage, the frequency detected may not be the same as the one you play, but the number should be closed. You can use this command to analyze signals whose frequencies range from 20~20000Hz.

28.1.4 DAAD

DMIC related problems can be caused by internal codec or DMIC itself. This section explains how to use DAAD loopback to determine whether the internal codec works in a normal way or not.

Fig 28-1 depicts how DAAD loopback works. The data transmitted from DA Digital IP is received directly by AD Digital IP without the interference of DMIC.

Type command "Audio_generate daad fft f " to generate a signal whose frequency is f . The main frequency detected and relative strength calculated will be printed out. If the frequency detected is closed to the one you generate, it means that the internal codec works well. The problem may be caused by DMIC.

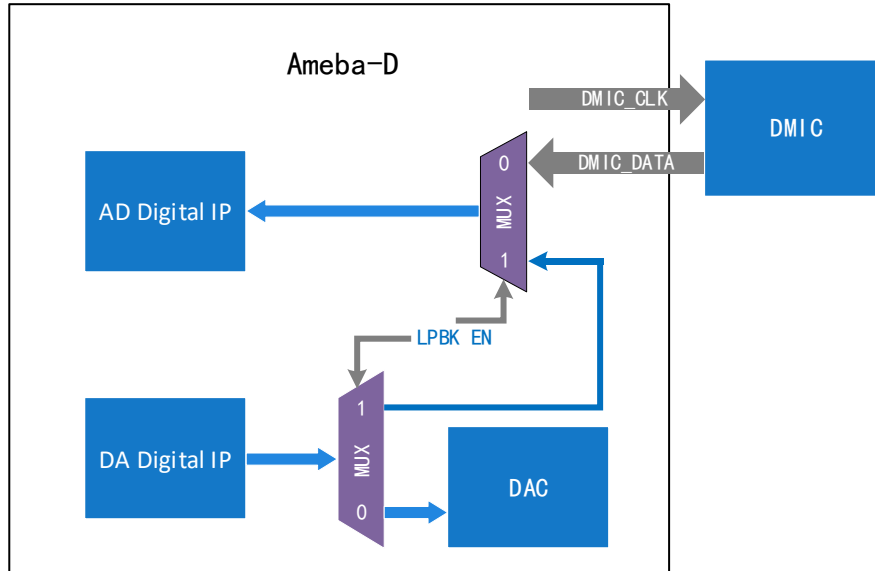


Fig 28-1 DAAD loopback diagram

28.1.5 Command

All the commands that can be used are listed in Table 28-1. Each command is composed of several parameters, and the parameters which are surrounded by () should be set by users.

Table 28-1 Available commands in audio signal generation and analysis

Command Format	Command Function	Example
Audio_generate tone spkr (<i>f</i>)	Generates a tone whose frequency is <i>f</i>	Audio_generate tone spkr 1000
Audio_generate dmic fft 1	Analyzes the signal collected by DMIC	Audio_generate dmic fft 1
Audio_generate daad fft (<i>f</i>)	DAAD loopback analysis	Audio_generate daad fft 1000
Audio_generate stop	Stops audio test	Audio_generate stop

29 Key-Scan

Ameba-D Key-Scan provides up to 6*6 (36) keypad array with 12 GPIOs, which can be configured for different applications, for example: 5*7 or 3*4 keypad arrays. Also multi-key detection and low power mode are supported.

This chapter introduces the APIs of Key-Scan, and how to use Key-Scan.

29.1 Pinmux

The pin assignments of Key-Scan are listed in Table 20-4.

Table 29-1 Key-Scan pin assignment

Port Name	Pin Name	QFN48	QFN68	QFN88
PA[12]	KEY_ROW0	Y	Y	Y
PA[13]	KEY_ROW1	Y	Y	Y
PA[14]	KEY_ROW2	Y	Y	Y
PA[15]	KEY_ROW3/KEY_COL6	Y	Y	Y
PA[16]	KEY_ROW4/KEY_COL5	N	Y	Y
PA[17]	KEY_ROW6/KEY_COL3	N	Y	Y
PA[18]	KEY_ROW5/KEY_COL4	N	Y	Y
PA[19]	KEY_COL2	N	Y	Y
PA[20]	KEY_COL7	N	N	Y
PA[21]	KEY_ROW7	N	N	Y
PA[25]	KEY_COL1	Y	Y	Y
PA[26]	KEY_COL0	Y	Y	Y

29.2 APIs

29.2.1 keyscan_array_pinmux

Items	Description
Introduction	Initialization of pinmux settings and pad settings
Parameters	- col: selected column number depending on KeyColumn (for example: col0 & col2, col is 5) - row: selected column number depending on KeyRow
Return	N/A

29.2.2 keyscan_getdatanum

Items	Description
Introduction	Gets data number of Key-Scan FIFO
Parameters	obj: Key-Scan object defined in application software
Return	Data number of Key-Scan FIFO

29.2.3 keyscan_read

Items	Description
Introduction	Reads data from Key-Scan FIFO
Parameters	- obj: Key-Scan object defined in application software. - pBuf: buffer to save data read from Key-Scan FIFO

	num: number of data to be read
Return	N/A

29.2.4 keyscan_init

Items	Description
Introduction	Initializes the Key-Scan device.
Parameters	obj: Key-Scan object defined in application software
Return	N/A

29.2.5 keyscan_set_irq

Items	Description
Introduction	Enables or disables the specified Key-Scan interrupts mask.
Parameters	- obj: Key-Scan object defined in application software. - keyscan_IT: specifies the Key-Scan interrupt sources to be enabled or masked. - newstate: new state of the specified Key-Scan interrupts mask.
Return	N/A

29.2.6 keyscan_clear_irq

Items	Description
Introduction	Clears the specified Key-Scan interrupt pending bit.
Parameters	- obj: Key-Scan object defined in application software. - keyscan_IT: specifies the Key-Scan interrupt to be cleared.
Return	N/A

29.2.7 keyscan_get_irq_status

Items	Description
Introduction	Gets Key-Scan interrupt status.
Parameters	obj: Key-Scan object defined in application software.
Return	Interrupt status

29.2.8 keyscan_set_irq_handler

Items	Description
Introduction	Sets Key-Scan interrupt handler.
Parameters	- obj: Key-Scan object defined in application software. - func: the interrupt handler function.
Return	N/A

29.2.9 keyscan_enable

Items	Description
Introduction	Enables the specified Key-Scan peripheral
Parameters	obj: Key-Scan object defined in application software.
Return	N/A

29.2.10 keyscan_disable

Items	Description
Introduction	Disables the specified Key-Scan peripheral
Parameters	obj: Key-Scan object defined in application software.
Return	N/A

29.2.11 keyscan_clearfifodata

Items	Description
Introduction	Clears the FIFO data
Parameters	obj: Key-Scan object defined in application software.
Return	N/A

29.3 Key-Scan Usage

There are two ways to use the Key-Scan: using default settings or using APIs listed in 29.2.

29.3.1 Using Default Configuration

When using default settings, only the keypad can be configured.

The Key-Scan initialization is implemented by the function **app_keyscan_init()** in **main.c**. User can follow these steps to use it.

- (1) Set the **km0_enable_key_touch** in **ps_config** (**Rtl8721dlp_sleepcfg.c**) to **BIT_KEY_ENABLE** to enable Key-Scan when KM0 boots.

```
PSCFG_TypeDef ps_config = {
    .km0_config_wifi_enable = TRUE,
    .km0_enable_key_touch = FALSE, //BIT_KEY_ENABLE | BIT_CAPTOUCH_ENABLE,
    .km0_tickles_debug = TRUE, // if open WIFI FW, should close it, or beacon will lost in WOWLAN */
    .km0_osc2m_close = TRUE,
    .km0_pg_enable = FALSE,
    .km0_rtc_calibration = FALSE,
    .km0_audio_pad_enable = TRUE,
};
```

- (2) Configure the GPIOs in the following structure (**touch_key.c**), and set valid to 1 for the pins which will be used in keypad.

```
static KeyMatrix_TypeDef KeyRow[8] =
{
    /* pinmux, valid */
    { _PA_12, 1 }, /* row 0 */
    { _PA_13, 1 }, /* row 1 */
    { _PA_14, 1 }, /* row 2 */
    { _PA_15, 1 }, /* row 3 */
    { _PA_16, 1 }, /* row 4 */
    { _PA_18, 0 }, /* row 5 */
    { _PA_17, 0 }, /* row 6 */
    { _PA_21, 0 }, /* row 7 */
};

static KeyMatrix_TypeDef KeyColumn[8] =
{
    /* pinmux, valid */
    { _PA_26, 1 }, /* Col 0 */
    { _PA_25, 1 }, /* Col 1 */
    { _PA_19, 1 }, /* Col 2 */
    { _PA_17, 1 }, /* Col 3 */
    { _PA_18, 1 }, /* Col 4 */
    { _PA_16, 0 }, /* Col 5 */
    { _PA_15, 0 }, /* Col 6 */
    { _PA_20, 1 }, /* Col 7 */
};
```

- (3) Rebuild SDK and re-burn images, the Key-Scan would work after boot up.

29.3.2 Using APIs

When using APIs, more parameters can be configured.

- (1) A Key-Scan object must be defined in application software first, then the necessary parameters must be configured.
For the parameter row, if row 0, row 2 and row 5 will be used, row must be set to 0x25 (100101), the same as parameter col. For the parameter clk, scan clock=bus clock/(clk+1).

```
struct keyscan_s {  
    u32 row;  
    u32 col;  
    u32 clk;  
    u32 workmode;    // 0 for regular scan mode, 1 for event trigger mode  
    u32 keylimit;  
    u32 overctrl;    // 0 for discard new, 1 for discard oldest  
};
```

- (2) Use **keyscan_init()** in **keyscan_api.c** to initialize Key-Scan.
- (3) Use **keyscan_set_irq()** in **keyscan_api.c** to enable the interrupt wanted.
- (4) Use **keyscan_set_irq_handler()** in **keyscan_api.c** to handle the triggered interrupt.
- (5) Use **keyscan_enable()** in **keyscan_api.c** to enable the Key-Scan. Then the Key-Scan would work.

30 Bluetooth

30.1 Overview

Bluetooth (BT) is one of the most popular short-range wireless communication technologies. The lower power feature of Bluetooth is redesigned and greatly enhanced since Bluetooth 4.0. The latest version of Bluetooth Low Energy (BLE) core specification is 5.2. BLE enables short-distance data exchange between fixed devices and mobile devices, and aims to build personal area networks (PAN) for its outstanding low power feature.

In Ameba-D, BLE shares WLAN's Radio Frequency (RF) circuit and antenna to transmit and receive data because BLE does not have a separate radio system. Users must ensure that WLAN is initialized before BLE's initialization, and BLE is de-initialized before WLAN's de-initialization.

When BLE is enabled for the first time, there is a compile error to remind users to configure the start address of Flash Translation layer (FTL). Users need to ensure that this area does not overlap with other areas.

Two ways of debug log are offered for BLE: the up level debug log and the low level debug log. The up level log is steaming out from PA[26], and the low level log is steaming out from PA[16]. Fortunately, these two GPIOs only work as UART pins when BLE log is enabled, so they can be used as normal GPIOs, only if a 0 ohm resistance is placed between the GPIOs and the peripheral circuit.

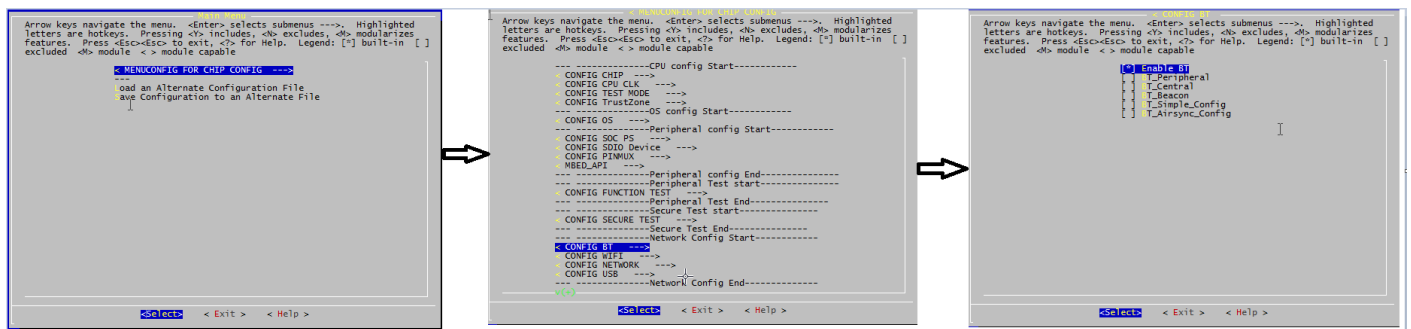
30.2 BT Examples

Ameba-D BT examples provide a set of BT functionality, such as BT Config, BT Peripheral, BT Central and BT Beacon. For more information, refer to *UM0201 Ameba Common BT Application User Manual EN.pdf*.

This section illustrates how to build and run BT examples in our SDK, including GCC and IAR environment.

30.2.1 GCC Project

- Enter SDK path: `project/realtek_amebaD_va0_example/GCC-RELEASE/project_hp`, input command `make menuconfig` to enable BT.



- Allocate FTL physical map by setting `FTL_MEM_CUSTEM` to 1 and setting FTL start address in `rtl8721dhp_intfcfg.c`.

```
#if defined(CONFIG_FTL_ENABLED)
#define FTL_MEM_CUSTEM 1
#endif
#error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, please refer to the FTL physical map configuration."
const u8 ftl_phy_page_num = 3; /* The number of physical map pages,
const u32 ftl_phy_page_start_addr = 0x00102000; /* The start offset of flash pages which
Users should modify it according to the FTL physical map configuration."
#endif
```

- Build images and use ImageTool to download images to your board.

Note: You can select one BLE example or all the examples at once. If all the examples are selected at once, the integrated image can support all BLE test commands, and the scatternet configuration will display only when both peripheral and central are selected.

30.2.2 IAR Project

- Enter SDK path: **project/realtek_amebaD_va0_example/inc/inc_hp**, edit **platform_autoconf.h** to enable BT by defining the corresponding macro to 1.

```
/*
 * < CONFIG_BT
 */
#define CONFIG_BT_EN 1
#define CONFIG_BT 1
#define CONFIG_BT 1
#define CONFIG_BT_PERIPHERAL 1
#define CONFIG_BT_CENTRAL 1
#define CONFIG_BT_SCATTERNET 1
#define CONFIG_BT_BEACON 1
#define CONFIG_BT_CONFIG 1
#define CONFIG_BT_AIRSYNC_CONFIG 1
```

- Allocate FTL physical map by setting FTL_MEM_CUSTEM to 1 and setting FTL start address in **rtl8721dhp_intfcfg.c**.

```
#if defined(CONFIG_FTL_ENABLED)
#define FTL_MEM_CUSTEM 1
#if FTL_MEM_CUSTEM == 0
#error "You should allocate flash sectors to for FTL physical map as following, then set FTL_MEM_CUSTEM to 1. For more information, please refer to the FTL physical map configuration."
#else
const u8 ftl_phy_page_num = 3; /* The number of physical map pages,
const u32 ftl_phy_page_start_addr = 0x00102000; /* The start offset of flash pages which
Users should modify it according to the FTL physical map configuration.
#endif
#endif
```

- Build image and use ImageTool to download images to your board.

Note: You can select one BLE example or all the examples at once. If all the examples are selected at once, the integrated image can support all BLE test commands, and the scatternet configuration will display only when both peripheral and central are selected.

30.3 BT Debug

30.3.1 Introduction

This section focuses on how to use log tools to capture log files. When user wants to debug an issue of BT SDK, four log files are needed, as shown in Table 30-1.

Table 30-1 Log files

File	File type	Description
FW LOG	RawData_*.log	BT controller log
BT TRACE LOG	.cfa & .bin	BT host log
LOGUART LOG	.txt	System log
APP.trace	.trace	The escaped index of BT host log

30.3.2 Hardware & Software Preparation

If user wants to capture log files to debug, the needed hardware and software are listed below.

30.3.2.1 Hardware

When using BT function, along with using the following GPIOs at the same time, it is best to series a 0 ohm resistor between these GPIOs and peripherals, so that you can easily pull out a test point when debugging BT. For more information, refer to the design of HDK.

Package	Log	Hardware pin	Default baud rate
---------	-----	--------------	-------------------

QFN48 (RTL8720xx)	FW LOG	PA[15]	115200
	BT TARCE LOG	PA[26]	150000
QFN68 (RTL8721xx)/QFN88 (RTL8722xx)	FW LOG	PA[16]	115200
	BT TARCE LOG	PA[26]	150000

30.3.2.2 Software

Name	Path	Description
BTDebugger.exe	\tools\bluetooth\BTFWDebugger	Used to capture the FW LOG
DebugAnalyzer	\tools\bluetooth\DebugAnalyzer	Used to capture the BT TRACE LOG
SecureCRT	-	Terminal emulator supporting SSH (SSH1 and SSH2), used to capture the LOGUART LOG
UartAssist	-	Serial port debugging tool, used to check that whether the log can be captured

30.3.3 Prerequisite – Opening the Log Switch

The log switch is closed by default. You need to open it when debugging the BT issue.

The debug bit is used to control the log switch. The default value of all bits are 1.

- 1: The log switch is closed.
- 0: The log switch is open.

Bit	Description	Comment
Bit[1]	The switch of FW LOG	For Ameba-D, the debug bit is located in the address of 0x8003028 in Flash. The value of the word is 0x9d, that is to say, bit[1], bit[5] and bit[6] are all 0. You can use the tool shown in Fig 30-1 to change the word value to 0x9d.
Bit[5]	The switch of BT TRACE LOG	
Bit[6]	The switch of DRIVER debug	

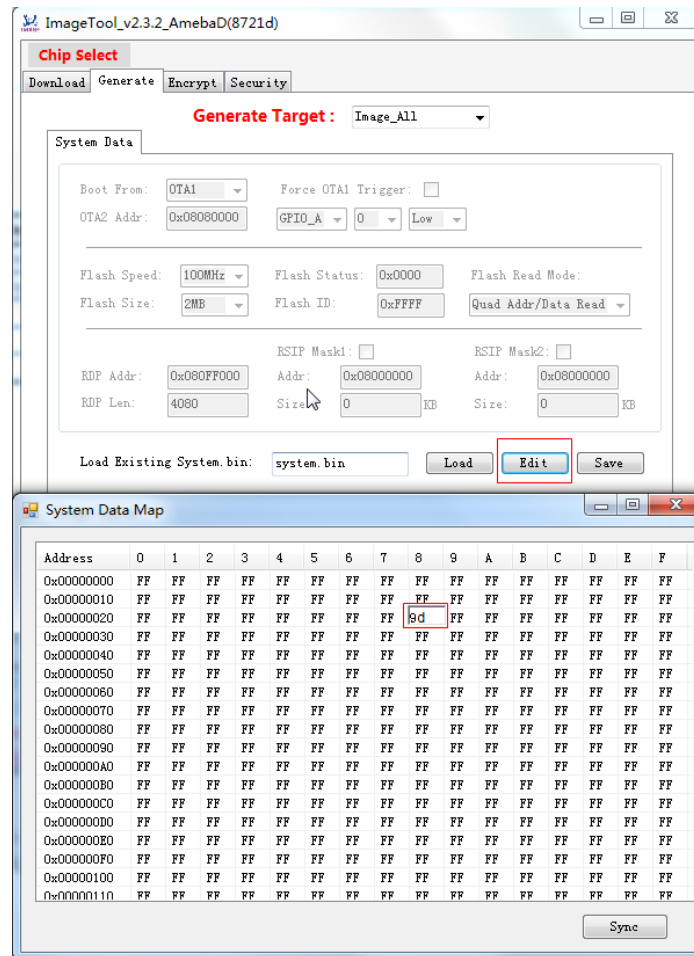


Fig 30-1 Changing the world value in Flash

Note:

- The debug bit can be written only once. You must erase the sector when writing the debug bit again, otherwise hardfault occurs.
- The debug bit is used to debug only. If you want to exit the debug mode, you need to erase the Flash.

30.3.4 Capturing Log Files

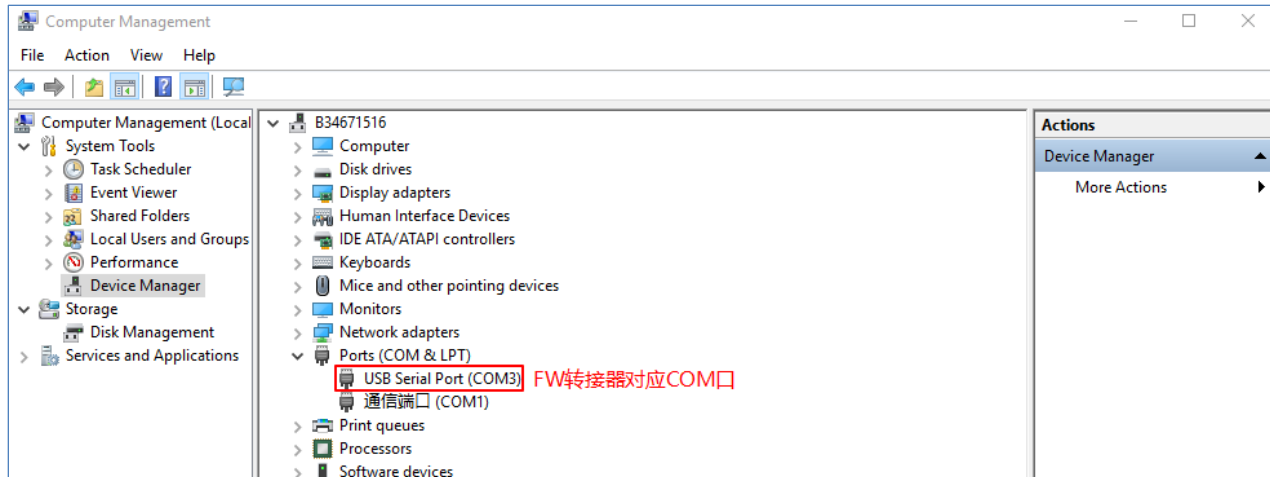
As mentioned above, four log files should be captured when debugging, the operations are described below.

30.3.4.1 FW LOG

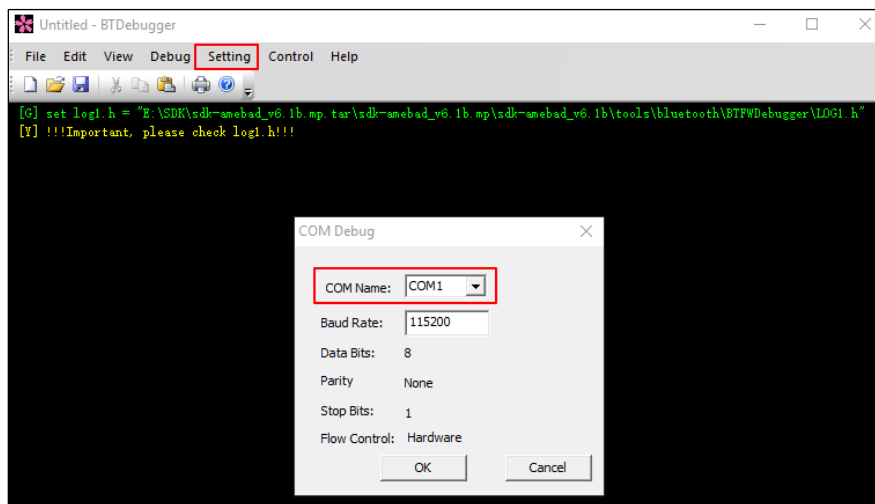
The FW LOG pin for Ameba-D is PA[15] or PA[16]. This is a hardware pin, you can't change it. So remember to reserve the pin when debugging the BT issue.

Follow the steps below to capture the FW LOG:

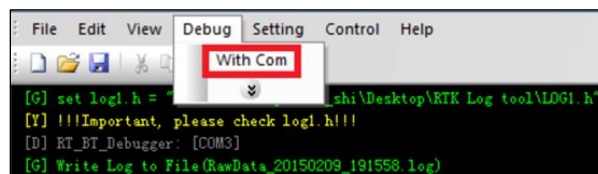
- (1) Open the Device Manager of your PC to check the number of the UART COM (e.g. COM3).



- (2) Launch the software – BTDebugger.exe, then click **Setting > Com Setting** to choose the corresponding COM of UART.



- (3) Click **Debug > With Com** to start the capture of FW LOG.



If the log capture is successful, you can see that the FW LOG (RawData*.log) under the tool directory is constantly increasing during the Bluetooth operation. Once the file size is found to be constant even though you refresh or operate the Bluetooth, re-check the above steps.

BTDebugger.exe	2016/7/22 16:15	应用程序	3,836 KB
RawData_20190416_111955.log	2019/4/16 11:21	文本文档	3 KB

30.3.4.2 BT TRACE LOG

The BT TRACE LOG pin for Ameba-D is PA[26]. This pin shares the software pin of UART, it can be changed to another pin according to the Datasheet.

Follow the steps below to capture the BT TRACE LOG:

- (1) Open the Device Manager of your PC to check the number of the UART COM.
- (2) Launch the software – DebugAnalyzer.exe, then choose the corresponding COM of Serial Port, and configure the items.

Serial Port COM1	Decoder Input ☒ Decode From Serial ☐ Decode From USB ☐ Decode From File	Decoder Output ☒ Save Decoded File ☒ Line Number ☒ Timestamp ☒ Save Snoop File ☒ Show Errors ☒ Save Raw File ☒ Sequence Number ☒ Internal Time ☒ Ellisys Inject ☒ Show Output Logs	Log ☒ Auto Scroll ☒ RT Decode Save Load Clear
Start			

- (3) Click **Settings** to change the details of settings.

Note:

- For Ameba-D, the baud rate is 1500000.
- The output directory of BT TRACE LOG can be defined by yourself.

Serial Setting

Serial Port COM1

Baud rate 1500000

Parity None

Stop Bits One

Flow Control None

Decoder Output

Max Display Rows 10000

Max Log File Size(M) 10

Output Directory E:\LogFile

ARM Trace File Path

ROM Trace File ROM.trace

ROM Index File ROM.h

App Trace File App.trace

Patch Trace File Patch.trace

Patch Index File Patch.h

Patch_extension Trace File Patch_extension.trace

DSP Trace File Path

App trace File App.trace

Audio Trace File Audio.trace

Voice Trace File Voice.trace

Patch Trace File Patch.trace

☒ Parse HCI Command and Event ☐ show ModuleId ☐ show undecoded data

Save Load Cancel Confirm

- (4) Click **Start** to start the capture of BT TRACE LOG.
- (5) Reset the device, and look over the log.

```

00009 04-16#14:23:38.916 008 01765 [APP] !**gaps_add_client: client ID = 0
00010 04-16#14:23:38.916 009 01765 [APP] !**tp_client_add_client: client ID = 1
00011 04-16#14:23:38.917 010 01765 [APP] !**vendor_uxpext_client_add: client ID = 2
00012 04-16#14:23:39.414 151 02247 [APP] !**profile callback PROFILE_EVT_SRV_REG_COMPLETE result
00013 04-16#14:23:39.416 154 02247 [APP] !**app_handle_dev_state_evt: init state 1 scan state 0 adv state 0 conn state 0
00014 04-16#14:23:39.416 155 02247 [APP] !**GAP stack ready
00001 2019-04-16#14:29:32.913 Realtek Semiconductor Corporation <-> File Version [2.1.8.2]
00001 2019-04-16#14:30:21.613 Realtek Semiconductor Corporation <-> File Version [2.1.8.2]

```

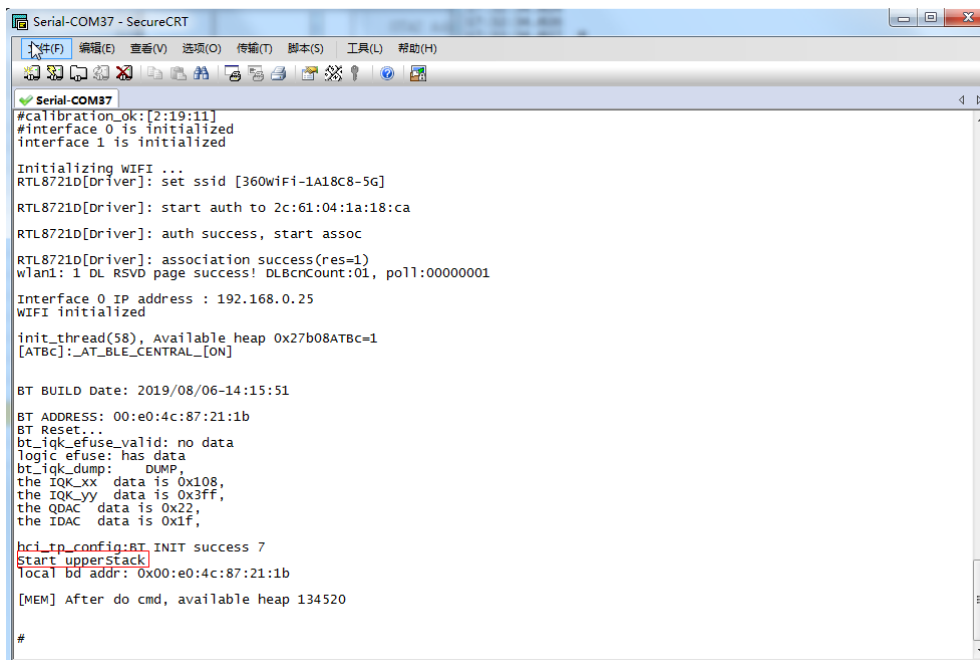
The log files located in the LogFile folder are COM*.cfa and COM*.bin, select them according to the operation time. If the COM*.bin file is constantly increasing, the log may be right.

COM14_btsnoop_2019-04-16_14-33-14.cfa	2019/4/16 14:36	Protocol Analyze...	1 KB
COM14_2019-04-16_14-33-14.bin	2019/4/16 14:36	UltraEdit Docum...	5 KB
COM14_2019-04-16_14-33-24.log	2019/4/16 14:36	文本文档	1 KB

30.3.4.3 LOGUART LOG

The LOGUART LOG can be seen in UART log tools such as SecureCRT. If the hardware level initialization has been done successfully, the log of “Start upperStack” will be printed.

The LOGUART LOG can be stored in a *.txt file.



```

Serial-COM37
#calibration_ok:[2:19:11]
#interface 0 is initialized
interface 1 is initialized

Initializing WIFI ...
RTL8721D[Driver]: set ssid [360WiFi-1A18C8-5G]
RTL8721D[Driver]: start auth to 2c:61:04:1a:18:ca
RTL8721D[Driver]: auth success, start assoc
RTL8721D[Driver]: association success(res=1)
wlan1: 1 DL RSVD page success! DLBcnCount:01, poll:00000001
Interface 0 IP address : 192.168.0.25
WIFI initialized

init_thread(58), Available heap 0x27b08ATBc=1
[ATBc]:_AT_BLE_CENTRAL_[ON]

BT BUILD Date: 2019/08/06-14:15:51
BT ADDRESS: 00:e0:4c:87:21:1b
BT Reset...
bt_igk_efuse_valid: no data
logic_efuse: has data
bt_igk_dump: DUMP,
the IQK_xx data is 0x108,
the IQK_yy data is 0x3ff,
the QDAC data is 0x22,
the IDAC data is 0x1f,

hci_tp_config:BT_INIT success 7
start upperStack
local bd addr: 0x00:e0:4c:87:21:1b

[MEM] After do cmd, available heap 134520

#
  
```

30.3.4.4 APP.trace

APP.trace is for BT TRACE LOG. For BT debug, App.trace must be offered.

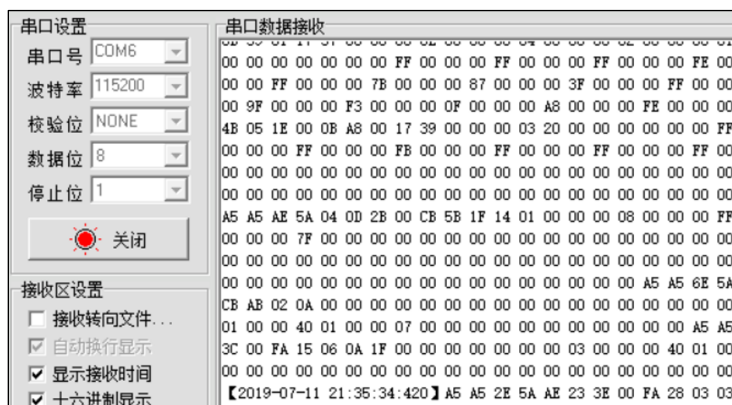
This file is located in project*\asdk\image or tools\bluetooth\DebugAnalyzer after building the project.

30.3.5 FAQ

Q: How to know quickly that whether the log can be captured?

A: UartAssist.exe tool can be used to check. Launch the UartAssist.exe, then select the corresponding serial port and baud rate.

- FW LOG: If you can receive a data packet in UartAssist every 10 seconds, the FW LOG can be captured.



- BT TRACE LOG: If you can see a data packet starting with “7E” in UartAssist when resetting the device every time, the BT TRACE LOG can be captured.



Revision History

Date	Version	Change
2021/8/6	v3	Added the section: Power Save
2021/8/3	v2	Updated the chapter Liquid Crystal Display Controller (LCDC): 16-bit mode is not supported.
2021/5/25	v1	Released version for open-source